

OPTIMAL CONTROL

OPTIMAL CONTROL

Third Edition

FRANK L. LEWIS

Department of Electrical Engineering
Automation & Robotics Research Institute
University of Texas at Arlington
Arlington, Texas

DRAGUNA L. VRABIE

United Technologies Research Center
East Hartford, Connecticut

VASSILIS L. SYRMOS

Department of Electrical Engineering
University of Hawaii at Manoa
Honolulu, Hawaii



WILEY

JOHN WILEY & SONS, INC.

This book is printed on acid-free paper. ☺

Copyright © 2012 by John Wiley & Sons, Inc. All rights reserved.

Published by John Wiley & Sons, Inc., Hoboken, New Jersey.

Published simultaneously in Canada.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 646-8600, or on the web at www.copyright.com. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at www.wiley.com/go/permissions.

Limit of Liability/Disclaimer of Warranty: While the publisher and the author have used their best efforts in preparing this book, they make no representations or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaim any implied warranties of merchantability or fitness for a particular purpose. No warranty may be created or extended by sales representatives or written sales materials. The advice and strategies contained herein may not be suitable for your situation. You should consult with a professional where appropriate. Neither the publisher nor the author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

For general information about our other products and services, please contact our Customer Care Department within the United States at (800) 762-2974, outside the United States at (317) 572-3993 or fax (317) 572-4002.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at <http://booksupport.wiley.com>. For more information about Wiley products, visit www.wiley.com.

Library of Congress Cataloging-in-Publication Data:

Lewis, Frank L.

Optimal control / Frank L. Lewis, Draguna L. Vrabie, Vassilis L. Syrmos.—3rd ed.
p. cm.

Includes bibliographical references and index.

ISBN 978-0-470-63349-6 (cloth); ISBN 978-1-118-12263-1 (ebk); ISBN 978-1-118-12264-8 (ebk); ISBN 978-1-118-12266-2 (ebk); ISBN 978-1-118-12270-9 (ebk); ISBN 978-1-118-12271-6 (ebk); ISBN 978-1-118-12272-3 (ebk)

1. Control theory. 2. Mathematical optimization. I. Vrabie, Draguna L. II. Syrmos, Vassilis L. III. Title.

QA402.3.L487 2012

629.8'312—dc23

2011028234

Printed in the United States of America

10 9 8 7 6 5 4 3 2 1

To Galina, Roma, and Chris, who make every day exciting
—Frank Lewis

*To my mother and my grandmother, for teaching me my potential and
supporting my every choice*
—Draguna Vrabie

To my father, my first teacher
—Vassilis Syrmos

CONTENTS

PREFACE	xi
1 STATIC OPTIMIZATION	1
1.1 Optimization without Constraints /	1
1.2 Optimization with Equality Constraints /	4
1.3 Numerical Solution Methods /	15
Problems /	15
2 OPTIMAL CONTROL OF DISCRETE-TIME SYSTEMS	19
2.1 Solution of the General Discrete-Time Optimization Problem /	19
2.2 Discrete-Time Linear Quadratic Regulator /	32
2.3 Digital Control of Continuous-Time Systems /	53
2.4 Steady-State Closed-Loop Control and Suboptimal Feedback /	65
2.5 Frequency-Domain Results /	96
Problems /	102
3 OPTIMAL CONTROL OF CONTINUOUS-TIME SYSTEMS	110
3.1 The Calculus of Variations /	110
3.2 Solution of the General Continuous-Time Optimization Problem /	112
3.3 Continuous-Time Linear Quadratic Regulator /	135

3.4	Steady-State Closed-Loop Control and Suboptimal Feedback / 154	
3.5	Frequency-Domain Results / 164	
	Problems / 167	
4	THE TRACKING PROBLEM AND OTHER LQR EXTENSIONS	177
4.1	The Tracking Problem / 177	
4.2	Regulator with Function of Final State Fixed / 183	
4.3	Second-Order Variations in the Performance Index / 185	
4.4	The Discrete-Time Tracking Problem / 190	
4.5	Discrete Regulator with Function of Final State Fixed / 199	
4.6	Discrete Second-Order Variations in the Performance Index / 206	
	Problems / 211	
5	FINAL-TIME-FREE AND CONSTRAINED INPUT CONTROL	213
5.1	Final-Time-Free Problems / 213	
5.2	Constrained Input Problems / 232	
	Problems / 257	
6	DYNAMIC PROGRAMMING	260
6.1	Bellman's Principle of Optimality / 260	
6.2	Discrete-Time Systems / 263	
6.3	Continuous-Time Systems / 271	
	Problems / 283	
7	OPTIMAL CONTROL FOR POLYNOMIAL SYSTEMS	287
7.1	Discrete Linear Quadratic Regulator / 287	
7.2	Digital Control of Continuous-Time Systems / 292	
	Problems / 295	
8	OUTPUT FEEDBACK AND STRUCTURED CONTROL	297
8.1	Linear Quadratic Regulator with Output Feedback / 297	
8.2	Tracking a Reference Input / 313	
8.3	Tracking by Regulator Redesign / 327	
8.4	Command-Generator Tracker / 331	
8.5	Explicit Model-Following Design / 338	
8.6	Output Feedback in Game Theory and Decentralized Control / 343	
	Problems / 351	

9	ROBUSTNESS AND MULTIVARIABLE FREQUENCY-DOMAIN TECHNIQUES	355
9.1	Introduction / 355	
9.2	Multivariable Frequency-Domain Analysis / 357	
9.3	Robust Output-Feedback Design / 380	
9.4	Observers and the Kalman Filter / 383	
9.5	LQG/Loop-Transfer Recovery / 408	
9.6	H_∞ DESIGN / 430 Problems / 435	
10	DIFFERENTIAL GAMES	438
10.1	Optimal Control Derived Using Pontryagin's Minimum Principle and the Bellman Equation / 439	
10.2	Two-player Zero-sum Games / 444	
10.3	Application of Zero-sum Games to H_∞ Control / 450	
10.4	Multiplayer Non-zero-sum Games / 453	
11	REINFORCEMENT LEARNING AND OPTIMAL ADAPTIVE CONTROL	461
11.1	Reinforcement Learning / 462	
11.2	Markov Decision Processes / 464	
11.3	Policy Evaluation and Policy Improvement / 474	
11.4	Temporal Difference Learning and Optimal Adaptive Control / 489	
11.5	Optimal Adaptive Control for Discrete-time Systems / 490	
11.6	Integral Reinforcement Learning for Optimal Adaptive Control of Continuous-time Systems / 503	
11.7	Synchronous Optimal Adaptive Control for Continuous-time Systems / 513	
	APPENDIX A REVIEW OF MATRIX ALGEBRA	518
A.1	Basic Definitions and Facts / 518	
A.2	Partitioned Matrices / 519	
A.3	Quadratic Forms and Definiteness / 521	
A.4	Matrix Calculus / 523	
A.5	The Generalized Eigenvalue Problem / 525	
	REFERENCES	527
	INDEX	535

PREFACE

This book is intended for use in a second graduate course in modern control theory. A background in the state-variable representation of systems is assumed. Matrix manipulations are the basic mathematical vehicle and, for those whose memory needs refreshing, Appendix A provides a short review.

The book is also intended as a reference. Numerous tables make it easy to find the equations needed to implement optimal controllers for practical applications.

Our interactions with nature can be divided into two categories: observation and action. While observing, we process data from an essentially uncooperative universe to obtain knowledge. Based on this knowledge, we act to achieve our goals. This book emphasizes the control of systems assuming perfect and complete knowledge. The dual problem of estimating the state of our surroundings is briefly studied in Chapter 9. A rigorous course in optimal estimation is required to conscientiously complete the picture begun in this text.

Our intention is to present optimal control theory in a clear and direct fashion. This goal naturally obscures the more subtle points and unanswered questions scattered throughout the field of modern system theory. What appears here as a completed picture is in actuality a growing body of knowledge that can be interpreted from several points of view and that takes on different personalities as new research is completed.

We have tried to show with many examples that computer simulations of optimal controllers are easy to implement and are an essential part of gaining an intuitive feel for the equations. Students should be able to write simple programs as they progress through the book, to convince themselves that they have confidence in the theory and understand its practical implications.

Relationships to classical control theory have been pointed out, and a root-locus approach to steady-state controller design is included. Chapter 9 presents

some multivariable classical design techniques. A chapter on optimal control of polynomial systems is included to provide a background for further study in the field of adaptive control. A chapter on robust control is also included to expose the reader to this important area. A chapter on differential games shows how to extend the optimality concepts in the book to multiplayer optimization in interacting teams.

Optimal control relies on solving the matrix design equations developed in the book. These equations can be complicated, and exact solution of the Hamilton-Jacobi equations for nonlinear systems may not be possible. The last chapter, on optimal adaptive control, gives practical methods for solving these matrix design equations. Algorithms are given for finding approximate solutions online in real-time using adaptive learning techniques based on data measured along the system trajectories.

The first author wants to thank his teachers: J. B. Pearson, who gave him the initial excitement and passion for the field; E. W. Kamen, who tried to teach him persistence and attention to detail; B. L. Stevens, who forced him to consider applications to real situations; R. W. Newcomb, who gave him self-confidence; and A. H. Haddad, who showed him the big picture and the humor behind it all. We owe our main thanks to our students, who force us daily to take the work seriously and become a part of it.

Acknowledgments

This work was supported by NSF grant ECCS-0801330, ARO grant W91NF-05-1-0314, and AFOSR grant FA9550-09-1-0278.

1

STATIC OPTIMIZATION

In this chapter we discuss optimization when time is not a parameter. The discussion is preparatory to dealing with time-varying systems in subsequent chapters. A reference that provides an excellent treatment of this material is Bryson and Ho (1975), and we shall sometimes follow their point of view.

Appendix A should be reviewed, particularly the section that discusses matrix calculus.

1.1 OPTIMIZATION WITHOUT CONSTRAINTS

A scalar performance index $L(u)$ is given that is a function of a control or decision vector $u \in R^m$. It is desired to determine the value of u that results in a minimum value of $L(u)$.

We proceed to solving this optimization problem by writing the Taylor series expansion for an increment in L as

$$dL = L_u^T du + \frac{1}{2} du^T L_{uu} du + O(3), \quad (1.1-1)$$

where $O(3)$ represents terms of order three. The gradient of L with respect to u is the column vector

$$L_u \triangleq \frac{\partial L}{\partial u}, \quad (1.1-2)$$

and the Hessian matrix is

$$L_{uu} \triangleq \frac{\partial^2 L}{\partial u^2}. \quad (1.1-3)$$

L_{uu} is called the *curvature matrix*. For more discussion on these quantities, see Appendix A.

Note. The gradient is defined throughout the book as a *column* vector, which is at variance with some authors, who define it as a row vector.

A *critical* or *stationary point* is characterized by a *zero increment dL to first order* for all increments du in the control. Hence,

$$L_u = 0 \quad (1.1-4)$$

for a critical point.

Suppose that we are at a *critical point*, so $L_u = 0$ in (1.1-1). For the critical point to be a *local minimum*, it is required that

$$dL = \frac{1}{2} du^T L_{uu} du + O(3) \quad (1.1-5)$$

is positive for all increments du . This is guaranteed if the curvature matrix L_{uu} is positive definite,

$$L_{uu} > 0. \quad (1.1-6)$$

If L_{uu} is negative definite, the critical point is a local maximum; and if L_{uu} is indefinite, the critical point is a saddle point. If L_{uu} is semidefinite, then higher terms of the expansion (1.1-1) must be examined to determine the type of critical point.

The following example provides a tangible meaning to our initial mathematical developments.

Example 1.1-1. Quadratic Surfaces

Let $u \in R^2$ and

$$L(u) = \frac{1}{2} u^T \begin{bmatrix} q_{11} & q_{12} \\ q_{12} & q_{22} \end{bmatrix} u + [s_1 \quad s_2] u \quad (1)$$

$$\triangleq \frac{1}{2} u^T Q u + S^T u. \quad (2)$$

The critical point is given by

$$L_u = Qu + S = 0 \quad (3)$$

and the optimizing control is

$$u^* = -Q^{-1} S. \quad (4)$$

By examining the Hessian

$$L_{uu} = Q \quad (5)$$

one determines the type of the critical point.

The point u^* is a minimum if $L_{uu} > 0$ and it is a maximum if $L_{uu} < 0$. If $|Q| < 0$, then u^* is a **saddle point**. If $|Q| = 0$, then u^* is a **singular point** and in this case L_{uu} does not provide sufficient information for characterizing the nature of the critical point.

By substituting (4) into (2) we find the **extremal value** of the performance index to be

$$\begin{aligned} L^* \triangleq L(u^*) &= \frac{1}{2} S^T Q^{-1} Q Q^{-1} S - S^T Q^{-1} S \\ &= -\frac{1}{2} S^T Q^{-1} S. \end{aligned} \quad (6)$$

Let

$$L = \frac{1}{2} u^T \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} u + [0 \quad 1] u. \quad (7)$$

Then

$$u^* = - \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (8)$$

is a minimum, since $L_{uu} > 0$. Using (6), we see that the minimum value of L is $L^* = -\frac{1}{2}$.

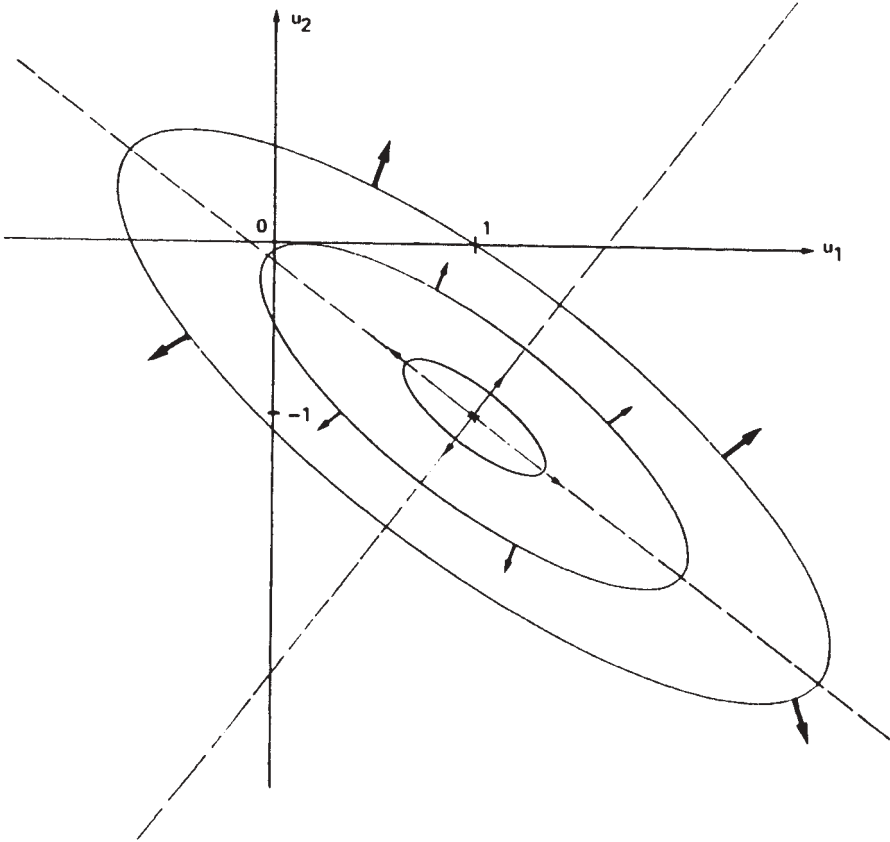


FIGURE 1.1-1 **Contours and the gradient vector.**

The contours of the $L(u)$ in (7) are drawn in Fig. 1.1-1, where $u = [u_1 \ u_2]^T$. The arrows represent the gradient

$$L_u = Qu + S = \begin{bmatrix} u_1 + u_2 \\ u_1 + 2u_2 + 1 \end{bmatrix}. \quad (9)$$

Note that the gradient is always perpendicular to the contours and pointing in the direction of increasing $L(u)$.

We shall use an asterisk to denote optimal values of u and L when we want to be explicit. Usually, however, the asterisk will be omitted. ■

Example 1.1-2. Optimization by Scalar Manipulations

We have discussed optimization in terms of vectors and the gradient. As an alternative approach, we could deal entirely in terms of scalar quantities. To demonstrate, let

$$L(u_1, u_2) = \frac{1}{2}u_1^2 + u_1u_2 + u_2^2 + u_2, \quad (1)$$

where u_1 and u_2 are scalars. A critical point is present where the derivatives of L with respect to *all arguments* are equal to zero:

$$\begin{aligned} \frac{\partial L}{\partial u_1} &= u_1 + u_2 = 0, \\ \frac{\partial L}{\partial u_2} &= u_1 + 2u_2 + 1 = 0. \end{aligned} \quad (2)$$

Solving this system of equations yields

$$u_1 = 1, \quad u_2 = -1; \quad (3)$$

thus, the critical point is $(1, -1)$. Note that (1) is an expanded version of (7) in Example 1.1-1, so we have just derived the same answer by another means.

Vector notation is a tool that simplifies the bookkeeping involved in dealing with multidimensional quantities, and for that reason it is very attractive for our purposes. ■

1.2 OPTIMIZATION WITH EQUALITY CONSTRAINTS

Now let the scalar performance index be $L(x, u)$, a function of the control vector $u \in R^m$ and an auxiliary (state) vector $x \in R^n$. The optimization problem is to determine the control vector u that minimizes $L(x, u)$ and at the same time satisfies the constraint equation

$$f(x, u) = 0. \quad (1.2-1)$$

The auxiliary vector x is determined for a given u by the relation (1.2-1). For a given u , (1.2-1) defines a set of n scalar equations.

To find necessary and sufficient conditions for a local minimum that also satisfies $f(x, u) = 0$, we proceed exactly as we did in the previous section, first expanding dL in a Taylor series and then examining the first- and second-order terms. Let us first gain some insight into the problem, however, by considering it from three points of view (Bryson and Ho 1975, Athans and Falb 1966).

Lagrange Multipliers and the Hamiltonian

Necessary Conditions At a stationary point, dL is equal to zero in the first-order approximation with respect to increments du when df is zero. Thus, at a critical point the following equations are satisfied:

$$dL = L_u^T du + L_x^T dx = 0 \quad (1.2-2)$$

and

$$df = f_u du + f_x dx = 0. \quad (1.2-3)$$

Since (1.2-1) determines x for a given u , the increment dx is determined by (1.2-3) for a given control increment du . Thus, the Jacobian matrix f_x is nonsingular and one can write

$$dx = -f_x^{-1} f_u du. \quad (1.2-4)$$

Substituting this into (1.2-2) yields

$$dL = (L_u^T - L_x^T f_x^{-1} f_u) du. \quad (1.2-5)$$

The derivative of L with respect to u holding f constant is therefore given by

$$\left. \frac{\partial L}{\partial u} \right|_{df=0} = (L_u^T - L_x^T f_x^{-1} f_u)^T = L_u - f_u^T f_x^{-T} L_x, \quad (1.2-6)$$

where f_x^{-T} means $(f_x^{-1})^T$. Note that

$$\left. \frac{\partial L}{\partial u} \right|_{dx=0} = L_u. \quad (1.2-7)$$

Thus, for dL to be zero in the first-order approximation with respect to arbitrary increments du when $df = 0$, we must have

$$L_u - f_u^T f_x^{-T} L_x = 0. \quad (1.2-8)$$

This is a necessary condition for a minimum. Before we derive a sufficient condition, let us develop some more insight by examining two more ways to obtain (1.2-8). Write (1.2-2) and (1.2-3) as

$$\begin{bmatrix} dL \\ df \end{bmatrix} = \begin{bmatrix} L_x^T & L_u^T \\ f_x & f_u \end{bmatrix} \begin{bmatrix} dx \\ du \end{bmatrix} = 0. \quad (1.2-9)$$

This set of linear equations defines a stationary point, and it must have a solution $[dx^T \ du^T]^T$. The critical point is obtained only if the $(n+1) \times (n+m)$ coefficient matrix has rank less than $n+1$. That is, its rows must be linearly dependent so there exists an n vector λ such that

$$[1 \ \lambda^T] \begin{bmatrix} L_x^T & L_u^T \\ f_x & f_u \end{bmatrix} = 0. \quad (1.2-10)$$

Then

$$L_x^T + \lambda^T f_x = 0, \quad (1.2-11)$$

$$L_u^T + \lambda^T f_u = 0. \quad (1.2-12)$$

Solving (1.2-11) for λ gives

$$\lambda^T = -L_x^T f_x^{-1}, \quad (1.2-13)$$

and substituting in (1.2-12) again yields the condition (1.2-8) for a critical point.

Note. The left-hand side of (1.2-8) is the transpose of the Schur complement of L_u^T in the coefficient matrix of (1.2-9) (see Appendix A for more details).

The vector $\lambda \in R^n$ is called a *Lagrange multiplier*, and it will turn out to be an extremely useful tool for us. To give it some additional meaning now, let $du = 0$ in (1.2-2), (1.2-3) and eliminate dx to get

$$dL = L_x^T f_x^{-1} df. \quad (1.2-14)$$

Therefore,

$$\left. \frac{\partial L}{\partial f} \right|_{du=0} = (L_x^T f_x^{-1})^T = -\lambda, \quad (1.2-15)$$

so that $-\lambda$ is the partial of L with respect to the constraint holding the control u constant. It shows the effect on the performance index of holding the control constant when the constraints are changed.

As a third method of obtaining (1.2-8), let us develop the approach we shall use for our analysis in subsequent chapters. Include the constraints in the performance index to define the *Hamiltonian function*

$$H(x, u, \lambda) = L(x, u) + \lambda^T f(x, u), \quad (1.2-16)$$



where $\lambda \in R^n$ is an as yet undetermined Lagrange multiplier. To determine x , u , and λ , which result in a critical point, we proceed as follows.

Increments in H depend on increments in x , u , and λ according to

$$dH = H_x^T dx + H_u^T du + H_\lambda^T d\lambda. \quad (1.2-17)$$

Note that

$$H_\lambda = \frac{\partial H}{\partial \lambda} = f(x, u), \quad (1.2-18)$$

so suppose we choose some value of u and demand that

$$H_\lambda = 0. \quad (1.2-19)$$

Then x is determined for the given u by $f(x, u) = 0$, which is the constraint relation. In this situation the Hamiltonian equals the performance index:

$$H|_{f=0} = L. \quad (1.2-20)$$

Recall that if $f = 0$, then dx is given in terms of du by (1.2-4). We should rather not take into account this coupling between du and dx , so it is convenient to choose λ so that

$$H_x = 0. \quad (1.2-21)$$

Then, by (1.2-17), increments dx do not contribute to dH . Note that this yields a value for λ given by

$$\frac{\partial H}{\partial x} = L_x + f_x^T \lambda = 0 \quad (1.2-22)$$

or (1.2-13).

If (1.2-19) and (1.2-21) hold, then

$$dL = dH = H_u^T du, \quad (1.2-23)$$

since $H = L$ in this situation. To achieve a stationary point, we must therefore finally impose the *stationarity condition*

$$H_u = 0. \quad (1.2-24)$$

In summary, necessary conditions for a minimum point of $L(x, u)$ that also satisfies the constraint $f(x, u) = 0$ are

$$\frac{\partial H}{\partial \lambda} = f = 0, \quad (1.2-25a)$$

$$\frac{\partial H}{\partial x} = L_x + f_x^T \lambda = 0, \quad (1.2-25b)$$

$$\frac{\partial H}{\partial u} = L_u + f_u^T \lambda = 0, \quad (1.2-25c)$$

with $H(x, u, \lambda)$ defined by (1.2-16). The way we shall often use them, these three equations serve to determine x , λ , and u in that respective order. The last two of these equations are (1.2-11) and (1.2-12). In most applications determining the value of λ is not of interest, but this value is required, since it is an intermediate variable that allows us to determine the quantities of interest, u , x , and the minimum value of L .

The usefulness of the Lagrange-multiplier approach can be summarized as follows. In reality dx and du are not independent increments, because of (1.2-4). By introducing an undetermined multiplier λ , however, we obtain an extra degree of freedom, and λ can be selected to make dx and du behave as if they were independent increments. Therefore, setting independently to zero the gradients of H with respect to all arguments as in (1.2-25) yields a critical point. By introducing Lagrange multipliers, the problem of minimizing $L(x, u)$ subject to the constraint $f(x, u) = 0$ is replaced with the problem of minimizing the Hamiltonian $H(x, u, \lambda)$ without constraints.

Sufficient Conditions Conditions (1.2-25) determine a stationary (critical) point. We are now ready to derive a test that guarantees that this point is a minimum. We proceed as we did in Section 1.1.

Write Taylor series expansions for increments in L and f as

$$dL = [L_x^T \quad L_u^T] \begin{bmatrix} dx \\ du \end{bmatrix} + \frac{1}{2} [dx^T \quad du^T] \begin{bmatrix} L_{xx} & L_{xu} \\ L_{ux} & L_{uu} \end{bmatrix} \begin{bmatrix} dx \\ du \end{bmatrix} + O(3), \quad (1.2-26)$$

$$df = [f_x \quad f_u] \begin{bmatrix} dx \\ du \end{bmatrix} + \frac{1}{2} [dx^T \quad du^T] \begin{bmatrix} f_{xx} & f_{xu} \\ f_{ux} & f_{uu} \end{bmatrix} \begin{bmatrix} dx \\ du \end{bmatrix} + O(3), \quad (1.2-27)$$

where

$$f_{xu} \triangleq \frac{\partial^2 f}{\partial u \partial x}$$

and so on. (What are the dimensions of f_{xu} ?) To introduce the Hamiltonian, use these equations to see that

$$\begin{bmatrix} 1 & \lambda^T \end{bmatrix} \begin{bmatrix} dL \\ df \end{bmatrix} = \begin{bmatrix} H_x^T & H_u^T \end{bmatrix} \begin{bmatrix} dx \\ du \end{bmatrix} + \frac{1}{2} [dx^T \quad du^T] \begin{bmatrix} H_{xx} & H_{xu} \\ H_{ux} & H_{uu} \end{bmatrix} \begin{bmatrix} dx \\ du \end{bmatrix} + O(3). \quad (1.2-28)$$

A critical point requires that $f = 0$, and also that dL is zero in the first-order approximation for all increments dx, du . Since f is held equal to zero, df is also zero. Thus, these conditions require $H_x = 0$ and $H_u = 0$ exactly as in (1.2-25).

To find sufficient conditions for a minimum, let us examine the second-order term. First, it is necessary to include in (1.2-28) the dependence of dx on du . Hence, let us suppose we are at a critical point so that $H_x = 0$, $H_u = 0$, and $df = 0$. Then by (1.2-27)

$$dx = -f_x^{-1} f_u du + O(2). \quad (1.2-29)$$

Substituting this relation into (1.2-28) yields

$$dL = \frac{1}{2} du^T \begin{bmatrix} -f_u^T f_x^{-T} & I \end{bmatrix} \begin{bmatrix} H_{xx} & H_{xu} \\ H_{ux} & H_{uu} \end{bmatrix} \begin{bmatrix} -f_x^{-1} f_u \\ I \end{bmatrix} du + O(3). \quad (1.2-30)$$

To ensure a minimum, dL in (1.2-30) should be positive for all increments du .

This is guaranteed if the curvature matrix with constant f equal to zero

$$\begin{aligned} L_{uu}^f &\triangleq L_{uu}|_f = \begin{bmatrix} -f_u^T f_x^{-T} & I \end{bmatrix} \begin{bmatrix} H_{xx} & H_{xu} \\ H_{ux} & H_{uu} \end{bmatrix} \begin{bmatrix} -f_x^{-1} f_u \\ I \end{bmatrix} \\ &= H_{uu} - f_u^T f_x^{-T} H_{xu} - H_{ux} f_x^{-1} f_u + f_u^T f_x^{-T} H_{xx} f_x^{-1} f_u \end{aligned} \quad (1.2-31)$$

is positive definite. Note that if the constraint $f(x, u)$ is identically zero for all x and u , then (1.2-31) reduces to L_{uu} in (1.1-6). If (1.2-31) is negative definite (indefinite), then the stationary point is a constrained maximum (saddle point).

Examples

To gain a feel for the theory we have just developed, let us consider some examples. The first example is a geometric problem that allows easy visualization, while the second involves a quadratic performance index and linear constraint. The second example is representative of the case that is used extensively in controller design for linear systems.

Example 1.2-1. Quadratic Surface with Linear Constraint

Suppose the performance index is as given in Example 1.1-1:

$$L(x, u) = \frac{1}{2} \begin{bmatrix} x & u \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix} + \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix}, \quad (1)$$

where we have simply renamed the old scalar components u_1, u_2 as x, u , respectively. Let the constraint be

$$f(x, u) = x - 3 = 0. \quad (2)$$

The Hamiltonian is

$$H = L + \lambda^T f = \frac{1}{2}x^2 + xu + u^2 + u + \lambda(x - 3), \quad (3)$$

where λ is a scalar. The conditions for a stationary point are (1.2-25), or

$$H_\lambda = x - 3 = 0, \quad (4)$$

$$H_x = x + u + \lambda = 0, \quad (5)$$

$$H_u = x + 2u + 1 = 0. \quad (6)$$

Solving in the order (4), (6), (5) yields $x = 3$, $u = -2$, and $\lambda = -1$. The stationary point is therefore

$$(x, u)^* = (3, -2). \quad (7)$$

To verify that (7) is a minimum, find the constrained curvature matrix (1.2-31):

$$L_{uu}^f = 2. \quad (8)$$

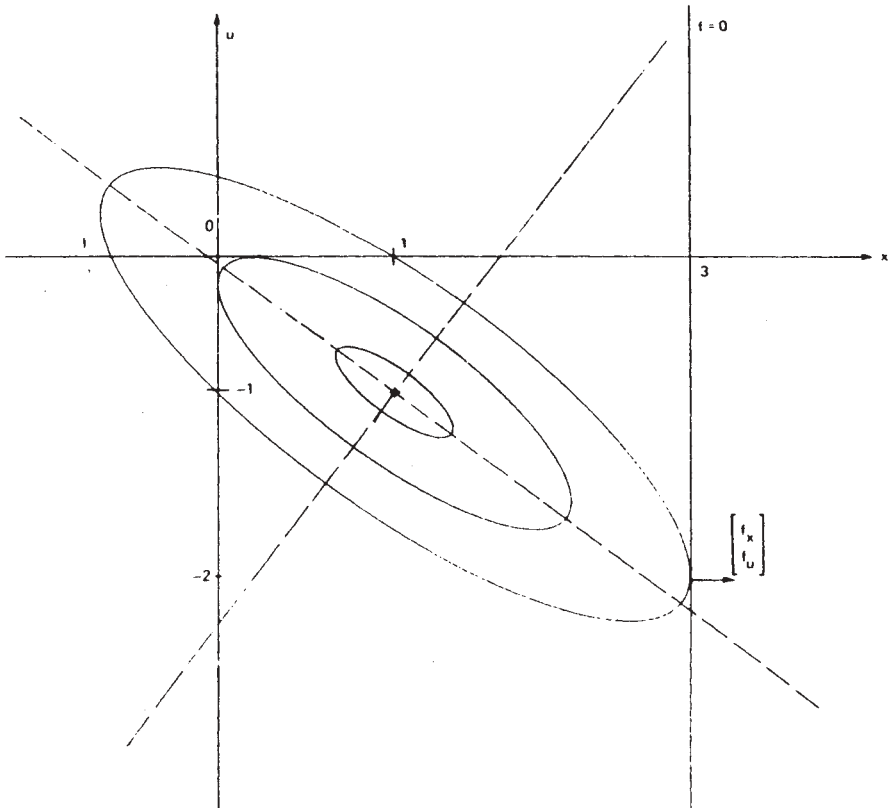


FIGURE 1.2-1 Contours of $L(x, u)$, and the constraint $f(x, u)$.

This is positive, so (7) is a minimum. The contours of $L(x, u)$ and the constraint (2) are shown in Fig. 1.2-1.

It is worthwhile to make an important point. The gradient of $f(x, u)$ in the (x, u) plane is

$$\begin{bmatrix} f_x \\ f_u \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad (9)$$

as shown in Fig. 1.2-1. The gradient of $L(x, u)$ in the plane is

$$\begin{bmatrix} L_x \\ L_u \end{bmatrix} = \begin{bmatrix} x + u \\ x + 2u + 1 \end{bmatrix} \quad (10)$$

(cf. (9) in Example 1.1-1). At the constrained minimum (3, -2), this has a value of

$$\begin{bmatrix} L_x \\ L_u \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (11)$$

Note that the gradients of f and L are *parallel* at the stationary point. This means that the constrained minimum occurs where the constraint (2) is tangent to an elliptical contour of L . Moving in either direction along the line $f = 0$ will then increase the value of L . The value of L at the constrained minimum is found by substituting $x = 3$, $u = -2$ into (1) to be $L^* = 0.5$. Since $\lambda = -1$, holding u constant at -2 and changing the constraint by df (i.e., moving the line in Fig. 1.2-1 to the right by df) will result in an *increase* in the value of $L(x, u)$ of $dL = -\lambda df = df$ (see (1.2-15)). ■

Example 1.2-2. Quadratic Performance Index with Linear Constraint

Consider the quadratic performance index

$$L(x, u) = \frac{1}{2}x^T Qx + \frac{1}{2}u^T Ru \quad (1)$$

with linear constraint

$$f(x, u) = x + Bu + c = 0, \quad (2)$$

where $x \in R^n$, $u \in R^m$, $f \in R^n$, $\lambda \in R^n$, Q , R , and B are matrices, and c is an n vector. We assume $Q > 0$ and $R > 0$ (with both symmetric). This *static linear quadratic* (LQ) *problem* will be further generalized in Chapters 2 and 3 to apply to time-varying systems.

The contours of $L(x, u)$ are hyperellipsoids, and $f(x, u) = 0$ defines a hyperplane intersecting them. The stationary point occurs where the gradients of f and L are parallel.

The Hamiltonian is

$$H = \frac{1}{2}x^T Qx + \frac{1}{2}u^T Ru + \lambda^T(x + Bu + c) \quad (3)$$

and the conditions for a stationary point are

$$H_\lambda = x + Bu + c = 0, \quad (4)$$

$$H_x = Qx + \lambda = 0, \quad (5)$$

$$H_u = Ru + B^T \lambda = 0. \quad (6)$$

12 STATIC OPTIMIZATION

To solve these, first use the stationarity condition (6) to find an expression for u in terms of λ ,

$$u = -R^{-1}B^T\lambda. \quad (7)$$

According to (5)

$$\lambda = -Qx, \quad (8)$$

and taking into account (4) results in

$$\lambda = QBu + Qc. \quad (9)$$

Using this in (7) yields

$$u = -R^{-1}B^T(QBu + Qc) \quad (10)$$

or

$$\begin{aligned} (I + R^{-1}B^TQB)u &= -R^{-1}B^TQc, \\ (R + B^TQB)u &= -B^TQc. \end{aligned} \quad (11)$$

Since $R > 0$ and $B^TQB > 0$, we can invert $R + B^TQB$ and so the optimal control is

$$u = -(R + B^TQB)^{-1}B^TQc. \quad (12)$$

Using (12) in (4) and (9) gives the optimal-state and multiplier values of

$$x = -(I - B(R + B^TQB)^{-1}B^TQ)c, \quad (13)$$

$$\lambda = (Q - QB(R + B^TQB)^{-1}B^TQ)c. \quad (14)$$

By the matrix inversion lemma (see Appendix A)

$$\lambda = (Q^{-1} + BR^{-1}B^T)^{-1}c \quad (15)$$

if $|Q| \neq 0$.

To verify that control (12) results in a minimum, use (1.2-31) to determine that the constrained curvature matrix is

$$L_{uu}^f = R + B^TQB, \quad (16)$$

which is positive definite by our restrictions on R and Q . Using (12) and (13) in (1) yields the optimal value

$$L^* = \frac{1}{2}c^T[Q - QB(R + B^TQB)^{-1}B^TQ]c, \quad (17)$$

$$L^* = \frac{1}{2}c^T\lambda, \quad (18)$$

so that

$$\frac{\partial L^*}{\partial c} = \lambda. \quad (19)$$

■

Effect of Changes in Constraints

Equation (1.2-28) expresses the increment dL in terms of df , dx , and du . In the discussion following that equation we let $df = 0$, found dx in terms of du , and expressed dL in terms of du . That gave us conditions for a stationary point ($H_x = 0$ and $H_u = 0$) and led to the second-order coefficient matrix L_{uu}^f in (1.2-31), which provided a test for the stationary point to be a constrained minimum.

In this subsection we are interested in dL as a function of an increment df in the constraint. We want to see how the performance index L changes in response to changes in the constraint f if we remain at a stationary point. We are therefore trying to find stationary points near a given stationary point. See Fig. 1.2-2, which shows how the stationary point moves with changes in f .

At the stationary point $(u, x)^*$ defined by $f(x, u) = 0$, the conditions $H_\lambda = 0$, $H_x = 0$, and $H_u = 0$ are satisfied. If the constraint changes by an increment so that $f(x, u) = df$, then the stationary point moves to $(u + du, x + dx)$. The partials in (1.2-25) change by

$$dH_\lambda = df = f_x dx + f_u du, \quad (1.2-32a)$$

$$dH_x = H_{xx} dx + H_{xu} du + f_x^T d\lambda, \quad (1.2-32b)$$

$$dH_u = H_{ux} dx + H_{uu} du + f_u^T d\lambda. \quad (1.2-32c)$$

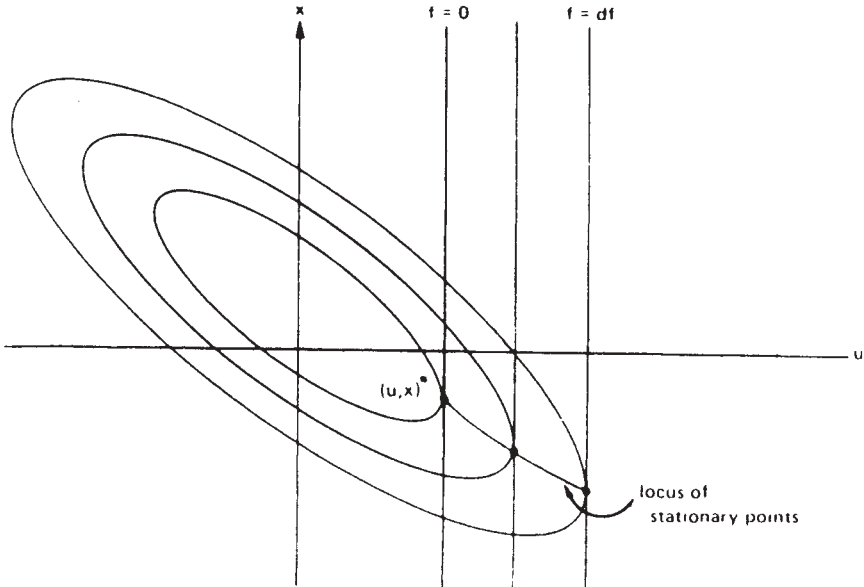


FIGURE 1.2-2 Locus of stationary points as the constraint varies.

In order that we remain at a stationary point, the increments dH_x and dH_u should be zero. This requirement imposes certain relations between the changes dx , du , and df , which we shall use in (1.2-28) to determine dL as a function of df .

To find dx and du as functions of df with the requirement that we remain at an optimal solution, use (1.2-32a) to find

$$dx = f_x^{-1} df - f_x^{-1} f_u du, \quad (1.2-33)$$

and set (1.2-32b) to zero to find

$$d\lambda = -f_x^{-T} (H_{xx} dx + H_{xu} du). \quad (1.2-34)$$

Now use these relations in (1.2-32c) to obtain

$$\begin{aligned} dH_u = & (H_{uu} - H_{ux} f_x^{-1} f_u - f_u^T f_x^{-T} H_{xu} + f_u^T f_x^{-T} H_{xx} f_x^{-1} f_u) du \\ & + (H_{ux} - f_u^T f_x^{-T} H_{xx}) f_x^{-1} df = 0 \end{aligned}$$

so that

$$du = - (L_{uu}^f)^{-1} (H_{ux} - f_u^T f_x^{-T} H_{xx}) f_x^{-1} df \triangleq -C df. \quad (1.2-35)$$

Using (1.2-35) in (1.2-33) yields

$$\begin{aligned} dx &= \left[I + f_x^{-1} f_u (L_{uu}^f)^{-1} (H_{ux} - f_u^T f_x^{-T} H_{xx}) \right] f_x^{-1} df \\ &= f_x^{-1} (I + f_u C) df. \end{aligned} \quad (1.2-36)$$

Equations (1.2-35) and (1.2-36) are the required expressions for the increments in the stationary values of control and state as functions of df . If $|L_{uu}^f| \neq 0$, then dx and du can be determined in terms of df , and the existence of neighboring optimal solutions as f varies is guaranteed.

To determine the increment dL in the optimal performance index as a function of df , substitute (1.2-35) and (1.2-36) into (1.2-28), using $H_x = 0$, $dH_u = 0$, since we began at a stationary point $(u, x)^*$. The result is found after some work to be

$$dL = -\lambda^T df + \frac{1}{2} df^T (f_x^{-T} H_{xx} f_x^{-1} - C^T L_{uu}^f C) df + O(3). \quad (1.2-37)$$

From this we see that the first and second partial derivatives of $L^*(x, u)$ with respect to $f(x, u)$ under the restrictions $dH_x = 0$, $dH_u = 0$ are

$$\left. \frac{\partial L^*}{\partial f} \right|_{H_x, H_u} = -\lambda, \quad (1.2-38)$$

$$\left. \frac{\partial^2 L^*}{\partial f^2} \right|_{H_x, H_u} = f_x^{-T} H_{xx} f_x^{-1} - C^T L_{uu}^f C. \quad (1.2-39)$$

Equation (1.2-38) allows a further interpretation of the Lagrange multiplier; it indicates the rate of change of the *optimal* value of the performance index with respect to the constraint.

1.3 NUMERICAL SOLUTION METHODS

Analytic solutions for the stationary point $(u, x)^*$ and minimal value L^* of the performance index cannot be found except for simple functions $L(x, u)$ and $f(x, u)$. In most practical cases, numerical optimization methods must be used. Many methods exist, but *steepest descent* or *gradient* (Luenberger 1969, Bryson and Ho 1975) methods are probably the simplest.

The steps in constrained minimization by the method of steepest descent are (Bryson and Ho 1975)

1. Select an initial value for u .
2. Determine x from $f(x, u) = 0$.
3. Determine λ from $\lambda = -f_x^{-T} L_x$.
4. Determine the gradient vector $H_u = L_u + f_u^T \lambda$.
5. Update the control vector by $\Delta u = -\alpha H_u$, where K is a positive scalar constant (to find a maximum use $\Delta u = \alpha H_u$).
6. Determine the predicted change in the value of L , $\Delta L = H_u^T \Delta u = -\alpha H_u^T H_u$. If ΔL is sufficiently small, stop. Otherwise, go to step 2.

There are many variations to this procedure. If the step-size constant K is too large, then the algorithm may overshoot the stationary point $(u, x)^*$ and convergence may not occur. The step size should usually be reduced as $(u, x)^*$ is approached, and several of the existing variations differ in the approach to adapting K .

Many software routines are available for unconstrained optimization. The numerical solution of the constrained optimization problem of minimizing $L(x, u)$ subject to $f(x, u) = 0$ can be obtained using the MATLAB function *constr.m* available under the Optimization Toolbox. This function takes in the user-defined subroutine *funct.m*, which computes the value of the function, the constraints, and the initial conditions.

PROBLEMS

Section 1.1

1.1-1. Find the critical points u^* (classify them) and the value of $L(u^*)$ in Example 1.1-1 if

a. $Q = \begin{bmatrix} -1 & 1 \\ 1 & -2 \end{bmatrix}$, $S^T = [0 \quad 1]$.

b. $Q = \begin{bmatrix} -1 & 1 \\ 1 & 2 \end{bmatrix}$, $S^T = [0 \quad 1]$.

Sketch the contours of L and find the gradient L_u .

1.1-2. Find the minimum value of

$$L(x_1, x_2) = x_1^2 - x_1x_2 + x_2^2 + 3x_1. \quad (1)$$

Find the curvature matrix at the minimum. Sketch the contours, showing the gradient at several points.

1.1-3. Failure of test for minimality. The function $f(x, y) = x^2 + y^4$ has a minimum at the origin.

- Verify that the origin is a critical point.
- Show that the curvature matrix is singular at the origin.
- Prove that the critical point is indeed a minimum.

Section 1.2

1.2-1. Ship closest point of approach. A ship is moving at 10 miles per hour on a course of 30° (measured clockwise from north, which is 0°). Find its closest point of approach to an island that at time $t = 0$ is 20 miles east and 30 miles north of it. Find the distance to the island at this point. Find the time of closest approach.

1.2-2. Shortest distance between two points. Let $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ be two given points. Find the third point $P_3 = (x_3, y_3)$ such that $d_1 = d_2$ is minimized, where d_1 is the distance from P_3 to P_1 and d_2 is the distance from P_3 to P_2 .

1.2-3. Meteor closest point of approach. A meteor is in a hyperbolic orbit described with respect to the earth at the origin by

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1. \quad (1)$$

Find its closest point of approach to a satellite that is in such an orbit that it has a constant position of (x_1, y_1) . Verify that the solution indeed yields a minimum.

1.2-4. Shortest distance between a parabola and a point. A meteor is moving along the path

$$y = x^2 + 3x - 6. \quad (1)$$

A space station is at the point $(x, y) = (2, 2)$.

- Use Lagrange multipliers to find a cubic equation for x at the closest point of approach.

- b. Find the closest point of approach (x, y) , and the distance from this point to $(2, 2)$.

1.2-5. Rectangles with maximum area, minimum perimeter

- a. Find the rectangle of maximum area with perimeter p . That is, maximize

$$L(x, y) = xy \quad (1)$$

subject to

$$f(x, y) = 2x + 2y - p = 0. \quad (2)$$

- b. Find the rectangle of minimum perimeter with area a^2 . That is, minimize

$$L(x, y) = 2x + 2y \quad (3)$$

subject to

$$f(x, y) = xy - a^2 = 0. \quad (4)$$

- c. In each case, sketch the contours of $L(x, y)$ and the constraint. Optimization problems related like these two are said to be *dual*.

1.2-6. Linear quadratic case. Minimize

$$L = \frac{1}{2}x^T \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} x + \frac{1}{2}u^T \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} u$$

if

$$x = \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 1 & 0 \end{bmatrix} u.$$

Find x^* , u^* , λ^* , L^* .

1.2-7. Linear quadratic case. In the LQ problem define the *Kalman gain*

$$K \triangleq (B^TQB + R)^{-1}B^TQ \quad (1)$$

- a. Express u^* , λ^* , x^* , and L^* in terms of K .

- b. Let

$$S_0 \triangleq Q - QB(B^TQB + R)^{-1}B^TQ \quad (2)$$

so that $L^* = c^T S_0 c / 2$. Show that

$$S_0 = Q(I - BK) = (I - BK)^T Q (I - BK) + K^T R K. \quad (3)$$

Hence, factor L^* as a perfect square. (Let $\sqrt{Q}$ and $\sqrt{R}$ be the square roots of Q and R .)

c. Show that

$$S_0 = (Q^{-1} + BR^{-1}B^T)^{-1}. \quad (4)$$

1.2-8. Geometric mean less than or equal to arithmetic mean

- Show that the minimum value of $x^2y^2z^2$ on the sphere $x^2 + y^2 + z^2 = r^2$ is $(r^2/3)^3$.
- Show that the maximum value of $x^2 + y^2 + z^2$ on the sphere $x^2y^2z^2 = (r^2/3)^3$ is r^2 .
- Generalize part a or b and so deduce that, for $a_i > 0$,

$$(a_1a_2 \cdots a_n)^{1/n} \leq (a_1 + a_2 + \cdots + a_n)/n.$$

Note: The problems in parts a and b are dual (Fulks 1967).

1.2-9. Find the point nearest the origin on the line $3x + 2y + z = 1$, $x + 2y - 3z = 4$.

1.2-10. Rectangle inside Ellipse

- Find the rectangle of maximum perimeter that can be inscribed inside an ellipse. That is, maximize $4(x + y)$ subject to constraint $x^2/a^2 + y^2/b^2 = 1$.
- Find the rectangle of maximum area $4xy$ that can be inscribed inside an ellipse.

2

OPTIMAL CONTROL OF DISCRETE-TIME SYSTEMS

We are now ready to extend the methods of Chapter 1 to the optimization of a performance index associated with a system developing dynamically through time. It is important to realize that we shall be making a fairly subtle change of emphasis. In Chapter 1, the focus of our attention was initially on the performance index, and we introduced the notion of constraints as the discussion proceeded. In this and subsequent chapters we are forced to begin with the constraint equations, since these represent the dynamics of the system. These constraint relations are *fixed by the physics of the problem*. The performance index is *selected by the engineer* as it represents the desired behavior of the dynamical system.

In Section 2.1 we derive the general solution of the optimization problem for discrete-time systems. In Section 2.2 we discuss the very important special case of linear systems with a quadratic performance index. We first discuss the case of *fixed final state*, which yields an open-loop control, followed by the situation of *free final state*, which yields a closed-loop control. In Section 2.3 we show how to apply these results to the digital control of continuous-time systems.

Some connections with classical root-locus design are given in Section 2.5.

2.1 SOLUTION OF THE GENERAL DISCRETE-TIME OPTIMIZATION PROBLEM

Problem Formulation

Let the plant be described by the very general nonlinear discrete-time dynamical equation

$$x_{k+1} = f^k(x_k, u_k) \quad (2.1-1)$$

with initial condition x_0 . The superscript on function f indicates that, in general, the system, and thus its model, can have time-varying dynamics. Let the state x_k be a vector of size n and the control input u_k be a vector of size m . Equation (2.1-1) represents the constraint, since it determines the state at time $k + 1$ given the control and state at time k . Clearly, f is a vector of n functions.

Let an associated scalar performance index, specified by the engineer, be given in the general form

$$J_i = \phi(N, x_N) + \sum_{k=i}^{N-1} L^k(x_k, u_k), \quad (2.1-2)$$

where $[i, N]$ is the time interval, on a discrete time scale with a fixed sample step, over which we are interested in the behavior of the system. $\phi(N, x_N)$ is a function of the final time N and the state at the final time, and $L^k(x_k, u_k)$ is a generally time-varying function of the state and control input at each intermediate time k in $[i, N]$.

The *optimal control problem* is to find the control u_k^* on the interval $[i, N]$ (i.e., $u_k^*, \forall k \in [i, N]$) that drives the system (2.1-1) along a trajectory x_k^* such that the value of the performance index (2.1-2) is optimized.

Here we note that relative to the meaning that it is attached to the performance index, the optimization problem can be either a minimization or a maximization problem. For the case that the performance index represents the costs accrued during the operation of the system over the time interval $[i, N]$, the optimal control input is determined to minimize the performance index, while in the situation related to accumulation of value over the time interval $[i, N]$, the optimal control input is determined to maximize the performance index (2.1-2). As in most industrial applications the optimal control problem deals with minimization of control errors as well as of control effort, without reducing the generality of the formulation, herein we will treat the optimal control problem as a minimization problem.

Example 2.1-1. Some Useful Performance Indices

To clarify the problem formulation, it is worthwhile to discuss some common performance indices that we can select for the given system (2.1-1).

a. Minimum-time Problems

Suppose we want to find the control u_k to drive the system from the given initial state x_0 to a desired final state $x \in R^n$ in minimum time. Then we could select the performance index

$$J = N = \sum_{k=0}^{N-1} 1 \quad (1)$$

and specify the *boundary condition*

$$x_N = x. \quad (2)$$

In this case one can consider either $\phi = N$ and $L = 0$, or equivalently $\phi = 0$ and $L = 1$.

b. *Minimum-fuel Problems*

To find the scalar control u_k to drive the system from x_0 to a desired final state x at a fixed time N using minimum fuel, we could use

$$J = \sum_{k=0}^{N-1} |u_k|, \quad (3)$$

since the fuel burned is proportional to the magnitude of the control vector. Then $\phi = 0$ and $L_k = |u_k|$. The boundary condition $x_N = x$ would again apply.

c. *Minimum-energy Problems*

Suppose we want to find u_k to minimize the energy of the final state and all intermediate states, and also that of the control used to achieve this. Let the final time N again be fixed. Then we could use

$$J = \frac{1}{2} s x_N^T x_N + \frac{1}{2} \sum_{k=0}^{N-1} (q x_k^T x_k + r u_k^T u_k), \quad (4)$$

where q , r , and s are scalar weighting factors. Then $\phi = \frac{1}{2} s x_N^T x_N$ and $L = \frac{1}{2} (q x_k^T x_k + r u_k^T u_k)$ are quadratic functions.

Minimizing the energy corresponds in some sense to keeping the state and the control close to zero. If it is more important to us that the intermediate state be small, then we should choose q large to weight it heavily in J , which we are trying to minimize. If it is more important that the control energy be small, then we should select a large value of r . If we are more interested in a small final state, then s should be large.

For more generality, we could select weighting *matrices* Q , R , S instead of scalars. The performance index can in this case be represented as

$$J = \frac{1}{2} x_N^T S x_N + \frac{1}{2} \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k). \quad (5)$$

■

At this point, several things should be clearly understood. First, the system dynamics (2.1-1) are *given* by the physics of the problem, while the performance index (2.1-2) is what we *choose* to achieve the desired system response. Second, to achieve different control objectives, different types of performance indices J are selected. Finally, the optimal control problem is characterized by *compromises and trade-offs*, with different weighting factors in J resulting in different balances between conformability with performance objectives and magnitude of the required optimal controls.

In practice, it is usually necessary to do a control design with a trial performance index J , compute the optimal control u_k^* , and then run a computer simulation to see how the system responds to this u_k^* . If the response is not acceptable, the entire process is repeated using another J with different state and control weightings. After several repetitions have been done to find an acceptable u_k^* , this final version of u_k^* is applied to the actual system.

Problem Solution

Let us now solve the optimal control problem for the general nonlinear system (2.1-1) with associated performance index (2.1-2). To determine the optimal control sequence $u_i^*, u_{i+1}^*, \dots, u_{N-1}^*$ minimizing J , we proceed basically as we did in Chapter 1, using the powerful Lagrange-multiplier approach. Since there is a constraint function $f^k(x_k, u_k)$ specified at each time k in the interval of interest $[i, N]$, we also require a Lagrange multiplier at each time. *Each constraint has an associated Lagrange multiplier.*

Thus, let $\lambda_k \in R^n$, and append the constraint (2.1-1) to the performance index (2.1-2) to define an augmented performance index J' by

$$J' = \phi(N, x_N) + \sum_{k=i}^{N-1} [L^k(x_k, u_k) + \lambda_{k+1}^T (f^k(x_k, u_k) - x_{k+1})]. \quad (2.1-3)$$

Note that we have associated with f^k the multiplier λ_{k+1} , not λ_k . This is done with the benefit of hindsight, as it makes the solution neater.

Defining the *Hamiltonian function* as

$$H^k(x_k, u_k) = L^k(x_k, u_k) + \lambda_{k+1}^T f^k(x_k, u_k), \quad (2.1-4)$$

we can write

$$J' = \phi(N, x_N) - \lambda_N^T x_N + H^i(x_i, u_i) + \sum_{k=i+1}^{N-1} [H^k(x_k, u_k) - \lambda_k^T x_k], \quad (2.1-5)$$

where some minor manipulations with indices have been performed. Note that the *Hamiltonian is defined slightly differently than in Chapter 1*, since we did not include x_{k+1} in H^k . Furthermore, a Hamiltonian is defined at each time k .

We now want to examine the increment in J' due to increments in all the variables x_k , λ_k , and u_k . We assume the *final time N is fixed*. According to the Lagrange-multiplier theory, at a constrained minimum this increment dJ' *should be zero*. Therefore, write

$$\begin{aligned} dJ' = & (\phi_{x_N} - \lambda_N)^T dx_N + (H_{x_i}^i)^T dx_i + (H_{u_i}^i)^T du_i \\ & + \sum_{k=i+1}^{N-1} [(H_{x_k}^k - \lambda_k)^T dx_k + (H_{u_k}^k)^T du_k] + \sum_{k=i+1}^N (H_{\lambda_k}^{k-1} - x_k)^T d\lambda_k, \end{aligned} \quad (2.1-6)$$

where

$$H_{x_k}^k \triangleq \frac{\partial H^k}{\partial x_k}$$

and so on. *Necessary conditions* for a constrained minimum are thus given by

$$\boxed{x_{k+1} = \frac{\partial H^k}{\partial \lambda_{k+1}}}, \quad k = i, \dots, N-1, \quad (2.1-7a)$$

$$\lambda_k = \frac{\partial H^k}{\partial x_k}, \quad k = i, \dots, N-1, \quad (2.1-7b)$$

$$0 = \frac{\partial H^k}{\partial u_k}, \quad k = i, \dots, N-1, \quad (2.1-7c)$$

which arise from the terms inside the summations and the coefficient of du_i , and

$$\left(\frac{\partial \phi}{\partial x_N} - \lambda_N \right)^T dx_N = 0, \quad (2.1-8a)$$

$$\left(\frac{\partial H^i}{\partial x_i} \right)^T dx_i = 0. \quad (2.1-8b)$$

Examining (2.1-3) and (2.1-4) one can see that λ_i does not appear in J' . We have defined it in such a manner that the lower index in (2.1-7b) can be taken as i , instead of $i+1$, solely as a matter of neatness.

These conditions are certainly not intuitively obvious, so we should discuss them a little to see what they mean. First, compare (2.1-7) with the conditions for a static minimum (1.2-25). They are very similar, except that our new conditions must hold at each time k in the interval of interest $[i, N-1]$, since x_k , u_k , and λ_k are now sequences. Equation (2.1-7c) is called the *stationarity condition*.

Writing (2.1-7) explicitly in terms of L^k and f^k using (2.1-4) yields the formulation in Table 2.1-1. Equality (2.1-9a) is just the constraint, or system, equation. It is a recursion for the state x_k that develops *forward* in time. Evidently, (2.1-9b) is a recursion for λ_k that develops *backward* in time! The (fictitious) Lagrange multiplier is thus a variable that is determined by its own dynamical equation. It is called the *costate* of the system, and (2.1-9b) is called the *adjoint system*. The system (2.1-9a) and the adjoint system (2.1-9b) are *coupled difference equations*. They define a *two-point boundary-value problem*, since the boundary conditions required for solution are the *initial* state x_i and the *final* costate λ_N . These problems are, in general, extremely difficult to solve. We consider some examples later.

The stationarity condition (2.1-9c) allows the optimal control u_k to be expressed in terms of the costate. We therefore have a rather curious situation: we do not really care what λ_k is, but this method of solution requires us to find λ_k as an intermediate step in finding the optimal control.

We have not yet discussed (2.1-8). The first of these equations holds only at final time $k=N$, whereas the second holds only at initial time $k=i$. They are not dynamical recursions like (2.1-7a) and (2.1-7b). These two equations specify the *split boundary conditions* needed to solve the recursions (2.1-9). Two possibilities exist for each of these equations.

If the *initial state* x_i is *fixed*, then $dx_i = 0$, so that (2.1-8b) holds regardless of the value of $H^i_{x_i}$. In the case of *free initial state*, dx_i is not zero, so (2.1-8b) demands that

$$\frac{\partial H^i}{\partial x_i} = 0. \quad (2.1-10)$$

TABLE 2.1-1 Discrete Nonlinear Optimal Controller*System model:*

$$x_{k+1} = f^k(x_k, u_k), \quad k > i$$

Performance index:

$$J_i = \phi(N, x_N) + \sum_{k=i}^{N-1} L^k(x_k, u_k)$$

Hamiltonian:

$$H^k = L^k + \lambda_{k+1}^T f^k$$

*Optimal controller**State equation:*

$$x_{k+1} = \frac{\partial H^k}{\partial \lambda_{k+1}} = f^k(x_k, u_k) \quad (2.1-9a)$$

Costate equation:

$$\lambda_k = \frac{\partial H_k}{\partial x_k} = \left(\frac{\partial f^k}{\partial x_k} \right)^T \lambda_{k+1} + \frac{\partial L^k}{\partial x_k} \quad (2.1-9b)$$

Stationarity condition:

$$0 = \frac{\partial H^k}{\partial u_k} = \left(\frac{\partial f^k}{\partial u_k} \right)^T \lambda_{k+1} + \frac{\partial L^k}{\partial u_k} \quad (2.1-9c)$$

Boundary conditions:

$$\begin{aligned} \left(\frac{\partial L^i}{\partial x_i} + \left(\frac{\partial f^i}{\partial x_i} \right)^T \lambda_{i+1} \right)^T dx_i &= 0 \\ \left(\frac{\partial \phi}{\partial x_N} - \lambda_N \right)^T dx_N &= 0 \end{aligned}$$

In our applications the system starts at a known initial state x_i . Thus, the first case holds, $dx_i = 0$, and there is no constraint on the value of $H_{x_i}^i$. We therefore ignore (2.1-8b) and use as the initial condition the given value of x_i .

We do need to deal with two possibilities for the final state x_N . In the case of a **fixed final state** we use the desired value of x_N as the **terminal condition**. Then x_N is not free to be varied in determining the optimal solution and $dx_N = 0$, so that (2.1-8a) holds. On the other hand, if we are not interested in a particular value for the final state, then x_N can be varied in determining the optimal solution. In this case dx_N is not zero. For this **free-final-state** situation, (2.1-8a) demands that

$$\lambda_N = \frac{\partial \phi}{\partial x_N}. \quad (2.1-11)$$

Then, the terminal condition is the value (2.1-11) of the final costate λ_N .

In summary, the initial condition for the two-point boundary-value problem (2.1-9) is the known value of x_i . The final condition is either a desired value of x_N or the value (2.1-11) of λ_N . These comments will become clearer as we proceed.

An Example

To develop some feel for the theory we have just derived, let us consider an example. We shall see that the solution of the optimal control problem is not straightforward even in the simplest cases, because of the two-point nature of the state and costate equations, but that once the solution is obtained it imparts a great deal of intuition about the control of the system.

We also show how to run software simulations to test our optimal control designs.

Example 2.1-2. Optimal Control for a Scalar Linear System

Consider the simple linear dynamical system

$$x_{k+1} = ax_k + bu_k, \quad (1)$$

where lowercase a and b are used to emphasize that we are dealing with the scalar case. Let the given initial condition be x_0 . Suppose the interval of interest is $[0, N]$ and that we are concerned with minimizing control energy so that

$$J_0 = \frac{r}{2} \sum_{k=0}^{N-1} u_k^2 \quad (2)$$

for some scalar weighting factor r .

Let us discuss two cases, corresponding to two ways in which we might want the system to behave.

a. Fixed Final State

First, suppose we want to make the system end up at time $k = N$ in exactly the particular (reference) state r_N :

$$x_N = r_N. \quad (3)$$

To find the optimal control sequence $u_0, u_1, \dots, u_{N-1}$ (note that x_N does not depend on u_N) that drives (1) from the given x_0 to the desired $x_N = r_N$ while minimizing (2), we can use Table 2.1-1. First, let us compute (2.1-9). The Hamiltonian is

$$H^k = L^k + \lambda_{k+1}^T f^k = \frac{r}{2} u_k^2 + \lambda_{k+1} (ax_k + bu_k), \quad (4)$$

so the conditions (2.1-9) are

$$x_{k+1} = \frac{\partial H^k}{\partial \lambda_{k+1}} = ax_k + bu_k, \quad (5)$$

$$\lambda_k = \frac{\partial H^k}{\partial x_k} = a\lambda_{k+1}, \quad (6)$$

$$0 = \frac{\partial H^k}{\partial u_k} = ru_k + b\lambda_{k+1}. \quad (7)$$

Solving the stationarity condition (7) for u_k in terms of the costate yields

$$u_k = -\frac{b}{r}\lambda_{k+1}. \quad (8)$$

If we can find the optimal λ_k , we can therefore use (8) to find the optimal control. To find λ_k , eliminate u_k in (5) using (8). Then

$$x_{k+1} = ax_k - \frac{b^2}{r}\lambda_{k+1} = ax_k - \gamma\lambda_{k+1}, \quad (9)$$

where

$$\gamma \triangleq \frac{b^2}{r} \quad (10)$$

is the ratio of “control effect” to control weighting.

Equation (6) is a simple homogeneous difference equation, with solution given by

$$\lambda_k = a^{N-k}\lambda_N. \quad (11)$$

This is all well and good, but we do not know λ_N . To find it, proceed as follows. Use (11) in (9) to get

$$x_{k+1} = ax_k - \gamma a^{N-k-1}\lambda_N. \quad (12)$$

This can be viewed as a difference equation with a forcing function of $-\gamma\lambda_N a^{N-k-1}$, so the solution in terms of x_0 is

$$\begin{aligned} x_k &= a^k x_0 - \sum_{i=0}^{k-1} a^{k-i-1} (\gamma \lambda_N a^{N-i-1}) \\ &= a^k x_0 - \gamma \lambda_N a^{N+k-2} \sum_{i=0}^{k-1} a^{-2i}. \end{aligned} \quad (13)$$

Using the formula for the sum of a geometric series we have

$$\begin{aligned} x_k &= a^k x_0 - \gamma \lambda_N a^{N+k-2} \frac{1 - a^{-2k}}{1 - a^{-2}} \\ &= a^k x_0 - \gamma \lambda_N a^{N-k} \frac{1 - a^{2k}}{1 - a^2}. \end{aligned} \quad (14)$$

The state at time k is thus a linear combination of the known initial state and the as yet unknown final costate. To find λ_N , we need to make use of the boundary conditions (2.1-8).

Since x_0 is fixed, $dx_0 = 0$ and (2.1-8b) is satisfied. Since we are also demanding the fixed final state (3), we have $dx_N = 0$ so that (2.1-8a) is satisfied. In words, the algorithm cannot vary either x_0 or x_N in determining the constrained minimum for this problem.

According to (14), the final state is expressed in terms of the unknown λ_N as

$$x_N = a^N x_0 - \frac{\gamma(1 - a^{2N})}{1 - a^2} \lambda_N = a^N x_0 - \Lambda \lambda_N, \quad (15)$$

where

$$\Lambda \triangleq \frac{\gamma(1 - a^{2N})}{1 - a^2} = \frac{b^2(1 - a^{2N})}{r(1 - a^2)}. \quad (16)$$

(Defining auxiliary variables is a good trick for making our results look neater than they actually are!) Solving for λ_N in terms of the given x_0 and the known desired $x_N = r_N$ yields

$$\lambda_N = -\frac{1}{\Lambda}(r_N - a^N x_0). \quad (17)$$

Note that $a^N x_0$ is the final state of the plant (1) in the case of zero input. The final costate λ_N is thus proportional to the desired final state r_N minus the final state $a^N x_0$, which the system would reach by itself with no control; it makes sense that the control required to drive x_0 to $x_N = r_N$ should depend on this difference!

At this point, we can determine the costate to be, using (11),

$$\lambda_k = -\frac{1}{\Lambda}(r_N - a^N x_0)a^{N-k} \quad (18)$$

and the optimal control to be, using (8),

$$u_k^* = \frac{b}{r\Lambda}(r_N - a^N x_0)a^{N-k-1}. \quad (19)$$

This is the solution to our problem, and u_k^* , for $k = 0, 1, \dots, N-1$ will drive x_0 to $x_N = r_N$ while minimizing the control energy (2).

It is worthwhile to **examine u_k^*** a little bit. Note that (19) can be written

$$u_k^* = \frac{1 - a^2}{b(1 - a^{2N})}(r_N - a^N x_0)a^{N-k-1}, \quad (20)$$

so that in the case of fixed final state **the optimal control is independent of the control weighting r** . It should also be stressed that u_k^* given by (19) is an **open-loop control**. It **depends on the initial and final states, but not on intermediate values x_k of the state**. This is discussed further in Section 2.2.

For completeness, let us determine the optimal state trajectory x_k^* and performance index J_0^* under the influence of u_k^* . Substituting (20) in (1) yields

$$x_{k+1}^* = ax_k^* + \frac{1 - a^2}{1 - a^{2N}}(r_N - a^N x_0)a^{N-k-1}. \quad (21)$$

The first observation worthy of note is that the optimal state trajectory x_k^* is independent of both r and b !

Equation (21) is a dynamical system with forcing function given by the second term, so its solution is

$$x_k^* = a^k x_0 + \frac{1 - a^2}{1 - a^{2N}}(r_N - a^N x_0) \sum_{i=0}^{k-1} a^{k-i-1} a^{N-i-1}. \quad (22)$$

Using the formula for the sum of a geometric series and simplifying yields

$$x_k^* = a^k x_0 + (r_N - a^N x_0) \frac{1 - a^{2k}}{1 - a^{2N}} a^{N-k} \quad (23)$$

or

$$x_k^* = \frac{(1 - a^{2(N-k)})a^k x_0 + (1 - a^{2k})a^{N-k} r_N}{1 - a^{2N}}. \quad (24)$$

Note that $x_0^* = x_0$ and $x_N^* = r_N$ as required. In fact, x_k^* is a time-varying linear combination of x_0 and r_N containing proportionately less of x_0 and more of r_N as k increases from 0 to N .

The optimal performance index is found by using (20) in (2):

$$J_0^* = \frac{r}{2} \frac{(1 - a^2)^2}{b^2(1 - a^{2N})^2} (r_N - a^N x_0)^2 \sum_{k=0}^{N-1} a^{2(N-k-1)}. \quad (25)$$

Using the formula for the sum of a geometric series and simplifying results in

$$J_0^* = \frac{1}{2\Lambda} (r_N - a^N x_0)^2. \quad (26)$$

Thus, the farther the plant zero-input response $a^N x_0$ is from the desired final state r_N , the larger the cost J_0^* .

b. Free Final State

Suppose that we still desire the system to end up in state r_N at time x_N , but we decide to choose quite a different method for ensuring this. We do not need x_N to be exactly equal to r_N , we only need x_N to be close to r_N . Let us, therefore, make the difference $x_N - r_N$ small by including it in the performance index, so that (2) becomes

$$J_0 = \frac{1}{2} (x_N - r_N)^2 + \frac{r}{2} \sum_{k=0}^{N-1} u_k^2. \quad (27)$$

Now the optimal control will attempt to make $|x_N - r_N|$ small while also using low control energy. (As we shall see, this will not guarantee that x_N will exactly equal r_N .) In this case

$$\phi = \frac{1}{2} (x_N - r_N)^2, \quad (28)$$

but f_k and L_k are not changed. The Hamiltonian is still given by (4), and conditions (2.1-9) are still (5)–(7). This means that all of our work in part a up through (15) is unchanged by adding to J the final-state weighting term. The only change is in the boundary conditions (2.1-8).

Since x_N is not constrained to take on an exact value, it can be varied in determining the optimal control. Hence, $dx_N \neq 0$, and so according to (2.1-8a) we must have

$$\lambda_N = \frac{\partial \phi}{\partial x_N} = x_N - r_N. \quad (29)$$

The final costate is now related to the final state x_N and the desired final state r_N by (29); this is our new terminal condition for the solution of (6) and (9). The initial condition is still the given value of x_i .

Returning to part a and picking up at equation (15), we must now use (29) (instead of (3)) to see that at the final time

$$x_N = a^N x_0 - \Lambda (x_N - r_N). \quad (30)$$

Solving for x_N gives

$$x_N = \frac{\Lambda r_N + a^N x_0}{1 + \Lambda}, \quad (31)$$

which should be compared to (17). Thus, fixing x_N by (3) allowed us to solve for the final *costate* in part a, while here the terminal relation (29) has allowed us to solve for the final *state* in terms of x_0 and the desired r_N .

According to (29) and (31),

$$\lambda_N = \frac{-(r_N - a^N x_0)}{1 + \Lambda}, \quad (32)$$

so the costate is given by (11) as

$$\lambda_k = \frac{-(r_N - a^N x_0)}{1 + \Lambda} a^{N-k} \quad (33)$$

(cf. (18)), and the optimal control is given by (8) as

$$u_k^* = \frac{b}{r(1 + \Lambda)} (r_N - a^N x_0) a^{N-k-1}. \quad (34)$$

This is the optimal control that solves our free-final-state problem. Note that, unlike (19), the control (34) does depend on r .

In the limit as $r \rightarrow 0$, we are concerned less and less about the control energy we use since u_k^2 is weighted less and less heavily in J_0 . In this case the free-final-state control (34) tends to the fixed-final-state control (19) (which is independent of r). Therefore, the less we are concerned about control energy (i.e., the smaller we make r in (27)), the closer the final state x_N comes to the desired r_N . This illustrates quite nicely the characteristic trade-off of optimal control that we discussed earlier.

As we let r go to infinity, meaning that we are expressing more concern about the control energy by weighting it more heavily in J_0 , the optimal control goes to zero. For completeness, let us determine the optimal state trajectory and performance index under the influence of (34). Substitute (34) into (1) to get

$$x_{k+1}^* = ax_k^* + \frac{b_2}{r(1 + \Lambda)} (r_N - a^N x_0) a^{N-k-1}. \quad (35)$$

Solving and manipulating yields

$$x_k^* = \frac{[(1 - a^2)/\gamma + (1 - a^{2(N-k)})]a^k x_0 + (1 - a^{2k})a^{N-k} r_N}{(1 - a^2)/\gamma + (1 - a^{2N})}. \quad (36)$$

Note that $x_0^* = x_0$, but

$$x_N^* = \frac{[(1 - a^2)/\gamma]a^N x_0 + (1 - a^{2N})r_N}{(1 - a^2)/\gamma + (1 - a^{2N})}. \quad (37)$$

As $r \rightarrow 0$, we have $\gamma \rightarrow \infty$, so that x_N^* approaches the desired r_N . In fact, if $\gamma \rightarrow \infty$, then the entire optimal state trajectory (36) approaches (24).

To find the optimal value of the performance index, use (34) in (2) and simplify to obtain

$$J_0^* = \frac{\Lambda}{2(1 + \Lambda)^2} (r_N - a^N x_0)^2. \quad (38)$$

As $r \rightarrow 0$, we have $\Lambda \rightarrow \infty$, so that this approaches the fixed-final-state cost (26).

c. Computer Simulation

In practical situations, the optimal control should be simulated to ensure that it results in acceptable system behavior before it is applied to the physical system. This is easy to do. Figure 2.1-1 shows a computer program in MATLAB to simulate the plant (1) with fixed-final-state optimal control (19). We are using an initial state of $x_0 = 0$ and a desired final state of $r_N = 10$. Figure 2.1-2 shows a computer program in MATLAB to simulate the plant (1) with free-final-state optimal control (19). Figure 2.1-3 shows the optimal state trajectory x_k^* for the fixed-final-state (i.e., $r = 0$) and for the free final state control with several values of weighting r , system parameters $a = .99$, $b = .1$, and $N = 100$. Figure 2.1-4 shows the corresponding optimal control sequences u_k^* . As expected, x_N approaches r_N and the control energy increases as r becomes small.

```
function [x, u] =scoptco_fixed (a, b, r, N, x0, rN)
% Simulation of Optimal Control for Scalar Systems
% Fixed Final State Case
x(1) =x0;
alam= (1-a^(2*N)) / (1-a^2); alam=alam*b^2;
u(1) =b*(rN-x(1)*a^N)*a^N/ (alam);
u(1) =u(1) / a;
for k=1:N
% Update the Plant State
x(k+1)=a*x(k) +b*u(k);
% Update the Optimal Control Input
u(k+1) =u(k) /a;
end
```

FIGURE 2.1-1 MATLAB simulation of fixed-final-state optimal control.

```
function [x, u]=scoptco_free (a, b, r, N, x0, rN)
% Simulation of Optimal Control for Scalar Systems
% Free Final State Case
x(1) =x0;
alam=(1-a^(2*N)) / (1-a^2); alam=alam*b^2/r;
u(1) =b*(rN-x(1)*a^N)*a^N/ (r*(alam+1));
u(1) =u(1) /a;
for k=1:N
% Update the Plant State
x(k+1) =a*x(k) +b*u(k);
% Update the Optimal Control Input
u(k+1) =u(k) /a;
end
```

FIGURE 2.1-2 MATLAB simulation of free-final-state optimal control.

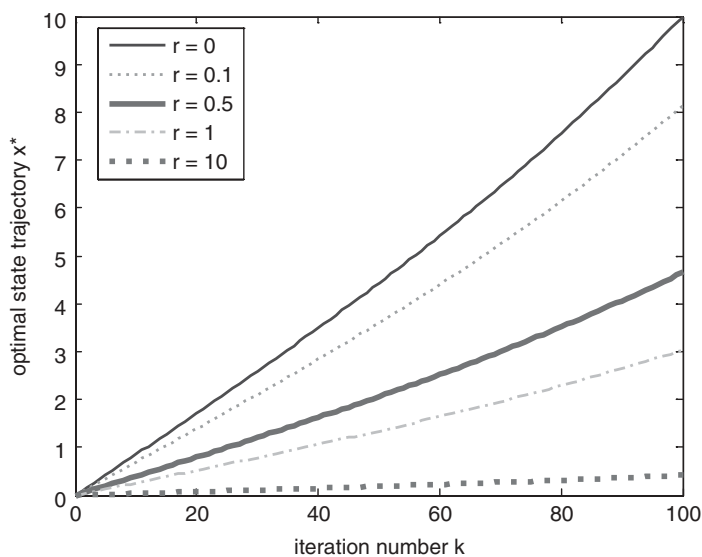


FIGURE 2.1-3 State trajectories for the fixed-final-state problem ($r = 0$) and for the free-final-state problem for several values of r .

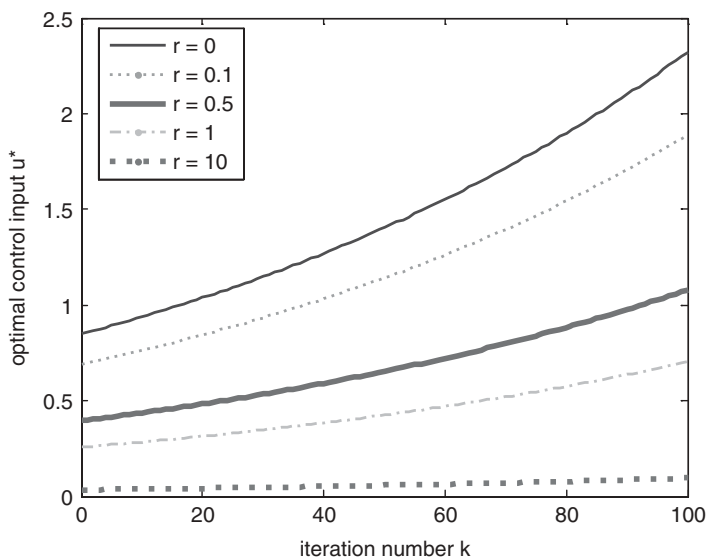


FIGURE 2.1-4 Optimal control functions corresponding to Fig. 2.1-3.

Using these simulation results, we can select r and the corresponding control u_k^* to yield an acceptable state trajectory and control energy for our particular application. This control is then used on the actual system. ■

2.2 DISCRETE-TIME **LINEAR QUADRATIC REGULATOR**

Table 2.1-1 provides the solution to the optimal control problem for nonlinear systems with very general performance indices, but explicit expressions for the optimal control are difficult to deduce. In this section we consider the extremely important special case of linear systems with quadratic performance indices. These performance indices can be considered as $(n + m)$ -dimensional quadratic surfaces, where n and m are the dimensions of the state and control vectors. The linear system is a hyperplane that intersects the quadratic surface and moves through the space as a function of time. This section is therefore the culmination of the natural progression begun by Examples 1.2-1 and 1.2-2.

We shall discover that very refined solutions can be given in two instances: the fixed-final-state situation, which leads to an open-loop control strategy, and the free-final-state situation, which leads to a closed-loop strategy.

The State and Costate Equations

Let the plant to be controlled be described by the linear equation

$$x_{k+1} = A_k x_k + B_k u_k, \quad (2.2-1)$$

with $x_k \in R^n$ and $u_k \in R^m$. The associated performance index is the quadratic function

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q_k x_k + u_k^T R_k u_k), \quad (2.2-2)$$

defined over the time interval of interest $[i, N]$. Note that both the plant and the cost-weighting matrices can, in general, be time-varying. The initial plant state is given as x_i . We assume that Q_k , R_k , and S_N are symmetric positive semidefinite matrices and that $|R_k| \neq 0$ for all k .

The objective is to find the control sequence $u_i, u_{i+1}, \dots, u_{N-1}$ that minimizes J_i . To solve this **linear quadratic (LQ) regulator problem**, we begin with the **Hamiltonian** function

$$H^k = \frac{1}{2} (x_k^T Q_k x_k + u_k^T R_k u_k) + \lambda_{k+1}^T (A_k x_k + B_k u_k). \quad (2.2-3)$$

Then Table 2.2-1 presents the state and costate equations

$$x_{k+1} = \frac{\partial H_k}{\partial \lambda_{k+1}} = A_k x_k + B_k u_k \quad (2.2-4)$$

$$\lambda_k = \frac{\partial H_k}{\partial x_k} = Q_k x_k + A_k^T \lambda_{k+1} \quad (2.2-5)$$

TABLE 2.2-1 Discrete Linear Quadratic Regulator (Final State Free)*System model:*

$$x_{k+1} = A_k x_k + B_k u_k, \quad k > i$$

Performance index:

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q_k x_k + u_k^T R_k u_k)$$

Assumptions:

$$S_N \geq 0, \quad Q_k \geq 0, \quad R_k > 0, \quad \text{and all three are symmetric}$$

Optimal feedback control:

$$S_k = A_k^T [S_{k+1} - S_{k+1} B_k (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1}] A_k + Q_k, \\ k < N, \quad S_N \text{ given}$$

$$K_k = (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k, \quad k < N$$

$$u_k = -K_k x_k, \quad k < N$$

$$J_i^* = \frac{1}{2} x_i^T S_i x_i$$

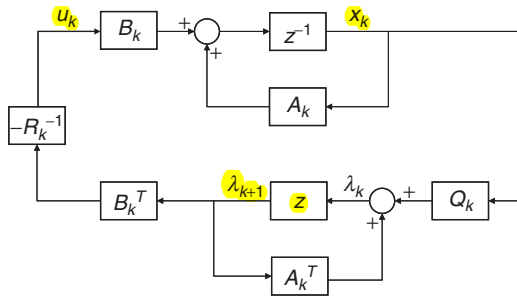
and the stationarity condition

$$0 = \frac{\partial H_k}{\partial u_k} = R_k u_k + B_k^T \lambda_{k+1}. \quad (2.2-6)$$

According to (2.2-6),

$$u_k = -R_k^{-1} B_k^T \lambda_{k+1}, \quad (2.2-7)$$

so the optimal control sequence is determined if we can find the costate sequence. A block diagram of the optimal controller appears in Fig. 2.2-1. We see, however, that it cannot be implemented in this form, since it is not causal.

**FIGURE 2.2-1 State-costate formulation** of the discrete linear quadratic optimal controller.

Using (2.2-7) to eliminate u_k in (2.2-4) gives

$$x_{k+1} = A_k x_k - B_k R_k^{-1} B_k^T \lambda_{k+1}. \quad (2.2-8)$$

At this point we drop the subscripts on the plant and weighting matrices to simplify the notation.

The coupled state and costate equations can be written as the single unforced system

$$\begin{bmatrix} x_{k+1} \\ \lambda_k \end{bmatrix} = \begin{bmatrix} A & -BR^{-1}B^T \\ Q & A^T \end{bmatrix} \begin{bmatrix} x_k \\ \lambda_{k+1} \end{bmatrix}. \quad (2.2-9)$$

Relative to the coefficient matrix describing (2.2-9) we must make an important observation. For this we introduce the following definition. A $2n \times 2n$ matrix H is said to be of Hamiltonian type if it satisfies

$$JHJ = H^T$$

where $J = \begin{bmatrix} 0 & -I_n \\ I_n & 0 \end{bmatrix}$.

It can be easily verified that the coefficient matrix in (2.2-9) is of Hamiltonian type and we shall refer to it as the *discrete Hamiltonian matrix*. The unforced system (2.2-9) will be known as the *discrete Hamiltonian system*. This system is difficult to solve since part of it develops forward and part backward in time.

If $|A| \neq 0$ (which is the case whenever it is obtained by sampling a continuous system), then we can write (2.2-8) as the *backward recursion*

$$x_k = A^{-1} x_{k+1} + A^{-1} B R^{-1} B^T \lambda_{k+1}. \quad (2.2-10)$$

Using this in (2.2-9) allows us to write (2.2-9) in the modified form

$$\begin{bmatrix} x_k \\ \lambda_k \end{bmatrix} = \begin{bmatrix} A^{-1} & A^{-1} B R^{-1} B^T \\ Q A^{-1} & A^T + Q A^{-1} B R^{-1} B^T \end{bmatrix} \begin{bmatrix} x_{k+1} \\ \lambda_{k+1} \end{bmatrix}. \quad (2.2-11)$$

This equation develops purely backward in time, so if we can determine x_N and λ_N , then we can find x_k and λ_k and hence the optimal control. Unfortunately, we are given x_0 , not λ_N .

Although we do not use equation (2.2-11) here, we discuss it again in Section 2.4, where we see that, in the time-invariant case, optimal controls can be computed from the eigenvectors of its coefficient matrix!

The costate equation (2.2-5) is an *adjoint system* for the plant (2.2-4). Let us discuss this notion to develop some more intuition on the relation between the state and the costate.

First suppose that $u_k = 0$ and $Q_k = 0$, and for simplicity let $i = 0$. Then the state and costate equations are

$$\begin{aligned} x_{k+1} &= A x_k \\ \lambda_k &= A^T \lambda_{k+1}, \end{aligned}$$

with solutions

$$x_k = A^k x_0, \quad (2.2-12)$$

$$\lambda_k = (A^T)^{N-k} \lambda_N. \quad (2.2-13)$$

Therefore,

$$\lambda_k^T x_k = \lambda_N^T (A)^{N-k} A^k x_0 = \lambda_N^T A^N x_0.$$

According to (2.2-12) and (2.2-13) this means that for all $k \in [0, N]$

$$\lambda_k^T x_k = \lambda_0^T x_0 = \lambda_N^T x_N, \quad (2.2-14)$$

so that in the case of zero intermediate-state weighting ($Q_k = 0$) and zero input u_k , the inner product of the state and the costate is invariant with time. Considering x_k and λ_k as vectors in n space, we have

$$\lambda_k^T x_k = |\lambda_k| \cdot |x_k| \cdot \cos \theta_k, \quad (2.2-15)$$

where $|\cdot|$ represents magnitude and θ_k is the angle between the vectors. As the state x_k develops through time, the magnitude and angle of the costate vary so that (2.2-15) is constant for all k .

To solve the optimal control problem, we need to solve for the costate λ_k and then use (2.2-7). To do this, we must use the boundary conditions (2.1-8). We know the initial state. We shall consider two special cases for the final condition (2.1-8a).

Before we do so, it is quite instructive to determine the value of the cost J_i when the control input u_k is zero.

Zero-input Cost and the Lyapunov Equation

Let the input u_k to the plant (2.2-1) be zero. We want to determine the value of the cost J_i in this uncontrolled situation.

Up to this point we have taken the initial time i as fixed, and k has been the variable time index, but it is now necessary to make a subtle shift of emphasis. In this subsection we want to find J_i as a function of i when $u_k = 0$ over $[i, N]$. To do this, we first let i equal the final time N and determine J_N . Then we increment i *backward* and find J_{N-1} , J_{N-2} , and so on. This amounts to taking i as our variable time index, considering successively longer time intervals with the final time N as the fixed quantity. The following discussion will make it clear why we wrote J_i and S_N in (2.2-2) as explicit functions of i and N , respectively.

To begin, note that

$$J_N = \frac{1}{2} x_N^T S_N x_N. \quad (2.2-16)$$

Now, let $i = N - 1$, $u_k = 0$ and write

$$J_{N-1} = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} x_{N-1}^T Q_{N-1} x_{N-1}. \quad (2.2-17)$$

Use the plant dynamics (2.2-1) to see that

$$J_{N-1} = \frac{1}{2} x_{N-1}^T (A_{N-1}^T S_N A_{N-1} + Q_{N-1}) x_{N-1}. \quad (2.2-18)$$

To make this look like (2.2-16), define a new intermediate variable (an $n \times n$ matrix) by

$$S_{N-1} = A_{N-1}^T S_N A_{N-1} + Q_{N-1}. \quad (2.2-19)$$

Then

$$J_{N-1} = \frac{1}{2} x_{N-1}^T S_{N-1} x_{N-1}. \quad (2.2-20)$$

It is clear that we can repeat this procedure for $i = N - 2, N - 3, \dots$ because (2.2-16) and (2.2-20) have the same form. The result is that we define an entirely new intermediate sequence of $n \times n$ matrices by the **backward recursion**

$$S_k = A_k^T S_{k+1} A_k + Q_k, \quad k < N, \quad (2.2-21)$$

with boundary condition S_N given as the final-state weighting matrix in (2.2-2). (Note that we have shifted from dummy index i to k .) This is a **discrete Lyapunov equation** for S_k , also known as the **observability Lyapunov equation**.

In terms of the new quantities S_k , the **zero-input performance** index over the interval $[k, N]$ is equal to

$$J_k = \frac{1}{2} x_k^T S_k x_k. \quad (2.2-22)$$

We call S_k the performance index **kernel sequence**.

The result (2.2-22) is quite interesting. The kernel S_k **can be computed offline, before we know the system state trajectory**, since it depends only on the plant and cost-weighting matrices. According to (2.2-22), then, we can calculate the zero-input cost over the interval $[k, N]$ by knowing the precomputed S_k and only the *initial* state x_k ! If we know where we start out, then we know what the ride will cost. Note that we can interpret $2J_k$ as the squared seminorm of x_k with respect to S_k , since $S_k \geq 0$. If $Q_k > 0$, then S_k is positive definite for all k , and $2J_k$ becomes a squared norm of x_k .

In the **time-invariant case**, we know that **the solution of the Lyapunov equation** (2.2-21) is

$$S_k = (A^T)^{N-k} S_N A^{N-k} + \sum_{i=k}^{N-1} (A^T)^{N-i-1} Q A^{N-i-1}. \quad (2.2-23)$$

By the Lyapunov stability theory, as $(N - k) \rightarrow \infty$ this converges to the *steady-state value*

$$S_\infty = \sum_{i=0}^{\infty} (A^T)^i Q A^i \quad (2.2-24)$$

if the plant is asymptotically stable. Figure 2.2-2 shows the limiting behavior of S_k for a scalar plant for the stable and unstable cases.

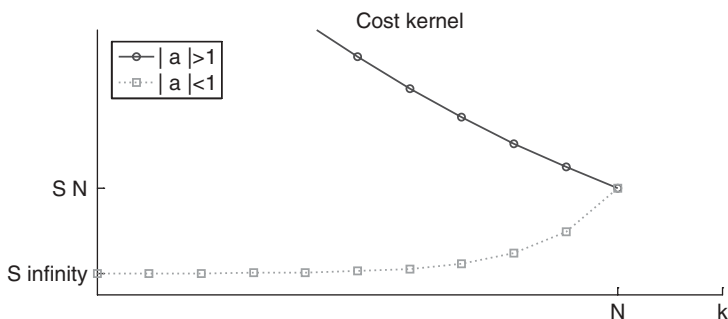


FIGURE 2.2-2 Zero-input cost kernel for a stable and an unstable plant.

If the plant is stable, the cost over the interval $[-\infty, N]$, or equivalently $[0, \infty]$, is given by the *steady-state cost*

$$J_{\infty} = \frac{1}{2} x_0^T S_{\infty} x_0. \quad (2.2-25)$$

It should be clearly understood that if A is stable, the uncontrolled cost over an infinite interval is *finite*. This is true because the state goes to zero with time.

If A is not stable, the zero-input steady-state cost can be infinite (depending on Q) since the norm of x_k is unbounded. In the steady-state case, $S \triangleq S_k = S_{k+1}$ for large k , so that (2.2-21) becomes the *algebraic Lyapunov equation*

$$S = A^T S A + Q. \quad (2.2-26)$$

By the Lyapunov theory, this equation has a positive semidefinite solution S_{∞} if A is stable, and this solution is given by (2.2-24).

If $\sqrt{Q}$ is defined by

$$Q = \sqrt{Q}^T \sqrt{Q}, \quad (2.2-27)$$

then S_{∞} is the *unique positive definite* solution if A is stable and $(A, \sqrt{Q})$ is *observable*. This latter condition means that Q was selected so that the plant state is “observable by the performance index,” so that variations in any direction of the state have an effect on J . If $(A, \sqrt{Q})$ is not observable, then the unobservable state components can tend to infinity with k , but the performance index may still be finite since these components have no effect on J . On the other hand, if $(A, \sqrt{Q})$ is observable and A is unstable, then J_k will grow without bound as $k \rightarrow -\infty$.

We shall see later that these results generalize to the case of nonzero input and provide a means for computing the optimal control sequence.

Fixed-final-state and Open-loop Control

Here we return to the problem of determining the optimal control for the plant (2.2-1) with the cost (2.2-2). The state and costate equations are given, once

u_k has been expressed in terms of λ_{k+1} , by (2.2-8) and (2.2-5). (For ease of notation, assume the time-invariant case; the results generalize to the time-varying situation.) To solve these, we need to determine the terminal conditions.

For simplicity, let the initial time be $i = 0$. The initial state x_0 is given. In this subsection our terminal objective will be to make x_N match *exactly* the desired final reference state r_N . The final condition is thus

$$x_N = r_N. \quad (2.2-28)$$

Since in this fixed-final-state case $dx_N = 0$, condition (2.1-8a) is automatically satisfied.

Since we are demanding that x_N be equal to a known desired r_N , the final-state contribution to J_0 in (2.2-2) always has a fixed value of $\frac{1}{2}r_N^T S_N r_N$. It is therefore redundant to include a final-state weighting term in J_0 . Accordingly, we may as well set $S_N = 0$.

Let the cost function be

$$J_0 = \frac{1}{2} \sum_{k=0}^{N-1} u_k^T R u_k, \quad (2.2-29)$$

and so we are asking for a control that drives x_0 exactly to $x_N = r_N$ using minimum control energy. The state and costate equations are now

$$x_{k+1} = A x_k - B R^{-1} B^T \lambda_{k+1}, \quad (2.2-30)$$

$$\lambda_k = A^T \lambda_{k+1}. \quad (2.2-31)$$

Since $Q = 0$, the costate equation is decoupled from the state equation, and the problem has an easy solution. To find it, write the solution of (2.2-31) in terms of the, as yet unknown, final costate as

$$\lambda_k = (A^T)^{N-k} \lambda_N. \quad (2.2-32)$$

Use this to eliminate λ_{k+1} in (2.2-30) to get

$$x_{k+1} = A x_k - B R^{-1} B^T (A^T)^{N-k-1} \lambda_N. \quad (2.2-33)$$

Considering this as a first-order difference equation with the second term as the input, we get

$$x_k = A^k x_0 - \sum_{i=0}^{k-1} A^{k-i-1} B R^{-1} B^T (A^T)^{N-i-1} \lambda_N. \quad (2.2-34)$$

To find λ_N , evaluate (2.2-34) at $k = N$,

$$x_N = A^N x_0 - \sum_{i=0}^{N-1} A^{N-i-1} B R^{-1} B^T (A^T)^{N-i-1} \lambda_N.$$

Therefore, the final costate, with $x_N = r_N$, is

$$\lambda_N = -G_{0,N}^{-1}(r_N - A^N x_0), \quad (2.2-35)$$

where

$$G_{0,N} = \sum_{i=0}^{N-1} A^{N-i-1} B R^{-1} B^T (A^T)^{N-i-1}. \quad (2.2-36)$$

Using (2.2-32) the costate is

$$\lambda_k = -(A^T)^{N-k} G_{0,N}^{-1}(r_N - A^N x_0), \quad (2.2-37)$$

and so by (2.2-7) the optimal control sequence is

$$u_k^* = R^{-1} B^T (A^T)^{N-k-1} G_{0,N}^{-1}(r_N - A^N x_0). \quad (2.2-38)$$

This is the minimum-control-energy solution to the fixed-final-state LQ regulator problem; the problem is solved.

We can easily demonstrate that u_k^* is a control that drives x_0 to $x_N = r_N$. The solution to state equation (2.2-1) is

$$x_k = A^k x_0 + \sum_{i=0}^{k-1} A^{k-i-1} B u_i. \quad (2.2-39)$$

Evaluating the state at $k = N$ and using $u_i = u_i^*$ as given by (2.2-38) yields

$$x_N = A^N x_0 + \sum_{i=0}^{N-1} A^{N-i-1} B R^{-1} B^T (A^T)^{N-i-1} G_{0,N}^{-1}(r_N - A^N x_0),$$

but $G_{0,N}^{-1}(r_N - A^N x_0)$ does not depend on i , and the remaining portion of the sum is just $G_{0,N}$. This means that

$$x_N = A^N x_0 + G_{0,N} G_{0,N}^{-1}(r_N - A^N x_0) = r_N \quad (2.2-40)$$

as desired!

To gain an understanding of our optimal control result, let us discuss it a little. First, note that in the absence of an input, the solution to the state equation (2.2-1) is

$$x_k = A^k x_0, \quad (2.2-41)$$

so that $x_N = A^N x_0$ is the final state with zero input. Thus, $r_N - A^N x_0$ is the difference between the desired and undriven final states; it makes sense that u_k^* should depend on this quantity.

Now we examine $G_{0,N}$. This is the weighted reachability gramian of the system. In terms of the system reachability matrix $U_k = [B \ AB \ \cdots \ A^{k-1}B]$,

it can be written as

$$G_{0,N} = U_N \begin{bmatrix} R^{-1} & & 0 \\ & \ddots & \\ 0 & & R^{-1} \end{bmatrix} U_N^T. \quad (2.2-42)$$

If $R = I$, then $G_{0,N} = U_N U_N^T$. The optimal control exists if and only if $|G_{0,N}| \neq 0$. Since we assumed $|R| \neq 0$, this is equivalent to U_N having full rank n , where n is the state dimension. Therefore, we can drive any given x_0 to any desired $x_N = r_N$ for some N if and only if the system is *reachable*! Since reachability implies that U_{n+j} has full rank for all $j \geq 0$, if the system is reachable, we can guarantee the existence of a control to drive x_0 to $x_N = r_N$ for any r_N by selecting $N \geq n$.

The following point should be clearly understood. The optimal control (2.2-38) is an *open-loop control*. It can be *precomputed knowing only the given x_0 and the desired r_N* , and it is independent of intermediate values of x_k within the interval $[0, N]$. This means that if we apply u_k^* as calculated by (2.2-38) to the actual system, all is well as long as (2.2-1) is an exact description of the dynamics and nothing occurs to cause x_k to deviate from the optimal state trajectory. In practice, however, nature is seldom cooperative. *Modeling uncertainties and noise cause errors in x_k , and the control (2.2-38) does not take these errors into account. Open-loop control schemes are not robust in most actual applications.*

To compute the reachability gramian, there is an attractive alternative to (2.2-36) or (2.2-42). The solution to the *Lyapunov equation*

$$P_{k+1} = AP_k A^T + BR^{-1}B^T, \quad k > 0 \quad (2.2-43)$$

is

$$P_k = A^k P_0 (A^k)^T + \sum_{i=0}^{k-1} A^{k-i-1} BR^{-1}B^T (A^T)^{k-i-1}, \quad (2.2-44)$$

so if we solve this equation with $P_0 = 0$, then $G_{0,k} = P_k$ for each k . First this recursion is solved to obtain $G_{0,N}$ for the final time of interest N , and then (2.2-38) is used to compute the optimal control u_k^* for each k in $[0, N]$. This *reachability Lyapunov equation* should be compared with the *observability Lyapunov equation* (2.2-21).

The next example illustrates the use of these results.

Example 2.2-1. Open-loop Control of a Scalar System

Consider the scalar plant

$$x_{k+1} = ax_k + bu_k \quad (1)$$

with cost

$$J_0 = \frac{r}{2} \sum_{k=0}^{N-1} u_k^2. \quad (2)$$

The system is reachable since

$$U_1 = b \neq 0. \quad (3)$$

The reachability gramian (2.2-36) is

$$G_{0,N} = \sum_{i=0}^{N-1} \frac{b^2}{r} a^{2(N-i-1)} = \frac{b^2}{r} \frac{1 - a^{2N}}{1 - a^2}, \quad (4)$$

which is exactly Λ from Example 2.1-2.

To drive any given x_0 to a desired r_N for any $N \geq n = 1$, we should use the optimal control sequence u_k^* for $k \in [0, N - 1]$ given by (2.2-38), so that

$$\begin{aligned} u_k^* &= \frac{a^{N-k-1}b}{r} \frac{r(1-a^2)}{b^2(1-a^{2N})} (r_N - a^N x_0) \\ &= \frac{1-a^2}{b(1-a^{2N})} (r_N - a^N x_0) a^{N-k-1}. \end{aligned} \quad (5)$$

This is exactly the result obtained by solving the state and costate equations in Example 2.1-2a.

Note that the optimal control at times k and $k+1$ are related by

$$u_{k+1}^* = u_k^*/a, \quad (6)$$

which makes the control sequence easy to calculate. See Fig. 2.1-1. ■

Free-final-state and Closed-loop Control

We have just found the minimum-energy optimal control for system (2.2-1) in the case where x_N is required to have a fixed given value. It is now desired to find the optimal control sequence that drives the system (2.2-1) along the trajectory, beginning at a given x_i , resulting in a minimum value of (2.2-2). We shall make here *no restriction on the value of the final state x_N* . This *free-final-state* problem will result in a radically different sort of control.

The state and costate equations with the input u_k eliminated are (2.2-8) and (2.2-5), which are reproduced here:

$$x_{k+1} = A_k x_k - B_k R_k^{-1} B_k^T \lambda_{k+1}, \quad (2.2-45)$$

$$\lambda_k = Q_k x_k + A_k^T \lambda_{k+1}. \quad (2.2-46)$$

The control is given as (2.2-7) or

$$u_k = -R_k^{-1} B_k^T \lambda_{k+1}. \quad (2.2-47)$$

The initial condition is given as x_i , and the final state x_N is free. This means that x_N can be varied in determining the constrained minimum. Hence, $dx_N \neq 0$.

According to (2.1-8), then, it is required that

$$\lambda_N = \frac{\partial \phi}{\partial x_N}. \quad (2.2-48)$$

The final state weighting function is $\phi = \frac{1}{2} x_N^T S_N x_N$, so that

$$\lambda_N = S_N x_N. \quad (2.2-49)$$

This relation between the final costate and state is the new terminal condition; in the fixed-final-state problem the terminal condition was (2.2-28).

To solve this two-point boundary-value problem, we shall use the *sweep method* (Bryson and Ho 1975). Thus, assume that a linear relation like (2.2-49) holds for all times $k \leq N$:

$$\lambda_k = S_k x_k \quad (2.2-50)$$

for some intermediate sequence of $n \times n$ matrices S_k . If we can find a consistent formula for these postulated S_k , then evidently (2.2-50) is a valid assumption. To do this, use (2.2-50) in (2.2-45) to get

$$x_{k+1} = A_k x_k - B_k R_k^{-1} B_k^T S_{k+1} x_{k+1}.$$

Solving for x_{k+1} yields

$$x_{k+1} = (I + B_k R_k^{-1} B_k^T S_{k+1})^{-1} A_k x_k, \quad (2.2-51)$$

which is a forward recursion for the state.

Now substitute (2.2-50) into costate equation (2.2-46) to see that

$$S_k x_k = Q_k x_k + A_k^T S_{k+1} x_{k+1},$$

or by (2.2-51)

$$S_k x_k = Q_k x_k + A_k^T S_{k+1} (I + B_k R_k^{-1} B_k^T S_{k+1})^{-1} A_k x_k.$$

Since x_k is generally nonzero, and this equation holds for all state sequences given any x_i , evidently

$$S_k = A_k^T S_{k+1} (I + B_k R_k^{-1} B_k^T S_{k+1})^{-1} A_k + Q_k, \quad (2.2-52)$$

or, using the matrix inversion lemma (Appendix A)

$$S_k = A_k^T [S_{k+1} - S_{k+1} B_k (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1}] A_k + Q_k. \quad (2.2-53)$$

This is a backward recursion for the postulated S_k , which completely specifies it in terms of S_{k+1} and the known system and weighting matrices. The boundary condition is known: it is just the final-state weighting matrix S_N . Therefore,

we have been able to discover an intermediate sequence such that the relation (2.2-50) holds.

The matrix quadratic equation (2.2-53) is called a *Riccati equation* in honor of the Italian mathematician Count Jacopo Riccati, who investigated a scalar version of its differential counterpart in 1724. If $|S_k| \neq 0$ for all k , then we can use the *matrix inversion lemma* to rewrite the Riccati equation as

$$S_k = A_k^T (S_{k+1}^{-1} + B_k R_k^{-1} B_k^T)^{-1} A_k + Q_k. \quad (2.2-54)$$

The intermediate sequence S_k can be computed *offline* knowing only the plant and parameters of the performance index. Then (2.2-51), with the given initial state x_i , yields the optimal state trajectory. We do not yet know, however, the control required to move the plant along that trajectory! To determine *the optimal control* write

$$u_k = -R_k^{-1} B_k^T \lambda_{k+1} = -R_k^{-1} B_k^T S_{k+1} x_{k+1}. \quad (2.2-55)$$

At first glance it appears that the problem is now solved, since the *Riccati equation gives sequence the S_k* , (2.2-51) *gives sequence x_k* , and we can use this to find sequence u_k . This is, *however, quite inconvenient*. Let us manipulate (2.2-55) to find a more satisfactory expression for the optimal control. To this end, use the plant equation $x_{k+1} = A_k x_k + B_k u_k$ in (2.2-55) to get

$$u_k = -R_k^{-1} B_k^T S_{k+1} (A_k x_k + B_k u_k)$$

or

$$(I + R_k^{-1} B_k^T S_{k+1} B_k) u_k = -R_k^{-1} B_k^T S_{k+1} A_k x_k.$$

Premultiply by R_k and then solve for the control to obtain

$$u_k = -(B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k x_k. \quad (2.2-56)$$

Defining the *Kalman gain sequence*

$$K_k = (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k, \quad (2.2-57)$$

the control takes the form

$$u_k = -K_k x_k. \quad (2.2-58)$$

Note that in these equations we could use the notation u_k^* , since they give the optimal control.

The form of (2.2-58) makes it particularly clear what we now have on our hands. The *Kalman gain is given in terms of the Riccati equation solution S_k and the system and weighting matrices*. It can therefore be computed and stored in computer memory *before the control* is ever applied to the plant. It *does not depend on the state trajectory*. Therefore, (2.2-58) is a *time-varying state-variable*

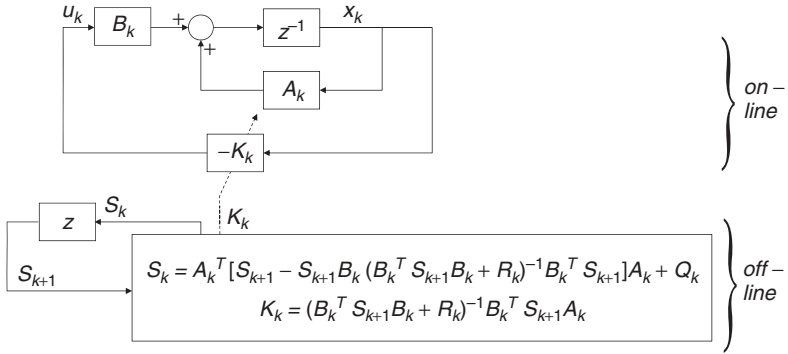


FIGURE 2.2-3 Free-final-state LQ regulator optimal control scheme.

feedback, which expresses the current required control in terms of the current state. In the free-final-state linear quadratic (LQ) regulator, the optimal control is thus given by a closed-loop control law. Closed-loop control is inherently more robust than open-loop control, because any deviations from the optimal state trajectory x_k^* , which is given by (2.2-51), are automatically accounted for.

The problem is solved. To determine the optimal control, we need only solve the Riccati equation for S_k and (2.2-57) for K_k , both of which can be done offline, and then use the feedback (2.2-58). These equations are summarized in Table 2.2-1, and a block diagram of the control scheme appears in Fig. 2.2-3.

The closed-loop system with the optimal feedback (2.2-58) is

$$x_{k+1} = (A_k - B_k K_k) x_k, \quad (2.2-59)$$

which provides an alternative to (2.2-51) for computing the optimal state trajectory. It is worth noting that an alternative more efficient way of computing S_k and K_k is to use the recursion

$$K_k = (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k, \quad (2.2-60)$$

$$S_k = A_k^T (S_{k+1} (A_k - B_k K_k) + Q_k). \quad (2.2-61)$$

It is not difficult to show (see the problems) that equation (2.2-61) is equivalent to the Joseph stabilized version of the Riccati equation

$$S_k = (A_k - B_k K_k)^T S_{k+1} (A_k - B_k K_k) + K_k^T R_k K_k + Q_k. \quad (2.2-62)$$

This equation has better numerical properties when it comes to computation and will soon be useful to us.

If the system and weighting matrices A , B , Q , R are all time invariant, then the feedback gain K_k is still a function of time, since, in general, the Riccati-equation solution S_k is time varying. Thus, even for time-invariant systems, the optimal control is a time-varying state feedback. This means that the optimal

closed-loop system $(A - BK_k)$ is time varying, and, of course, accounts for the fact that it cannot be found by classical frequency-domain methods.

For completeness, let us determine the optimal value of the performance index under the influence of the optimal control (2.2-58). First, observe that

$$\frac{1}{2} \sum_{k=i}^{N-1} (x_{k+1}^T S_{k+1} x_{k+1} - x_k^T S_k x_k) = \frac{1}{2} x_N^T S_N x_N - \frac{1}{2} x_i^T S_i x_i. \quad (2.2-63)$$

We can therefore add zero, in the form of the left-hand side of (2.2-63) minus its right-hand side, to the performance index (2.2-2). The result is

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} [x_{k+1}^T S_{k+1} x_{k+1} + x_k^T (Q_k - S_k) x_k + u_k^T R_k u_k]. \quad (2.2-64)$$

If we take into account the state equation (2.2-1), this is equivalent to

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} [x_k^T (A_k^T S_{k+1} A_k + Q_k - S_k) x_k + x_k^T A_k^T S_{k+1} B_k u_k + u_k^T B_k^T S_{k+1} A_k x_k + u_k^T (B_k^T S_{k+1} B_k + R_k) u_k]. \quad (2.2-65)$$

According to the Riccati equation (2.2-53), this is

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} [x_k^T A_k^T S_{k+1} B_k (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k x_k + x_k^T A_k^T S_{k+1} B_k u_k + u_k^T B_k^T S_{k+1} A_k x_k + u_k^T (B_k^T S_{k+1} B_k + R_k) u_k]. \quad (2.2-66)$$

The summand can be written as the perfect square of a norm with respect to $(B_k^T S_{k+1} B_k + R_k)$ (McReynolds 1966):

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} \left\| (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1} A_k x_k + u_k \right\|_{(B_k^T S_{k+1} B_k + R_k)}^2. \quad (2.2-67)$$

If we now select the optimal control (2.2-56), then the optimal value of the performance index is seen to be

$$J_i^* = \frac{1}{2} x_i^T S_i x_i. \quad (2.2-68)$$

This result deserves some discussion. The sequence S_k can be computed offline before the optimal control is applied, so that S_i is known a priori. Given any initial state x_i , then, we can use (2.2-68) to compute the optimal cost of applying the control before we ever apply it! In general, we can treat any time k in $[i, N]$

as the initial time of a subinterval, so that

$$J_k^* = \frac{1}{2} x_k^T S_k x_k. \quad (2.2-69)$$

What this means is that given the current state x_k we can compute the cost to go (i.e., remaining cost) of applying the optimal control from times k through N .

Because of (2.2-69), we call S_k the performance index kernel matrix. The optimal value of the performance index J_k is simply one-half of the semi-norm squared of the current state x_k with respect to S_k . (If $|S_k| \neq 0$, the semi-norm becomes a norm.)

It is worth noting that, according to (2.2-67), $B_k^T S_{k+1} B_k + R_k$ is equal to $\partial^2 J_i / \partial u_k^2$, the second derivative of J_i with respect to the k th input under the constraint (2.2-1). Compare this with the constrained curvature matrix L_{uu}^f in Example 1.2-3. Evidently, $B_k^T S_{k+1} B_k + R_k$ is a time-varying curvature matrix.

To contribute to the development of an intuitive grasp of the Riccati-equation-based control law, let us point out a few links with our previous work. First, use the matrix inversion lemma to write the optimal state trajectory recursion (2.2-51) as

$$x_{k+1} = [I - B_k(B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1}] A_k x_k \quad (2.2-70)$$

(which is, in fact, the same as (2.2-59)). It is now clear that this equation and (2.2-69) (with S_k given by (2.2-53)) are simply generalizations to the case where time is a parameter of our static results in Example 1.2-3.

We can also link our present results to the zero-input cost discussion, for if control matrix B_k is zero, then the (quadratic) Riccati equation reduces to the (linear) Lyapunov equation (2.2-21). The control-dependent term in the Riccati equation makes the value of S_k smaller than $A_k^T S_{k+1} A_k + Q_k$ at each step, and expresses the decrease in the value of the performance index that results if we are allowed to control the plant.

There is a second way to reduce a Riccati equation to a Lyapunov equation, and it provides the connection between the fixed-final-state and free-final-state control laws. Suppose $Q_k = 0$, so that there is no intermediate-state weighting. For simplicity, assume time-invariant plant and cost-weighting matrices. Suppose also that $|A| \neq 0$. Under these circumstances, (2.2-54) can be written

$$S_k^{-1} = A^{-i} S_{k+1}^{-1} A^{-T} + A^{-1} B R^{-1} B^T A^{-T}, \quad (2.2-71)$$

which is a backward-developing Lyapunov recursion for S_k^{-1} .

Now, if we want to ensure that x_N approaches exactly a desired final value of $r_N = 0$, in the performance index we can let the final-state weighting matrix S_N go to infinity. This tells the optimal control to make $x_N = 0$ in order to keep J_i finite. In this limit, $S_N^{-1} = 0$, which provides the terminal condition for (2.2-71). It is easy to show (by writing out explicitly the first few steps in the recursion and then using induction) that the solution to (2.2-71) with $S_N^{-1} = 0$ is just

$$S_{N-k}^{-1} = A^{-k} G_{0,k} (A^T)^{-k}, \quad (2.2-72)$$

where $G_{0,k}$ is the weighted reachability gramian (2.2-36).

It takes a few more steps to prove it, but it is fairly evident at this point that if $Q = 0$ and $S_N \rightarrow \infty$, then the free-final-state closed-loop control (2.2-56) reduces to the fixed-final-state open-loop control (2.2-38) in the case $r_N = 0$. See the related discussion in Example 2.1-2. (Note that $S_N \rightarrow \infty$ is equivalent to $R \rightarrow 0$ if $Q = 0$.)

As a final connection, examine part b of Example 2.1-2. It can be demonstrated that the control law given in equation (34) of that example is just an alternative formulation of (2.2-56). In practical applications the latter closed-loop formulation would be used.

As a final comment on the closed-loop LQ regulator, we make the following very important point. In the fixed-final-state problem, reachability of the system was required to be able to solve for the optimal control to drive any given initial state to any desired final state. In our discussion of the free-final-state problem, however, reachability never came up. In fact, it is *not* necessary for the system to be reachable for the control in Table 2.2-1 to exist. If the system is not reachable, the control will still do its best to minimize J_i . Clearly, if the system is, in fact, reachable, the control will do a better job of minimizing J_i , since then all vectors in R^n are candidates for the optimal x_N .

We shall see later that reachability does become important in the steady-state control problem, where N tends to infinity. Let us now work through an example.

Example 2.2-2. Optimal Feedback Control of a Scalar System

The plant to be controlled is the time-invariant scalar system

$$x_{k+1} = ax_k + bu_k \quad (1)$$

with performance index

$$J_i = \frac{1}{2} s_N x_N^2 + \frac{1}{2} \sum_{k=i}^{N-1} (qx_k^2 + ru_k^2). \quad (2)$$

In Example 2.1-2 we considered a special case ($q = 0$) of this problem, and we found the optimal controls for two different terminal conditions by direct solution of the state and costate equations. Here we shall find the optimal control in the state feedback formulation using the results we have just derived.

In this scalar case the Riccati equation is

$$s_k = a^2 s_{k+1} - \frac{a^2 b^2 s_{k+1}^2}{b^2 s_{k+1} + r} + q \quad (3)$$

or

$$s_k = \frac{a^2 r s_{k+1}}{b^2 s_{k+1} + r} + q. \quad (4)$$

The Kalman gain is

$$K_k = \frac{abs_{k+1}}{b^2 s_{k+1} + r} = \frac{a/b}{1 + r/b^2 s_{k+1}} \quad (5)$$

and the optimal control is

$$u_k = -K_k x_k. \quad (6)$$

The optimal value of the performance index is

$$J_k = \frac{1}{2} s_k x_k^2. \quad (7)$$

The optimal closed-loop system (2.2-59) is

$$x_{k+1} = (a - bK_k)x_k = \frac{a}{1 + (b^2/r)s_{k+1}} x_k. \quad (8)$$

Even in this simple case a closed-form solution to (3) is hard to find. Let us therefore consider three special cases. Then we shall demonstrate that even if we cannot solve (3) analytically, for particular values of a , b , q , r , s_N , it is very easy to compute the optimal control sequence u_k^* and to simulate applying it to the plant on a digital computer.

a. No Control Weighting

Let $r = 0$, meaning that we do not care how much control is used (i.e., u_k is not weighted in J_i so that the optimal solution will make no attempt to keep it small). Then (4) is

$$s_k = q, \quad (9)$$

the feedback gain (5) is $K_k = a/b$, and the optimal control becomes

$$u_k = -\frac{a}{b} x_k. \quad (10)$$

Under the influence of this control, the performance index is

$$J_k = \frac{1}{2} q x_k^2, \quad (11)$$

and the closed-loop system (8) is $x_{k+1} = 0$!

We can understand this as follows. If we have a given value x_k for the state at time k , then a naive approach to minimizing the magnitude of the state vector (which is all we require since $r = 0$) is to solve the state equation (1) for the u_k required to make x_{k+1} equal to zero, so that $0 = x_{k+1} = ax_k + bu_k$. This yields the control (10).

b. Very Large Control Weighting

If we are very concerned not to use too much control energy, we can let $r \rightarrow \infty$. Then (4) becomes

$$s_k = a^2 s_{k+1} + q. \quad (12)$$

The solution to this (Lyapunov) difference equation is

$$s_k = s_N a^{2(N-k)} + \sum_{i=k}^{N-1} q a^{2(N-i-1)} = s_N a^{2(N-k)} + \left(\frac{1 - a^{2(N-k)}}{1 - a^2} \right) q. \quad (13)$$

The Kalman gain is $K_k = 0$, and so the optimal control is $u_k = 0$. The closed-loop system is $x_{k+1} = ax_k$.

If we are very concerned about using too much control, the best policy is to use none at all!

c. *No Intermediate-state Weighting*

Let us set $q = 0$. Then we are concerned only about making x_N^2 small without using too much control energy. It is easier in this case to deal with the inverse of the cost kernel, s_k^{-1} , so use (2.2-71) to rewrite (4) as

$$s_k^{-1} = \frac{s_{k+1}^{-1}}{a^2} + \frac{b^2}{a^2 r}. \quad (14)$$

The solution to this (Lyapunov) difference equation is

$$s_k^{-1} = a^{-2(N-k)} s_N^{-1} + \sum_{i=k}^{N-1} \frac{b^2}{a^2 r} a^{-2(N-i-1)}, \quad (15)$$

or, changing variables,

$$s_k^{-1} = a^{-2(N-k)} s_N^{-1} + \frac{b^2}{a^2 r} \sum_{i=0}^{N-k-1} a^{-2i} = s_N^{-1} a^{-2(N-k)} + \frac{b^2}{r a^{2(N-k)}} \frac{1 - a^{2(N-k)}}{1 - a^2}. \quad (16)$$

After a few lines of work we get

$$s_k = \frac{s_N a^{2(N-k)}}{1 + s_N (b^2/r) [(1 - a^{2(N-k)})/(1 - a^2)]}. \quad (17)$$

This is the *Riccati-equation solution for $q = 0$* .

d. *Implementation of Optimal Control*

To actually compute and implement the control described by (4)–(6), none of the analysis subsequent to equation (6) is needed. The implementation of optimal controls has two phases: computing the control sequence and applying it to the plant. We show here how to compute u_k^* and then simulate its application using a digital computer.

```
function [x, u, K, S]=scaopt (a, b, q, r, s, x0, N)
% Program to Compute and Simulate Optimal Feedback Control
% Compute and Store Optimal Feedback Sequence
% (Backward Iteration)
S(N+1) =s;
for k=N:-1:1
K(k)=(a*b*s)/(r+s*b^2);
s=q+(r*s*a^2)/(r+s*b^2);
S(k)=s;
end
% Apply Optimal Control to Plant (Forward Iteration)
x(1)=x0;
for k=1:N
% Compute Optimal Control
u(k)=-K(k)*x(k);
% Update the Plant State
x(k+1)=a*x(k)+b*u(k);
end
```

FIGURE 2.2-4 MATLAB code to compute and simulate optimal feedback control.

A routine implementing (4) and (5), for $a = 1.05$, $b = 0.01$, $q = r = 1$, $x_0 = 10$, $s_N = 5$, and $N = 100$, is shown in Fig. 2.2-4. This must solve the **Riccati equation backward** on the desired interval $[0, N]$, using as a starting value s_N . The optimal feedback gains K_k are also computed for all $k \in [0, N]$, and they are stored in memory. (The s_k need not be stored. They are stored in this example *only* so they can be plotted for illustrative purposes. In an application the storage of s_k can be avoided.) This completes the computation of the control law.

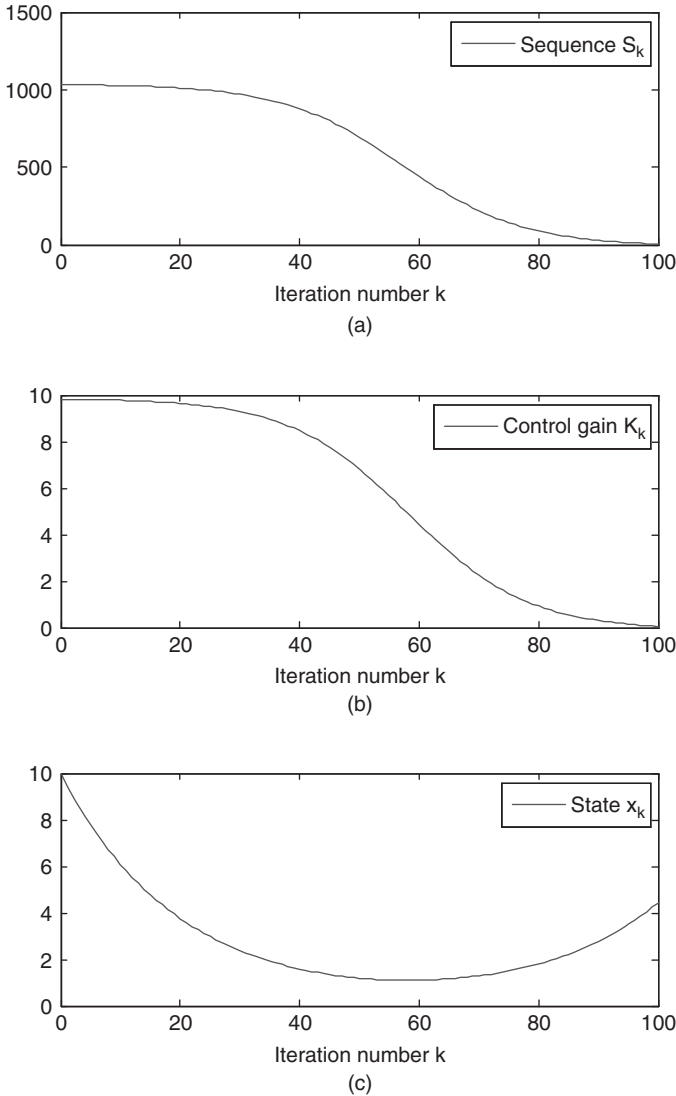


FIGURE 2.2-5 Optimal control simulations for $s_N = 5$. (a) Cost kernel s_k . (b) Optimal feedback gains K_k . (c) Optimal trajectory x_k^* .

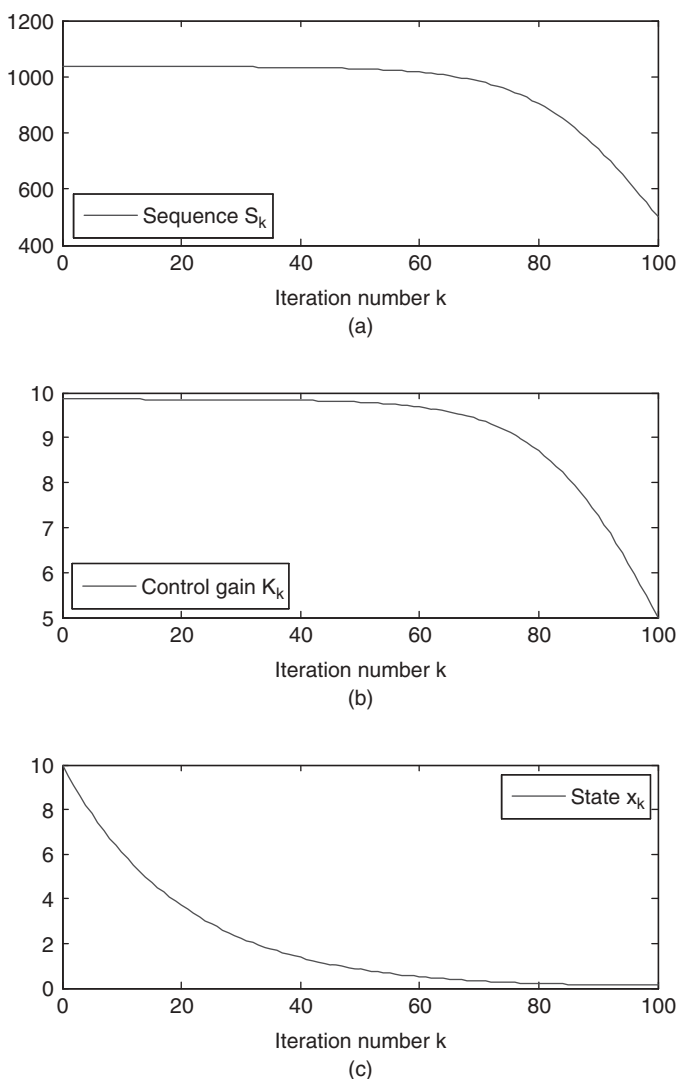


FIGURE 2.2-6 Optimal control simulations for $s_N = 500$. (a) Cost kernel s_k . (b) Optimal feedback gains K_k . (c) Optimal trajectory x_k^* .

A MATLAB routine simulating the application of the feedback control to the plant (1) is also shown in Fig. 2.2-4. Beginning at the given x_0 , it steps *forward* in time using (1) and (6), with precomputed stored values of K_k , to calculate the resulting trajectory x_k^* . The backward-computed kernel and gain sequences s_k and K_k and the forward-computed state sequence x_k^* are plotted in Fig. 2.2-5. Note that if s_N is finite, x_N will not be exactly equal to zero, but that as $s_N \rightarrow \infty$, the final state x_N approaches zero more closely. The value of s_N for Fig. 2.2-5 was 5. Figure 2.2-6 is the simulation using $s_N = 500$. ■

Software for the efficient solution of the Riccati equation is included in Bierman (1977). However, this example has made the point that a little *preliminary analysis* can result in simplified algorithms (i.e., we used (4) in the example, not (3), for implementation). This is especially true in the case where the number of states n is greater than one and the matrices are sparse.

To see why, note that the Riccati equation is symmetric (the transpose of the right-hand side is equal to itself if S_{k+1} is symmetric). Therefore, S_k is symmetric for all k if $S_N^T = S_N$, which we have assumed. This means that S_k has n^2 elements, only $n(n+1)/2$ of which are distinct. Some preliminary analysis can yield $n(n+1)/2$ *scalar* recursions for the components of S_k , which are easier to use than one $n \times n$ *matrix* recursion. We shall see examples of this in Section 2.3.

Exercise 2.2-3. LQ Regulator with Weighting of **State–input Inner Product**

Let the plant be given by

$$x_{k+1} = A_k x_k + B_k u_k, \quad (1)$$

but consider the modified performance index

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} \begin{bmatrix} x_k^T & u_k^T \end{bmatrix} \begin{bmatrix} Q_k & T_k \\ T_k^T & R_k \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix}, \quad (2)$$

where the block coefficient matrix in the sum is positive definite. This index allows us to weight products of state and input components to keep them small.

- a. Show that the Hamiltonian system is

$$\begin{bmatrix} x_{k+1} \\ \lambda_k \end{bmatrix} = \begin{bmatrix} \bar{A}_k & -B_k R_k^{-1} B_k^T \\ \bar{Q}_k & \bar{A}_k^T \end{bmatrix} \begin{bmatrix} x_k \\ \lambda_{k+1} \end{bmatrix}, \quad (3)$$

where

$$\bar{A}_k = A_k - B_k R_k^{-1} T_k^T, \quad (4)$$

$$\bar{Q}_k = Q_k - T_k R_k^{-1} T_k^T. \quad (5)$$

- b. Show that the optimal control is given by

$$u_k = -(B_k^T S_{k+1} B_k + R_k)^{-1} (B_k^T S_{k+1} A_k + T_k^T) x_k, \quad (6)$$

where sequence S_k is give by the Riccati equation

$$S_k = \bar{A}_k^T [S_{k+1} - S_{k+1} B_k (B_k^T S_{k+1} B_k + R_k)^{-1} B_k^T S_{k+1}] \bar{A}_k + \bar{Q}_k, \quad k < N \quad (7)$$

with boundary condition of S_N .

- c. Show that (7) can alternatively be written

$$S_k = A_k^T S_{k+1} A_k - K_k^T (B_k^T S_{k+1} B_k + R_k) K_k + Q_k \quad (8)$$

where the Kalman gain is

$$K_k = (B_k^T S_{k+1} B_k + R_k)^{-1} (B_k^T S_{k+1} A_k + T_k^T). \quad (9)$$

- d. Show that the optimum cost to go on the subinterval $[k, N]$ is given in terms of the state x_k as

$$J_k = \frac{1}{2} x_k^T S_k x_k. \quad (10)$$

- e. Show that the Riccati equation (2.2-53) can be written as (8) with K_k given by (2.2-57). Hence, the only changes introduced by the off-diagonal weighting term T_k are that the Kalman gain must be modified as in (9) and that the Riccati equation formulation (8) should be used. ■

2.3 DIGITAL CONTROL OF CONTINUOUS-TIME SYSTEMS

With the increasing sophistication of microprocessors, more and more control schemes are being implemented digitally. In these schemes, the control input is switched to new values at discrete time steps, with a zero-order hold usually used between switchings so that the control is constant during these intervals. Such controls must be designed using a discretized version of the continuous plant.

Design of Digital Controls

The design of digital control laws is very straightforward. If the continuous time-invariant plant is given by

$$\dot{x}(t) = Ax(t) + Bu(t), \quad (2.3-1)$$

then the discretized version of the plant, using a sampling period of T , is

$$x_{k+1} = A^s x_k + B^s u_k, \quad (2.3-2)$$

where the sampled plant and control matrices are

$$A^s = e^{AT}, \quad (2.3-3a)$$

$$B^s = \int_0^T e^{A\tau} B d\tau. \quad (2.3-3b)$$

This discretization process assumes that the control input $u(t)$ to the continuous plant is switched only at times kT , and that it is held constant (with a zero-order hold) between switchings so that $u(t) = u_k$, for $kT \leq t < (k+1)T$. This is within our power to guarantee, since we select the control input. If we do manufacture $u(t)$ from u_k in this fashion, then the continuous state is related to the discrete state according to $x(kT) = x_k$.

To guarantee that the samples of the state, $x_k = x(kT)$, and the control, $u_k = u(kT)$, display a desired behavior, it is necessary only to apply our optimal

control law computation methods to the sampled system (2.3-2) to compute the optimal sequence u_k^* . The actual continuous input that is sent to the plant is then manufactured from u_k^* .

The values of $x(t)$ between sampling instants kT cannot be specified using a control law design based on (2.3-2), but they can be determined in a very easy manner, for between the sampling instants kT and $(k+1)T$ the state propagates according to the plant dynamics with a constant input:

$$\dot{x}(t) = Ax(t) + Bu_k, \quad kT \leq t < (k+1)T. \quad (2.3-4)$$

The solution to this equation is

$$x(t) = e^{A(t-kT)}x_k + \int_{kT}^t e^{A(t-\tau)}B d\tau u_k, \quad kT \leq t < (k+1)T. \quad (2.3-5)$$

If the continuous plant is time varying, then the discretization process yields a time-varying discrete plant

$$A_k^s = \phi((k+1)T, kT), \quad (2.3-6a)$$

$$B_k^s = \int_{kT}^{(k+1)T} \phi((k+1)T, \tau)B(\tau) d\tau, \quad (2.3-6b)$$

where $\phi(t, t_0)$, is the state transition matrix of the time invariant plant described by (2.3-1). The optimal control approach just described still applies.

Simulation of Digital Controls

Once the optimal control sequence u_k^* has been designed based on (2.3-2), we should like to simulate the application of the resulting digital control to the plant (2.3-1) to verify that it has a satisfactory behavior. For completeness, we should like to observe the behavior of the state *between* sampling instants as well as at the sampling instants.

To do this, we can use the simulation scheme shown in Fig. 2.3-1. The continuous state equation (2.3-1) is represented in the figure as

$$F(t, X, \dot{X}) \quad (2.3-7)$$

and is integrated using the MATLAB routine *lsim.m*. The control input $u(t)$ is updated at each time kT and then held until time $(k+1)T$. It is important to realize that *two* sampling time intervals are involved, the plant sampling interval T and the simulation sampling interval T_s . The states are evaluated by the routine at the sampling interval and hence we choose T_s to be a divisor of T .

Note that we are using discrete system (2.3-2) to design the control law, but that, to find $x(t)$ at all values of t , we are using the continuous plant (2.3-1) to *simulate* the control law.

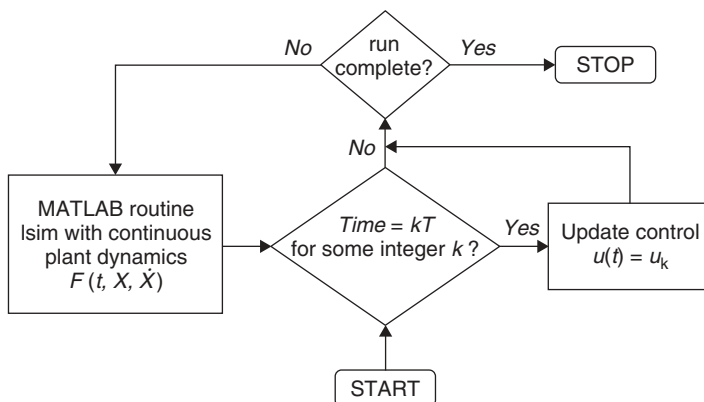


FIGURE 2.3-1 Digital control simulation scheme.

Some Examples

Let us demonstrate how easy this procedure is by considering some examples. The examples also illustrate how to simplify the coding for the computation of optimal control u_k by doing some preliminary analysis; by tailoring the Riccati equation to each problem, no matrix manipulations are used.

Example 2.3-1. Digital Control of an RC Circuit

The electric circuit in Fig. 2.3-2 provides a scalar example that nicely illustrates our approach. The continuous state equation is

$$\dot{x} = \frac{-1}{\tau}x + \frac{1}{\tau}u, \quad (1)$$

with time constant

$$\tau = \frac{1}{RC}. \quad (2)$$

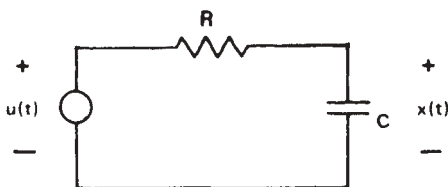


FIGURE 2.3-2 RC circuit.

Let $\tau = 5$, so that

$$\dot{x} = -0.2x + 0.2u. \quad (3)$$

It is desired to control the capacitor voltage $x(t)$ by a scheme in which input $u(t)$ is switched only at discrete instants kT by a microprocessor. The microprocessor also

samples $x(t)$ at each sampling period T . Let $T = 0.5$ sec. (For good control, we should select $T < \tau/10$.)

a. Design of Digital Control Law for Free Final State

The discretized system is

$$\begin{aligned} x_{k+1} &= e^{-T/\tau} x_k + \int_0^T e^{-\lambda/\tau} \frac{1}{\tau} d\lambda \cdot u_k \\ &= e^{-T/\tau} x_k + (1 - e^{-T/\tau}) u_k, \end{aligned} \quad (4)$$

or

$$x_{k+1} = ax_k + bu_k, \quad (5)$$

with $a = 0.9048$ and $b = 0.0952$. Suppose that we want the control and state samples u_k and x_k to be small over a 5-sec interval for any initial voltage $x(0)$. Then $N = 5/T = 10$. To express these control objectives mathematically, select the performance index

$$J = \frac{1}{2} s_N x_N^2 + \frac{1}{2} \sum_{k=0}^{N-1} (ax_k^2 + ru_k^2). \quad (6)$$

This is the same system and cost we examined in Example 2.2-2, so the optimal control is just given by the Riccati equation as

$$K_k = \frac{abs_{k+1}}{b^2 s_{k+1} + r}, \quad (7)$$

$$s_k = \frac{a^2 r s_{k+1}}{b^2 s_{k+1} + r} + q, \quad (8)$$

$$u_k = K_k x_k. \quad (9)$$

By making s_N large, we can force the final state $x_N = x(5)$ to be small.

b. Simulation of Digital Control Law for Free Final State

It is quite simple to simulate the digital control law being applied to the continuous plant (3). First notice that equations (7), (8), and (9) can be computed by using the *scaopt.m* function. The overall driver program for the implementation of the digital control law is given in Fig. 2.3-3, where $a_c = -0.2$, $b_c = 0.2$, $a_d = 0.9048$, $b_d = 0.0952$, $q = r = 1$, $s = 100$, $N = 10$, and $x_0 = 10$.

Results of the simulation run are shown in Fig. 2.3-4, where the state $x(t)$ with no control is shown along with the state resulting on application of our digital control. Also shown in Fig. 2.3-5 is $u(t)$, the digital control manufactured from u_k in (9).

c. Design of Control Law for Fixed Final State

To achieve a fixed final value of $x(5) = 0$, we can use a very large value of s_N in parts a and b. To achieve a nonzero value r_N for $x(5)$, however, we must use the open-loop control of (2.2-38). (When we discuss the *function of final-state-fixed problem* in Section 4.5, we shall see how to achieve a nonzero fixed final state using a Riccati-equation-based design.)

```

function ex2_3_1b(a_c, b_c, a_d, b_d, q, r, s, x0, N)
% Compute Optimal Control Input
[x, u, K, S] =scaopt(a_d, b_d, q, r, s, x0, N);
% Define the time interval T
T=0:0.05:5;
% Expand the input u to the specified interval T
U=kron(u, ones(1, 10));
U=[U u(length(u))];
% Simulate the plant dynamics
system=ss(a_c,b_c,1,0);
figure(1)
[Y,T,X]=lsim(system, U, T, x0); plot(T,Y); hold;
%Simulate the plant dynamics with zero input
[Y,T,X]=lsim(system, [0 kron(u, zeros(1, 10))], T, x0); plot(T,Y);
legend('x(t) with zero control input','x(t) with control');
    xlabel('Time [s]');
% Plot the input u(t)
figure(2)
T=0:0.05:5;
plot(T, U); legend('Control input'); xlabel('Time [s]');
end

```

FIGURE 2.3-3 Driver program to compute u_k^* and simulate the resulting digital control scheme.

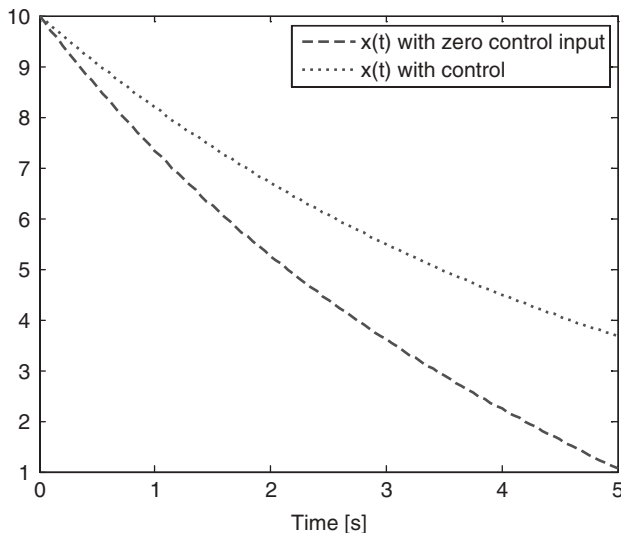


FIGURE 2.3-4 Simulation of continuous plant dynamics comparing zero-input response to controlled response.

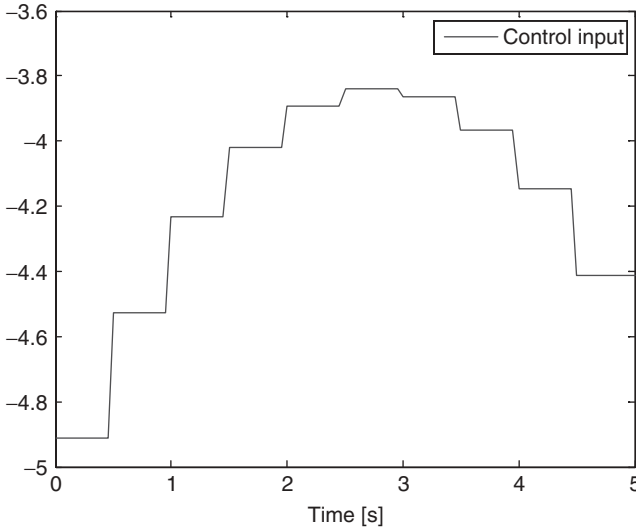


FIGURE 2.3-5 The control input obtained from u_k .

Thus, suppose we want to drive the capacitor voltage from $x(0) = x_0 = 10$ V exactly to $x(5) = r_N = 20$ V while minimizing the energy

$$J_0 = \frac{r}{2} \sum_{k=0}^{N-1} u_k^2 \quad (10)$$

of the control samples. The design of the control to make the sampled system (5) achieve this objective is exactly the problem we solved in Example 2.2-1, so the optimal discrete control sequence is

$$u_k = \frac{(1-a)^2}{b(1-a^{2N})} (r_N - a^N x_0) a^{N-k-1}. \quad (11)$$

d. Simulation of Digital Control for Fixed Final State

In this open loop scheme, the simulation is as easy as before. We just use the driver program as before with the exception that in this case we cannot use *scaopt.m* to compute u_k ; instead, we wrote a simple script to implement (11). The driver program is shown in Fig. 2.3-6.

The simulation results are shown in Fig. 2.3-7. Note that $-u(t)$ (not $u(t)$) is plotted there.

While the open-loop control is easier to implement, it is not as robust as the closed-loop control. If at a time of 2 sec, for example, a noise source drives the state off the optimal trajectory shown in Fig. 2.3-7, the final state $x(5)$ will no longer be 20V.

```

function ex2_3_1d (a_c, b_c, a_d, b_d, q, r, s, rN, x0, N)
% Compute Optimal Control Input
for k=1:N
u(k)=((1-a_d^2)/(b_d*(1-a_d^(2*N))))*(rN-x0*a_d^N)*a_d^(N-k);
end
% Define the time interval T
T=0.05:0.05:5;
% Expand the input u to the specified interval T
U=kron(u,ones(1,10));
% Simulate the plant dynamics
lsim(a_c, b_c, 1, 0, U, T, x0); hold
%Simulate the plant dynamics with zero input
lsim (a_c, b_c, 1, 0, kron (u, zeros (1, 10)), T, x0);
% Plot the input -u(t)
plot (T, U);

```

FIGURE 2.3-6 Driver program for on-line computation of open-loop control.

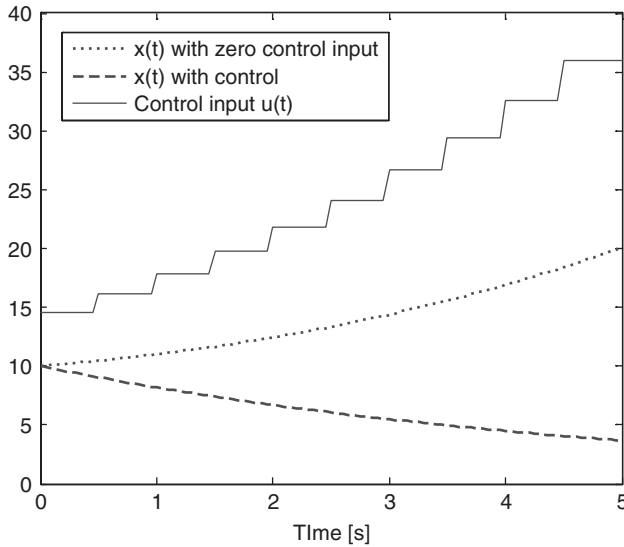


FIGURE 2.3-7 Simulation of continuous plant dynamics comparing zero-input response to controlled response. ■

Example 2.3-2. Digital Control of Systems Obeying Newton's Laws

Newton's laws $m\ddot{d} = F$ can be expressed in state-variable form as

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u, \quad (1)$$

where $x \triangleq [d \ v]^T$, with $d(t)$ and $v(t)$ representing position and velocity. Input $u(t)$ is an acceleration; to find the input in units of force, we can multiply $u(t)$ by m , the mass of the body.

The optimal control law we shall derive applies to any system obeying (1). To lend more interest to our example, however, let us formulate a particular problem to solve.

a. The Rendezvous Problem

See Fig. 2.3-8. A target aircraft A_t is moving in the y_1 direction with a constant velocity of V_t . Its initial y_1 coordinate is Y_t . Our aircraft A is moving in the y_1 direction with a constant velocity of $V > V_t$. Our initial y_1 coordinate is 0. Thus, our velocity relative to A_t is $(V - V_t)$. Clearly, at time

$$t_f = \frac{Y_t}{V - V_t} \quad (2)$$

the two aircraft A and A_t will be abreast of each other (i.e., have the same y_1 coordinate). The y_1 velocities V and V_t are fixed throughout the problem, and hence the final time t_f is known.

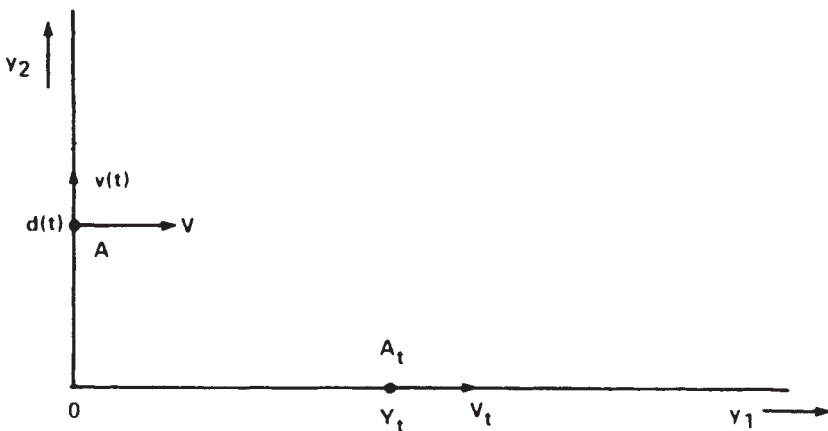


FIGURE 2.3-8 Rendezvous problem geometry.

The optimal control problem is as follows. The y_2 position and velocity of aircraft A relative to A_t are $d(t)$ and $v(t)$, and the y_2 dynamics of A are described by (1). It is required to determine the control acceleration $u(t)$ needed in the y_2 direction so that aircraft A will rendezvous with the target A_t at time t_f . This means that we must determine $u(t)$ so that $d(t_f)$ and $v(t_f)$ are both zero.

b. Design of Digital Control Law

To find such a control $u(t)$, discretize the y_2 dynamics (1) to get (note that $e^{AT} = I + AT$ since $A^2 = 0$):

$$x_{k+1} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} T^2/2 \\ T \end{bmatrix} u_k \quad (3)$$

for some sampling period T . Suppose that $t_f = 5$ sec. Then $T = 0.5$ sec is reasonable.

Associate with (3) the performance index

$$J_0 = \frac{1}{2} x_N^T \begin{bmatrix} s_d & 0 \\ 0 & s_v \end{bmatrix} x_N + \frac{1}{2} \sum_{k=0}^{N-1} \left(x_k^T \begin{bmatrix} q_d & 0 \\ 0 & q_v \end{bmatrix} x_k + r u_k^2 \right). \quad (4)$$

Select control weighting $r = 1$, position weighting $q_d = 1$, and velocity weighting $q_v = 1$. To ensure that the final y_2 position d_N and velocity V_N are very small, select the final-state component weights as $s_d = 100$, $s_v = 100$. The number of iterations is

$$N = t_f/T = 10. \quad (5)$$

Now the optimal control is given by (2.2-58), (2.2-60), and (2.2-61). Let us do some preliminary analysis on these equations, defining a few intermediate variables to simplify things.

Since we know that S_k is symmetric for all k , let

$$S_k \triangleq \begin{bmatrix} s_1 & s_2 \\ s_2 & s_3 \end{bmatrix}. \quad (6)$$

(We shall not require time subscripts on the cost kernel components s_i since they will be updated at each k by MATLAB replacement.) Then the feedback gain is updated by

$$\begin{aligned} \delta &= B^T S_{k+1} B + r \\ &= r + \frac{s_1 T^4}{4} + s_2 T^3 + s_3 T^2, \end{aligned} \quad (7)$$

$$\begin{aligned} K_k &= B^T S_{k+1} A / \delta \\ &= \frac{1}{\delta} \left[\frac{s_1 T^2}{2} + s_2 T \frac{s_1 T^3}{2} + \frac{3s_2 T^2}{2} + s_3 T \right]. \end{aligned} \quad (8)$$

Letting

$$K_k \triangleq [k_1 \quad k_2] \quad (9)$$

we can write

$$k_1 = \left(\frac{s_1 T^2}{2} + s_2 T \right) / \delta, \quad (10a)$$

$$k_2 = \left(\frac{s_1 T^3}{2} + \frac{3s_2 T^2}{2} + s_3 T \right) / \delta. \quad (10b)$$

The closed-loop plant matrix is

$$A_k^{\text{cl}} \triangleq A - B K_k = \begin{bmatrix} 1 - k_1 T^2/2 & T - k_2 T^2/2 \\ -k_1 T & 1 - k_2 T \end{bmatrix}. \quad (11)$$

Defining the components of A_k^{cl} as

$$A_k^{\text{cl}} \triangleq \begin{bmatrix} a_{11}^{\text{cl}} & a_{12}^{\text{cl}} \\ a_{21}^{\text{cl}} & a_{22}^{\text{cl}} \end{bmatrix}, \quad (12)$$

we have the four scalar updates:

$$a_{11}^{\text{cl}} = 1 - k_1 T^2 / 2, \quad (13a)$$

$$a_{12}^{\text{cl}} = T - k_2 T^2 / 2, \quad (13b)$$

$$a_{21}^{\text{cl}} = -k_1 T, \quad (13c)$$

$$a_{22}^{\text{cl}} = 1 - k_2 T. \quad (13d)$$

The updated cost kernel is

$$\begin{aligned} S_k &= A^T S_{k+1} A_k^{\text{cl}} + Q \\ &= \begin{bmatrix} s_1 a_{11}^{\text{cl}} + s_2 a_{21}^{\text{cl}} + q_d & s_1 a_{12}^{\text{cl}} + s_2 a_{22}^{\text{cl}} \\ (s_1 T + s_2) a_{11}^{\text{cl}} + (s_2 T + s_3) a_{21}^{\text{cl}} & (s_1 T + s_2) a_{12}^{\text{cl}} + (s_2 T + s_3) a_{22}^{\text{cl}} + q_v \end{bmatrix}, \end{aligned} \quad (14)$$

which yields the scalar updates:

$$s_1 = s_1 a_{11}^{\text{cl}} + s_2 a_{21}^{\text{cl}} + q_d, \quad (15a)$$

$$s_2 = s_1 a_{12}^{\text{cl}} + s_2 a_{22}^{\text{cl}}, \quad (15b)$$

$$s_3 = (s_1 T + s_2) a_{12}^{\text{cl}} + (s_2 T + s_3) a_{22}^{\text{cl}} + q_v. \quad (15c)$$

Note that (15b) and

$$s_2 = (s_1 T + s_2) a_{11}^{\text{cl}} + (s_2 T + s_3) a_{21}^{\text{cl}} \quad (16)$$

are evidently equivalent since S_k is symmetric. (Prove this.) For numerical stability in the face of computer roundoff error, we could use the average of (15b) and (16) for s_2 .

The optimal feedback gains are therefore found by iterating (7), (10), (13), and (15) for time index $k = N - 1, N - 2, \dots, 0$. Software for this is contained in subroutine *ex2_3_2c* in Fig. 2.3-10.

c. Simulation of Digital Control

Suppose that the initial y_2 position and velocity of aircraft A are $d(0) = 10$, $v(0) = 10$. By using subroutine *lsim* to simulate the dynamics (1) and driving with a zero input as follows

```
T=0:0.05:5;
U=zeros(1,101);
A=[0 1;0 0]; b=[0;1]; c=eye(2); d=zeros(2, 1);
system=ss(A,b,c,d);
ic=[10 10];
[Y,T,X]=lsim(system, U, T, ic);
plot(T,Y)
axis([0 5 0 60]);
legend('d(t)', 'v(t)'); xlabel('Time [s]');
```

the uncontrolled state plot in Fig. 2.3-9 was obtained.

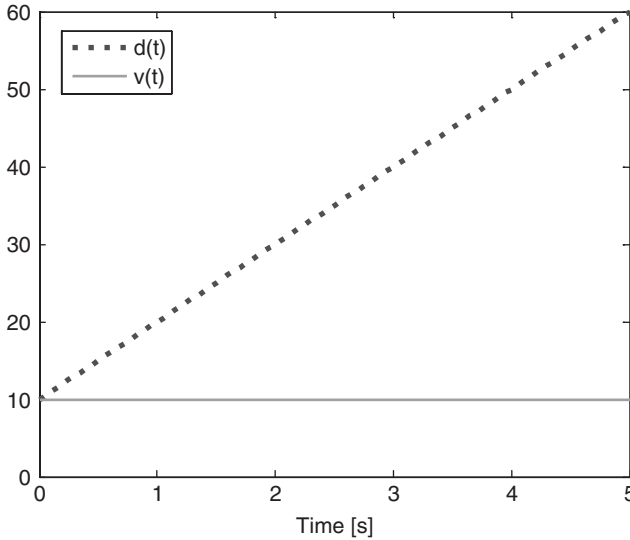


FIGURE 2.3-9 Uncontrolled response of plant.

This plot represents the zero input solution

$$d(t) = d(0) + v(0)t,$$

$$v(t) = v(0).$$

Clearly, this behavior is not exactly what we had in mind!

Let us examine the behavior of (1) manufactured under the influence of a digital control law manufactured from the optimal discrete control found in part b according to

$$u(t) = u_k, \quad kT \leq t < (k+1)T.$$

To do this, we can use the driver program shown in Fig. 2.3-10.

Since we computed the digital control law, we can use *lsim* for simulating the dynamics of the continuous plant. The resulting state trajectory is shown in Fig. 2.3-11. The first thing the control input does is go negative to -20 in order to decrease the y_2 velocity $v(t)$ to zero by a time of 0.5 sec. Then velocity $v(t)$ becomes negative so that our aircraft *A* begins to approach the target (i.e., $d(t)$ begins to decrease). Velocity $v(t)$ is then gradually returned to zero to achieve a rendezvous at $t_f = 5$ sec.

Apparently, our choice of final weighting $s_d = 100$ and $s_v = 100$ was satisfactory. The simulation shows that at $t = 5$ sec, $d(t)$ and $v(t)$ are indeed very close to zero as required.

Preliminary simplification of the Riccati equation can lead to some nice, simple implementations, as this example shows. For greater than $n = 2$ states, however, it can be more trouble than it is worth. It should be clearly realized that preliminary analysis is only for convenience; it is never actually required, since the software in Bierman (1977) can be used to solve the Riccati equation in the general case.

```

function u = ex2_3_2c(A_d, b_d, q, r, s, N, T, x0)
% Backward iteration for Cost Kernel and FB Gains
k=N;
K=zeros(N,2);
while k>0,
T2=T^2;
div=r+(s(1)*T2^2)/4 + s(2)*T*T2 + s(3)*T2;
% Feedback Gains
K(k, 1) = (s(1)*T2/2+s(2)*T)/div;
K(k, 2) = (s(1)*T2*T/2 +3*s(2)*T2/2 + s(3)*T)/div;
% Closed-loop Plant matrix
Acl=[ 1-K(k,1)*T2/2  T-K(k,2)*T2/2; -K(k,1)*T  1-K(k,2)*T]
% Cost Kernel Update
s(3)=(s(1)*T+s(2))*Acl(1, 2) + (s(2)*T+s(3))*Acl(2, 2)+q(2);
temp=s(2);
s(2)=s(1)*Acl(1,2)+temp*Acl(2,2);
s(1)=s(1)*Acl(1,1)+temp*Acl(2,1)+q(1);
k=k-1;
end
% Apply Optimal Control (Forward Iteration)
x(:,1)=x0;
for k=1:N
% Compute Optimal Control Law
u(k)=-K(k,:)*x(:,k);
% Update the Plant State
x(:,k+1)=A_d*x(:,k)+b_d*u(k);
end
end

```

FIGURE 2.3-10 Driver program to compute u_k^* and simulate the resulting control scheme.

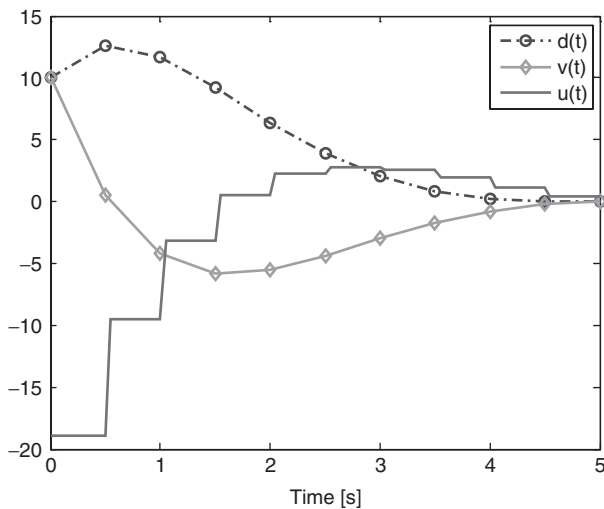


FIGURE 2.3-11 Optimal state and control trajectories for rendezvous problem.

2.4 STEADY-STATE CLOSED-LOOP CONTROL AND SUBOPTIMAL FEEDBACK

We have seen that the solution to the LQ optimal control problem is a state feedback of the form

$$u_k = -K_k x_k, \quad (2.4-1)$$

with gain sequence K_k given in terms of the solution sequence S_k to the Riccati equation as

$$S_k = A^T [S_{k+1} - S_{k+1} B (B^T S_{k+1} B + R)^{-1} B^T S_{k+1}] A + Q, \quad (2.4-2)$$

$$K_k = (B^T S_{k+1} B + R)^{-1} B^T S_{k+1} A. \quad (2.4-3)$$

In most of this section we assume the time-invariant case. Even in this situation, the closed-loop system

$$x_{k+1} = (A - BK_k)x_k \quad (2.4-4)$$

is time varying since the optimal feedback gains K_k are time varying.

This time-varying feedback is not always convenient to implement; it requires the storage of an entire sequence of $m \times n$ matrices. We might be interested in using instead a suboptimal feedback gain that does not actually minimize the performance index but is a constant so that

$$u_k = -K x_k. \quad (2.4-5)$$

Such a feedback is certainly easier to implement than (2.4-1).

As one candidate for a constant feedback gain, we might consider the limit of the optimal K_k as the final time N goes to infinity (or equivalently as $k \rightarrow -\infty$). We shall see that when this limit exists, it provides a constant feedback that is often satisfactory.

Let us first consider the effect of using an arbitrary feedback to control the plant.

Suboptimal Feedback Gains

The plant, which we shall assume time invariant in this subsection only for notational simplicity, is

$$x_{k+1} = Ax_k + Bu_k. \quad (2.4-6)$$

Let us use as a control the state feedback (2.4-1) for some arbitrary given matrix sequence K_k . We are not yet concerned about how to select K_k ; all we want to know is the resulting value of the performance index

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q x_k + u_k^T R u_k). \quad (2.4-7)$$

We can find this value by using a derivation like the one leading to (2.2-66). Thus, add the left-hand side minus the right-hand side of (2.2-63) (i.e., add zero) to (2.4-7) and use (2.4-1) to get

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} [x_{k+1}^T S_{k+1} x_{k+1} + x_k^T (Q - S_k + K_k^T R K_k) x_k]. \quad (2.4-8)$$

The sequence S_k is at this point undefined. Taking into account the state equation (2.4-6) with the control (2.4-1) yields

$$J_i = \frac{1}{2} x_i^T S_i x_i + \frac{1}{2} \sum_{k=i}^{N-1} x_k^T [(A - BK_k)^T S_{k+1} (A - BK_k) + Q + K_k^T R K_k - S_k] x_k. \quad (2.4-9)$$

Now suppose the sequence S_k satisfies the matrix equation (2.2-62). The sum is then zero, so that finally $J_i = \frac{1}{2} x_i^T S_i x_i$.

We can summarize this result as follows. Let the feedback (2.4-1) for any given K_k be applied to the plant. Then the resulting cost on $[k, N]$ is given for each time k by

$$J_k = \frac{1}{2} x_k^T S_k x_k, \quad (2.4-10)$$

where the kernel is the solution to

$$S_k = (A - BK_k)^T S_{k+1} (A - BK_k) + K_k^T R K_k + Q \quad (2.4-11)$$

with boundary condition S_N .

We should be sure we know exactly what is going on here. Equation (2.4-11) is *not* the Joseph-stabilized Riccati equation! It becomes the Joseph-Riccati equation *only if* the optimal gain sequence (2.4-3) is used. If K_k is an arbitrary given gain sequence, then (2.4-11) is simply a (linear) *Lyapunov* equation in terms of the known closed-loop plant matrix $(A - BK_k)$.

Note in particular that K_k can be a *constant* feedback matrix K , as in (2.4-5). If K_k is not the optimal gain, then J_k given by (2.4-10) and (2.4-11) is, in general, greater than the optimal cost J_k^* .

The next example illustrates the use of these new results.

Example 2.4-1. Suboptimal Feedback Control of a Scalar System

Let us reconsider the system of Example 2.2-2. The plant is

$$x_{k+1} = ax_k + bu_k \quad (1)$$

with performance index

$$J_0 = \frac{1}{2} s_N x_N^2 + \frac{1}{2} \sum_{k=0}^{N-1} (q x_k^2 + r u_k^2). \quad (2)$$

The optimal control is a time-varying state feedback

$$u_k = -K_k x_k, \quad (3)$$

with gain determined by the Riccati equation as

$$s_k = \frac{a^2 r s_{k+1}}{b^2 s_{k+1} + r} + q, \quad (4)$$

$$K_k = \frac{a b s_{k+1}}{b^2 s_{k+1} + r}. \quad (5)$$

For parameters of $a = 1.05$, $b = 0.01$, $q = r = s_N = 5$, with final time $N = 100$, a simulation was run to obtain the Kalman gain sequence shown in Fig. 2.2-5b. For $N - k = 100$, a steady-state value of $K_\infty \triangleq K_0 = 9.808$ has been reached. The corresponding Riccati-equation solution s_k^* is shown in Fig. 2.4-1a. If we apply the feedback (3) to the plant with $x_0 = 10$, the optimal state trajectory x_k^* in Fig. 2.4-1b results. The optimal cost

$$J_k^* = \frac{1}{2} s_k^* x_k^{*2} \quad (6)$$

along this trajectory is shown in Fig. 2.4-1c.

Now, let us suppose we want a simpler feedback control than (3) with K_k as in Fig. 2.2-5b. Suppose we try to use the constant state feedback

$$u_k = -K_\infty x_k = -9.808 x_k. \quad (7)$$

Then the cost is given by

$$J_k = \frac{1}{2} s_k x_k^2, \quad (8)$$

where s_k is the solution to the Lyapunov equation (2.4-11), which becomes

$$s_k = s_{k+1}(a - bK_\infty)^2 + rK_\infty^2 + q, \quad (9)$$

with boundary condition s_N . This suboptimal kernel sequence is shown in Fig. 2.4-1a. Note that it is an upper bound for the optimal sequence s_k^* . Thus, for any state x_k at any time k , the cost to go J_k using feedback (7) satisfies

$$J_k^* = \frac{1}{2} s_k^* x_k^{*2} \leq \frac{1}{2} s_k x_k^2 = J_k. \quad (10)$$

Simulating the plant (1) with the input (7) yields the suboptimal state trajectory x_k shown in Fig. 2.4-1b. The associated suboptimal cost J_k is shown in Fig. 2.4-1c. It is greater than J_k^* . The suboptimal trajectory x_k has less energy than x_k^* , but J_k is larger than J_k^* because of the larger control energy required to achieve x_k .

Note that initially, at times well removed from the final time N , x_k and x_k^* are about the same. As k approaches N , K_k^* deviates more from K_∞ , and the trajectories differ markedly.

Using the constant gain K_∞ , the closed-loop system is

$$x_{k+1} = (a - bK_\infty)x_k = 0.952x_k, \quad (11)$$

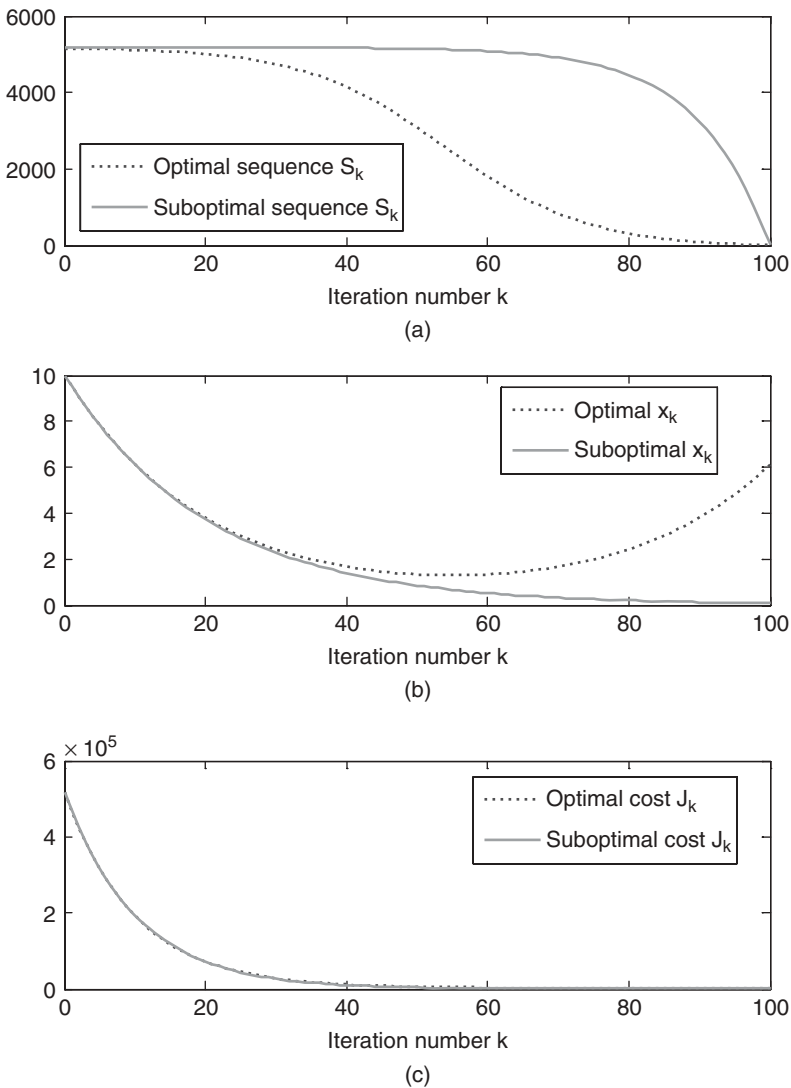


FIGURE 2.4-1 Optimal and suboptimal closed-loop behavior. (a) Cost kernels. (b) State trajectories. (c) Costs.

so the suboptimal trajectory is simply given by

$$x_k = 0.952^k x_0 = 10(0.952)^k. \quad (12)$$

By examining these graphs, we can determine whether the behavior under the influence of the simplified gain (7) is satisfactory and whether we are willing to use the extra control energy required. If so, we can go ahead and use this simplified feedback on the actual physical plant. ■

The Algebraic Riccati Equation

The comments in this subsection apply only for time-invariant plant and cost matrices. Equation (2.4-2) is solved backward in time beginning at time N . As $k \rightarrow -\infty$, the sequence S_k can have several types of behavior, as symbolized in Fig. 2.4-2. It can converge to a steady-state matrix S_∞ , which may be zero, positive semi-definite, or positive definite. It can also fail to converge to a finite matrix.

If S_k does converge, then for large negative k , evidently $S \triangleq S_k = S_{k+1}$. Thus, in the limit, (2.4-2) becomes the *algebraic Riccati equation* (ARE)

$$S = A^T[S - SB(B^T SB + R)^{-1} B^T S]A + Q, \quad (2.4-12)$$

which has no time dependence. The limiting solution S_∞ to (2.4-2) is clearly a solution of (2.4-12).

Note that if S_N is symmetric and positive semidefinite, then the solution S_k to (2.4-2) is also symmetric and positive semidefinite for all k (transpose both sides of the equation). The algebraic equation (2.4-12), on the other hand, can have nonpositive semidefinite, nonsymmetric, and even complex solutions. Thus, all solutions to the algebraic Riccati equation are not also limiting solutions to the (time-varying) Riccati equation for some S_N .

If the limiting solution to (2.4-2) exists and is denoted by S_∞ , then the corresponding *steady-state Kalman gain* is

$$K_\infty = (B^T S_\infty B + R)^{-1} B^T S_\infty A. \quad (2.4-13)$$

This is a *constant* feedback gain. Under some circumstances it may be acceptable to use the time-invariant feedback law

$$u_k = -K_\infty x_k \quad (2.4-14)$$

instead of the optimal control (2.4-1)–(2.4-3). The cost associated with such a control strategy is given by (2.4-10), where S_k satisfies (2.4-11) with $K_k = K_\infty$.

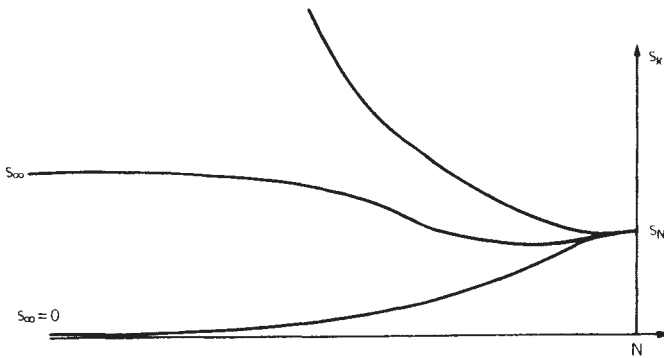


FIGURE 2.4-2 Limiting behavior of Riccati-equation solution.

To examine the consequences of using this steady-state feedback, let us discuss the limiting behavior of the closed-loop system (2.4-4) using the optimal time-varying feedback (2.4-1)–(2.4-3).

Limiting Behavior of the Riccati-equation Solution

This subsection applies only for time-invariant plant and cost matrices. We are interested in answering three questions here:

1. When does there exist a bounded limiting solution S_∞ to the Riccati equation for all choices of S_N ?
2. In general, the limiting solution S_∞ depends on the boundary condition S_N . When is S_∞ the same for all choices of S_N ?
3. When is the closed-loop plant (2.4-4) asymptotically stable?

Question 3 is particularly important. We have designed the feedback K_k to minimize a performance index, but as $k \rightarrow -\infty$ (or equivalently, as final time $N \rightarrow \infty$), we should certainly like for the closed-loop system to be stable! In some circumstances this can be guaranteed.

We can answer these questions in terms of the dynamical properties of the original system (2.4-6) with associated performance index (2.4-7). Recall that (A, B) is *reachable* if the eigenvalues of $(A - BK)$ can be arbitrarily assigned by appropriate choice of the feedback matrix K . (A, B) is *stabilizable* if there exists a matrix K such that $(A - BK)$ is asymptotically stable. This is equivalent to the reachability of all the unstable modes of A . Recall also that (A, C) is *observable* if the eigenvalues of $(A - LC)$ can be arbitrarily assigned by appropriate choice of the output injection matrix L . (A, C) is *detectable* if $(A - LC)$ can be made asymptotically stable by some matrix L . This is equivalent to the observability of the unstable modes of A .

Up to this point, we have been interested only in the state equation and not in an output, because our performance index was expressed in terms of the state and the input. Let us now define a *fictitious output* for our system. Let C and D denote the square roots of Q and R , such that $Q = C^T C$ and $R = D^T D$. In addition, let $S_N = C_N^T C_N$. Then define a fictitious output by the output equations

$$y_k = \begin{bmatrix} C \\ 0 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ D \end{bmatrix} u_k, \quad k = 0, \dots, N-1, \quad (2.4-15)$$

$$y_N = C_N x_N.$$

In terms of this “output,” the performance index can be written

$$J_i = \frac{1}{2} y_N^T y_N + \frac{1}{2} \sum_{k=i}^{N-1} y_k^T y_k. \quad (2.4-16)$$

We shall see that the answers to our three questions can be given in terms of the stabilizability of (A, B) and the observability of (A, C) . The following theorem tells us when there is a finite limiting cost kernel S_∞ .

Theorem 2.4-1. Let (A, B) be stabilizable. Then for every choice of S_N , there is a bounded limiting solution S_∞ to (2.4-2). Furthermore, S_∞ is a positive semidefinite solution to the algebraic Riccati equation (2.4-12).

Proof: Since (A, B) is stabilizable, there exists a constant feedback L so that

$$u_k = -Lx_k$$

and

$$x_{k+1} = (A - BL)x_k$$

is asymptotically stable. Thus, x_k is bounded and goes to zero as $k \rightarrow \infty$. Therefore, the associated cost

$$J_i = \frac{1}{2}x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q x_k + u_k^T R u_k)$$

is finite as $i \rightarrow \infty$. It is given by (2.4-10) with $k = i$, where S_i satisfies the Lyapunov equation (2.4-11) (with index k replaced by i) with K_i there equal to L , and using S_N as boundary condition.

The *optimal* cost, however, is given for all i by

$$J_i^* = \frac{1}{2}x_i^T S_i^* x_i,$$

where S_i^* is the solution to (2.4-2) with S_N as boundary condition. Since $J_i^* \leq J_i$ for all initial states x_i , S_i provides an upper bound for S_i^* for all i ($S_i^* \leq S_i$ means $(S_i - S_i^*) \geq 0$). Hence, the solution sequence to (2.4-2) is bounded above by a finite sequence.

It can be shown that the solution S_i^* to (2.4-2) is smooth, so that if it is bounded above by a finite S_i , then it converges to a constant limit S_∞ . For details, see Casti (1977) or Kwakernaak and Sivan (1972).

Since equation (2.4-2) is symmetric, then so is S_i for all i if S_N is symmetric, which we have assumed. The structure of the equation and the assumptions on Q and R also imply positive semidefiniteness.

Clearly, S_∞ is a solution to the limiting equation (2.4-12). ■

An important point should be noted here. In presenting the solution in Table 2.2-1 to the free-final-state LQ regulator and in the subsequent examples we discussed, we did not make any controllability assumptions on the plant. Regardless of the controllability properties of the plant, the optimal control will do the best it can to minimize the performance index. We have just shown that if the plant is, in fact, stabilizable, then there is a finite limiting solution S_∞ to

the Riccati equation. This means that as the time interval $[i, N]$ goes to infinity, the optimal cost J_i^* stays bounded. Since $R > 0$, this in turn guarantees that the optimal control u_k^* itself does not go to infinity.

We can often show the stronger condition of reachability, which implies stabilizability, and for which there is a simple test based on the full rank of the reachability matrix

$$U_n = [B \quad AB \quad \cdots \quad A^{n-1}B], \quad (2.4-17)$$

where $x \in R^n$.

Theorem 2.4-1 is based on system stabilizability, and it makes intuitive sense because we are dealing with the optimal control problem. The next result initially seems very strange; it provides answers to our second and third questions in terms of the observability of the plant through the fictitious output!

Theorem 2.4-2. Let C be a square root of the intermediate-state weighting matrix, so that $Q = C^T C \geq 0$, and suppose $R > 0$. Suppose that (A, C) is observable; then (A, B) is stabilizable if and only if:

- There is a unique positive definite limiting solution S_∞ to the Riccati equation (2.4-2). Furthermore, S_∞ is the unique positive definite solution to the algebraic Riccati equation (2.4-12).
- The closed-loop plant

$$x_{k+1} = (A - BK_\infty)x_k$$

is asymptotically stable, where K_∞ is given by (2.4-13).

Proof:

Necessity

Define D by $R = D^T D$. Then $|D| \neq 0$, so that $B = MD$ for some M . One, therefore, has

$$\text{rank} \begin{bmatrix} zI - A \\ C \\ DK \end{bmatrix} = \text{rank} \begin{bmatrix} I & 0 & M \\ 0 & I & 0 \\ 0 & 0 & I \end{bmatrix} \begin{bmatrix} zI - A \\ C \\ DK \end{bmatrix} = \text{rank} \begin{bmatrix} zI - (A - BK) \\ C \\ DK \end{bmatrix}. \quad (1)$$

If (A, C) is observable, then by the PBH (Popov–Belevitch–Hautus) rank test (Kailath 1980),

$$\text{rank} \begin{bmatrix} zI - A \\ C \end{bmatrix} = n \quad \text{for every } z, \quad (2)$$

so that by (1)

$$\left((A - BK), \begin{bmatrix} C \\ DK \end{bmatrix} \right)$$

is observable for any K .

Now, stabilizability implies the existence of a feedback control $u_k = -Lx_k$, so that

$$x_{k+1} = (A - BL)x_k \quad (3)$$

is asymptotically stable. The cost of such a control on $[i, \infty]$ is

$$J_i = \frac{1}{2} x_i^T S x_i \quad (4)$$

with S the limiting solution to (2.4-11) with $K_i = L$. The optimal closed-loop system (2.4-2)–(2.4-4) has an associated cost on $[i, \infty]$ of

$$J_i^* = \frac{1}{2} \sum_{k=i}^{\infty} (x_k^T Q x_k + u_k^T R u_k) = \frac{1}{2} x_i^T S^* x_i \leq J_i. \quad (5)$$

with S^* the limiting solution to (2.4-2). Therefore, $Cx_k \rightarrow 0$ and, since $|R| \neq 0$, $u_k \rightarrow 0$. Select an N so that Cx_k and u_k are negligible for $k > N$. Then for $k > N$,

$$0 = \begin{bmatrix} Cx_k \\ Cx_{k+1} \\ \vdots \\ Cx_{k+n-1} \end{bmatrix} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} x_k, \quad (6)$$

and so observability of (A, C) requires $x_k \rightarrow 0$. Hence, the optimal closed-loop system (2.4-4) is asymptotically stable.

Write (2.4-12) as

$$S = (A - BK_{\infty})^T S (A - BK_{\infty}) + \begin{bmatrix} C \\ DK_{\infty} \end{bmatrix}^T \begin{bmatrix} C \\ DK_{\infty} \end{bmatrix} \quad (7)$$

with K_{∞} the optimal feedback. (Prove that this can be done.) Then (7) is a Lyapunov equation with

$$\left((A - BK_{\infty}), \begin{bmatrix} C \\ DK_{\infty} \end{bmatrix} \right)$$

observable and $(A - BK_{\infty})$ stable. Therefore, there is a unique positive definite solution S^* to (2.4-12).

Sufficiency

If $x_{k+1} = (A - BK_{\infty})x_k$ is asymptotically stable, then (A, B) is certainly stabilizable. ■

The structure of this result should be examined. All it is, is a restatement of Theorem 2.4-1 under the observability hypothesis. The observability condition has made our previous theorem quite a bit stronger!

Let us discuss two aspects of this theorem: what it does for us, and how we can guarantee that it holds in a particular problem. Part a of the theorem says

that if interval $[i, N]$ is a large enough, then the optimal cost of control is

$$J_i^* = \frac{1}{2} x_i^T S_\infty x_i, \quad (2.4-18)$$

which is both finite and independent of the value we selected for the final-state weighting S_N . Thus, whether we weight the final state heavily or lightly in J_i , the cost of optimal control over $[i, N]$ is the same. Since $R > 0$, the finiteness of J_i means that the optimal control is also finite; all our objectives can be achieved with finite control energy.

The theorem also guarantees the existence of a steady-state gain K_∞ that stabilizes the plant. This means two things. First, at the beginning of a control run, far from the final time N , the closed-loop plant $(A - BK_k)$ is nearly $(A - BK_\infty)$, so it starts out stable. Second, if we should decide to use the constant suboptimal gain $u_k = -K_\infty x_k$ for all k instead of the harder to implement optimal time-varying feedback, we are guaranteed at least closed-loop stability.

These properties are all quite desirable. To guarantee that they hold, we need only to ensure that the plant is stabilizable and to be judicious in our choice of the state weighting matrix Q . We should select $Q = C^T C$ for some C such that (A, C) is observable. Note that this is always the case if Q is selected to be positive *definite*, for (A, C) is observable for any C of rank n .

Intuitively, all of this means the following. If the plant is observable through the fictitious output, then motions in all of the directions of the state space R^n have an influence on the performance index. If any state component begins to increase, then so does the cost J_k . Hence, if J_k is small, necessarily the state is also. Any control that makes J_k small will also make small the excursions of the state from the origin.

On the other hand, if (A, C) is unobservable, then if the state tends to infinity in an unobservable direction of R^n , this motion will not be sensed in J_k . Thus, the boundedness of J_k does not in this case guarantee the boundedness of the state trajectory.

We have made the hypothesis of Theorem 2.4-2 unnecessarily strong. All we really require for the theorem to hold is the *detectability* of (A, C) . Thus, only the unstable modes need be observable through the performance index. In this case, however, the unique limiting solution to the Riccati equation can be guaranteed only to be positive *semidefinite*.

One result of these theorems is that we now have a way of stabilizing any multi-input plant. If Q and R are *any* positive definite matrices of appropriate dimension, then the state feedback $u_k = -K_\infty x_k$ based on the steady-state gain (2.4-13) determined from the unique positive definite solution S_∞ to the algebraic Riccati equation (2.4-12) will result in a stable closed-loop system. Different matrices Q and R will result in different closed-loop poles for $A - BK_\infty$, but these poles will always be inside the unit circle.

It would be quite useful to be able to select Q and R to yield a desired set of closed-loop poles. We present an example demonstrating the relation between Q , R , and the closed-loop poles for a simple system (Example 2.4-3), and then

in Section 2.5 we discuss a technique similar to root locus for selecting Q and R to yield desired closed-loop poles.

One more result on the limiting, or *infinite-horizon* optimal control problem $((N - k) \rightarrow \infty)$ should be mentioned. Let (A, B) be stabilizable and $(A, \sqrt{Q})$ observable. Then the steady-state feedback $u_k = -K_\infty x_k$ is the *optimal* control for a particular problem, for it is the control that minimizes the cost over the infinite time interval $[0, \infty]$. That is, it minimizes

$$J_0 = \frac{1}{2} \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k). \quad (2.4-19)$$

This follows from our discussion.

The next examples illustrate these results.

Example 2.4-2. Steady-state Control of a Scalar System

Let the plant be

$$x_{k+1} = ax_k + bu_k, \quad (1)$$

with performance index

$$J_0 = \frac{1}{2} \sum_{k=0}^{\infty} (qx_k^2 + ru_k^2). \quad (2)$$

The optimal control minimizing J_0 is the constant feedback

$$u_k = -k_\infty x_k, \quad (3)$$

where the gain (2.4-13) is

$$k_\infty = \frac{abs_\infty}{b^2 s_\infty + r} \quad (4)$$

and the steady-state kernel is the unique positive (definite) root of the algebraic Riccati equation

$$s = a^2 s - \frac{a^2 b^2 s^2}{b^2 s + r} + q. \quad (5)$$

Under the influence of the control (3), the closed-loop system is

$$a^{\text{cl}} = a - bk_\infty = \frac{a}{1 + (b^2/r)s_\infty}. \quad (6)$$

The ARE can be written

$$b^2 s^2 + [(1 - a^2)r - b^2 q]s - qr = 0. \quad (7)$$

If we define the auxiliary variable

$$\Lambda = \frac{b^2 q}{(1 - a^2)r}, \quad (8)$$

this becomes

$$\frac{\Lambda}{q}s^2 + (1 - \Lambda)s - \frac{\Lambda r}{b^2} = 0, \quad (9)$$

which has two solutions, given by

$$s = \frac{q}{2\Lambda} \left[\pm \sqrt{(1 - \Lambda)^2 + \frac{4\Lambda}{(1 - a^2)}} - (1 - \Lambda) \right]. \quad (10)$$

We must now consider two cases.

a. Original System Stable

If $|a| < 1$, then $(1 - a^2) > 0$ and $\Lambda > 0$. In this case the unique non-negative solution to (7) is

$$s_\infty = \frac{q}{2\Lambda} \left[\sqrt{(1 - \Lambda)^2 + \frac{4\Lambda}{(1 - a^2)}} - (1 - \Lambda) \right], \quad (11)$$

and the steady-state gain is given by (4).

In the scalar case, observability of $(a, \sqrt{q})$ is equivalent to $q \neq 0$, and plant reachability is equivalent to $b \neq 0$. Therefore, the observability and reachability conditions imply $\Lambda > 0$ and hence $s_\infty > 0$. Then, according to (6)

$$|a^{\text{cl}}| < |a|, \quad (12)$$

so that the closed-loop system is stable.

Since $|a| < 1$, if $q = 0$, the system is still detectable (the unobservable mode is stable). If $q = 0$, then $\Lambda = 0$, but according to (6) $a^{\text{cl}} = a$ is still stable. Note that in this case, $s_\infty = 0$ (i.e., positive *semidefinite*).

b. Original System Unstable

If $|a| > 1$, then $(1 - a^2) < 0$ and $\Lambda < 0$. Then the unique non-negative solution to the ARE is

$$s_\infty = \frac{-q}{2\Lambda} \left[\sqrt{(1 - \Lambda)^2 + \frac{4\Lambda}{(1 - a^2)}} + (1 - \Lambda) \right]. \quad (13)$$

Again, reachability and observability imply that Λ is strictly negative, so that according to (13), $s_\infty > 0$.

According to (6), if $\Lambda < 0$, we still have $|a^{\text{cl}}| < |a|$, but it is not easy to prove that $|a^{\text{cl}}| < 1$. Note, however, that

$$a^{\text{cl}} = \frac{a}{1 - \frac{1 - a^2}{2} \left[\sqrt{(1 - \Lambda)^2 + \frac{4\Lambda}{1 - a^2}} + (1 - \Lambda) \right]}, \quad (14)$$

so that if $|a| \gg 1$, then $\Lambda \simeq 0$ and

$$a^{\text{cl}} \simeq \frac{1}{a}. \quad (15)$$

This is certainly stable. Note that a^{cl} does not depend on b , q , and r individually, but only on the quantity b^2q/r . (This is also true if a is stable.) If $|a| > 1$, then detectability of $(a, \sqrt{q})$ is equivalent to $q \neq 0$, which is necessary for (14) to be stable.

c. a^{cl} as a Function of a

Figure 2.4-3 shows a plot of the closed-loop plant matrix a^{cl} as a function of the original plant matrix a for $b^2q/r = 1$. Note that a^{cl} is stable for all values of a .

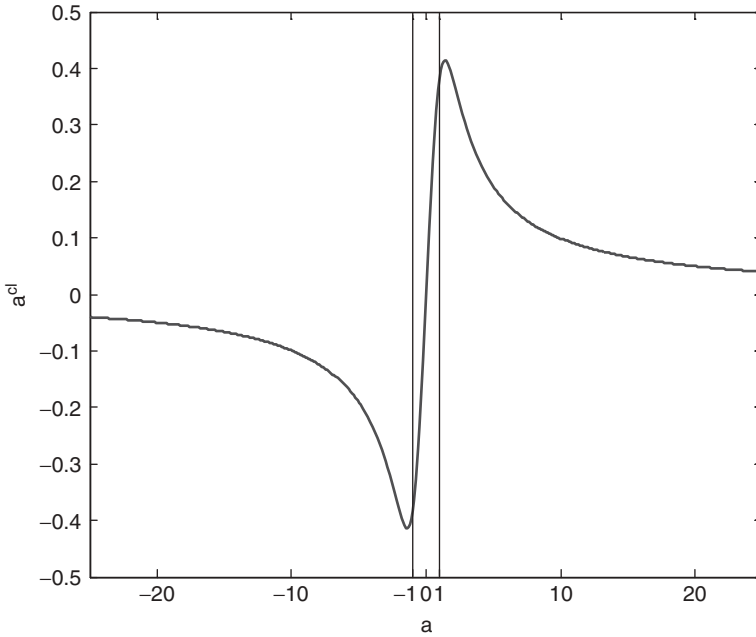


FIGURE 2.4-3 Closed-loop plant matrix as a function of an open-loop plant matrix. ■

Example 2.4-3. Limiting Control Behavior for Systems Obeying Newton's Laws

Let us reconsider the discretized *Newton's system* of Example 2.3-2,

$$x_{k+1} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} T^2/2 \\ T \end{bmatrix} u_k, \quad (1)$$

where $x_k = [d_k, v_k]^T$, with d_k and v_k the k th samples of position and velocity. The sampling period is $T = 0.5$ sec for this example, so that

$$x_{k+1} = \begin{bmatrix} 1 & 0.5 \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} 0.125 \\ 0.5 \end{bmatrix} u_k. \quad (2)$$

Let $Q = I$, so that the cost is

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (d_k^2 + v_k^2 + u_k^2). \quad (3)$$

Using the solution to the optimal control problem given in Example 2.3-2 and the associated software, we can determine the optimal cost kernel S_k , feedback gain K_k , closed-loop plant matrix A_k^{cl} , and the optimal control u^* and state trajectory x^* . By adding a polynomial root finder we can determine the poles of A_k^{cl} for each time k .

First, let us use final-state weighting of $S_N = 100I$. This is the simulation run in Example 2.3-2, and the optimal state trajectory and control are shown in Fig. 2.3-11. This behavior is quite satisfactory, and a good rendezvous is achieved. The Kalman gain K_k is shown in Fig. 2.4-4a. Note that it is defined only at integer values of k . In the graph its values are connected by lines to distinguish more easily between the two components of K_k , which are denoted by K_k^1 and K_k^2 .

The poles of the closed-loop system A_k^{cl} are a function of time index k , since A_k^{cl} is a time-varying system. They are illustrated in Fig. 2.4-4b. When $k = N$, the system is marginally stable with poles at $z = 0$ and $z = 1$. As k decreases, the closed-loop poles become complex and move as shown.

For $N - i = 10$, corresponding to a run time of 5 sec, the Riccati-equation solution has reached a steady-state value that is given by S_0 . It is equal to

$$S_\infty \triangleq \begin{bmatrix} 4.035 & 2.0616 \\ 2.0616 & 4.1438 \end{bmatrix}. \quad (4)$$

The corresponding steady-state gain is

$$K_\infty = \begin{bmatrix} 0.6514 \\ 1.3142 \end{bmatrix}, \quad (5)$$

which agrees with Fig. 2.4-4a. The steady-state closed-loop plant matrix is

$$A_\infty^{\text{cl}} = (A - BK_\infty) = \begin{bmatrix} 0.9185 & 0.3357 \\ -0.3257 & 0.3429 \end{bmatrix}, \quad (6)$$

which has poles at

$$z = 0.6307 \pm j0.1628. \quad (7)$$

These steady-state closed-loop poles agree with Fig. 2.4-4b.

We might now be interested in trying to use the suboptimal feedback

$$u_k = -K_\infty x_k \quad (8)$$

with K_∞ given by (5). This control is easier to implement than the optimal control $u_k = -K_k x_k$ since the Kalman gain sequence K_k need not be stored. If we simulate this suboptimal control, the suboptimal state trajectory in Fig. 2.4-4c results. This trajectory is barely distinguishable from the optimal path in Fig. 2.3-11, except that the final values $d(5)$ and $v(5)$ are not quite equal to zero. It would probably be suitable for most purposes.

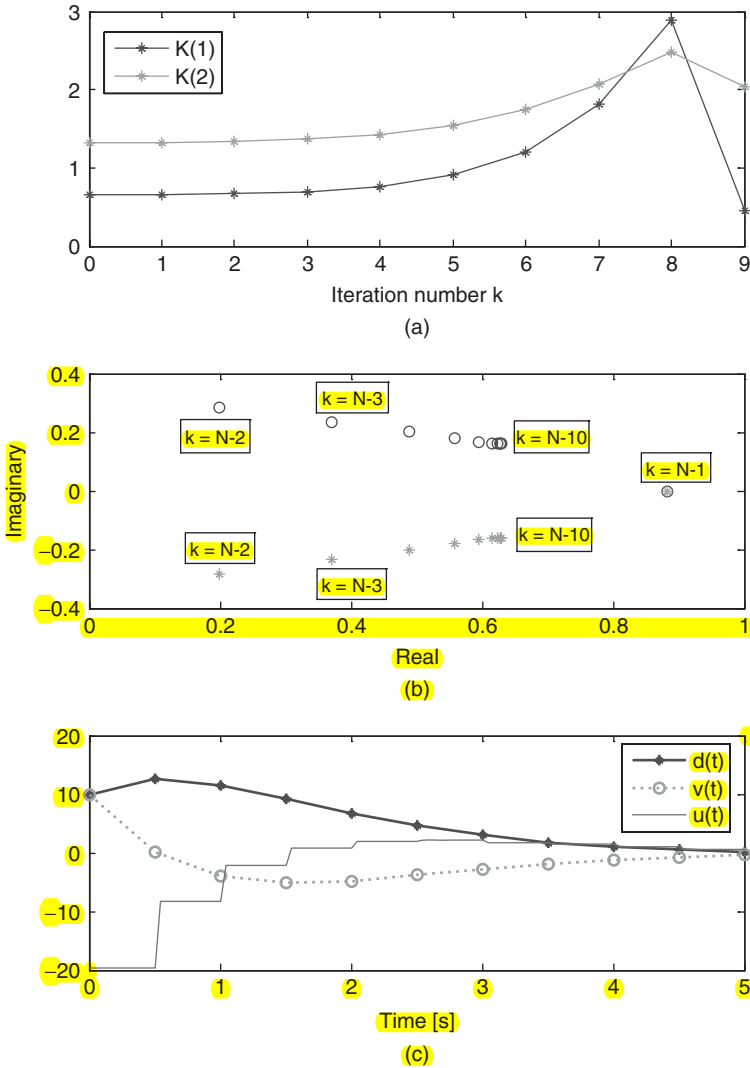


FIGURE 2.4-4 Position and velocity weighting, $S_N = 100I$. (a) Optimal feedback gain sequence. (b) Locus of the closed-loop poles. (c) Suboptimal state trajectory using steady-state feedback gain K_∞ .

Let us now check our theorems. Since (A, B) is reachable, the solution S_k to the Riccati equation converges to a finite S_∞ . A root of Q is given by $C = I$. Since (A, C) is certainly observable, S_∞ is positive definite. Furthermore, A_∞^{cl} is asymptotically stable.

An important consequence of these facts is that if we use control law (8) and give the system enough time, the state will always go to zero.

To show that (8) may not always be satisfactory, suppose we want to rendezvous in 2 sec, corresponding to a discrete interval of length $N - i = 4$. According to Fig. 2.4-4a,

K_k has not yet reached steady state by four iterations backward from $N = 10$ (i.e., K_6 is not yet equal to $K_0 = K_\infty$). Thus, we might anticipate problems in using the steady-state control law (8). Figure 2.4-5 shows the optimal trajectories using $u_k = -K_k x_k$ and the suboptimal trajectories using (8). For this short rendezvous time, the steady-state gains would not be satisfactory.

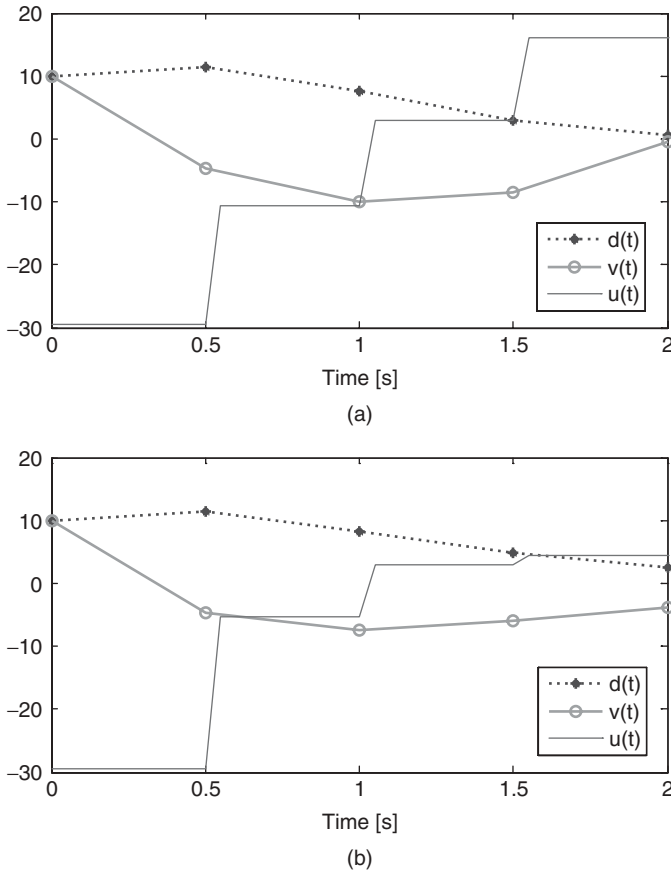


FIGURE 2.4-5 System trajectories for a short run time. (a) Trajectories using optimal control. (b) Trajectories using suboptimal steady-state feedback gains.

An Analytic Solution to the Riccati Equation

The optimal control is given in terms of the Riccati equation (2.4-2) with boundary condition S_N . One way to solve this equation is by iteration. We discuss here a nonrecursive solution for S_k with important theoretical uses that applies in the case of time-invariant plant and cost-weighting matrices. These results are due to Vaughan (1970). These results are for the case where A is nonsingular, as it is whenever a continuous-time system is discretized. In some cases, however, A

can be singular (e.g., system with pure delays). The solution of the discrete time ARE for the case of singular A is covered in a subsequent subsection.

We wrote the LQ regulator Hamiltonian system as the backward recursion (2.2-11), which is

$$\begin{bmatrix} x_k \\ \lambda_k \end{bmatrix} = H \begin{bmatrix} x_{k+1} \\ \lambda_{k+1} \end{bmatrix}, \quad (2.4-20)$$

with

$$H \triangleq \begin{bmatrix} A^{-1} & A^{-1}BR^{-1}B^T \\ QA^{-1} & A^T + QA^{-1}BR^{-1}B^T \end{bmatrix}. \quad (2.4-21)$$

The final condition for (2.4-20) if the final state is free is $\lambda_N = S_N x_N$. The initial condition is x_0 .

We assumed that

$$\lambda_k = S_k x_k \quad (2.4-22)$$

for all $k \leq N$ (cf. (2.2-50)), and based on this assumption derived the results in Table 2.2-1. Matrix S_k in (2.4-22) turned out to be given by (2.4-2). Let us now demonstrate that the Riccati-equation solution S_k can be computed in terms of the eigenvalues and eigenvectors of H . Define

$$J = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}. \quad (2.4-23)$$

Then, it can be shown with only a few lines of work that

$$H^T J H = J. \quad (2.4-24)$$

A matrix H satisfying (2.4-24) is called *symplectic*. Since H is symplectic, its inverse is very easy to find, because by (2.4-24)

$$\begin{aligned} H^T J &= J H^{-1}, \\ J^{-1} H^T J &= H^{-1}, \end{aligned}$$

so that (since $J^{-1} = -J$)

$$H^{-1} = -J H^T J. \quad (2.4-25)$$

Performing these multiplications yields

$$H^{-1} = \begin{bmatrix} A + BR^{-1}B^T A^{-T} Q & -BR^{-1}B^T A^{-T} \\ -A^{-T} Q & A^{-T} \end{bmatrix}. \quad (2.4-26)$$

(Remember that Q and R are symmetric. A^{-T} means $(A^{-1})^T$.) Now it can be shown that if μ is an eigenvalue of H , then so is $1/\mu$. If μ is an eigenvalue with

eigenvector $\begin{bmatrix} f \\ g \end{bmatrix}$ partitioned comfortably with H , then

$$\begin{bmatrix} A^{-1} & A^{-1}BR^{-1}B^T \\ QA^{-1} & A^T + QA^{-1}BR^{-1}B^T \end{bmatrix} \begin{bmatrix} f \\ g \end{bmatrix} = \mu \begin{bmatrix} f \\ g \end{bmatrix}. \quad (2.4-27)$$

This can be rearranged to read

$$\begin{bmatrix} A^T + QA^{-1}BR^{-1}B^T & -QA^{-1} \\ -A^{-1}BR^{-1}B^T & A^{-1} \end{bmatrix} \begin{bmatrix} g \\ -f \end{bmatrix} = \mu \begin{bmatrix} g \\ -f \end{bmatrix}. \quad (2.4-28)$$

Now note that the coefficient matrix on the left-hand side is just H^{-T} . This means that μ is also an eigenvalue of H^{-T} , and hence of H^{-1} . Therefore, $1/\mu$ is an eigenvalue of H , as we wanted to show (see Appendix A).

What this means is that the $2n$ eigenvalues of H can be arranged in a matrix

$$D = \begin{bmatrix} M & 0 \\ 0 & M^{-1} \end{bmatrix}, \quad (2.4-29)$$

where M is a diagonal matrix containing n eigenvalues outside the unit circle. (Hence, M^{-1} is stable.)

There is a nonsingular matrix W whose columns are the eigenvectors of H such that

$$W^{-1}HW = D. \quad (2.4-30)$$

Define a state space transformation W^{-1} so that for each k ,

$$\begin{aligned} \begin{bmatrix} x_k \\ \lambda_k \end{bmatrix} &= W \begin{bmatrix} w_k \\ z_k \end{bmatrix} \\ &= \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix} \begin{bmatrix} w_k \\ z_k \end{bmatrix}, \end{aligned} \quad (2.4-31)$$

where W_{ij} are partitions of W . Then the Hamiltonian system (2.4-20) takes on its Jordan normal form

$$\begin{bmatrix} w_k \\ z_k \end{bmatrix} = D \begin{bmatrix} w_{k+1} \\ z_{k+1} \end{bmatrix}. \quad (2.4-32)$$

The solution to (2.4-32) in terms of the final conditions is

$$\begin{bmatrix} w_k \\ z_k \end{bmatrix} = \begin{bmatrix} M^{N-k} & 0 \\ 0 & M^{-(N-k)} \end{bmatrix} \begin{bmatrix} w_N \\ z_N \end{bmatrix}. \quad (2.4-33)$$

The problem with this solution is that M^{N-k} does not go to zero as $(N-k) \rightarrow \infty$, since M is not stable. Therefore, rewrite (2.4-33) as

$$\begin{bmatrix} w_N \\ z_k \end{bmatrix} = \begin{bmatrix} M^{-(N-k)} & 0 \\ 0 & M^{-(N-k)} \end{bmatrix} \begin{bmatrix} w_k \\ z_N \end{bmatrix} \quad (2.4-34)$$

Now we consider the relations between x_k and λ_k (e.g., (2.4-22)) and between w_k and z_k . According to (2.4-22) and (2.4-31), at the final time N

$$\lambda_N = W_{21}W_N + W_{22}z_N = S_N x_N = S_N(W_{11}w_N + W_{12}z_N).$$

Solving for z_N in terms of w_N yields

$$z_N = T w_N, \quad (2.4-35)$$

where

$$T = -(W_{22} - S_N W_{12})^{-1}(W_{21} - S_N W_{11}). \quad (2.4-36)$$

Now, by (2.4-34)

$$z_k = M^{-(N-k)} z_N = M^{-(N-k)} T w_N = M^{-(N-k)} T M^{-(N-k)} w_k, \quad (2.4-37)$$

so that at each value of k

$$z_k = T_k w_k, \quad (2.4-38)$$

where

$$T_k = M^{-(N-k)} T M^{-(N-k)}. \quad (2.4-39)$$

It remains to relate S_k in (2.4-22) to T_k in (2.4-38). To do this, use (2.4-31) to write

$$\lambda_k = W_{21}w_k + W_{22}z_k = S_k x_k = S_k(W_{11}w_k + W_{12}z_k),$$

so that by (2.4-38)

$$(W_{21} + W_{22}T_k)w_k = S_k(W_{11} + W_{12}T_k)w_k. \quad (2.4-40)$$

Since this must hold for all x_0 , and hence for all trajectories w_k , it implies that

$$S_k = (W_{21} + W_{22}T_k)(W_{11} + W_{12}T_k)^{-1}. \quad (2.4-41)$$

Equations (2.4-36), (2.4-39), and (2.4-41) give a nonrecursive analytic solution to the Riccati equation for any $k \leq N$ in terms of S_N and the eigenvalues and eigenvectors of Hamiltonian matrix H . One important special case is the following. As $N \rightarrow \infty$, the Riccati equation tends to the ARE (2.4-12). We have seen that if (A, B) is reachable and $(A, \sqrt{Q})$ is observable, then the steady-state feedback gain K_∞ defined in terms of the ARE solution by (2.4-13) is often a satisfactory choice as a suboptimal and easy-to-implement control law. One way to find the positive definite solution S_∞ to the ARE is to select any S_N and iterate (2.4-2) until S_k converges. Our new results provide an alternative way to find S_∞ , which is important theoretically. If $(N - k) \rightarrow \infty$, then $M^{-(N-k)}$ goes

to zero since M^{-1} is stable. This means that $T_k \rightarrow 0$, so that in the steady-state limit, (2.4-41), yields

$$S_\infty = W_{21} W_{11}^{-1} \quad (2.4-42)$$

as an expression for the positive definite ARE solution. Thus, S_∞ can be manufactured from the *unstable* eigenvectors

$$\begin{bmatrix} W_{11} \\ W_{21} \end{bmatrix}$$

of H (or the *stable* eigenvectors of H^{-1}). Hence, the optimal steady-state feedback K_∞ can be found (by (2.4-13)) *without solving* a Riccati equation.

Example 2.4-4. Analytic Solution to a Scalar Riccati Equation

This example is from Vaughan (1970). Let the plant and cost function be

$$x_{k+1} = x_k + u_k, \quad (1)$$

$$J_i = \frac{10}{2} x_N^2 + \frac{1}{2} \sum_{k=1}^{N-1} (x_k^2 + u_k^2). \quad (2)$$

Then

$$H = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \quad (3)$$

has eigenvalues of 0.382, 2.618, so

$$M = 2.618. \quad (4)$$

The matrix of eigenvectors (the unstable one first!) is

$$W = \begin{bmatrix} 1.0 & 1.0 \\ 1.618 & -0.618 \end{bmatrix}. \quad (5)$$

Since $S_N = 10$,

$$T = -0.789 \quad (6)$$

and

$$T_k = -0.789(0.382)^{2(N-k)}. \quad (7)$$

Therefore, (2.4-41) yields

$$s_k = \frac{1.618 + 0.488(0.382)^{2(N-k)}}{1 - 0.789(0.382)^{2(N-k)}} \quad (8)$$

as the analytic solution to the Riccati equation (2.4-2). In the limiting case $(N - k) \rightarrow \infty$ this yields the ARE solution

$$s_\infty = 1.618, \quad (9)$$

so the steady-state feedback is given by (2.4-13) as

$$K_{\infty} = \frac{1.618}{2.618} = 0.618. \quad (10)$$

The control law

$$u_k = -0.618x_k \quad (11)$$

results in a stable closed-loop plant of

$$a_{\infty}^{\text{cl}} = (a - bK_{\infty}) = 0.382. \quad (12)$$

We point out now that the closed-loop pole of 0.382 is the stable eigenvalue of Hamiltonian matrix H . This is not a coincidence, and we shall have more to say about it in Section 2.5. ■

Analytic Solution to the Discrete Riccati Equation: System Matrix A Singular

In several digital control problems the system matrix A may either be singular or ill conditioned with respect to inversion. In that case the method described previously may either fail or produce misleading results due to the fact the A is ill conditioned with respect to inversion. This problem can be circumvented by employing the generalized eigenvalue problem for the analytic solution of the discrete Riccati equation presented by Pappas, Laub, and Sandell (1980).

In the previous section we defined the Hamiltonian matrix H as

$$H = \begin{bmatrix} A^{-1} & A^{-1}BR^{-1}B^T \\ QA^{-1} & A^T + QA^{-1}BR^{-1}B^T \end{bmatrix},$$

associated with the backward recursion (2.4-20). By noting that the Hamiltonian matrix H can be decomposed as the product of two matrices as

$$H = \begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix}^{-1} \begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix},$$

we can rewrite the backward recursion (2.4-20) as a forward generalized recursion

$$\begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix} \begin{bmatrix} x_{k+1} \\ \lambda_{k+1} \end{bmatrix} = \begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix} \begin{bmatrix} x_k \\ \lambda_k \end{bmatrix}. \quad (2.4-43)$$

The key idea of this method is to study the generalized eigenvalue problem

$$Gz = \mu Fz \quad (2.4-44)$$

where

$$F = \begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix} \quad \text{and} \quad G = \begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix}. \quad (2.4-45)$$

As before, we can extend the definition of a symplectic matrix H to the definition of a symplectic pair of matrices (F, G) . It is easy to show that (F, G) have the following property:

$$GJG^T = FJF^T = \begin{bmatrix} 0 & A \\ -A^T & 0 \end{bmatrix}, \quad (2.4-46)$$

where J is defined in (2.4-23). A pair that satisfies equation (2.4-46) is called a *symplectic* pair.

We now show that the solution to the discrete algebraic Riccati equation can be obtained from the eigenvectors of the Hamiltonian. Before we proceed, we show that the Hamiltonian has n stable and n unstable eigenvalues. Under the assumption of stabilizability of (A, B) and detectability of $(\sqrt{Q}, A)$, none of the eigenvalues of the Hamiltonian H given by (2.4-21), which corresponds to the discrete algebraic Riccati equation, lie on the unit circle. To see this, consider the generalized eigenvalue problem

$$Gz = \mu Fz$$

where

$$F = \begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix} \quad \text{and} \quad G = \begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix}.$$

Suppose that $|\mu| = 1$, then there exists some $z \neq 0$ such that

$$\begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \mu \begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix}.$$

This implies

$$Az_1 = \mu z_1 + \mu BR^{-1}B^T z_2 \quad \text{and} \quad -Qz_1 + z_2 = \mu A^T z_2.$$

By premultiplying the first equation by $\mu^* z_2^H$ and postmultiplying the conjugate transpose of the second equation by z_1 , one gets

$$\mu^* z_2^H A z_1 = |\mu|^2 z_2^H z_1 + |\mu|^2 z_2^H B R^{-1} B^T z_2, \quad \text{and} \quad z_2^H z_1 = \mu^* z_2^H A z_1 + z_1^H Q z_1.$$

By adding the above two equations and noting that $|\mu|^2 = 1$, we get

$$z_2^H B R^{-1} B^T z_2 + z_1^H Q z_1 = 0,$$

which, in turn, implies that

$$B^T z_2 = 0 \quad \text{and} \quad \sqrt{Q} z_1 = 0.$$

Note that under these conditions

$$A z_1 = \mu z_1 \quad \text{and} \quad A^T z_2 = (1/\mu) z_2,$$

which implies that the system is unstabilizable and undetectable, which is a contradiction; so there are no eigenvalues on the unit circle.

Further, it is straightforward to show that $\det(\mu F - G) \neq 0$. Note that if the determinant were identically zero, it would also be zero for $|\mu| = 1$, which contradicts the fact that the pair (F, G) has no eigenvalues on the unit circle. Next we show, as in the case of the Hamiltonian H , that if $\nu \neq 0$ is an eigenvalue of $(\mu F - G)$, then $1/\nu$ is also an eigenvalue. To show this, assume that y is a left eigenvector corresponding to eigenvalue ν . Then

$$\begin{aligned} y^H G &= \nu y^H F, \\ y^H G J G^T &= \nu y^H F J G^T, \\ y^H F J F^T &= \nu y^H F J G^T, \\ (1/\nu) x^T F^T &= x^H G^T, \\ Gx &= (1/\nu) Fx \end{aligned}$$

(where $x^H = y^H F J$), which shows that if $\nu \neq 0$ is an eigenvalue of $(\mu F - G)$, then $1/\nu$ is also an eigenvalue. What this means is that there exist matrices V and W such that the $2n$ eigenvalues can be arranged as (see Appendix A.5)

$$\begin{bmatrix} \mu I - M_0 & 0 & 0 & 0 \\ 0 & \mu I - M_f & 0 & 0 \\ 0 & 0 & \mu I - M_f^{-1} & 0 \\ 0 & 0 & 0 & \mu N_0 - I \end{bmatrix} = \begin{bmatrix} \mu N_s - M_s & 0 \\ 0 & \mu N_u - M_u \end{bmatrix}$$

where the n generalized eigenvalues of $(N_s = I, M_s)$ are stable, i.e., $|\mu_i| < 1$, and the eigenvalues of (N_u, M_u) are the reciprocals of the generalized eigenvalues of (N_s, M_s) , i.e., $|\mu_i| > 1$; that is, the n generalized eigenvalues of (N_u, M_u) are unstable. Note that in the case where A is singular some of the unstable eigenvalues lie at infinity.

Let V_s be the $2n \times n$ matrix corresponding to the stable eigenvalues, that is, the first n columns of the matrix V . Then

$$GV_s = FV_s M_s, \quad (2.4-47)$$

where M_s is in Jordan canonical form, corresponding to all stable eigenvalues. By substituting for F, G and conformably partitioning V_s , equation (2.4-47) can be rewritten as

$$\begin{bmatrix} A & 0 \\ -Q & I \end{bmatrix} \begin{bmatrix} V_{1s} \\ V_{2s} \end{bmatrix} = \begin{bmatrix} I & BR^{-1}B^T \\ 0 & A^T \end{bmatrix} \begin{bmatrix} V_{1s} M_s \\ V_{2s} M_s \end{bmatrix}$$

or

$$AV_{1s} = V_{1s} M_s + BR^{-1}B^T V_{2s} M_s \quad (2.4-48)$$

$$-QV_{1s} + V_{2s} = A^T V_{2s} M_s. \quad (2.4-49)$$

Since V_{1s} is nonsingular (see Appendix A), equation (2.4-48) can be rewritten as

$$A = V_{1s} M_s V_{1s}^{-1} + BR^{-1} B^T V_{2s} M_s V_{1s}^{-1} = (V_{1s} + BR^{-1} B^T V_{2s}) M_s V_{1s}^{-1} \quad (2.4-50)$$

and (2.4-49) can be written as

$$S = V_{2s} V_{1s}^{-1} = A^T V_{2s} M_s V_{1s}^{-1} + Q. \quad (2.4-51)$$

In the sequel it is shown that S described in (2.4-51) is a solution to the discrete ARE. Note that using the inversion lemma, the ARE can be written as

$$S = A^T S (I + BR^{-1} B^T S)^{-1} A + Q. \quad (2.4-52)$$

Substituting (2.4-51), the ARE (2.4-52) becomes

$$A^T V_{2s} M_s V_{1s}^{-1} = A^T V_{2s} V_{1s}^{-1} (I + BR^{-1} B^T V_{2s} V_{1s}^{-1})^{-1} A.$$

Substituting A from equation (2.4-50), the ARE becomes

$$A^T V_{2s} M_s = A^T V_{2s} (V_{1s} + BR^{-1} B^T V_{2s})^{-1} (V_{1s} M_s + BR^{-1} B^T V_{2s} M_s),$$

or

$$A^T V_{2s} M_s = A^T V_{2s} (V_{1s} + BR^{-1} B^T V_{2s})^{-1} (V_{1s} + BR^{-1} B^T V_{2s}) M_s,$$

which shows that (2.4-51) is a solution to the ARE.

It remains to show that S is the stabilizing solution to the ARE. That is, the closed-loop system matrix $(A - B(B^T S B + R)^{-1} B^T S A)$ has stable eigenvalues. The closed-loop matrix is

$$\begin{aligned} A - B(B^T S B + R)^{-1} B^T S A &= (I - B(B^T S B + R)^{-1} B^T S) A \\ &= (I + BR^{-1} B^T S)^{-1} A \\ &= (I + BR^{-1} B^T V_{2s} V_{1s}^{-1})^{-1} A \\ &= V_{1s} (V_{1s} + BR^{-1} B^T V_{2s})^{-1} A \\ &= V_{1s} M_s V_{1s}^{-1}, \end{aligned}$$

which shows that the closed-loop system matrix $(A - B(B^T S B + R)^{-1} B^T S A)$ and M_s have the same spectrum. Therefore, the closed-loop spectrum corresponds to the stable eigenvalues of the symplectic pair (F, G) .

The calculation of generalized eigenvectors has significant difficulties that may lead to inaccurate results; in particular, in the case of multiple eigenvalues. To avoid this problem, the solution to the ARE is computed by using the QZ algorithm and the generalized Schur form. For more details on efficient and stable

computations of solutions to algebraic Riccati equations see Laub (1979), Pappas, Laub, and Sandell (1980), and Bittanti, Laub, and Willems (1991).

Example 2.4-5. Discrete Riccati Equation: System Matrix A Singular

This example is from Åström and Wittenmark (1984). Let the plant be described by

$$\begin{aligned} x_{k+1} &= \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u_k \\ y_k &= \begin{bmatrix} 1 & 0 \end{bmatrix} x_k, \end{aligned}$$

and the cost function by

$$J = \frac{1}{2} \sum_{k=0}^{\infty} (y_k^T y_k + u_k^T u_k).$$

Then the matrices for the generalized eigenvalue problem are (see equation (2.4-45))

$$F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad G = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}.$$

The finite eigenvalues of (F, G) are the roots of the equation

$$\det(\mu F - G) = \mu(\mu^2 - 3\mu + 1) = 0,$$

which are $\mu_1 = 0$, $\mu_2 = 0.3820$, and $\mu_3 = 2.6180$. The fourth eigenvalue is an infinite one. The eigenvectors corresponding to the two stable eigenvalues are

$$V_s = \begin{bmatrix} V_{1s} \\ V_{2s} \end{bmatrix} = \begin{bmatrix} 0.4777 & -0.5774 \\ -0.2952 & 0.5774 \\ 0.7730 & -0.5774 \\ 0.2952 & 0.0000 \end{bmatrix}.$$

Then the solution to the ARE is

$$\begin{aligned} S &= V_{1s} V_{2s}^{-1} = \begin{bmatrix} 0.7730 & -0.5774 \\ 0.2952 & 0.0000 \end{bmatrix} \begin{bmatrix} 0.4777 & -0.5774 \\ -0.2952 & 0.5774 \end{bmatrix}^{-1} \\ &= \begin{bmatrix} 2.6180 & 1.6180 \\ 1.6180 & 1.6180 \end{bmatrix}. \end{aligned}$$

The feedback gain is

$$K = (B^T S B + I)^{-1} B^T S A = \begin{bmatrix} 0.6180 & 0.6180 \end{bmatrix}.$$

■

Design of Steady-state Regulators by Eigenstructure Assignment

The results of the previous subsection gave us an alternative design procedure for the optimal steady-state LQ regulator that did not involve solving a Riccati equation. It involved finding S_∞ from the unstable eigenvectors of the Hamiltonian matrix H in (2.4-21). Let us now discuss a method for finding the optimal steady-state gain K_∞ directly from the eigenstructure of the Hamiltonian system.

We assume here that (A, B) is stabilizable and $(A, \sqrt{Q})$ is detectable. First, write the Hamiltonian system (2.4-20) as the forward recursion

$$\begin{bmatrix} x_{k+1} \\ \lambda_{k+1} \end{bmatrix} = \begin{bmatrix} A + BR^{-1}B^T A^{-T}Q & -BR^{-1}B^T A^{-T} \\ -A^{-T}Q & A^{-T} \end{bmatrix} \begin{bmatrix} x_k \\ \lambda_k \end{bmatrix}, \quad (2.4-53)$$

where the coefficient matrix is H^{-1} in (2.4-26). Let μ be an eigenvector of H^{-1} . Then the eigenvectors of H^{-1} corresponding to μ are the eigenvectors of H corresponding to $1/\mu$ (Appendix A). Hence, W_{11} , W_{21} in (2.4-42) can alternatively be found by partitioning the *stable* eigenvectors of H^{-1} . In terms of the state vector only, the steady-state closed-loop system with the optimal control $u_k = -K_\infty x_k$ is

$$x_{k+1} = (A - BK_\infty)x_k. \quad (2.4-54)$$

Both (2.4-53) and (2.4-54) characterize the optimal closed-loop plant. We want to demonstrate that the eigenvalues of the optimal closed-loop system (2.4-54) are simply the stable eigenvalues of H^{-1} and that the n eigenvectors of (2.4-54) are given by the columns of W_{11} .

Suppose that μ_i is an eigenvalue of the optimal closed-loop system (Kailath 1980). Then, if only the mode corresponding to μ_i is excited, the state, control, and costate are described by

$$x_k = X_i \mu_i^k, \quad (2.4-55a)$$

$$u_k = U_i \mu_i^k, \quad (2.4-55b)$$

$$\lambda_k = \Lambda_i \mu_i^k, \quad (2.4-55c)$$

for some vectors X_i , U_i , Λ_i . But $x_{k+1} = Ax_k + Bu_k$, or

$$X_i \mu_i^{k+1} = AX_i \mu_i^k + BU_i \mu_i^k,$$

so that

$$(\mu_i I - A)X_i = BU_i. \quad (2.4-56)$$

The optimal control is $u_k = -K_\infty x_k$, so that

$$U_i = -K_\infty X_i \quad (2.4-57)$$

and

$$(\mu_i I - A + BK_\infty)X_i = 0. \quad (2.4-58)$$

Thus, X_i is an eigenvector of the closed-loop plant (2.4-54) for eigenvalue μ_i .

Now focus on the representation (2.4-53). Using (2.4-55a) and (2.4-55c) we have

$$\mu_i \begin{bmatrix} X_i \\ \Lambda_i \end{bmatrix} = H^{-1} \begin{bmatrix} X_i \\ \Lambda_i \end{bmatrix}. \quad (2.4-59)$$

Hence,

$$\begin{bmatrix} X_i \\ \Lambda_i \end{bmatrix}$$

is an eigenvector of H^{-1} for eigenvalue μ_i .

According to Theorem 2.4-2, $(A - BK_\infty)$ is stable, so that $|\mu_i| < 1$. Hence, the eigenvalues of (2.4-54) are the stable eigenvalues of H^{-1} and the eigenvectors of (2.4-54) are the top halves of the stable eigenvectors of H^{-1} .

To see why this is useful, observe that H^{-1} can be written down by inspection. Its stable eigenvalues and eigenvectors can be found, and these are the desired pole locations and associated eigenvectors of the optimal closed-loop plant.

If the plant is single input, we do not need to determine the eigenvectors of H^{-1} , since given only the desired closed-loop eigenvalues we can use *Ackermann's formula* to find the required feedback K_∞ . According to this formula, the state feedback K is required to assign a desired closed-loop characteristic polynomial $\Delta^d(s)$ is

$$K = e_n^T U_n^{-1} \Delta^d(A), \quad (2.4-60)$$

where e_n is the last column of the $n \times n$ identity matrix, U_n is the reachability matrix (2.4-17), and $\Delta^d(A)$ is the desired characteristic polynomial evaluated at A (Franklin and Powell 1980).

In the multivariable case, the desired closed-loop eigenvalues are not sufficient to determine the required feedback—the closed-loop eigenvectors are also required. In general, we can compute K_∞ from the eigenstructure of H^{-1} as follows.

Suppose the closed-loop eigenvalues are distinct. Then the optimal control is $u_k = -R^{-1}B^T \lambda_{k+1}$, so that

$$U_i = -R^{-1}B^T \mu_i \Lambda_i. \quad (2.4-61)$$

The optimal feedback satisfies (2.4-57), so that

$$K_\infty X_i = R^{-1}B^T \mu_i \Lambda_i. \quad (2.4-62)$$

Let X be a matrix whose columns are the X_i , and Λ be a matrix whose columns are the corresponding Λ_i (where

$$\begin{bmatrix} X_i \\ \Lambda_i \end{bmatrix}$$

is an eigenvector of the stable eigenvalue μ_i of H^{-1}). Let $M = \text{diag}[\mu_1, \dots, \mu_n]$. Then, evidently,

$$K_\infty = R^{-1} B^T \Lambda M X^{-1}. \quad (2.4-63)$$

Compare this with (2.4-42) ($W_{11} = X$, $W_{21} = \Lambda$).

If μ_i is complex, then so are X_i and Λ_i . In this event, there is a block in (2.4-63) of the form

$$[\Lambda_i \quad \lambda_i^*] \begin{bmatrix} \mu_i & 0 \\ 0 & \mu_i^* \end{bmatrix} [X_i \quad X_i^*]^{-1}.$$

By premultiplying and postmultiplying $\text{diag}[u_i, \mu_i^*]$ by

$$I = \frac{1}{2} \begin{bmatrix} 1 & -j \\ 1 & j \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ j & -j \end{bmatrix},$$

we see that this block can be replaced by

$$[\text{Re}(\Lambda_i) \quad \text{Im}(\Lambda_i)] \begin{bmatrix} \text{Re}(\mu_i) & \text{Im}(\mu_i) \\ -\text{Im}(\mu_i) & \text{Re}(\mu_i) \end{bmatrix} [\text{Re}(X_i) \quad \text{Im}(X_i)]^{-1},$$

which results in a real feedback gain K_∞ .

Note that $|X| \neq 0$ since the μ_i were assumed to be distinct. If this is not the case, then generalized eigenvectors must be used in manufacturing X .

It is worth remarking that if the state-weighting matrix Q is zero, then

$$H^{-1} = \begin{bmatrix} A & -BR^{-1}B^T A^{-T} \\ 0 & A^{-T} \end{bmatrix}. \quad (2.4-64)$$

In this case the eigenvalues of H^{-1} are the eigenvalues of A plus those of A^{-1} (i.e., those of A^{-T}). The optimal closed-loop poles are therefore found simply by taking the stable poles of A , and the reciprocals of the unstable poles of A , since this yields the set of stable eigenvalues of H^{-1} . These are also the optimal closed-loop poles in the case of infinite control weighting $R \rightarrow \infty$.

Example 2.4-6. Eigenstructure Design of Steady-state Regulator for Harmonic Oscillator

Suppose our plant is the harmonic oscillator

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\delta\omega_n \end{bmatrix} x + \begin{bmatrix} 0 \\ 10 \end{bmatrix} u \quad (1)$$

with natural frequency $\omega_n = \sqrt{2}$ and damping ratio $\delta = -1/\sqrt{2}$, so that

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -2 & 2 \end{bmatrix} x + \begin{bmatrix} 0 \\ 10 \end{bmatrix} u. \quad (2)$$

The plant is unstable with poles at $1 \pm j$. Discretizing with $T = 25$ msec yields (we give only three decimal places)

$$x_{k+1} = \begin{bmatrix} 0.999 & 0.026 \\ -0.051 & 1.051 \end{bmatrix} x_k + \begin{bmatrix} 0.003 \\ 0.256 \end{bmatrix} u_k$$

$$\triangleq Ax_k + Bu_k. \quad (3)$$

The open-loop poles are at

$$z = 1.025 \pm j0.026. \quad (4)$$

Let us associate with (3) the infinite-horizon performance index

$$J_0 = \frac{1}{2} \sum_{k=0}^{\infty} (qx_k^T x_k + u_k^2) \quad (5)$$

(i.e., $Q = qI$), where the infinite time interval $[0, \infty]$ means we are seeking the optimal steady-state control.

a. Locus of Optimal Closed-loop Poles versus q

Here we investigate the effect of q on the optimal closed-loop poles. To do this, we can look at the eigenstructure of the Hamiltonian matrix ($R = 1$)

$$H^{-1} = \begin{bmatrix} A + BB^T A^{-T} q & -BB^T A^{-T} \\ -A^{-T} q & A^{-T} \end{bmatrix}. \quad (6)$$

A program was written to plot the eigenvalues of H^{-1} as q varies. The resulting root-locus is shown in Fig. 2.4-6. The poles of the optimal closed-loop plant

$$x_{k+1} = (A - BK_{\infty})x_k \triangleq A_{\infty}^{\text{cl}} x_k \quad (7)$$

are given by the stable poles of H^{-1} . For $q = 0$, these are

$$z = 0.975 \pm j0.024, \quad (8)$$

which are simply the poles of the original plant reflected inside the unit circle (i.e., their reciprocals). As $q \rightarrow \infty$ the optimal closed-loop poles tend to

$$z = 0, 0.975. \quad (9)$$

We now discuss three ways to find the optimal steady-state feedback

$$u_k = -K_{\infty} x_k \quad (10)$$

for the case of $q = 0.07$.

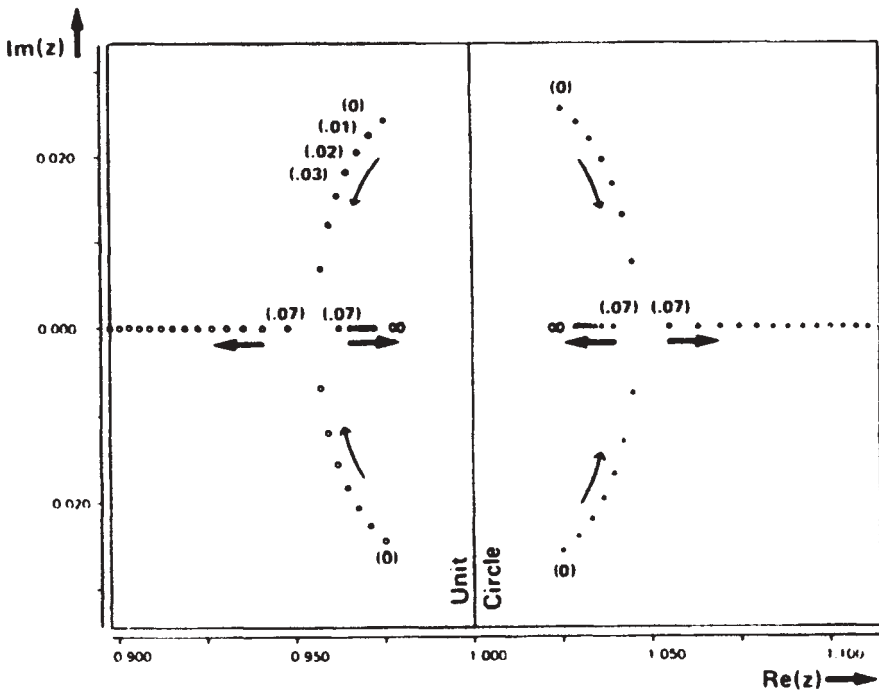


FIGURE 2.4-6 Locus of eigenvalues of H^{-1} as state weighting q varies. (Values of q are in parentheses. The increment in q is 0.01.)

b. Solution of the Algebraic Riccati Equation

To solve the ARE (2.4-12), we use a final condition of $S = I$ (any final condition will do since (A, B) is reachable and $(A, \sqrt{Q})$ is observable) in (2.4-2) and iterate until the solution converges. This occurs after 200 iterations (5 sec) and yields

$$S_{\infty} = \begin{bmatrix} 6.535 & 0.528 \\ 0.528 & 2.314 \end{bmatrix}. \quad (11)$$

We now use (2.4-13) to find the optimal steady-state feedback gain

$$K_{\infty} = [0.109 \quad 0.545]. \quad (12)$$

The resulting closed-loop plant is

$$A_{\infty}^{cl} = (A - BK_{\infty}) = \begin{bmatrix} 0.999 & 0.024 \\ -0.079 & 0.911 \end{bmatrix}, \quad (13)$$

which has stable poles of

$$z = 0.962, 0.948. \quad (14)$$

These correspond to $q = 0.07$ in Fig. 2.4-6.

c. Ackermann's Formula

We can avoid solving the ARE as follows. If $q = 0.07$, the optimal closed-loop poles are the stable eigenvalues of H^{-1} . These are found by the program that generated Fig. 2.4-6 to be 0.962, 0.948. Therefore, the desired closed-loop characteristic polynomial is

$$\Delta^{\text{cl}}(z) = (z - 0.962)(z - 0.948) = z^2 - 1.910z + 0.912. \quad (15)$$

The reachability matrix is

$$U_2 = [B \quad AB] = \begin{bmatrix} 0.003 & 0.010 \\ 0.256 & 0.269 \end{bmatrix}. \quad (16)$$

According to Ackermann's formula,

$$K_\infty = [0 \quad 1]U_2^{-1}\Delta^{\text{cl}}(A), \quad (17)$$

where $\Delta^{\text{cl}}(A) = A^2 - 1.910A + 0.912I$ is just a 2×2 real matrix. This again results in (12).

d. Eigenstructure Assignment

Another way to avoid solving the ARE is to use (2.4-63). The diagonal matrix of stable eigenvalues of H^{-1} for $q = 0.07$ is

$$M = \begin{bmatrix} 0.962 & 0 \\ 0 & 0.948 \end{bmatrix}, \quad (18)$$

and the associated eigenvectors are the columns in

$$\begin{bmatrix} 0.148 & 0.764 \\ -0.229 & -1.640 \\ 0.849 & 4.130 \\ -0.452 & -3.392 \end{bmatrix} \triangleq \begin{bmatrix} X \\ \Lambda \end{bmatrix}. \quad (19)$$

Using these matrices in (2.4-63) yields exactly (12).

We can also check our analytic ARE solution (2.4-42). If we compute

$$S_\infty = W_{21}W_{11}^{-1} = \Lambda X^{-1}, \quad (20)$$

we get exactly (11). ■

Time-varying Plant

If the original plant is time varying, we need to redefine observability and reachability to discuss the asymptotic LQ regulator. Let the plant and cost function be given by

$$x_{k+1} = A_k x_k + B_k u_k, \quad (2.4-65)$$

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q_k x_k + u_k^T R_k u_k). \quad (2.4-66)$$

Define the discrete state transition matrix as

$$\phi(k, i) = A_{k-1} A_{k-2} \cdots A_i \quad \text{for } k > i, \quad \text{with } \phi(k, k) = I. \quad (2.4-67)$$

Then we say the plant is *uniformly completely observable* if for every N the *observability gramian* satisfies

$$\alpha_0 I \leq \sum_{k=i}^{N-1} \phi^T(k, i) Q_k \phi(k, i) \leq \alpha_1 I \quad (2.4-68)$$

for some $I < N$, $\alpha_0 > 0$, and $\alpha_1 > 0$. This guarantees the positive definiteness of both the gramian and its inverse. Compare (2.4-68) with (2.2-24). We say the plant is *uniformly completely reachable* if for every i the *reachability gramian* satisfies

$$\alpha_0 I \leq \sum_{k=i}^{N-1} \phi(N, k+1) B_k R_k^{-1} B_k^T \phi^T(N, k+1) \leq \alpha_1 I \quad (2.4-69)$$

for some $N > i$, $\alpha_0 > 0$, and $\alpha_1 > 0$. Compare with the time-invariant reachability gramian (2.2-36).

If the plant is time varying, then there is, in general, no constant steady-state solution to the Riccati equation. However, uniform complete observability and reachability (and boundedness of A_k , B_k , Q_k , R_k) guarantee that for large $(N - k)$ the behavior of S_k is unique, independent of S_N . They also guarantee uniform asymptotic stability of the closed-loop plant $(A - BK_k)$. See Kalman and Bucy (1961) and Kalman (1963).

2.5 FREQUENCY-DOMAIN RESULTS

In the steady-state case with time-invariant plant, the optimal closed-loop system is also time invariant, and we can work in the frequency domain to derive two important results. One of these yields further insight on the fictitious output (2.4-15), and the other gives a frequency-domain approach to the design of optimal regulators that is similar to the classical root-locus technique.

A Factorization Result

The optimal steady-state LQ regulator is given by the constant feedback

$$u_k = -K x_k, \quad (2.5-1)$$

where

$$K = (B^T S_\infty B + R)^{-1} B^T S_\infty A. \quad (2.5-2)$$

(For notational convenience, we write K instead of K_∞ .) S_∞ is the unique positive definite solution to the ARE

$$S = A^T[S - SB(B^T SB + R)^{-1} B^T S]A + Q \quad (2.5-3)$$

(we assume (A, B) is stabilizable and $(A, \sqrt{Q})$ is observable). The resulting time-invariant closed-loop system

$$x_{k+1} = (A - BK)x_k \quad (2.5-4)$$

is asymptotically stable.

To derive a relation between the open-loop characteristic polynomial $\Delta(z) = |zI - A|$ and the optimal closed-loop characteristic polynomial $\Delta^{cl}(z)$, note that (Appendix A)

$$\begin{aligned} \Delta^{cl}(z) &= |zI - A + BK| \\ &= |I + BK(zI - A)^{-1}| \cdot |zI - A| \\ &= |I + K(zI - A)^{-1}B| \cdot \Delta(z). \end{aligned} \quad (2.5-5)$$

This identity will be useful shortly. According to Fig. 2.5-1, $-K(zI - A)^{-1}B$ can be interpreted as a loop gain matrix, so that $I + K(zI - A)^{-1}B$ is a return difference matrix.

To derive the result on which this section is based, note that

$$S - A^T S A = (z^{-1}I - A)^T S (zI - A) + (z^{-1}I - A)^T S A + A^T S (zI - A). \quad (2.5-6)$$

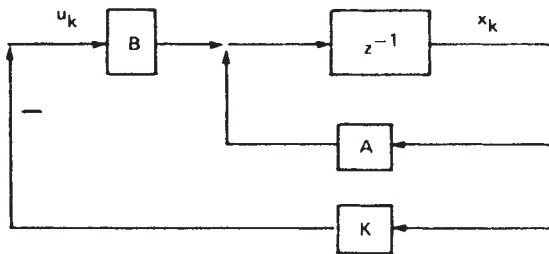


FIGURE 2.5-1 Optimal closed-loop system with control drawn as a state feedback.

Use the ARE to write

$$\begin{aligned} & (z^{-1}I - A)^T S(zI - A) + (z^{-1}I - A)^T SA + A^T S(zI - A) \\ & + A^T SB(B^T SB + R)^{-1} B^T SA = Q. \end{aligned}$$

Premultiply this by $B^T(z^{-1}I - A)^{-T}$ and postmultiply it by $(zI - A)^{-1}B$ to get

$$\begin{aligned} & B^T SB + B^T SA(zI - A)^{-1}B + B^T(z^{-1}I - A)^{-T} A^T SB \\ & + B^T(z^{-1}I - A)^{-T} A^T SB(B^T SB + R)^{-1} B^T SA(zI - A)^{-1}B \\ & = B^T(z^{-1}I - A)^{-T} Q(zI - A)^{-1}B. \end{aligned}$$

Substitute from (2.5-2) to obtain

$$\begin{aligned} & B^T SB + (B^T SB + R)K(zI - A)^{-1}B + B^T(z^{-1}I - A)^{-T} K^T (B^T SB + R) \\ & + B^T(z^{-1}I - A)^{-T} K^{-T} (B^T SB + R)K(zI - A)^{-1}B \\ & = B^T(z^{-1}I - A)^{-T} Q(zI - A)^{-1}B. \end{aligned}$$

Finally, add R to both sides and factor to see that

$$\begin{aligned} & B^T(z^{-1}I - A)^{-T} Q(zI - A)^{-1}B + R \\ & = [I + K(z^{-1}I - A)^{-1}B]^T (B^T SB + R) [I + K(zI - A)^{-1}B]. \end{aligned} \quad (2.5-7)$$

Let us briefly discuss this result to try to get a feel for it. From (2.4-6) and (2.4-15) we can write the transfer-function relation in the plant from the control input to the fictitious output as

$$Y(z) = \begin{bmatrix} C(zI - A)^{-1}B \\ D \end{bmatrix} U(z), \quad (2.5-8)$$

where $Q = C^T C$ and $R = D^T D$. But

$$\begin{bmatrix} C(z^{-1}I - A)^{-1}B \\ D \end{bmatrix}^T \begin{bmatrix} C(zI - A)^{-1}B \\ D \end{bmatrix} = B^T(z^{-1}I - A)^{-T} Q(zI - A)^{-1}B + R; \quad (2.5-9)$$

evidently, then, (2.5-7) shows that we can factor this transfer-function product in terms of the return difference matrix.

Another point of view is obtained from Figs. 2.5-1 and 2.5-2. Define

$$H(z) = C(zI - A)^{-1}B \quad (2.5-10)$$

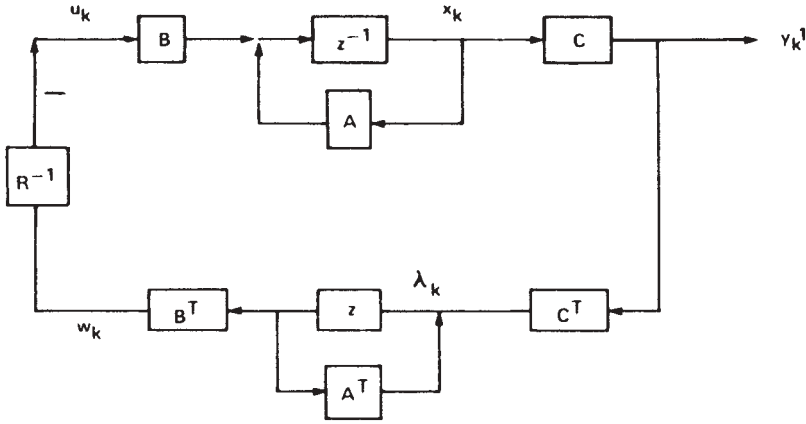


FIGURE 2.5-2 Optimal closed-loop system with control drawn as a costate feedback.

as the transfer function in the original state system from the control input to y_k^1 , which is the partition of the fictitious output corresponding to Cx_k (see (2.4-15)). Then it is evident that

$$H^T(z^{-1}) = B^T(z^{-1}I - A)^{-T}C^T = B^T(z^{-1}I - A^T)^{-1}C^T \quad (2.5-11)$$

is the transfer function of the *costate* system from y_k^1 to the intermediate signal w_k . Therefore, (2.5-7) simply expresses an equivalence between a transfer-function product in Fig. 2.5-2 and a transfer-function (i.e., return difference) product in Fig. 2.5-1. It is simply another way of expressing the equivalence of the two formulations of the LQ regulator (2.2-9) and Table 2.2-1.

We can use (2.5-5) and (2.5-7) to derive a frequency-domain method for designing steady-state regulators, as we now show.

Chang–Letov Design Procedure for the Steady-state LQ Regulator

By using the characteristic polynomial relation (2.5-5) and the factorization result (2.5-7) we can write the *Chang–Letov equation*

$$\Delta^{\text{cl}}(z^{-1})\Delta^{\text{cl}}(z) = |H^T(z^{-1})H(z) + R| \cdot \Delta(z^{-1})\Delta(z) \cdot |B^TSB + R|^{-1}, \quad (2.5-12)$$

where $H(z)$ is defined in (2.5-10) ($Q = C^TC$) (Chang 1961, Letov 1960). This is a very useful result, since it provides an alternative frequency-domain method for steady-state regulator design that is very similar to the classical root-locus technique.

To see this, note that $H(z)$ and the open-loop characteristic equation $\Delta(z)$ can be computed immediately, given the plant and performance index. The term

$|B^T S B + R|$ depends on the as yet unknown ARE solution S (i.e., S_∞), but it is irrelevant since it only provides a normalizing constant. Thus, the entire right-hand side of the Chang-Letov equation is known to within a multiplicative constant.

The term $\Delta^{\text{cl}}(z^{-1})\Delta^{\text{cl}}(z)$ on the left-hand side is a polynomial with quite interesting properties. Its roots are the roots z_i of $\Delta^{\text{cl}}(z)$ and their reciprocals $1/z_i$ (which are the roots of $\Delta^{\text{cl}}(z^{-1})$). The closed-loop plant (2.5-4) is stable, so that the optimal closed-loop poles can be determined by selecting the stable roots of the right-hand polynomial in (2.5-12)!

The importance of the Chang-Letov equation is now clear. It allows us to determine directly from A , B , Q , and R , which are all known, the optimal closed-loop poles. Then, in the single-input case, Ackermann's formula or an equivalent pole-placement technique can be used to determine the required optimal feedback K . (In the multivariable case we need to know the optimal closed-loop eigenvectors also before K can be uniquely determined. Chang-Letov cannot tell us these, but the techniques of Section 2.4 can be used.)

In the single-input case with $Q = qI$, we have

$$H(z) = \sqrt{q} \frac{[\text{adj}(zI - A)]B}{\Delta(z)} \triangleq \sqrt{q} \frac{N(z)}{\Delta(z)}, \quad (2.5-13)$$

where $N(z)$ is a column vector. Then (2.5-12) becomes

$$\Delta^{\text{cl}}(z^{-1})\Delta^{\text{cl}}(z) = \frac{(q/r)N^T(z^{-1})N(z) + \Delta(z^{-1})\Delta(z)}{1 + B^T S B / r}. \quad (2.5-14)$$

The roots of the right-hand side are the zeros of

$$1 + \frac{q}{r} \frac{N^T(z^{-1})N(z)}{\Delta(z^{-1})\Delta(z)}, \quad (2.5-15)$$

which is in exactly the form required for root-locus analysis. That entire body of theory therefore applies here. Evidently, then, as q/r varies from 0 (no state weighting) to ∞ (no control weighting), the optimal closed-loop poles move from the stable poles of

$$G(z) \triangleq H^T(z^{-1})H(z) \quad (2.5-16)$$

to its stable zeros. The ratio of cost weights q/r can therefore be selected to yield suitable closed-loop poles.

The next example illustrates these ideas. Good references are Schultz and Melsa (1967) and Kailath (1980).

Example 2.5-1. Root-locus Design of Steady-state Regulator for Harmonic Oscillator

To compare the Chang-Letov method with the steady-state regulator design methods in Section 2.4, let us reconsider Example 2.4-6. The only thing we need to begin our design

is the “fictitious” transfer function

$$H(z) = \sqrt{Q}(zI - A)^{-1}B \quad (1)$$

Using A and B from Example 2.4-6, and $Q = qI$ and $R = 1$ as in that example, this is found to be (we show only three decimal places)

$$H(z) \triangleq \sqrt{q} \frac{N(z)}{\Delta(z)} = \sqrt{q} \frac{\begin{bmatrix} 0.003z + 0.003 \\ 0.256z - 0.256 \end{bmatrix}}{z^2 - 2.050z + 1.051}. \quad (2)$$

The design is based on the rational function

$$G(z) = \frac{N^T(z^{-1})N(z)}{\Delta(z^{-1})\Delta(z)}, \quad (3)$$

since the zeros of

$$1 + \frac{q}{r}G(z) \quad (4)$$

are the roots of $\Delta^{\text{cl}}(z^{-1})\Delta^{\text{cl}}(z)$. This function is

$$\begin{aligned} G(z) &= \frac{\begin{bmatrix} 0.003z^{-1} + 0.003 \\ 0.256z^{-1} - 0.256 \end{bmatrix}^T \begin{bmatrix} 0.003z + 0.003 \\ 0.256z - 0.256 \end{bmatrix}}{(z^{-2} - 2.050z^{-1} + 1.051)(z^2 - 2.050z + 1.051)} \quad (5) \\ &= \frac{-0.066z^3 + 0.131z^2 - 0.066z}{1.051z^4 - 4.205z^3 + 6.308z^2 - 4.205z + 1.051}. \end{aligned}$$

Note the symmetric form of the coefficients of the numerator and denominator; this means that if z is a root of either of these polynomials, then z^{-1} is also.

$G(z)$ can be factored as

$$G(z) = \frac{-0.063z(z - 0.975)(z - 1.025)}{[(z - 0.975)^2 + 0.024^2][(z - 1.025)^2 + 0.026^2]}. \quad (6)$$

If we draw the root locus of (4) as q varies from 0 to ∞ , we obtain exactly Fig. 2.4-6! Thus, the eigenvalues of the Hamiltonian matrix H^{-1} in (2.4-43) are exactly the roots of $(q/r)N^T(z^{-1})N(z) + \Delta(z^{-1})\Delta(z)$.

According to the Chang-Letov equation, the optimal poles of the closed-loop plant $(A - BK)$ with LQ regulator are the stable zeros of (4) for any given q and r . When $q = 0$, they are the original plant *poles* reflected inside the unit circle (i.e., unstable plant poles z_i are replaced by z_i^{-1}), and when $q \rightarrow \infty$, they are the *zeros* of $H(z)$ reflected inside the unit circle.

Suppose we examine the root locus and decide that the closed-loop poles corresponding to $q = 0.07$ are satisfactory for our purposes. Then we can use Ackermann's formula to find the required optimal feedback K as in part c of Example 2.4-6. ■

PROBLEMS

Section 2.1

2.1-1. Optimal control of a bilinear system. Let the scalar plant

$$x_{k+1} = x_k u_k + 1 \quad (1)$$

have performance index

$$J = \frac{1}{2} \sum_{k=0}^{N-1} u_k^2, \quad (2)$$

with final time $N = 2$. Given x_0 , it is desired to make $x_2 = 0$.

- Write state and costate equations with u_k eliminated.
- Assume the final costate λ_2 is known. Solve for λ_0, λ_1 in terms of λ_2 and the state. Use this to express x_2 in terms of λ_2 and x_0 . Hence, find a quartic equation for λ_2 in terms of initial state x_0 .
- If $x_0 = 1$, find the optimal state and costate sequences, the optimal control, and the optimal value of the performance index.

2.1-2. Optimal control of a bilinear system. Consider the bilinear system

$$x_{k+1} = Ax_k + Dx_k u_k + bu_k, \quad (1)$$

where $x_k \in R^n$, $u_k \in R$, with quadratic performance index

$$J = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=0}^{N-1} (x_k^T Q x_k + r u_k^2), \quad (2)$$

where $S_N \geq 0$, $Q \geq 0$, $r > 0$. Show that the optimal control is the bilinear state–costate feedback,

$$u_k = -(b + Dx_k)^T \lambda_{k+1} / r, \quad (3)$$

and that the state and costate equations after eliminating u_k are

$$x_{k+1} = Ax_k - (b + Dx_k)(b + Dx_k)^T \lambda_{k+1} / r, \quad (4)$$

$$\lambda_k = Qx_k + A^T \lambda_{k+1} - (b + Dx_k)^T \lambda_{k+1} D^T \lambda_{k+1} / r. \quad (5)$$

2.1-3. Optimal control of a generalized state-space system. Rederive the equations in Table 2.1-1 to find the optimal controller for the nonlinear *generalized state-space* (or *descriptor*) system

$$Ex_{k+1} = f^k(x_k, u_k), \quad (6)$$

where E is singular. These systems often arise in circuit analysis, economics, and similar areas.

Section 2.2

2.2-1. Writing the Lyapunov equation as a vector equation. Show that the Lyapunov equation (2.2-21) can be written as the vector equation

$$s(S_k) = (A_k^T \otimes A_k^T)s(S_{k+1}) + s(Q_k), \quad (1)$$

where the Kronecker product and stacking operator are defined in Appendix A.

2.2-2. Solutions to the algebraic Lyapunov equation.

a. Find all possible solutions to (2.2-26) if

$$A = \begin{bmatrix} \frac{1}{2} & 1 \\ 0 & -\frac{1}{2} \end{bmatrix}, \quad C = [2 \quad 0], \quad Q = C^T C.$$

(Hint: Let

$$P = \begin{bmatrix} p_1 & p_2 \\ p_3 & p_4 \end{bmatrix},$$

substitute into (2.2-26), and solve for the scalars p_i . Alternatively, the results of Problem 2.2-1 can be used.)

b. Now find the symmetric solutions.

2.2-3. Prove that (2.2-57) and the Joseph-stabilized Riccati equation (2.2-62) are equivalent to (2.2-53).

2.2-4. Control of a scalar system. Let $x_{k+1} = 2x_k + u_k$.

- Find the homogeneous solution x_k for $k = 0, 5$ if $x_0 = 3$.
- Find the minimum-energy control sequence u_k required to drive $x_0 = 3$ to $x_5 = 0$. Check your answer by finding the resulting state trajectory.
- Find the optimal feedback gain sequence K_k to minimize the performance index

$$J_0 = 5x_5^2 + \frac{1}{2} \sum_{k=0}^4 (x_k^2 + u_k^2).$$

Find the resulting state trajectory and the costs to go J_k^* for $k = 0, 5$.

2.2-5. Comparison of different discrete controllers

$$x_{k+1} = \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} \\ 3 & -1 \end{bmatrix} x_k + \begin{bmatrix} 2 & 1 \\ 2 & 0 \end{bmatrix} u_k, \quad x_0 = \begin{bmatrix} 8 \\ 4 \end{bmatrix}.$$

- Find the open-loop control u_0, u_1 to drive the initial state to $x_2 = 0$ while minimizing the cost

$$J_a = \frac{1}{2} \sum_{k=0}^1 u_k^T \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} u_k.$$

Check your answer by “simulation” (i.e., apply your u_0, u_1 to the plant to verify that $x_2 = 0$).

- b. Find a constant state-variable feedback to input component one of the form

$$u_k^1 = -Kx_k,$$

where $u_k = [u_k^1 \ u_k^2]^T$, to yield a deadbeat control (all closed-loop poles at the origin). Find the closed-loop state trajectory.

- c. Let

$$J_c = 10x_2^T x_2 + J_a,$$

with J_a as in part a. Solve the Riccati equation to determine the optimal control u_0, u_1 . Find the optimal cost.

- d. Compare the state trajectories of parts a, b, and c.
e. Now suppose $x_0 = [1 \ 2]^T$. How must the controls of parts a, b, and c be modified?

2.2-6. Linear performance index. Let

$$x_{k+1} = Ax_k + Bu_k,$$

$$J = S_N x_N + \sum_{K=0}^{N-1} (Qx_k + Ru_k),$$

with J a scalar. Write state and costate equations and stationarity condition. What is the problem? (We shall learn how to deal with linear cost indices in Section 5.2.)

2.2-7. Cubic performance index. Let

$$x_{k+1} = ax_k + bu_k,$$

where x_k and u_k are scalars, and

$$J = \frac{1}{3}s_N x_N^3 + \frac{1}{3} \sum_{k=0}^{N-1} (qx_k^3 + ru_k^3).$$

- a. Write state and costate equations and stationarity condition.
b. When can we solve for u_k ? Under this condition, eliminate u_k from the state equation.
c. Solve the open-loop control problem (i.e., x_N fixed, $s_N = 0$, $q = 0$).

2.2-8. Optimal control with weighting of state-input inner product

- a. Redo Problem 2.2-4c if the cost index is

$$J = 5x_5^2 + \frac{1}{2} \sum_{k=0}^4 (x_k^2 + u_k^2 + 2x_k u_k).$$

- b. Redo Problem 2.2-5c if the term $2x_k^T u_k$ is added to J_a .

2.2-9. Information formulation of the Riccati equation. If S_N is very large, then it is convenient to use the *information formulation* of the discrete Riccati equation, which propagates S_k^{-1} instead of S_k . (The name derives from the filtering application of the Riccati equation.)

Separate the Riccati equation into two parts by defining the intermediate matrix

$$\bar{S}_{k+1} \triangleq S_{k+1} - S_{k+1} B (B^T S_{k+1} B + R)^{-1} B^T S_{k+1}. \quad (1)$$

Then

$$S_k = A^T \bar{S}_{k+1} A + Q. \quad (2)$$

The first of these equations incorporates the effect of the control input on the performance index J_k ; note that $\bar{S}_{k+1} \leq S_{k+1}$. The second equation shows the effects of the plant dynamics and state weighting Q on J_k ; these effects generally make S_k larger than $\bar{S}_{k+1}$.

a. Show that *input update* (1) can be written

$$\bar{S}_{k+1}^{-1} = S_{k+1}^{-1} + B R^{-1} B^T. \quad (3)$$

b. Assume $|A| \neq 0$, $Q > 0$, and define

$$F_{k+1} = A^{-1} \bar{S}_k^{-1} A^{-T}. \quad (4)$$

Show that the *state update* (2) can be written

$$S_k^{-1} = F_{k+1} - F_{k+1} (F_{k+1} + Q^{-1})^{-1} F_{k+1}. \quad (5)$$

If the information Kalman gain is defined as

$$G_k = (F_{k+1} + Q^{-1})^{-1} F_{k+1}, \quad (6)$$

this becomes

$$S_k^{-1} = F_{k+1} (I - G_k). \quad (7)$$

The information update is given by (3), (4), (6), and (7).

2.2-10. Square-root Riccati formulations. Split the Riccati equation into two parts as in Problem 2.2-9 and define the roots $P_k, \bar{P}_k$ by $S_k = P_k^T P_k$, $\bar{S}_k = \bar{P}_k^T \bar{P}_k$. The following results show how to propagate $\bar{P}_k, P_k$ instead of $\bar{S}_k, S_k$ and yield numerically stable algorithms. See Schmidt (1967, 1970), Businger and Golub (1965), Dyer and McReynolds (1969), Bierman (1977), Morf and Kailath (1975), and Kaminski et al. (1971).

a. Show that the state update is equivalent to

$$T_1 \begin{bmatrix} \bar{P}_{k+1} A \\ C \end{bmatrix} = \begin{bmatrix} P_k \\ 0 \end{bmatrix}, \quad (1)$$

where $Q = C^T C$, and T_1 is any orthogonal transformation (i.e., $T_1^T T_1 = I$) selected so that P_k has n rows.

b. Show that the input update is equivalent to

$$T_2 \begin{bmatrix} D & 0 \\ P_{k+1} B & P_{k+1} \end{bmatrix} = \begin{bmatrix} (B^T S_{k+1} B + R)^{1/2} & \bar{K}_{k+1} \\ 0 & \bar{P}_{k+1} \end{bmatrix}, \quad (2)$$

where $R = D^T D$, and T_2 is any orthogonal matrix such that $\bar{P}_{k+1}$ has n rows.

c. Show that both updates can be expressed together in square-root form as

$$T_3 \begin{bmatrix} D & 0 \\ P_{k+1} B & P_{k+1} A \\ 0 & C \end{bmatrix} = \begin{bmatrix} (B^T S_{k+1} B + R)^{1/2} & \bar{K}_{k+1} A \\ 0 & P_k \\ 0 & 0 \end{bmatrix}, \quad (3)$$

where T_3 is orthogonal.

Section 2.3

2.3-1. Digital control of harmonic oscillator. A harmonic oscillator is described by

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\omega_n^2 x_1 + u. \end{aligned} \quad (1)$$

- Discretize the plant using a sampling period of T .
- With the discretized plant, associate a performance index of

$$J = \frac{1}{2} [s_1 (x_N^1)^2 + s_2 (x_N^2)^2] + \frac{1}{2} \sum_{k=0}^{N-1} [q_1 (x_k^1)^2 + q_2 (x_k^2)^2 + r u_k^2], \quad (2)$$

where the state is $x_k = [x_k^1 \ x_k^2]^T$. Write scalar equations for a digital optimal controller.

- Write a MATLAB subroutine to simulate the plant dynamics, and use the time response program *lsim.m* to obtain zero-input state trajectories.
- Write a MATLAB subroutine to compute and store the optimal control gains and to update the control u_k given the current state x_k . Write a MATLAB driver program to obtain time response plots for the optimal controller.

2.3-2. Digital control of an unstable system. Repeat the previous problem for

$$\begin{aligned} \dot{x}_1 &= x_2, \\ \dot{x}_2 &= a^2 x_1 + b u. \end{aligned} \quad (3)$$

2.3-3. Digital controller with weighting of state–input inner product. Modify your controller in Problem 2.3-1 to include terms like

$$v_1 x_k^1 u_k + v_2 x_k^2 u_k$$

in the performance index, where v_1 and v_2 are scalar weightings.

2.3-4. General digital control subroutine. For a general time-invariant plant and performance index as in Table 2.2-1, use MATLAB to write m-files to compute and store the optimal feedback gains K_k .

Section 2.4

2.4-1. Steady-state behavior. In this problem we consider a rather unrealistic discrete system because it is simple enough to allow an analytic treatment. Thus, let the plant

$$x_{k+1} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x_k + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u_k \quad (1)$$

have performance index of

$$J_0 = \frac{1}{2} x_N^T x_N + \frac{1}{2} \sum_{k=0}^{N-1} \left(x_k^T \begin{bmatrix} q_1 & q_2 \\ q_2 & q_1 \end{bmatrix} x_k + r u_k^2 \right). \quad (2)$$

- Find the optimal steady-state (i.e., $N \rightarrow \infty$) Riccati solution S_∞^* and show that it is positive definite. Find the optimal steady-state gain K_∞^* and determine when it is nonzero.
- Find the optimal steady-state closed-loop plant and demonstrate its stability.
- Now the suboptimal constant feedback

$$u_k = -K_\infty^* x_k \quad (3)$$

is applied to the plant. Find scalar updates for the components of the suboptimal cost kernel S_k . Find the suboptimal steady-state cost kernel S_∞ and demonstrate that $S_\infty = S_\infty^*$.

2.4-2. Analytic Riccati solution. Let

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad S_N = I, \quad Q = I.$$

- a. Let $r = 0.1$. Find the Hamiltonian matrix H and its eigenvalues and eigenvectors. Find the analytic expression for Riccati solution S_k . Find the steady-state solution S_∞ using (2.4-42). Find the optimal steady-state gain K_∞ using (2.4-63) and also using Ackermann's formula.
- b. Let $r = 1$. Find the Hamiltonian matrix and its eigenstructure. Find the steady-state solution S_∞ and gain K_∞ . (*Hint*: See the discussion following (2.4-63).)

2.4-3. Software for plotting optimal closed-loop poles

- a. For the LQ case, write a computer program to
 1. Compute the Hamiltonian matrix H .
 2. Find eigenstructure of H . You can use MATLAB routine *eig.m*.
 3. Compute the steady-state Riccati solution S_∞ .
 4. Compute the steady-state gain K_∞ and closed-loop system A^{cl} .
- b. Modify your program to find the optimal steady-state closed-loop poles as a function of r , the control weighting. (Note that you only need to do 1 and 2 for this.) You now have a design tool to select the cost-weighting matrices $Q = qI$ and $R = rI$ to yield desired closed-loop performance at steady state.

Section 2.5

2.5-1. Chang–Letov design.

- a. For the system in Problem 2.4-2, use the Chang–Letov procedure and Ackermann's formula to find the optimal steady-state feedback gain K_∞ and closed-loop plant if $r = 0.1$.
- b. Plot a root locus of the closed-loop poles as r varies from ∞ to 0.

2.5-2. Reciprocal polynomials. Let $\phi(z) = z^2 + 2\alpha z + \omega^2$. Find and sketch the roots z_1, z_2 of $\phi(z)$. Define the *reciprocal polynomial* as $z^2\phi(z^{-1}) = \omega^2 z^2 + 2\alpha z + 1$. Show that the roots of $z^2\phi(z^{-1})$ are the roots of $\phi(z)$ reflected about the unit circle. That is, they are equal to $1/z_1$ and $1/z_2$.

2.5-3. Decomposition of polynomials using a reciprocal polynomial.

- a. Show that any polynomial $P(z)$ of degree n with real coefficients can be decomposed as

$$P(z) = P_1(z) + P_2(z), \quad (1)$$

where the *mirror-image* and *anti-mirror-image* polynomials are defined by

$$P_1(z) = \frac{1}{2}(P(z) + z^n P(z^{-1})), \quad (2)$$

$$P_2(z) = \frac{1}{2}(P(z) - z^n P(z^{-1})). \quad (3)$$

b. Show that

$$P_1(z) = z^n P_1(z^{-1}), \quad (4)$$

$$P_2(z) = -z^n P_2(z^{-1}). \quad (5)$$

(Compare this with the decomposition of a real matrix into symmetric and antisymmetric matrices. Hence, note that the reciprocal polynomial $z^n P(z^{-1})$ plays a role similar to that of the transpose of a matrix.)

3

OPTIMAL CONTROL OF CONTINUOUS-TIME SYSTEMS

We shall now discuss optimal control for systems with a continuous time index. From a glance at the table of contents, it is apparent that this chapter will follow the development of Chapter 2.

There are several distinctions between the optimal control problems for continuous and discrete systems, the most noticeable of which is that the continuous control laws are based on equations of a simpler form than their discrete counterparts. That will allow us to obtain some analytic solutions in this chapter. Another distinction arises in the initial stages of the derivation of the control law. For continuous systems, we must distinguish between *differentials* and *variations* in a quantity, which we did not need to do in Chapter 2. This means that we shall need to use the calculus of variations, which is briefly reviewed in Section 3.1.

The continuous dependence on time also makes it fairly simple to talk about minimum-time problems, which we do in Chapter 5. The derivations in this chapter are for the most part similar to those for discrete systems, and we shall attempt to set them down in a manner that makes clear what is going on without duplicating too much of our work from Chapter 2.

3.1 THE CALCULUS OF VARIATIONS

Only a few ideas from the calculus of variations will be needed, so our review will be short. For an in-depth discussion, see Athans and Falb (1966) or Kirk (1970). In Section 3.2 we shall be concerned with minimizing an augmented performance index J' exactly as we were in Chapter 2. To perform this minimization, we need

to find the change induced in J' by independent changes in all of its arguments (cf. (2.1-6)). Unfortunately, we shall run into a slight problem. The change in J' will depend on the time and state differentials dt and dx . However, these quantities are not independent. The purpose of this section is to clear up this point and to derive a relation that will soon be useful.

If $x(t)$ is a continuous function of time t , then the differentials $dx(t)$ and dt are not independent. We can, however, define a small change in $x(t)$ that is independent of dt . Let us define the variation in $x(t)$, $\delta x(t)$, as the incremental change in $x(t)$ when time t is held fixed.

To find the relations among dx , δx , and dt , examine Fig. 3.1-1. Here we show the original function $x(t)$ and a neighboring function $x(t) + dx(t)$ over an interval specified by initial time t_0 and final time T (Bryson and Ho 1975). In addition to the increment $dx(t)$ at each time t , the final time has been incremented by dT . It is clear from the illustration that the overall increment in x at T , $dx(T)$, depends on dT . According to our definition, the variation $\delta x(T)$ occurs at the fixed value of $t = T$, as shown and is independent of dT . Since $x(t)$ and $x(t) + dx(t)$ have approximately the same slope $\dot{x}(T)$ at $t = T$, and since dT is small, we have

$$dx(T) = \delta x(T) + \dot{x}(T) dT. \quad (3.1-1)$$

This relation is the one we shall need later.

Another relation we shall need is *Leibniz's rule* for functionals: if $x(t) \in R^n$ is a function of t and

$$J(x) = \int_{t_0}^T h(x(t), t) dt, \quad (3.1-2)$$

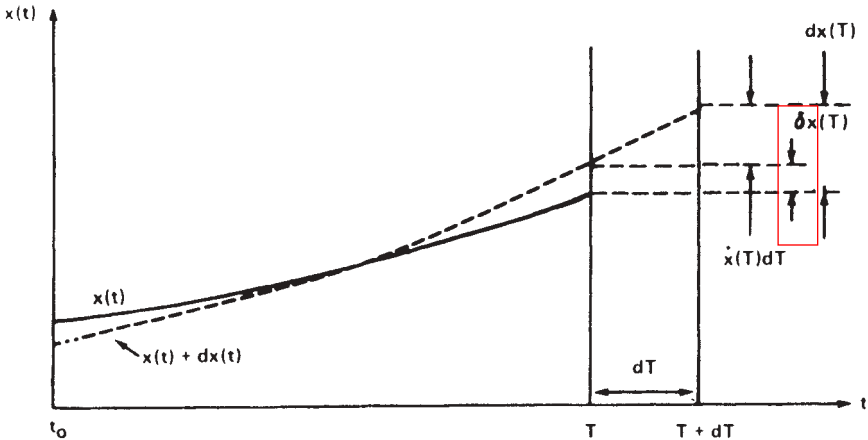


FIGURE 3.1-1 Relation between the variation δx and the differential dx .

where $J(\cdot)$ and $h(\cdot)$ are both real scalar functionals (i.e., functions of the function $x(t)$), then

$$dJ = h(x(T), T) dT - h(x(t_0), t_0) dt_0 + \int_{t_0}^T [h_x^T(x(t), t) \delta x] dt. \quad (3.1-3)$$

Our notation is

$$h_x \triangleq \frac{\partial h}{\partial x}.$$

3.2 SOLUTION OF THE GENERAL CONTINUOUS-TIME OPTIMIZATION PROBLEM

The philosophy in this chapter is to derive the solution to the continuous optimal control problem in the most general case. This is accomplished in the present section. Then, in subsequent sections, we consider various special cases of the general solution. The discussion at the beginning of Chapter 2 and the comments in Section 2.1 also apply here; they provide some insight on the formulation of the optimal control problem.

Problem Formulation and Solution

Suppose the plant is described by the nonlinear time-varying dynamical equation

$$\dot{x}(t) = f(x, u, t), \quad (3.2-1)$$

with state $x(t) \in R^n$ and control input $u(t) \in R^m$. With this system let us associate the performance index

$$J(t_0) = \phi(x(T), T) + \int_{t_0}^T L(x(t), u(t), t) dt, \quad (3.2-2)$$

where $[t_0, T]$ is the time interval of interest. The final weighting function $\phi(x(T), T)$ depends on the final state and final time, and the weighting function $L(x, u, t)$ depends on the state and input at intermediate times in $[t_0, T]$.

The performance index is selected to make the plant exhibit a desired type of performance. Some different possibilities for $J(t_0)$ are discussed in Example 2.1-1, which carries over to the continuous case.

The optimal control problem is to find the input $u^*(t)$ on the time interval $[t_0, T]$ that drives the plant (3.2-1) along a trajectory $x^*(t)$ such that the cost function (3.2-2) is minimized, and such that

$$\psi(x(T), T) = 0 \quad (3.2-3)$$

for a given function $\psi \in R^p$. This corresponds to the function-of-final-state-fixed discrete problem solved in Section 4.5.

The roles of the **final weighting function ϕ** and **the fixed final function ψ** should not be confused. $\phi(x(T), T)$ is a function of the final state, which we want to make *small*. An illustration might be the energy, which is $[x^T(T)S(T)x(T)]/2$, where $S(T)$ is a given weighting matrix. On the other hand, $\psi(x(T), T)$ is a function of the final state, which we want *fixed* at exactly zero. As an illustration, consider a satellite with state $x = [r\dot{r}\theta\dot{\theta}]^T$, where r and θ are radius and angular position. If we want to place the satellite in a circular orbit with radius R , then the final state function to be zeroed would be

$$\psi(x(T), T) = \begin{bmatrix} r(T) - R \\ \dot{r}(T) \\ \dot{\theta}(T) = \sqrt{\frac{\mu}{R^3}} \end{bmatrix},$$

with $\mu = GM$ the gravitational constant of the attracting mass M .

To solve the continuous optimal control problem, we shall use Lagrange multipliers to adjoin the constraints (3.2-1) and (3.2-3) to the performance index (3.2-2). Since (3.2-1) holds at each $t \in [t_0, T]$, we require an associated multiplier $\lambda(t) \in R^n$, which is a function of time. Since (3.2-3) holds only at one time, we require only a constant associated multiplier $\nu \in R^p$. The augmented performance index is thus

$$\begin{aligned} J' &= \phi(x(T), T) + \nu^T \psi(x(T), T) \\ &+ \int_{t_0}^T [L(x, u, t) + \lambda^T(t)(f(t)(x, u, t) - \dot{x})] dt. \end{aligned} \quad (3.2-4)$$

If we define the **Hamiltonian function** as

$$H(x, u, t) = L(x, u, t) + \lambda^T f(x, u, t), \quad (3.2-5)$$

then we can rewrite (3.2-4) as

$$\begin{aligned} J' &= \phi(x(T), T) + \nu^T \psi(x(T), T) \\ &+ \int_{t_0}^T [H(x, u, t) - \lambda^T \dot{x}] dt. \end{aligned} \quad (3.2-6)$$

Using **Leibniz's** rule, the increment in J' as a function of increments in x , λ , ν , u , and t is

$$\begin{aligned} dJ' &= (\phi_x + \psi_x^T \nu)^T dx|_T + (\phi_t + \psi_t^T \nu) dt|_T + \psi^T|_T d\nu \\ &+ (H - \lambda^T \dot{x}) dt|_T - (H - \lambda^T \dot{x}) dt|_{t_0} \\ &+ \int_{t_0}^T [H_x^T \delta x + H_u^T \delta u - \lambda^T \delta \dot{x} + (H_\lambda - \dot{x})^T \delta \lambda] dt. \end{aligned} \quad (3.2-7)$$

To eliminate the variation in $\dot{x}$, integrate by parts to see that

$$-\int_{t_0}^T \lambda^T \delta \dot{x} dt = -\lambda^T \delta x|_T + \lambda^T \delta x|_{t_0} + \int_{t_0}^T \dot{\lambda}^T \delta x dt. \quad (3.2-8)$$

If we substitute this into (3.2-7), there result terms at $t = T$ dependent on both $dx(t)$ and $\delta x(T)$. We can express $\delta x(T)$ in terms of $dx(t)$ and dT using (3.1-1). The result after these two substitutions is

$$\begin{aligned} dJ' = & (\phi_x + \psi_x^T v - \lambda)^T dx|_T + (\phi_t + \psi_t^T v + H - \lambda^T \dot{x} + \dot{\lambda}^T \dot{x}) dt|_T \\ & + \psi^T|_T dv - (H - \lambda^T \dot{x} + \dot{\lambda}^T \dot{x}) dt|_{t_0} + \lambda^T dx|_{t_0} \\ & + \int_{t_0}^T [(H_x + \dot{\lambda})^T \delta x + H_u^T \delta u + (H_\lambda - \dot{x})^T \delta \lambda] dt. \end{aligned} \quad (3.2-9)$$

According to the Lagrange theory, the constrained minimum of J is attained at the unconstrained minimum of J' . This is achieved when $dJ' = 0$ for all independent increments in its arguments. Setting to zero the coefficients of the independent increments dv , δx , δu , and $\delta \lambda$ yields necessary conditions for a minimum as shown in Table 3.2-1. For our applications, t_0 and $x(t_0)$ are both fixed and known, so that dt_0 and $dx(t_0)$ are both zero. The two terms evaluated at $t = t_0$ in (3.2-9) are thus automatically equal to zero.

The final condition (3.2-10) in the table needs further discussion. We have seen that $dx(T)$ and dT are not independent (Fig. 3.1-1). Therefore, we cannot simply set the coefficients of the first two terms on the right-hand side of (3.2-9) separately equal to zero. Instead, the entire expression (3.2-10) must be zero at $t = T$. Compare it with (2.7-7). The extra term in (3.2-10) arises in the present situation since we have allowed for possible variations in the final time T . This will allow us to deal with minimum-time problems, which we shall do in Chapter 5.

For convenience, we have shown the conditions in the table both in terms of H and in terms of L and f . Compare these results with Table 2.1-1 and see the associated discussion for further insight. Note that the discrete and continuous costate equations are both dynamical equations that develop backward in time. In the continuous case, this amounts to making the rate of change (i.e., $\dot{\lambda}$) negative. The costate equation is also called the adjoint to the state equation.

As in the discrete case, the optimal control in Table 3.2-1 depends on the solution to a two-point boundary-value problem, since $x(t_0)$ is given and $\lambda(T)$ is determined by (3.2-10). It is, in general, very difficult to solve these problems. We do not really care about the value of $\lambda(t)$, but it must evidently be determined as an intermediate step in finding the optimal control $u^*(t)$, which depends on $\lambda(t)$ through the stationarity condition.

An important point is worth noting. The time derivative of the Hamiltonian is

$$\dot{H} = H_t + H_x^T \dot{x} + H_u^T \dot{u} + \dot{\lambda}^T f = H_t + H_u^T \dot{u} + (H_x + \dot{\lambda})^T f. \quad (3.2-11)$$

TABLE 3.2-1 Continuous Nonlinear Optimal Controller with Function of Final State Fixed*System model:*

$$\dot{x} = f(x, u, t), \quad t \geq t_0, \quad t_0 \text{ fixed}$$

Performance index:

$$J(t_0) = \phi(x(T), T) + \int_{t_0}^T L(x, u, t) dt$$

Final-state constraint:

$$\psi(x(T), T) = 0$$

*Optimal controller**Hamiltonian:*

$$H(x, u, t) = L(x, u, t) + \lambda^T f(x, u, t)$$

State equation:

$$\dot{x} = \frac{\partial H}{\partial \lambda} = f, \quad t \geq t_0$$

Costate equation:

$$-\dot{\lambda} = \frac{\partial H}{\partial x} = \frac{\partial f^T}{\partial x} \lambda + \frac{\partial L}{\partial x}, \quad t \leq T$$

Stationarity condition:

$$0 = \frac{\partial H}{\partial u} = \frac{\partial L}{\partial u} + \frac{\partial f^T}{\partial u} \lambda$$

Boundary conditions: $x(t_0)$ given

$$(\phi_x + \psi_x^T v - \lambda)^T|_T dx(T) + (\phi_t + \psi_t^T v + H)|_T dT = 0 \quad (3.2-10)$$

If $u(t)$ is an optimal control, then

$$\dot{H} = H_{t^*} \quad (3.2-12)$$

Now, in the time-invariant case, f and L are not explicit functions of t , and so neither is H . In this situation

$$\dot{H} = 0. \quad (3.2-13)$$

Hence, for time-invariant systems and cost functions, the Hamiltonian is a *constant* on the optimal trajectory.

Let us begin to develop a feel for the continuous optimal controller by looking at some examples.

Examples

The first two examples make the point that the solution to the optimization problem given in Table 3.2-1 is very general; it does not only apply in system theory. The next examples illustrate the computation of the optimal controller for dynamical systems, the last example emphasizing that the optimal control equations apply for general nonlinear systems.

Example 3.2-1. *Hamilton's Principle in Classical Dynamics*

Hamilton's principle for conservative systems in classical physics says that “of all possible paths along which a dynamical system may move from one point to another within a specified time interval (consistent with any constraints), the actual path followed is that which minimizes the time integral of the difference between the kinetic and potential energies” (Marion 1965).

a. Lagrange's Equations of Motion

We can derive Lagrange's equations of motion from this principle by defining (Bryson and Ho 1975)

$$\begin{aligned} q &\triangleq \text{generalized coordinate vector,} \\ u = \dot{q} &\triangleq \text{generalized velocities,} \\ U(q) &\triangleq \text{potential energy,} \\ T(q, u) &\triangleq \text{kinetic energy,} \\ L(q, u) &\triangleq T(q, u) - U(q), \text{ the Lagrangian of the system.} \end{aligned}$$

The “plant” is then described by

$$\dot{q} = u \triangleq f(q, u), \quad (1)$$

where the function f is given by the physics of the problem. To find the trajectories of the motion, Hamilton's principle says that we must minimize the performance index

$$J(0) = \int_0^T L(q, u) dt. \quad (2)$$

Therefore, the Hamiltonian is

$$H = L + \lambda^T u. \quad (3)$$

According to Table 3.2-1, for a minimum we require

$$-\dot{\lambda} = \frac{\partial H}{\partial q} = \frac{\partial L}{\partial q} \quad (4)$$

and

$$0 = \frac{\partial H}{\partial u} = \frac{\partial L}{\partial u} + \lambda. \quad (5)$$

Combining these equations yields Lagrange's equations of motion

$$\frac{\partial L}{\partial q} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} = 0. \quad (6)$$

It is worth emphasizing that in this context, the costate equation and the stationarity condition are equivalent to Lagrange's equation. In the general context of variational problems, (6) is called *Euler's equation*. The costate equation and stationarity condition in Table 3.2-1 are, therefore, an alternative formulation of Euler's equation.

In the context of this example, condition (3.2-13) is nothing more than a statement of the conservation of energy!

b. Hamilton's Equations of Motion

If we define the *generalized momentum vector* by

$$\lambda = -\frac{\partial L}{\partial \dot{q}}, \quad (7)$$

then the equations of motion can be written in Hamilton's form as

$$\dot{q} = \frac{\partial H}{\partial \lambda}, \quad (8)$$

$$-\dot{\lambda} = \frac{\partial H}{\partial q}. \quad (9)$$

Hence, in the optimal control problem, the state and costate equations are a generalized formulation of Hamilton's equations of motion! ■

Example 3.2-2. Shortest Distance between Two Points

The length of a curve $x(t)$ dependent on a parameter t between $t = a$ and $t = b$ is given by

$$J = \int_a^b \sqrt{1 + \dot{x}^2(t)} dt. \quad (1)$$

To specify that the curve join two points $(a, A)(b, B)$ in the plane, we need to impose the boundary conditions

$$x(a) = A, \quad (2)$$

$$x(b) = B. \quad (3)$$

See Shultz and Melsa (1967).

It is desired to find the curve $x(t)$ joining (a, A) and (b, B) that minimizes (1). To put this into the form of an optimal control problem, define the "input" by

$$\dot{x} = u. \quad (4)$$

This is the "plant." Then (1) becomes

$$J = \int_a^b \sqrt{1 + u^2} dt. \quad (5)$$

The Hamiltonian is

$$H = \sqrt{1 + u^2} + \lambda u. \quad (6)$$

Now, Table 3.2-1 yields the conditions

$$\dot{x} = H_\lambda = u, \quad (7)$$

$$-\dot{\lambda} = H_x = 0, \quad (8)$$

$$0 = H_u = \lambda + \frac{u}{\sqrt{1 + u^2}}. \quad (9)$$

To solve these for the optimal slope u , note that by (9)

$$u = \frac{-\lambda}{\sqrt{1 - \lambda^2}}, \quad (10)$$

but according to (8), λ is constant. Hence,

$$u = \text{const} \quad (11)$$

is the optimal “control.” Now use (7) to get

$$x(t) = c_1 t + c_2. \quad (12)$$

To determine c_1 and c_2 , use the boundary conditions (2) and (3) to see that

$$x(t) = \frac{(A - B)t + (aB - bA)}{a - b}. \quad (13)$$

The optimal trajectory (13) between two points is thus a straight line. ■

Example 3.2-3. Temperature Control in a Room

It is desired to heat a room using the least possible energy. If $\theta(t)$ is the temperature in the room, θ_a the ambient air temperature outside (a constant), and $u(t)$ the rate of heat supply to the room, then the dynamics are

$$\dot{\theta} = -a(\theta - \theta_a) + bu \quad (1)$$

for some constants a and b , which depend on the room insulation and so on. By defining the state as

$$x(t) \triangleq \theta(t) - \theta_a, \quad (2)$$

we can write the state equation

$$\dot{x} = -ax + bu. \quad (3)$$

See McClamroch (1980). To control the temperature on the fixed time interval $[0, T]$ with the least possible supplied energy, define the performance index as

$$J(0) = \frac{1}{2} \int_0^T u^2(t) dt. \quad (4)$$

We shall discuss two possible control objectives in parts *a* and *b* below. The Hamiltonian is

$$H = \frac{u^2}{2} + \lambda(-ax + bu). \quad (5)$$

According to Table 3.2-1, the optimal control $u(t)$ is determined by solving

$$\dot{x} = H_\lambda = -ax + bu, \quad (6)$$

$$\dot{\lambda} = -H_x = a\lambda, \quad (7)$$

$$0 = H_u = u + b\lambda. \quad (8)$$

The stationarity condition (8) says that the optimal control is given by

$$u(t) = -b\lambda(t), \quad (9)$$

so to determine $u^*(t)$ we need only find the optimal costate $\lambda^*(t)$.

Substituting (9) into (6) yields the state–costate equations

$$\dot{x} = -ax - b^2\lambda, \quad (10a)$$

$$\dot{\lambda} = a\lambda, \quad (10b)$$

which must now be solved for $\lambda^*(t)$ and the optimal state trajectory $x^*(t)$. We do not yet know the final costate $\lambda(T)$, but let us solve (10) as if we did. The solution to (10b) is

$$\lambda(t) = e^{-a(T-t)}\lambda(T). \quad (11)$$

Using this in (10a) yields

$$\dot{x} = -ax - b^2\lambda(T)e^{-a(T-t)}. \quad (12)$$

Using Laplace transforms to solve this gives

$$\begin{aligned} X(s) &= \frac{x(0)}{s+a} - \frac{b^2\lambda(T)e^{-aT}}{(s+a)(s-a)} \\ &= \frac{x(0)}{s+a} - \frac{b^2}{a}\lambda(T)e^{-aT} \left(\frac{-1/2}{s+a} + \frac{1/2}{s-a} \right) \end{aligned} \quad (13)$$

so that

$$x(t) = x(0)e^{-at} - \frac{b^2}{a}\lambda(T)e^{-aT} \sinh at. \quad (14)$$

Equations (11) and (14) give the optimal costate $\lambda^*(t)$ and state $x^*(t)$ in terms of the as yet unknown final costate $\lambda(T)$. The initial state $x(0)$ is given.

Now we consider two possible control objectives, which will give two ways to determine $\lambda(T)$.

a. Fixed Final State

Suppose that the initial temperature of the room is equal to $\theta_a = 60^\circ$. Then

$$x(0) = 0^\circ. \quad (15)$$

Let our control objective be to drive the final temperature $\theta(T)$ exactly to 70° at the given final time of T seconds. Then the final state is required to take on the fixed value of

$$x(T) = 10^\circ. \quad (16)$$

Note that since the final time and final state are both fixed, dT and $dx(T)$ are both zero, so that (3.2-10) is satisfied.

Using (15) and (16), we must determine $\lambda(T)$; then we can find $\lambda(t)$ by using (11) and the optimal control by using (9). To find $\lambda(T)$, use (14) to write

$$x(T) = x(0)e^{-aT} - \frac{b^2}{2a}\lambda(T)(1 - e^{-2aT}). \quad (17)$$

Taking into account (15) and (16) shows that the final costate is

$$\lambda(T) = \frac{20a}{b^2(1 - e^{-2aT})}, \quad (18)$$

and so the optimal costate trajectory is

$$\lambda^*(t) = -\frac{10ae^{at}}{b^2 \sinh aT} aT. \quad (19)$$

Finally, the optimal rate of heat supply to the room is given by (9) or

$$u^*(t) = \frac{10ae^{at}}{b \sinh aT} \quad 0 \leq t \leq T. \quad (20)$$

To check our answer, apply $u^*(t)$ to the system (3). Solving for the state trajectory yields

$$x^*(t) = 10 \frac{\sinh at}{\sinh aT}. \quad (21)$$

Indeed $x^*(T) = 10$ as desired.

b. Free Final State

Now suppose that we are not so concerned that the final state $x(T)$ be exactly 10° . Let us demand only that the control $u(t)$ minimize

$$J(0) = \frac{1}{2}s(x(T) - 10)^2 + \frac{1}{2} \int_0^T u^2(t) dt \quad (22)$$

for some weighting s (i.e., some real number s) to be selected later. If s is large, then the optimal solution will have $x(T)$ near 10° , since only then will the first term make a small contribution to the cost.

According to Table 3.2-1, the state and costate equations are still given by 10, and the optimal control by (9). Therefore, (11) and (14) are still valid.

The initial condition is still (15), but the final condition must be determined by using (3.2-10). The final time T is fixed, so $dT = 0$ and the second term of (3.2-10) is automatically equal to zero. Since $x(T)$ is not fixed, $dx(T)$ is not zero (as it was in part a). Therefore, it is required that

$$\lambda(T) = \left. \frac{\partial \phi}{\partial x} \right|_T = s(x(T) - 10). \quad (23)$$

(Note that there is no function ψ in this problem.) This is our new terminal condition, and from (15) and (23) we must determine $\lambda(T)$. To do this, note that

$$x(T) = \frac{\lambda(T)}{s} + 10, \quad (24)$$

and use this and (15) and (17). Solving for the final costate gives

$$\lambda(T) = \frac{-20as}{2a + b^2s(1 - e^{-2aT})}. \quad (25)$$

Using (11) gives the optimal costate trajectory

$$\lambda^*(t) = \frac{-10ase^{at}}{ae^{aT} + sb^2 \sinh aT}. \quad (26)$$

Finally (9) yields the optimal control

$$u^*(t) = \frac{10abse^{at}}{ae^{aT} + sb^2 \sinh aT}. \quad (27)$$

To check our answer, we “simulate” the control by using $u^*(t)$ in the plant (3). Solving for the optimal state trajectory yields

$$x^*(t) = \frac{10sb^2 \sinh at}{ae^{aT} + sb^2 \sinh aT}. \quad (28)$$

At the final time,

$$x^*(T) = \frac{10sb^2 \sinh aT}{ae^{aT} + sb^2 \sinh aT}. \quad (29)$$

c. Discussion

The final value $x^*(T)$ in (29) is not equal to the desired 10° . It is a function of the final-state weighting s in the performance index. As s becomes larger, we are making it more important relatively for $x(T)$ to equal 10° than for $u^2(t)$ to be small on $[0, T]$. In fact, in the limit $s \rightarrow \infty$, the costate (26), control (27), and state trajectory (28) tend to the expressions found in part a. In this limit, the final state $x^*(T)$ in (29) does indeed become exactly 10° .

By examining (29), we can determine $x^*(T)$ for various values of s and select a value that gives a good compromise between driving $x(t)$ to the desired final value and conserving control energy. Using this value of s in (27) yields the optimal control that we would actually apply to heat the room. ■

Example 3.2-4. The Intercept and Rendezvous Problems*a. Problem Formulation*

In Example 2.3-2 we constructed a digital controller for the rendezvous problem. Let us now find an analytical expression for the continuous-time optimal control. In the next subsection we show an easy way to program the continuous optimal controller on a digital computer that does not require the analysis done here.

The geometry of the problem is shown in Fig. 3.2-1, where $y(t)$ and $v(t)$ are the vertical position and velocity of the pursuit aircraft A relative to the target aircraft A_t , which we can assume is at rest. Its initial horizontal distance from the pursuer is D . The horizontal velocity of the pursuit aircraft relative to A_t is V ; so the final time T , at which the two aircraft will have the same horizontal distance, is fixed and known to be

$$T = t_0 + \frac{D}{V}. \quad (1)$$

The *line-of-sight angle* is $\sigma(t)$. See Bryson and Ho (1975).

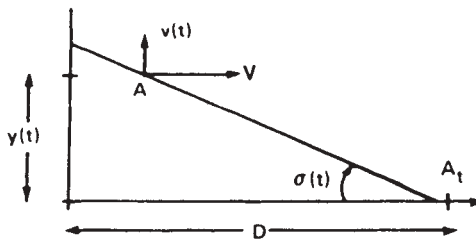


FIGURE 3.2-1 Intercept and rendezvous geometry.

In the *rendezvous* problem it is desired that final position $y(T)$ and velocity $v(T)$ both be zero. In the *intercept* problem we are not concerned with final velocity, and it is only desired that the final position $y(T)$ be zero.

The vertical dynamics are described by the state equations

$$\dot{y} = v, \quad (2)$$

$$\dot{v} = u, \quad (3)$$

where $u(t)$ is the vertical control input acceleration. Let the performance index be

$$J(t_0) = \frac{s_y y^2(T)}{2} + \frac{s_v v^2(T)}{2} + \frac{1}{2} \int_{t_0}^T u^2(t) dt. \quad (4)$$

For intercept, $s_v = 0$ and s_y is made large so the optimal control will make $y^2(T)$ small. For rendezvous, s_v and s_y are both selected large.

b. Problem Solution

The optimal control must now be selected to minimize (4). Each state component must have an associated scalar Lagrange multiplier; hence let $\lambda \triangleq [\lambda_y \lambda_v]^T$. Then the

Hamiltonian is

$$H = \frac{1}{2}u^2 + v\lambda_y + u\lambda_v. \quad (5)$$

The costate equations are therefore

$$\dot{\lambda}_y = -\frac{\partial H}{\partial y} = 0, \quad (6)$$

$$\dot{\lambda}_v = -\frac{\partial H}{\partial v} = -\lambda_y. \quad (7)$$

The stationarity condition is

$$0 = \frac{\partial H}{\partial u} = u + \lambda_v, \quad (8)$$

so the optimal control is the negative of the velocity multiplier

$$u(t) = -\lambda_v(t). \quad (9)$$

The initial conditions are

$$y(t_0), v(t_0) \text{ given.} \quad (10)$$

The final conditions are determined by (3.2-10). Since the final time is fixed, $dT = 0$ and so only the first term gives binding conditions. They are

$$\lambda_y(T) = \frac{\partial \phi}{\partial y}(T) = s_y y(T), \quad (11)$$

$$\lambda_v(T) = \frac{\partial \phi}{\partial v}(T) = s_v v(T). \quad (12)$$

We must now solve the two-point boundary-value problem defined by the state and costate equations, with u as in (9) and with boundary conditions (10)–(12). To do this, we proceed as we did in Example 3.2-3, assuming at the outset that $\lambda_y(T)$ and $\lambda_v(T)$ are known. The costate equation is first solved *backward* in time (i.e., in terms of $\lambda(T)$), and the state equation is then solved *forward* in time (i.e., in terms of $x(t_0)$).

Integrating both sides of (6) from t to T yields the constant costate component

$$\lambda_y(t) = \lambda_y(T) \triangleq \lambda_y. \quad (13)$$

Integrating (7) then gives

$$\lambda_v(T) - \lambda_v(t) = -(T - t)\lambda_y$$

or

$$\lambda_v(t) = \lambda_v(T) + (T - t)\lambda_y. \quad (14)$$

Now, to simplify things, let us assume for a few moments that $t_0 = 0$. Substituting the control (9) into (3) gives

$$\dot{v} = -\lambda_v(t). \quad (15)$$

Using (14) and integrating both sides from 0 to t yields the quadratic expression

$$v(t) = v(0) - t(\lambda_v(T) + T\lambda_y) + \frac{t^2}{2}\lambda_y. \quad (16)$$

Taking this into account and integrating (2) yields the cubic expression

$$y(t) = y(0) + tv(0) - \frac{t^2}{2}(\lambda_v(T) + T\lambda_y) + \frac{t^3}{6}\lambda_y. \quad (17)$$

The state and costate equations have now been solved in terms of $\lambda(T)$ and the given $y(0)$, $v(0)$. Unfortunately, the final costate is unknown. To find it, we must use the relations (11) and (12) between the final state and costate. Thus, use these relations and (16), (17) to get

$$\lambda_y = s_y \left[y(0) + Tv(0) - \frac{T^2}{2}(\lambda_v(T) + T\lambda_y) + \frac{T^3}{6}\lambda_y \right] \quad (18)$$

and

$$\lambda_v(T) = s_v \left[v(0) - T(\lambda_v(T) + T\lambda_y) + \frac{T^2}{2}\lambda_y \right]. \quad (19)$$

These two equations can be written as

$$\begin{bmatrix} 1 + \frac{s_y T^3}{3} & \frac{s_y T^2}{2} \\ \frac{s_v T^2}{2} & 1 + s_v T \end{bmatrix} \begin{bmatrix} \lambda_y \\ \lambda_v(T) \end{bmatrix} = \begin{bmatrix} s_y & s_y T \\ 0 & s_v \end{bmatrix} \begin{bmatrix} y(0) \\ v(0) \end{bmatrix}. \quad (20)$$

Solving this yields the final costate

$$\begin{bmatrix} \lambda_y \\ \lambda_v(T) \end{bmatrix} = \frac{1}{\Delta(T)} \begin{bmatrix} \bar{s}_v + T & T \left(\bar{s}_v + \frac{T}{2} \right) \\ -T^2/2 & \bar{s}_v - \frac{T^3}{6} \end{bmatrix} \begin{bmatrix} y(0) \\ v(0) \end{bmatrix}, \quad (21)$$

where

$$\Delta(T) = (\bar{s}_y + T^3/3)(\bar{s}_v + T) - T^4/4 \quad (22)$$

and the reciprocal final weights are

$$\bar{s}_y \equiv \frac{1}{s_y}, \quad (23a)$$

$$\bar{s}_v \equiv \frac{1}{s_v}. \quad (23b)$$

The initial time is, in fact t_0 not 0. Since the state and costate equations are linear, all we need do to correct this is to substitute $(T - t_0)$ for T on the right-hand side of (21). Before we do this, however, consider the following. At the current time $t \leq T$ we know

$y(t)$ and $v(t)$, so we can take the current time t as the initial time. This corresponds to minimizing $J(t)$, the *remaining* cost on the interval $[t, T]$.

Substituting $(T - t)$ for T in (21) yields an expression for the final costate in terms of the *current* state:

$$\begin{bmatrix} \lambda_y \\ \lambda_v(T) \end{bmatrix} = \frac{1}{\Delta(T-t)} \begin{bmatrix} \bar{s}_v + (T-t) & (T-t) \left[\bar{s}_v + \frac{T-t}{2} \right] \\ -\frac{(T-t)^2}{2} & \bar{s}_y - \frac{(T-t)^3}{6} \end{bmatrix} \begin{bmatrix} y(t) \\ v(t) \end{bmatrix}. \quad (24)$$

We are finally in a position to compute the optimal control, for according to (9) and (14),

$$u(t) = - \begin{bmatrix} T-t & 1 \end{bmatrix} \begin{bmatrix} \lambda_y \\ \lambda_v(T) \end{bmatrix}. \quad (25)$$

Taking into account (24) therefore yields the optimal control

$$\begin{aligned} u(t) = & - \frac{(T-t)\bar{s}_v + (T-t)^2/2}{\Delta(T-t)} y(t) \\ & - \frac{\bar{s}_y + (T-t)^2\bar{s}_v + (T-t)^3/3}{\Delta(T-t)} v(t). \end{aligned} \quad (26)$$

This is a *feedback* control law since the current control is given in terms of the current state.

c. Proportional Navigation

For the intercept problem, we select $s_v = 0$ and $s_y \rightarrow \infty$. Taking the limit in (26) yields

$$u(t) = - \frac{3}{(T-t)^2} y(t) - \frac{3}{(T-t)} v(t) \quad (27)$$

as the optimal control for intercept. To make this look neater, note that for a small line-of-sight angle

$$\sigma(t) \simeq \tan \sigma(t) = \frac{y(t)}{(T-t)V}, \quad (28)$$

so that

$$\dot{\sigma} \simeq \frac{\dot{y}(t)}{(T-t)V} + \frac{y(t)}{(T-t)^2 V}. \quad (29)$$

Therefore, the optimal control is

$$u(t) = -3V\dot{\sigma}. \quad (30)$$

This is the *proportional navigation* control law. Every pilot knows that for an intercept it is necessary only to keep the angle to the target constant so that there is no relative bearing drift!

See Bryson and Ho (1975) for further discussion. ■

Example 3.2-5. Thrust Angle Programming

This example emphasizes that the optimal controller in Table 3.2-1 applies to general nonlinear systems.

a. The Bilinear Tangent Law

A particle of mass m is acted on by a constant thrust F applied at a variable angle of $\gamma(t)$. Its position is $(x(t), y(t))$, and its x and y velocities are $u(t)$ and $v(t)$. See Fig. 3.2-2. The nonlinear state equations $\dot{X} = f(X, \gamma, t)$ are

$$\dot{x} = u, \quad (1)$$

$$\dot{y} = v, \quad (2)$$

$$\dot{u} = a \cos \gamma, \quad (3)$$

$$\dot{v} = a \sin \gamma, \quad (4)$$

where the state is $X = [x \ y \ u \ v]^T$, $a \triangleq F/m$ is the known thrust acceleration, and thrust angle $\gamma(t)$ is the control input.

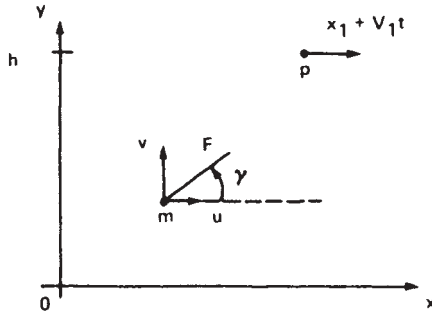


FIGURE 3.2-2 Thrust angle programming.

Let the performance index be a function only of the final time and state, so that

$$J = \phi(X(T), T) \quad (5)$$

(i.e., $L(X, \gamma, t) = 0$). Suppose that a given function ψ of the final state must be zeroed, so that

$$\psi(X(T), T) = 0. \quad (6)$$

The form of the control $\gamma(t)$ required to minimize J and satisfy (6) is easy to determine. The Hamiltonian is

$$H = L + \lambda^T f = \lambda_x u + \lambda_y v + \lambda_u a \cos \gamma + \lambda_v a \sin \gamma, \quad (7)$$

where the Lagrange multiplier $\lambda(t) = [\lambda_x \ \lambda_y \ \lambda_u \ \lambda_v]^T$ has a component associated with each state component.

According to Table 3.2-1, the costate equations are $\dot{\lambda} = -f_X^T \lambda$ or

$$\dot{\lambda}_x = -f_x^T \lambda = 0, \quad (8)$$

$$\dot{\lambda}_y = -f_y^T \lambda = 0, \quad (9)$$

$$\dot{\lambda}_u = -f_u^T \lambda = -\lambda_x, \quad (10)$$

$$\dot{\lambda}_v = -f_v^T \lambda = -\lambda_y. \quad (11)$$

(Note that subscripts on f indicate partial derivatives, while subscripts on λ indicate the costate components.) The stationarity condition is

$$0 = H\gamma = -\lambda_u a \sin \gamma + \lambda_v a \cos \gamma, \quad (12)$$

or

$$\tan \gamma(t) = \frac{\lambda_v(t)}{\lambda_u(t)}. \quad (13)$$

Integrating the costate equations backward from the final time T yields

$$\lambda_x(t) = \lambda_x(T) \triangleq \lambda_x, \quad (14)$$

$$\lambda_y(t) = \lambda_y(T) \triangleq \lambda_y, \quad (15)$$

$$\lambda_u(t) = (\lambda_u(T) + T\lambda_x) - t\lambda_x \triangleq c_1 - t\lambda_x, \quad (16)$$

$$\lambda_v(t) = (\lambda_v(T) + T\lambda_y) - t\lambda_y \triangleq c_2 - t\lambda_y. \quad (17)$$

Substituting into (13) yields the optimal control law

$$\tan \gamma(t) = \frac{t\lambda_y - c_2}{t\lambda_x - c_1}. \quad (18)$$

This is called the *bilinear tangent law* for the optimal thrust direction $\gamma(t)$.

To determine the constants λ_x , λ_y , c_1 , and c_2 , we may substitute for $\gamma(t)$ into the state equations using (18), solve them, and then use the boundary conditions. To determine the boundary conditions, we need to specify ϕ and ψ , which depend on the particular control objectives. There are many possible objectives we might have in mind for the behavior of the particle m . See Bryson and Ho (1975). One that leads to an interesting and fairly simple solution is discussed next.

b. Minimum-time Intercept

Suppose m represents an aircraft that wants to intercept a target P in minimum time. P has an initial position of x_1 and a constant velocity in the x direction of V_1 , so that its x position at time t is $x_1 + V_1 t$. Its y position h is constant.

The minimum-time objective can be expressed by demanding that the optimal control minimize

$$J = T = \int_0^T 1 \, dt. \quad (19)$$

Since $L = 1$, the Hamiltonian is now

$$H(t) = 1 + \lambda_x u + \lambda_y v + \lambda_u a \cos \gamma + \lambda_v a \sin \gamma; \quad (20)$$

however, since L is constant, the other results in part a remain valid.

If m starts out at $t_0 = 0$ at rest at the origin, the initial conditions are

$$x(0) = 0, \quad y(0) = 0, \quad u(0) = 0, \quad v(0) = 0. \quad (21)$$

The final-state function is

$$\psi(X(T), T) = \begin{bmatrix} x(T) - (x_1 + V_1 T) \\ y(T) - h \end{bmatrix} = 0, \quad (22)$$

so that

$$x(T) = x_1 + V_1 T, \quad (23)$$

$$y(T) = h. \quad (24)$$

To find the remaining terminal conditions, we need to use (3.2-10). Both the final state and the final time are free (i.e., different choices of $\gamma(t)$ will result in different values for T and the state components $u(T)$, $v(T)$). Therefore, $dx(T) \neq 0$ and $dT \neq 0$. In this problem, however, $dx(T)$ and dT are *independent* so that (3.2-10) yields the two separate conditions

$$(\phi_x + \psi_x^T v - \lambda)|_T = 0 \quad (25)$$

and

$$(\phi_t + \psi_t^T v + H)|_T = 0, \quad (26)$$

where $v = [v_x \ v_y]^T$ is a new constant Lagrange multiplier.

Taking into account (22) (note that $\phi(x(T), T) = 0$), (25) becomes

$$\lambda(T) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

or

$$\lambda_x(T) = v_x, \quad (27)$$

$$\lambda_y(T) = v_y, \quad (28)$$

$$\lambda_u(T) = 0, \quad (29)$$

$$\lambda_v(T) = 0. \quad (30)$$

Note that the components of $\lambda(T)$ corresponding to the *fixed* final state components $x(T)$ and $y(T)$ are unknown variables, and the components of $\lambda(T)$ corresponding to the *free* final state components $u(T)$ and $v(T)$ are fixed at zero.

Using (20) and (22), the final condition (26) becomes

$$H(T) = -\psi_T^T v|_T = -[-V_1 \quad 0] \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

or

$$1 + v_x u(T) + v_y v(T) = V_1 v_x. \quad (31)$$

We have used (27)–(30).

Now we need to solve the state equations (1)–(4), taking into account (18) and the costate solutions (14)–(17) and the boundary conditions (21), (23), (24), and (27)–(30). We also need condition (31) to allow us to solve for the unknown optimal final time T^* .

First, note that in light of (27)–(30), the costate solutions are

$$\lambda_x(t) = v_x, \quad (32)$$

$$\lambda_y(t) = v_y, \quad (33)$$

$$\lambda_u(t) = (T - t)v_x, \quad (34)$$

$$\lambda_v(t) = (T - t)v_y, \quad (35)$$

where the terminal multipliers v_x , v_y still need to be determined. The bilinear tangent law (18) therefore takes on the simple form

$$\tan \gamma = v_y/v_x. \quad (36)$$

For this minimum-time intercept problem, the optimal thrust angle is a *constant*!

To find the optimal control $\gamma^*(t)$, all that remains is to find v_x and v_y . We shall see that this still requires a little work.

Since γ is a constant, it is easy to integrate the state equations forward from $t_0 = 0$ to get

$$v(t) = at \sin \gamma, \quad (37)$$

$$u(t) = at \cos \gamma, \quad (38)$$

$$y(t) = \frac{at^2}{2} \sin \gamma, \quad (39)$$

$$x(t) = \frac{at^2}{2} \cos \gamma, \quad (40)$$

where we have used the initial conditions (21). Evaluating (39) and (40) at $t = T$ yields

$$\tan \gamma = \frac{y(T)}{x(T)}, \quad (41)$$

and final conditions (23) and (24) then give an expression for the control in terms of the final time:

$$\tan \gamma = \frac{h}{x_1 + V_1 T}. \quad (42)$$

We still need to determine the optimal final time T^* to use in (42). The role of equation (31) is to allow us to solve for T^* , but to use it we would first need to find v_x and v_y . In this particular problem, we can use a shortcut that does not require v_x , v_y .

Indeed, note that (39), (40), (23), and (24) imply that

$$\sin \gamma = \frac{2y(T)}{aT^2} = \frac{2h}{aT^2}, \quad (43)$$

$$\cos \gamma = \frac{2x(T)}{2T^2} = \frac{2(x_1 + V_1 T)}{aT^2}. \quad (44)$$

Hence, $\sin^2 \gamma + \cos^2 \gamma = 1$, or

$$4h^2 + 4(x_1 + V_1 T)^2 = a^2 T^4, \quad (45)$$

which is

$$-\frac{a^2 T^4}{4} + V_1^2 T^2 + 2V_1 x_1 T + (x_1^2 + h^2) = 0. \quad (46)$$

This is a quartic equation, which can be solved for T^* given the initial information x_1 , V_1 , h about the target. Only one solution to (46) will make physical sense. The optimal control is determined by simply solving (46) for T^* and then solving (42) for the optimal thrust angle γ^* .

It is not difficult to see what our solution means intuitively. See Fig. 3.2-3, where the hypotenuse can be expressed in terms of the motion of the target as

$$d^2 = h^2 + (x_1 + V_1 T^*)^2, \quad (47)$$

or in terms of the motion of the pursuit aircraft as

$$d^2 = \left(\frac{1}{2}a(T^*)^2\right)^2. \quad (48)$$

Equation (45) is just an expression of the requirement that the two aircraft be at the same point at the final time! Of course, if we had not gone through our rigorous derivation of (45), we could not be sure from Fig. 3.2-3 that its solution yields the *optimal* final time (courtesy of E. Verriest).

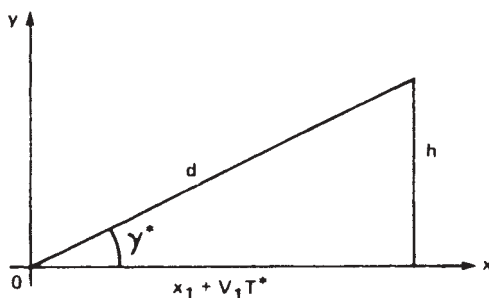


FIGURE 3.2-3 Interpretation of minimum-time intercept control law. ■

Solution of Two-point Boundary-value Problems

There are many computational methods for solving the optimal control problem. Our purpose is not to provide a survey of methods, for this would occupy more space than we have available. Instead, we present a few approaches that have immediate practical appeal.

Suppose it is desired to solve the optimal control problem for the nonlinear plant (3.2-1) with **quadratic performance index**

$$J(t_0) = \frac{1}{2}(x(T) - r(T))^T S(T)(x(T) - r(T)) + \frac{1}{2} \int_{t_0}^T (x^T Q x + u^T R u) dt, \quad (3.2-14)$$

where $S(T) \geq 0$, $Q \geq 0$, $R > 0$, and the desired final-state value $r(T)$ is given. Thus, we want to find the control $u(t)$ over the interval $[t_0, T]$ to minimize $J(t_0)$. **The final state** is constrained to satisfy (3.2-3) for some given function $\psi \in R^p$. For simplicity, let the final time T be fixed.

According to Table 3.2-1, we must solve the state equation (3.2-1) and the Euler equations

$$-\dot{\lambda} = \frac{\partial f^T}{\partial x} \lambda + Qx, \quad (3.2-15)$$

$$0 = Ru + \frac{\partial f^T}{\partial u} \lambda. \quad (3.2-16)$$

In general, the Jacobians $\partial f/\partial x$ and $\partial f/\partial u$ depend on the control $u(t)$, so that (3.2-16) is an implicit equation for $u(t)$. If **$\partial f/\partial u$ is independent of $u(t)$** , then we have

$$u = -R^{-1} \frac{\partial f^T}{\partial u} \lambda, \quad (3.2-17)$$

which we can use to eliminate $u(t)$ in the state equation and the costate equation (3.2-15), obtaining the **Hamiltonian system**

$$\dot{x} = f \left(x, -R^{-1} \frac{\partial f^T}{\partial u} \lambda, t \right), \quad (3.2-18a)$$

$$-\dot{\lambda} = \frac{\partial f^T}{\partial x} \lambda + Qx. \quad (3.2-18b)$$

The Hamiltonian system is a nonlinear ordinary differential equation in $x(t)$ and $\lambda(t)$ of order $2n$ with split boundary conditions, which are

$$n \text{ conditions: } x(t_0) = r(t_0) \text{ given,} \quad (3.2-19)$$

$$p \text{ conditions: } \psi(x(T), T) = 0, \quad (3.2-20)$$

$$n - p \text{ conditions: } \lambda(T) = S(T)(x(T) - r(T)) + \left. \frac{\partial \psi^T}{\partial x} \right|_T v. \quad (3.2-21)$$

The undetermined multipliers $v \in R^p$ allow some freedom, which means that (3.2-21) provides only the required number $n - p$ of final conditions.

There are several ways to solve the Hamiltonian system numerically. An excellent discussion is provided in Bryson and Ho (1975). One method is the following algorithm:

1. **Guess** the n unspecified initial conditions $\lambda(t_0)$.
2. **Integrate the Hamiltonian** system forward from t_0 to T .
3. Using the resulting values of $x(T)$ and $\lambda(T)$, evaluate

$$\psi(x(T), T) \quad (3.2-22)$$

and

$$\lambda(T) - S(T)(x(T) - r(T)) - \left. \frac{\partial \psi^T}{\partial x} \right|_T v. \quad (3.2-23)$$

4. If there is no $v \in R^p$ that makes (3.2-22) and (3.2-23) equal to zero, determine changes in the final state and costate $\delta x(T)$ and $\delta \lambda(T)$ to bring these functions closer to zero.
5. Find the sensitivity matrix

$$\begin{bmatrix} \partial \mu(T) \\ \partial \lambda(t_0) \end{bmatrix}, \quad \text{where } \mu(T) = \begin{bmatrix} x(T) \\ \lambda(T) \end{bmatrix}$$

and

$$\delta \mu(T) = \begin{bmatrix} \delta x(T) \\ \delta \lambda(T) \end{bmatrix} = \frac{\partial \mu(T)}{\partial \lambda(t_0)} \delta \lambda(t_0).$$

(Several ways of doing this are given in Bryson and Ho (1975). See also our discussion of unit solutions.)

6. Calculate the change in $\lambda(t_0)$ required to produce the desired changes in the final values $x(T)$, $\lambda(T)$ by solving (3.2-24).
7. Repeat steps 2 through 6 until (3.2-22), (3.2-23) are close enough to zero for the application.

Another way to solve the two-point boundary-value problem (3.2-18)–(3.2-21) **for linear systems** is first to solve several initial-condition problems and then solve a system of simultaneous equations. This *unit solution method* proceeds as follows:

1. Integrate the Hamiltonian system using as initial conditions $\lambda(t_0) = 0$ and $x(t_0) = r(t_0)$, where $r(t_0)$ is the given initial state. Call the resulting solutions $x_0(t)$, $\lambda_0(t)$.

2. Suppose $\lambda \in R^n$, and let e_i represent the i th column of the $n \times n$ identity matrix. Determine n unit solutions by integrating the Hamiltonian system n times, using as initial conditions

$$\begin{aligned} x(t_0) &= 0, \\ \lambda t_0 &= e_i, \quad i = 1, \dots, n. \end{aligned} \quad (3.2-2)$$

Call the resulting unit solutions $x_i(t)$, $\lambda_i(t)$ for $i = 1, \dots, n$.

3. General initial conditions can be expressed as

$$x(t_0) = r(t_0) \text{ given}, \quad (3.2-3)$$

$$\lambda(t_0) = \sum_{i=1}^n c_i e_i$$

for constants c_i . The overall solutions for these general initial conditions are

$$\begin{aligned} x(t) &= x_0(t) + \sum_{i=1}^n c_i x_i(t), \\ \lambda(t) &= \lambda_0(t) + \sum_{i=1}^n c_i \lambda_i(t). \end{aligned}$$

Evaluate these solutions at the final time $t = T$, and then solve for the initial-costate values c_i required to ensure that the terminal conditions (3.2-20) and (3.2-21) are satisfied. The unit solutions show the effect on $x(T)$, $\lambda(T)$ of each of the n individual components of $\lambda(t_0)$, and so they can be used to find the sensitivity matrix in (3.2-24).

The next example illustrates this approach.

Example 3.2-6. Unit Solution Method for Scalar System

The scalar plant

$$\dot{x} = ax + bu \quad (1)$$

has performance index of

$$J = \frac{s(x(T) - r(T))^2}{2} + \frac{1}{2} \int_0^T (qx^2 + ru^2) dt. \quad (2)$$

The desired final-state value $r(T)$ is given. The initial state $x(0) = r(0)$ is known, the final time is fixed, and the final state is free. To determine the optimal control $u(t)$ on $[0, T]$ by the method of unit solutions, we proceed as follows.

From Table 3.2-1, the Hamiltonian is

$$H = \frac{qx^2}{2} + \frac{ru^2}{2} + \lambda(ax + bu). \quad (3)$$

The Euler equations are

$$\dot{\lambda} = -\frac{\partial H}{\partial x} = -qx - a\lambda, \quad (4)$$

$$0 = \frac{\partial H}{\partial u} = ru + b\lambda. \quad (5)$$

Therefore, the optimal control is

$$u = -\frac{b}{r}\lambda. \quad (6)$$

Eliminating $u(t)$ in (1) yields the Hamiltonian system

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = \begin{bmatrix} a & -b^2/r \\ -q & -a \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} \triangleq A \begin{bmatrix} x \\ \lambda \end{bmatrix} \quad (7)$$

The split boundary conditions are

$$x(0) = r(0) \text{ given}, \quad (8)$$

$$\lambda(T) = s(x(T) - r(T)). \quad (9)$$

Instead of solving the split boundary-value problem (7)–(9), we solve two (i.e., $n+1$) initial-value problems, one with initial conditions $x(0) = r(0)$, $\lambda(0) = 0$, and one with $x(0) = 0$, $\lambda(0) = 1$. Then we solve for the $\lambda(0)$ required to make (9) hold.

If $x(0) = r(0)$, $\lambda(0) = 0$, then the solution can be found by Laplace transforms to be

$$\begin{bmatrix} x_0(t) \\ \lambda_0(t) \end{bmatrix} = \frac{r(0)}{2\alpha} \begin{bmatrix} \alpha - a \\ q \end{bmatrix} e^{-\alpha t} + \frac{r(0)}{2\alpha} \begin{bmatrix} \alpha + a \\ -q \end{bmatrix} e^{\alpha t}, \quad t \geq 0, \quad (10)$$

where $\alpha = \sqrt{a^2 + \frac{qb^2}{r}}$. If $x(0) = 0$, $\lambda(0) = 1$, the unit solution is

$$\begin{bmatrix} x_1(t) \\ \lambda_1(t) \end{bmatrix} = \frac{1}{2\alpha} \begin{bmatrix} \frac{b^2}{r} \\ \alpha + a \end{bmatrix} e^{-\alpha t} + \frac{1}{2\alpha} \begin{bmatrix} -\frac{b^2}{r} \\ \alpha - a \end{bmatrix} e^{\alpha t}, \quad t \geq 0. \quad (11)$$

Now consider the general initial condition $x(0) = r(0)$, $\lambda(0) = c$ for some constant c . The solution with these initial conditions is

$$\begin{aligned} \begin{bmatrix} x(t) \\ \lambda(t) \end{bmatrix} &= \begin{bmatrix} x_0(t) \\ \lambda_0(t) \end{bmatrix} + c \begin{bmatrix} x_1(t) \\ \lambda_1(t) \end{bmatrix} \\ &= \frac{1}{2\alpha} \begin{bmatrix} r(0)(\alpha - a) + \frac{cb^2}{r} \\ r(0)q + c(\alpha + a) \end{bmatrix} e^{-\alpha t} \\ &\quad + \frac{1}{2\alpha} \begin{bmatrix} r(0)(\alpha + a) - \frac{cb^2}{r} \\ -r(0)q + c(\alpha - a) \end{bmatrix} e^{\alpha t}, \quad t \geq 0. \end{aligned} \quad (12)$$

Now it remains only to determine the initial-costate value c so that boundary condition (9) holds. Evaluating (12) at $t = T$, and substituting $x(T)$ and $\lambda(T)$ into (9) yields the required initial-costate value of

$$\lambda(0) = c = \frac{r(0)[(q + sa) \sinh \alpha T + s\alpha \cosh \alpha T] - r(T)s\alpha}{(sb^2/r - a) \sinh \alpha T + \alpha \cosh \alpha T}. \quad (13)$$

Note that the initial costate is a linear combination of the initial and final states.

Using this value of c in (12) yields the optimal state and costate trajectories. Then (6) yields the optimal control. This method yields the optimal control as on open-loop control law, that is, as a function of time, not of the current state. ■

3.3 CONTINUOUS-TIME LINEAR QUADRATIC REGULATOR

Table 3.2-1 provides the optimal controller for general nonlinear systems, but explicit expressions for the control law are hard to compute. In this section we consider the linear time-varying plant

$$\dot{x} = A(t)x + B(t)u, \quad (3.3-1)$$

where $x \in R^n$, $u \in R^m$ with associated quadratic performance index

$$J(t_0) = \frac{1}{2}x^T(T)S(T)x(T) + \frac{1}{2} \int_{t_0}^T (x^T Q(t)x + u^T R(t)u) dt. \quad (3.3-2)$$

The time interval over which we are interested in the behavior of the plant is $[t_0, T]$. We shall determine the control $u^*(t)$ on $[t_0, T]$ that minimizes J for two cases: fixed final state and free final state. In the former case, u^* will turn out to be an open-loop control, and in the latter case a feedback control.

We assume in this section that the final time T is fixed and known, and that no function of the final state ψ is specified. The initial plant state $x(t_0)$ is given. Weighting matrices $S(T)$ and $Q(T)$ are symmetric and positive semi-definite, and $R(t)$ is symmetric and positive definite, for all $t \in [t_0, T]$.

Let us use Table 3.2-1 to write down the solution to this *linear quadratic regulator problem*.

The State and Costate Equations

The **Hamiltonian** is

$$H(t) = \frac{1}{2}(x^T Qx + u^T Ru) + \lambda^T(Ax + Bu), \quad (3.3-3)$$

where $\lambda(t) \in R^n$ is an undetermined multiplier. The state and costate equations are

$$\dot{x} = \frac{\partial H}{\partial \lambda} = Ax + Bu, \quad (3.3-4)$$

$$-\dot{\lambda} = \frac{\partial H}{\partial x} = Qx + A^T \lambda, \quad (3.3-5)$$

and the stationarity condition is

$$0 = \frac{\partial H}{\partial u} = Ru + B^T \lambda. \quad (3.3-6)$$

Solving (3.3-6) yields the optimal control in terms of the costate

$$u(t) = -R^{-1} B^T \lambda(t). \quad (3.3-7)$$

The control structure defined by these equations is identical to Fig. 2.2-1; however, the continuous LQ regulator cannot be implemented in this noncausal state–costate form.

Using (3.3-7) in the state equation yields the homogeneous *Hamiltonian system*.

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = \begin{bmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix}. \quad (3.3-8)$$

The coefficient matrix is called the continuous *Hamiltonian matrix*, which we discuss further in Section 3.4.

To find the optimal control, we must take into account the boundary conditions and solve (3.3-8). We shall presently do this for two special cases: fixed and free final state. First, it is instructive to investigate the value of the performance index $J(t_0)$ when the control input $u(t)$ is zero.

Zero-input Cost and the Lyapunov Equation

We want to determine the value of the performance index J if the plant control input $u(t)$ is zero. Suppose the $n \times n$ matrix function $S(t)$ is defined as the solution to the continuous *Lyapunov* equation

$$-\dot{S} = A^T S + S A + Q, \quad t \leq T, \quad (3.3-9)$$

with final condition $S(T)$ as given in (3.3-2). This equation is integrated backward in time from $t = T$. Then it is easy to show that the cost to go on any interval $[t, T]$ is given by

$$J(t) = \frac{1}{2} x^T(t) S(t) x(t), \quad (3.3-10)$$

where $x(t)$ is the current state t .

To wit, note that

$$\frac{1}{2} \int_{t_0}^T \frac{d}{dx} (x^T S x) dt = \frac{1}{2} x^T(T) S(T) x(T) - \frac{1}{2} x^T(t_0) S(t_0) x(t_0). \quad (3.3-11)$$

Now add zero, in the form of the left-hand side of (3.3-11) minus its right-hand side, to $J(t_0)$ in (3.3-2) to see that $u(t) = 0$

$$J(t_0) = \frac{1}{2}x^T(t_0)S(t_0)x(t_0) + \frac{1}{2} \int_{t_0}^T (\dot{x}^T Sx + x^T \dot{S}x + x^T S\dot{x} + x^T Qx) dt. \quad (3.3-12)$$

Taking into account the state equation (3.3-1) results in

$$J(t_0) = \frac{1}{2}x^T(t_0)S(t_0)x(t_0) + \frac{1}{2} \int_{t_0}^T (x^T(A^T S + \dot{S} + SA + Q)x) dt; \quad (3.3-13)$$

but $S(t)$ satisfies the Lyapunov equation, so that (3.3-10) follows since the current time t can be interpreted as the initial time of the remaining interval $[t, T]$.

This result allows us to compute, in terms of the known current state, the cost to go till time T of failing to apply any control to the plant. Note that $S(t)$ does not depend on the state, so it can be precomputed off-line and stored. Because of the form of (3.3-10), we call $S(t)$ the cost kernel function. The cost is just one-half the semi-norm squared of the state with respect to the weighting $S(t)$.

The solution to (3.3-9) is given by

$$S(t) = e^{A^T(T-t)} S(T) e^{A(T-t)} + \int_t^T e^{A^T(T-\tau)} Q e^{A(T-\tau)} d\tau, \quad (3.3-14)$$

which can be verified by Leibniz's rule. According to the Lyapunov stability theory, this converges to the steady-state value as $(T - t) \rightarrow \infty$ of

$$S_\infty = \int_0^\infty e^{A^T \tau} Q e^{A \tau} d\tau \quad (3.3-15)$$

if the plant is asymptotically stable. In this event, the cost over any interval $[t, \infty]$, is given by the steady-state cost (3.3-10) with $S(t)$ replaced by S_∞ , which is finite. If A is unstable and $(A, \sqrt{Q})$ is observable, where $Q = \sqrt{Q^T} \sqrt{Q}$, then the cost tends to infinity as the time interval grows.

In the steady-state case, $\dot{S} = 0$, so that (3.3-9) becomes the algebraic Lyapunov equation

$$0 = A^T S + SA + Q. \quad (3.3-16)$$

If A is stable, then (3.3-15) is a positive semi-definite solution to (3.3-16). If $(A, \sqrt{Q})$ is observable, the steady-state solution S_∞ is positive definite and is the unique positive definite solution to the algebraic Lyapunov equation.

See the discussion on the discrete time counterparts to these results in Section 2.2.

We shall soon see that the optimal closed-loop control depends on an equation like (3.3-9), but with an extra term to account for the effect of the input.

Example 3.3-1. Propagation of Cost for Uncontrolled Scalar System

Let

$$\dot{x} = ax \quad (1)$$

be an uncontrolled scalar system with cost on $[t, T]$ defined by

$$J(t) = \frac{1}{2}S(T)x^2(T) + \frac{1}{2} \int_t^T qx^2(\tau) d\tau. \quad (2)$$

The Lyapunov equation is

$$-\dot{s} = 2as + q, \quad (3)$$

with solution

$$s(t) = e^{2a(T-t)}S(T) + \int_t^T e^{2a(T-\tau)}q d\tau \quad (4)$$

or

$$s(t) = \left(s(T) + \frac{q}{2a}\right)e^{2a(T-t)} - \frac{q}{2a}. \quad (5)$$

If $a < 0$, then as $(T - t) \rightarrow \infty$, $s(t)$ converges to the steady-state value of

$$s_\infty = -\frac{q}{2a} > 0. \quad (6)$$

Note that this is the solution to the algebraic Lyapunov equation

$$0 = 2as + q. \quad (7)$$

If a is unstable, then $s(t)$ grows without bound as the time interval of interest grows.

The steady-state cost on $[0, \infty]$ of applying no control input to the plant (1) is

$$J_\infty = \frac{1}{2}s_\infty x^2(0) = -\frac{q}{4a}x^2(0) \quad (8)$$

when $a < 0$; otherwise it is infinite. In neither case does it depend on the final-state weighting $s(T)$. ■

Fixed-final-state and Open-loop Control

Let us now return to the problem of determining the control required in (3.3-1) to minimize the cost (3.3-2). The state and costate equations are given by (3.3-8) and the optimal control is given by (3.3-7). It remains only to solve (3.3-8) given the boundary conditions.

Suppose that the initial state is known to be $x(t_0)$ and that the control objective is to drive the state *exactly* to the given fixed reference value of $r(T)$ at the final time. Then the final condition is

$$x(T) = r(T). \quad (3.3-17)$$

Since $dx(T) = 0$ and $dT = 0$, condition (3.2-10) is automatically satisfied.

Since $x(T)$ is fixed at $r(T)$, it is redundant to include a final-state weighting in the cost index, so let $S(T) = 0$. To allow us to get an analytic solution, let $Q = 0$ also. Then the cost function is

$$J(t_0) = \frac{1}{2} \int_{t_0}^T u^T R u dt, \quad (3.3-18)$$

and so we are trying to find a control that drives $x(t_0)$ to $x(T) = r(T)$ using minimum control energy.

The state and costate equations are now

$$\dot{x} = Ax - BR^{-1}B^T\lambda, \quad (3.3-19)$$

$$\dot{\lambda} = -A^T\lambda. \quad (3.3-20)$$

Setting $Q = 0$ has decoupled the costate equation from the state equation, so its solution is just

$$\lambda(t) = e^{A^T(T-t)}\lambda(T), \quad (3.3-21)$$

where $\lambda(T)$ is still unknown. Using this expression in the state equation yields

$$\dot{x} = Ax - BR^{-1}B^T e^{A^T(T-t)}\lambda(T), \quad (3.3-22)$$

whose solution is

$$x(t) = e^{A(t-t_0)}x(t_0) = \int_{t_0}^t e^{A(t-\tau)}BR^{-1}B^T e^{A^T(T-\tau)}\lambda(T) d\tau. \quad (3.3-23)$$

To find $\lambda(T)$, evaluate this at $t = T$ to get

$$x(T) = e^{A(T-t_0)}x(t_0) - G(t_0, T)\lambda(T), \quad (3.3-24)$$

where the weighted *continuous reachability gramian* is

$$G(t_0, T) = \int_{t_0}^T e^{A(T-\tau)}BR^{-1}B^T e^{A^T(T-\tau)} d\tau. \quad (3.3-25)$$

According to final condition (3.3-17), then,

$$\lambda(T) = -G^{-1}(t_0, T)[r(T) - e^{A(T-t_0)}x(t_0)]. \quad (3.3-26)$$

Finally, the optimal control can be written using (3.3-7), (3.3-21), and (3.3-26) as

$$u^*(t) = R^{-1}B^T e^{A^T(T-t)}G^{-1}(t_0, T)[r(T) - e^{A(T-t_0)}x(t_0)]. \quad (3.3-27)$$

This is our result; it is the **minimum-energy control** that drives the given initial state $x(t_0)$ to the desired final reference value of $x(T) = r(T)$. Note that

$$x(T) = e^{A(T-t_0)}x(t_0) \quad (3.3-28)$$

is the **final state in the absence of an input**, so the **optimal control** is proportional to the **difference between this homogeneous solution and the desired final state**.

Since $u^*(t)$ is found by using $G(t_0, T)$, the optimal control exists for arbitrary $x(t_0)$ and $r(T)$ if and only if $|G(t_0, T)| \neq 0$. This corresponds to **reachability** of the plant. If (A, B) is **reachable**, there exists a **minimum-energy control** that drives any $x(t_0)$ to any desired $r(T)$.

The control (3.3-27) is an **open-loop control**, since $u^*(t)$ does not depend on the current state $x(t)$. It depends only on the initial and final states, and it is **precomputed** and then applied for all t in $[t_0, T]$. If, for some reason, the state is perturbed off the predicted optimal trajectory, then such an open-loop control will not, in general, result in $x(T) = r(T)$ as desired.

To compute the **reachability gramian in practice**, we do not need to do the integration (3.3-25), which can be very messy. The solution to the **Lyapunov equation** is

$$\dot{P} = AP + PA^T + BR^{-1}B^T, \quad t > t_0, \quad (3.3-29)$$

$$P(t) = e^{A(t-t_0)}P(t_0)e^{A^T(t-t_0)} + \int_{t_0}^t e^{A(t-\tau)}BR^{-1}B^Te^{A^T(t-\tau)}d\tau.$$

Hence, if $P(t_0) = 0$, then $G(t_0, t) = P(t)$.

To determine $u^*(t)$, then, we would first solve (3.3-29) off-line to get $G(t_0, T)$. This can be done numerically using a Runge-Kutta integrator (Appendix B.1). Then for each $t \in [t_0, T]$, we would use (3.3-27) to find $u^*(t)$, which is then applied to the plant (3.3-1).

Compare the “**reachability Lyapunov equation**” (3.3-29) to the “**observability Lyapunov equation**” (3.3-9). The former describes the interaction between **plant and input**, and the latter describes the interaction between **plant and cost function** when $u(t) = 0$.

It is a simple matter to determine the value of the cost index (3.3-18) under the influence of the optimal control (3.3-27). Representing the **final-state difference** as

$$d(t_0, T) = r(T) - e^{A(T-t_0)}x(t_0), \quad (3.3-30)$$

we have

$$J^*(t_0) = \frac{1}{2} \int_{t_0}^T d^T G^{-1} e^{A(T-t)} BR^{-1} R R^{-1} B^T e^{A^T(T-t)} G^{-1} d dt,$$

where we have used the symmetry of $G^{-1}(t_0, T)$ and R^{-1} . Realizing that $d(t_0, T)$ and $G^{-1}(t_0, T)$ do not depend on t and using the definition of the gramian yields

$$J^*(t_0) = \frac{1}{2} d^T(t_0, T) G^{-1}(t_0, T) d(t_0, T), \quad (3.3-31)$$

or

$$J^*(t_0) = \frac{1}{2} d^T(t_0, T) P^{-1}(T) d(t_0, T), \quad (3.3-32)$$

where $P(t)$ satisfies (3.3-29). Compare this with (3.3-10).

Example 3.3-2. Open-loop Control of a Scalar System

Let the scalar plant be

$$\dot{x} = ax + bu, \quad t \geq 0, \quad (1)$$

with cost

$$J(0) = \frac{1}{2} \int_0^T r u^2 dt. \quad (2)$$

The Lyapunov equation (3.3-29) is

$$\dot{p} = 2ap + \frac{b^2}{r}, \quad p(0) = 0, \quad (3)$$

so the reachability gramian on $[0, t]$ is

$$G(0, t) = p(t) = \frac{b^2}{r} \int_0^t e^{2a(t-\tau)} d\tau$$

or

$$G(0, t) = \frac{b^2}{2ar} (e^{2at} - 1). \quad (4)$$

Compare this to the solution of the **Lyapunov equation** in Example 3.3-1.

According to (3.3-27), the optimal control taking $x(0)$ to $x(T) = r(T)$ for a given $r(T)$ is

$$\begin{aligned} u(t) &= \frac{b}{r} e^{a(T-t)} \cdot \frac{2ar}{b^2(e^{2aT} - 1)} (r(T) - e^{aT} x(0)) \\ &= \frac{a}{b} \frac{e^{-at}}{\sinh aT} (r(T) - e^{aT} x(0)). \end{aligned} \quad (5)$$

Interestingly enough, this is independent of the control weighting r . Compare (5) to the control (20) in Example 3.2-3, which was found by direct solution of the state and costate equations. ■

Example 3.3-3. Open-loop Control of Motion Obeying Newton's Laws

A particle obeying Newton's laws satisfies

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u \quad (1)$$

where $x = [d \ v]^T$ with $d(t)$ the position, $v(t)$ the velocity, and $u(t)$ an acceleration input. It is easy to find an analytic expression for the control required to drive any given $x(0)$ to any desired $x(T)$, while minimizing

$$J(0) = \frac{1}{2} \int_0^T r u^2 dt. \quad (2)$$

To find the reachability gramian, we solve the Lyapunov equation (3.3-29). Let

$$P(t) = \begin{bmatrix} p_1(t) & p_2(t) \\ p_2(t) & p_3(t) \end{bmatrix} \quad (3)$$

Then (3.3-29) is

$$\dot{P} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} P + P \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1/r \end{bmatrix}, \quad (4)$$

which yields the scalar equations

$$\dot{p}_1 = 2p_2, \quad (5)$$

$$\dot{p}_2 = p_3, \quad (6)$$

$$\dot{p}_3 = 1/r. \quad (7)$$

For the gramian, we integrate (7), (6), and then (5) with $P(0) = 0$ to get

$$p_3 = \frac{t}{r}, \quad (8)$$

$$p_2 = \frac{t^2}{2r}, \quad (9)$$

$$p_1 = \frac{t^3}{3r}, \quad (10)$$

so that

$$G(0, t) = P(t) = \begin{bmatrix} \frac{t^3}{3r} & \frac{t^2}{2r} \\ \frac{t^2}{2r} & \frac{t}{r} \end{bmatrix}. \quad (11)$$

The state transition matrix is

$$e^{At} = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}. \quad (12)$$

To find the optimal control, we use (3.3-27), which becomes

$$u(t) = \frac{1}{r} [T - t \quad 1] \begin{bmatrix} \frac{12r}{T^3} & -\frac{6r}{T^2} \\ -\frac{6r}{T^2} & \frac{4r}{T} \end{bmatrix} \left(x(T) - \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x(0) \right), \quad (13)$$

or

$$u(t) = \begin{bmatrix} \frac{6T-12t}{T^3} & \frac{-2T+6t}{T^2} \end{bmatrix} \left(x(T) - \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x(0) \right). \quad (14)$$

Once again, since $u(t)$ is a scalar, it is independent of r . Note also that the control magnitude decreases as the control interval $[0, T]$ increases. More control is required to move the system more quickly from one state to another. ■

Free-final-state and Closed-loop Control

We can find an optimal control law in the form of a state feedback by changing our control objectives for the plant (3.3-1). Instead of fixing the final state at a desired final value, let us require only that the control minimize the performance index (3.3-2). Thus, the final state is *free*, and its value can be varied in the optimization process implicit in the solution presented in Table 3.2-1.

The state and costate equations (3.3-8) are reproduced here for convenience:

$$\dot{x} = Ax - BR^{-1}B^T\lambda, \quad (3.3-33)$$

$$-\dot{\lambda} = Qx + A^T\lambda. \quad (3.3-34)$$

The control input is

$$u(t) = -R^{-1}B^T\lambda. \quad (3.3-35)$$

The given initial state is $x(t_0)$, and the final state $x(T)$ is free. Thus, $dx(T) \neq 0$, and $dT = 0$ (the final time is fixed and known here) in (3.2-10), so the coefficient of $dx(T)$ must be zero:

$$\lambda(T) = \left. \frac{\partial \phi}{\partial x} \right|_T = S(T)x(T). \quad (3.3-36)$$

This is the terminal condition.

To solve the two-point boundary-value problem specified by (3.3-33) and (3.3-34), given $x(t_0)$ and (3.3-36), we shall use the *sweep method* (Bryson and Ho 1975). Thus, assume that $x(t)$ and $\lambda(t)$ satisfy a linear relation like (3.3-36) for all $t \in [t_0, T]$ for some as yet unknown matrix function $S(t)$:

$$\lambda(t) = S(t)x(t). \quad (3.3-37)$$

If we can find such a $S(t)$, then this assumption is valid.

To find the intermediate function $S(t)$, differentiate the costate to get

$$\dot{\lambda} = \dot{S}x + S\dot{x} = \dot{S}x + S(Ax - BR^{-1}B^TSx), \quad (3.3-38)$$

where we have used the state equation. Now, taking into account the costate equation, we must have

$$-\dot{S}x = (A^T S + SA - SBR^{-1}B^T S + Q)x \quad (3.3-39)$$

for all t . Since this holds for all state trajectories given any $x(t_0)$, it is necessary that

$$-\dot{S} = A^T S + SA - SBR^{-1}B^T S + Q, \quad t \leq T. \quad (3.3-40)$$

This is a matrix *Riccati equation*, and if $S(t)$ is its solution with final condition $S(T)$, then (3.3-37) holds for all $t \leq T$. Our assumption was evidently a good one.

In terms of the Riccati-equation solution, the optimal control is given by (3.3-35) and (3.3-37) as

$$u(t) = -R^{-1}B^T Sx(t). \quad (3.3-41)$$

Defining the *Kalman gain* as

$$K(t) = R^{-1}B^T S(t), \quad (3.3-42)$$

we have

$$u(t) = -K(t)x(t). \quad (3.3-43)$$

The optimal control is determined by solving the Riccati equation (3.3-40) *backward* in time for $S(t)$. This can be done offline *before* the control run since $x(t)$ is not required to find $S(t)$. The gain $K(t)$ can be computed and stored. Finally, during the control run, $u^*(t)$ is found using (3.3-43) and applied to the plant.

The continuous optimal LQ regulator is summarized in Table 3.3-1. A block diagram of this scheme has the same structure shown in Figure 2.2-3.

In terms of the Kalman gain, the Riccati equation can be written

$$-\dot{S} = A^T S + SA - K^T R K + Q. \quad (3.3-44)$$

The control (3.3-43) is a *time-varying state feedback*, since even if A , B , Q , and R are time invariant, $K(t)$ varies with time. The closed-loop plant is

$$\dot{x} = (A - BK)x, \quad (3.3-45)$$

and this equation can be used to find the optimal state trajectory $x^*(t)$ given any $x(t_0)$.

In terms of the closed-loop plant matrix, the Riccati equation can be written in the *Joseph-stabilized formulation*

$$-\dot{S} = (A - BK)^T S + S(A - BK) + K^T R K + Q, \quad t \leq T. \quad (3.3-46)$$

TABLE 3.3-1 Continuous Linear Quadratic Regulatory (Final State Free)*System model:*

$$\dot{x} = Ax + Bu, \quad t \geq t_0$$

Performance index:

$$J(t_0) = \frac{1}{2}x^T(T)S(T)x(T) + \frac{1}{2} \int_{t_0}^T (x^T Q x + u^T R u) dt$$

Assumptions:

$$S(T) \geq 0, Q \geq 0, R > 0, \quad \text{with all three symmetric}$$

Optimal feedback control:

$$-\dot{S} = A^T S + SA - SBR^{-1}B^T S + Q, \quad t \leq T, \text{ given } S(T)$$

$$K = R^{-1}B^T S$$

$$u = -Kx$$

$$J^*(t_0) = \frac{1}{2}x^T(t_0)S(t_0)x(t_0)$$

By a derivation like the ones leading to (3.3-10) and (2.2-69) we can show that the cost on any interval $[t, T]$ satisfies

$$J(t) = \frac{1}{2}x^T(t)S(t)x(t) + \frac{1}{2} \int_t^T \|R^{-1}B^T Sx + u\|_R^2 dt, \quad (3.3-47)$$

where $S(t)$ is the solution to the Riccati equation.

If we now select the optimal control (3.3-43), then the value of the performance index on $[t, T]$ is just

$$J(t) = \frac{1}{2}x^T(t)S(t)x(t). \quad (3.3-48)$$

This result is important since, if we know the current state $x(t)$, then by solving the Riccati equation we can determine the optimal cost of controlling the plant on $[t, T]$ before we apply the control or even compute it! If this cost is too high, we should select another control scheme, or at least change the weighting matrices $S(T)$, Q , and R and find a new feedback gain $K(t)$.

If $B = 0$, then the Riccati equation reduces to the zero-input Lyapunov equation (3.3-9).

Note from (3.3-47) that

$$\frac{\partial^2 J}{\partial u^2} = R, \quad (3.3-49)$$

so that the curvature matrix in the continuous case is R . Since $R > 0$, the optimal control minimizes $J(t_0)$. In the discrete case, the curvature matrix is

$(B^T S_{k+1} B + R)$, where S_k satisfies the discrete Riccati equation. This accounts for the simplicity of the continuous LQ regulator as compared to the discrete LQ regulator.

By selecting $S(T)$ very large, we can guarantee that the optimal control will drive $x(T)$ very close to zero to keep $J(t_0)$ small. In the limit as $S(T) \rightarrow \infty$, it can be shown that the control scheme in Table 3.3-1 tends to the fixed-final-state scheme (3.3-27) for the case $r(T) = 0$. See the discussion in Section 2.2.

It should be clearly realized that reachability of the plant is *not* required for the free-final-state LQ regulator. Even if (A, B) is not reachable, $u^*(t)$ will do its best to keep $J(t_0)$ small. If (A, B) is reachable, it can be expected to do a better job. In fact, we shall see in Section 3.4 that reachability results in some very desirable properties as the control interval $[t_0, T]$ becomes large.

A few examples will impart some intuition on the LQ regulator. First, let us briefly discuss a software implementation of Table 3.3-1.

Software Implementation of the LQ Regulator

In the discrete case, the Riccati equation is a simple backward recursion that can easily be programmed, as we have seen. In the continuous case, however, the Riccati equation must be *integrated backward*. Most Runge-Kutta integration routines run forward in time. The best policy is therefore to convert (3.3-40) into an equation that is integrated forward. This is easy to do.

Changing variables by

$$\tau = T - t, \quad (3.3-50)$$

we have $d\tau = -dt$, so the Riccati equation becomes (in the time-invariant case)

$$\dot{S}_b = A^T S_b + S_b A - S_b B R^{-1} B^T S_b + Q, \quad (3.3-51)$$

where

$$S(t) = S_b(T - t). \quad (3.3-52)$$

All we must do to solve (3.3-40) is to integrate it *forward from $t = 0$ without the minus sign* on its left-hand side, then reverse the resulting solution and shift it to $t = T$.

The control scheme in Table 3.3-1 has two parts. The first is the *control law computation* by backward integration of the Riccati equation to find $S(t)$ and then $K(t)$. Only $K(t)$ must be stored. This integration we have just discussed.

The second part is the *simulation of the control law* found in part one by applying $u = -K(t)x$ to the plant. This is accomplished by a forward integration of the state equation $\dot{x} = Ax + Bu$.

The complete simulation procedure is shown in Fig. 3.3-1. The simulation portion can be compared with Fig. 2.3-1.

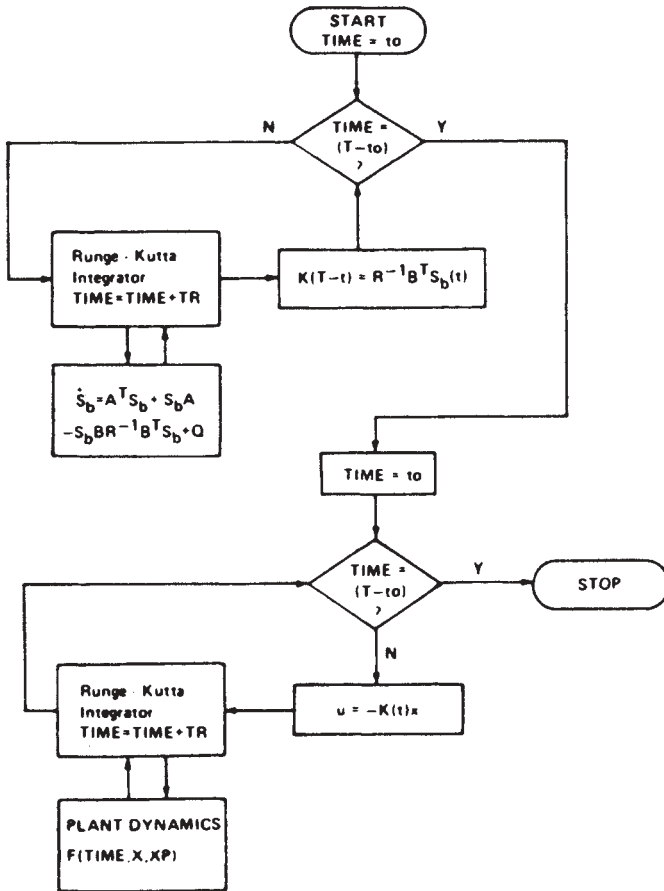


FIGURE 3.3-1 Continuous LQ regulator simulation procedure.

Examples and Exercises

Example 3.3-4. Optimal Feedback Control of a Scalar System

This is the continuous counterpart of Example 2.2-3. Let the scalar plant be

$$\dot{x} = ax + bu \quad (1)$$

with performance index

$$J(t_0) = \frac{1}{2} s(T) x^2(T) + \frac{1}{2} \int_{t_0}^T (qx^2 + ru^2) dt. \quad (2)$$

a. Analytic Solution

The Riccati equation is

$$-\dot{s} = 2as + q - \frac{b^2 s^2}{r}, \quad t \leq T. \quad (3)$$

Using separation of variables we have

$$\int_{s(t)}^{s(T)} \frac{ds}{(b^2/r)s^2 - 2as - q} = \int_t^T dt,$$

and integrating yields

$$s(t) = s_2 + \frac{s_1 + s_2}{[(s(T) + s_1)/(s(T) - s_2)]e^{2\beta(T-t)} - 1} \quad (4)$$

where

$$\beta = \sqrt{a^2 + \frac{b^2 q}{r}}, \quad (5)$$

$$s_1 = \frac{r}{b^2}(\beta - a), \quad s_2 = \frac{r}{b^2}(\beta + a). \quad (6)$$

The steady-state value as $(T - t) \rightarrow \infty$ is given by s_2 or, if $a > 0$,

$$s_\infty = \frac{q}{\gamma} \left(1 + \sqrt{1 + \frac{\gamma}{a}} \right) \quad (7)$$

where

$$\gamma = b^2 q / ar \quad (8)$$

is a *control-effectiveness-to-plant-inertia* ratio. If $a < 0$, a similar expression holds. The steady-state value is independent of the final-state weighting $s(T)$. It is also bounded if $b \neq 0$, which corresponds to reachability, even if a is unstable. Contrast this with Example 3.3-1.

b. No Intermediate-state Weighting

Let us consider the rather interesting special case of $q = 0$. Then $\beta = |a|$ and

$$s(t) = \frac{s(T)}{b^2 s(T)/2ar + (1 - b^2 s(T)/2ar)e^{-2a(T-t)}}. \quad (9)$$

If we want to ensure that the optimal control drives $x(T)$ exactly to zero, we can let $s(T) \rightarrow \infty$ to weight $x(T)$ more heavily in $J(t_0)$. In this limit we have

$$s(t) = \frac{2ar/b^2}{1 - e^{-2a(T-t)}}, \quad (10)$$

so the optimal control is $(K(t) = bs(t)/r)$

$$u(t) = -K(t)x(t) = -\frac{2a/b}{1 - e^{-2a(T-t)}}x(t)$$

or

$$u(t) = -\frac{a}{b} \frac{e^{a(T-t)}}{\sinh a(T-t)} x(t). \quad (11)$$

Compare this with the fixed-final-state control in Example 3.3-2 for the case $r(T) = 0$. We have just discovered a *feedback* formulation of that control law.

If the plant is stable, $a < 0$, then the steady-state value $((T - t) \rightarrow \infty)$ of the cost kernel is

$$s_{\infty} = 0, \quad (12)$$

so that the steady-state closed-loop system

$$\dot{x} = (a - bK)x = ax \quad (13)$$

is stable. On the other hand, if $a > 0$, then

$$s_{\infty} = \frac{2ar}{b^2}, \quad (14)$$

and the steady-state closed-loop system is

$$\dot{x} = \left(a - \frac{b^2}{r}s_{\infty} \right) x = -ax. \quad (15)$$

This is still stable.

Figure 3.3-2 shows the behavior of the general solution (4) for the case of stable and unstable plant.

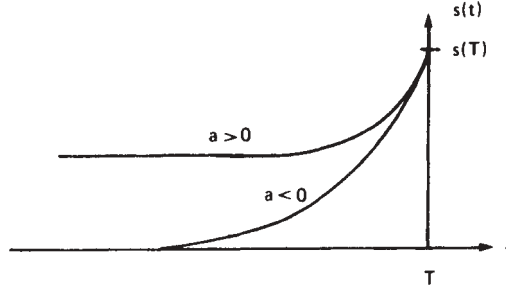


FIGURE 3.3-2 Limiting behavior of the Riccati-equation solution.

c. Simulation

To implement the LQ regulator, none of the analysis subsequent to (3) is required. The Kalman gain is

$$K(t) = \frac{b}{r}s(t), \quad (16)$$

and the complete LQ regulator is shown in Fig. 3.3-3. First, the Riccati equation (3) is integrated backward to get $s(t)$. This is accomplished by integrating

$$\dot{s}_b = 2as_b + q - b^2s_b^2/r \quad (17)$$

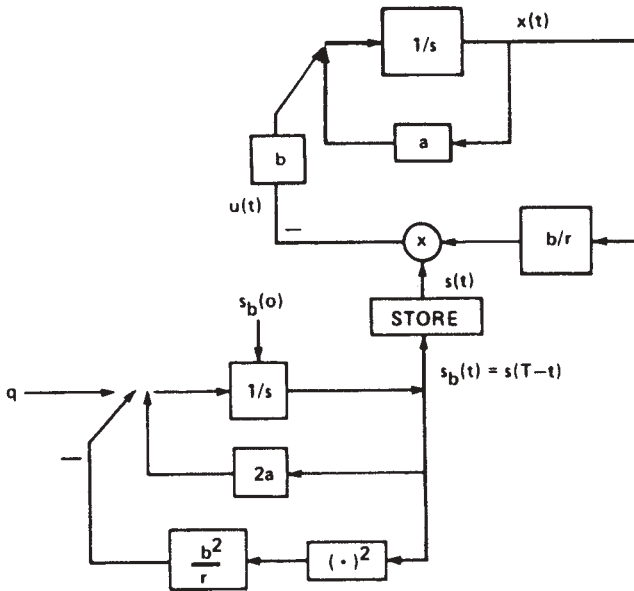


FIGURE 3.3-3 Scalar continuous LQ regulator.

forward from zero with $s_b(0) = s(T)$ and then using

$$s(t) = s_b(T - t). \quad (18)$$

The kernel $s(t)$ is stored and then used to compute $u(t)$ that takes the form

$$u = -K(t)x \quad (19)$$

as the plant dynamics are integrated in the simulation. Note that the Riccati-equation system is a sort of “doubled” or “squared” version of the plant. ■

Example 3.3-5. Optimal Feedback Control of a Damped Harmonic Oscillator

Let the plant be

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\delta\omega_n \end{bmatrix} x + \begin{bmatrix} 0 \\ b \end{bmatrix} u \quad (1)$$

with performance index

$$\begin{aligned} J(t_0) = & \frac{1}{2} x^T(T) \begin{bmatrix} s_1(T) & 0 \\ 0 & s_3(T) \end{bmatrix} x(T) \\ & + \frac{1}{2} \int_{t_0}^T \left(x^T \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix} x + ru^2 \right) dt. \end{aligned} \quad (2)$$

If we let

$$S(t) \triangleq \begin{bmatrix} s_1(t) & s_2(t) \\ s_2(t) & s_3(t) \end{bmatrix} \quad (3)$$

and simplify the Riccati equation, we get the coupled scalar differential equations

$$-\dot{s}_1 = -2\omega_n^2 s_2 - \frac{b^2}{r} s_2^2 + q_1, \quad (4)$$

$$-\dot{s}_2 = s_1 - 2\delta\omega_n s_2 - \omega_n^2 s_3 - \frac{b^2}{r} s_2 s_3, \quad (5)$$

$$-\dot{s}_3 = 2s_2 - 4\delta\omega_n s_3 - \frac{b^2}{r} s_3^2 + q_2. \quad (6)$$

Writing the optimal feedback gain as

$$K(t) = [k_1(t) \quad k_2(t)], \quad (7)$$

we have $K = R^{-1}B^T S$ or

$$k_1 = \frac{bs_2}{r}, \quad (8)$$

$$k_2 = \frac{bs_3}{r}. \quad (9)$$

```
function [x, u, Sf, tf] = ex3_3_5(a, b, r, x0)
% Control of a Harmonic Oscillator
% Compute the solution to Riccati Equation
[tb, S]=ode45(@fex3_3_5,[-10:0.1:0],zeros(3,1));
% Compute Optimal Feedback Gains
Sf=flipud(S);
tf=-flipud(tb);
K=-b/r*Sf(:,2:3);
x(:,1)=x0;
u(1)=K(1,:)*x(:,1) ;
% compute Closed-loop Response
for k=1:length(tf)-1,
% Harmonic Oscillator System State Equations
x(:,k+1) =expm((a+[0; b]*K(k,:))*(tf(k+1)-tf(k)))*x(:,k);
u(k+1)=K(k+1,:)*x(:,k+1);
end

function sd=fex3_3_5(t,s)
q=1*eye(2); om=0.8; del=0.1; b=1; r=1;
sd =[-2*om^2*s(2)-b^2*s(2)^2+q(1,1);
s(1)-4*del*om*s(2)-om^2*s(3)-b^2*s(2)*s(3);
2*s(2)-4*del*om*s(3)-b^2*s(3)^2+q(2,2)];
```

FIGURE 3.3-4 MATLAB code to use for the control of a harmonic oscillator.

The optimal control is then

$$u = -Kx = -k_1x_1 - k_2x_2, \tag{10}$$

where $x = [x_1 \ x_2]^T$.

a. Software Implementation

To implement the optimal LQ controller, we need subroutines to describe the Riccati-equation dynamics (4)–(6) (without the minus signs on the left-hand side of the equalities) and to compute the gains (8)–(9). These are used in the control law computation. For the forward integration to simulate the control law, we need subroutines to compute the control (10) and provide the plant dynamics (1). The MATLAB code is shown in Fig. 3.3-4.

Using this software, the optimal state trajectories and controls for several values of $q = q_1 = q_2$ were plotted (see Fig. 3.3-5). Also shown are the cost kernel elements for two values of q .

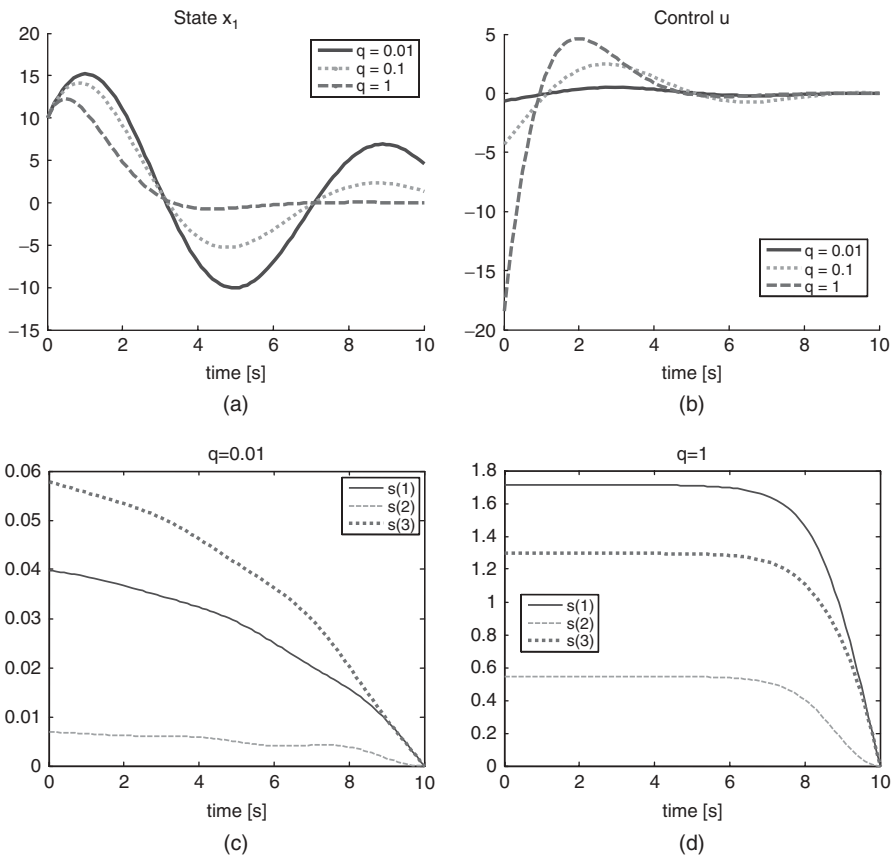


FIGURE 3.3-5 Results of a harmonic oscillator simulation using MATLAB. (a) State trajectories. (b) Optimal control inputs $u(t)$. (c) Riccati solutions $s(t)$ for $q = 0.01$. (d) Riccati solutions for $q = 1.0$.

b. Steady-state Riccati Solution

Equations (4)–(6) are difficult to solve, but a steady-state solution is easy to obtain. Letting $\dot{s}_1 = \dot{s}_2 = \dot{s} = 0$, three algebraic equations are obtained. These can be solved to give

$$s_2(\infty) = \frac{q_1}{\gamma_1} \left(\sqrt{1 + \frac{\gamma_1}{\omega_n^2}} - 1 \right), \quad (11)$$

$$s_3(\infty) = \frac{q_2}{\gamma_2} \left(\sqrt{1 + \frac{\gamma_2}{2\delta\omega_n} \left(1 + \frac{2s_2}{q_2} \right)} - 1 \right), \quad (12)$$

$$s_1(\infty) = \frac{b^2}{r} \left(\frac{q_2}{\gamma_2} s_2 + \frac{q_1}{\gamma_1} s_3 + s_2 s_3 \right), \quad (13)$$

where

$$\gamma_1 \triangleq \frac{b^2 q_1}{\omega_n^2 r}, \quad (14)$$

$$\gamma_2 \triangleq \frac{b^2 q_2}{2\delta\omega_n r} \quad (15)$$

are “control effectiveness” ratios. In solving for the steady-state kernel, we select positive square roots, since $S(\infty) \geq 0$. ■

Exercise 3.3-6. LQ Regulator with Weighting of State/Input Inner Product

This is the continuous-time counterpart to Exercise 2.2-4. Let the plant

$$\dot{x} = Ax + Bu \quad (1)$$

have the modified performance index

$$J(t_0) = \frac{1}{2} x^T(T) S(T) x(T) + \frac{1}{2} \int_{t_0}^T \begin{bmatrix} x^T & u^T \end{bmatrix} \begin{bmatrix} Q & V \\ V^T & R \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix}, \quad (2)$$

where the block matrix is positive semidefinite and $R > 0$.

a. Define a modified Kalman gain as

$$K = R^{-1}(V^T + B^T S), \quad (3)$$

where $S(t)$ is the solution to the Riccati equation

$$-\dot{S} = A^T S + SA - K^T R K + Q. \quad (4)$$

(Compare (4) with (3.3-44).) Show that the optimal control is

$$u(t) = -K(t)x(t). \quad (5)$$

b. Show that the optimal remaining cost on any subinterval $[t, T]$ is

$$J(t) = \frac{1}{2}x^T(t)S(t)x(t). \quad (6)$$

In summary, if the cost index contains a weighting V that picks up the state-input inner product, the only required modification to the LQ regulator is that the Kalman gain must be modified, and the Riccati-equation formulation (4) should be used. ■

3.4 STEADY-STATE CLOSED-LOOP CONTROL AND SUBOPTIMAL FEEDBACK

In this section we present the continuous counterparts to the results in Section 2.4. It would be a very good exercise to fill in the derivations of these equations, both to become more familiar with them and to compare the continuous and discrete situations, which have subtle and interesting distinctions.

Suboptimal Feedback Gains

If the plant

$$\dot{x} = Ax + Bu \quad (3.4-1)$$

has the feedback

$$u = -Kx, \quad (3.4-2)$$

then the closed-loop plant becomes

$$\dot{x} = (A - BK)x. \quad (3.4-3)$$

The optimal feedback gain $K(t)$ is time varying and depends on the Riccati-equation solution as in Table 3.3-1.

If the gain K in (3.4-2) is arbitrary, then the resulting cost on $[t, T]$ for any t is

$$J(t) = \frac{1}{2}x^T(t)S(t)x(t), \quad (3.4-4)$$

where $S(t)$ satisfies

$$-\dot{S} = (A - BK)^T S + S(A - BK) + K^T R K + Q, \quad t \leq T, \quad (3.4-5)$$

with $S(T)$ equal to the final-state weighting.

If $K(t)$ is given, then (3.4-5) is a *Lyapunov equation* in terms of the closed-loop plant matrix. If $K(t)$ is the optimal gain in Table 3.3-1, then (3.4-5) is the *Joseph stabilized Riccati equation*.

If K in (3.4-2) is selected as a *constant matrix*, then the cost given by (3.4-4) can be examined to see if it is reasonable. If it is, and if the plant behavior is satisfactory in a simulation run, then the constant feedback can be used on the actual plant.

The Algebraic Riccati Equation

In this subsection we assume time-invariant plant and weighting matrices.

As $(T - t)$ goes to infinity, the solution to the Riccati equation can exhibit several types of behavior. It can be unbounded, or it can converge to a limiting solution $S(\infty)$, which can be zero, positive semi-definite, or positive definite.

If $S(t)$ does converge, then for $t \ll T$, $\dot{S} = 0$, so that there results in the limit the *algebraic Riccati equation* (ARE)

$$0 = A^T S + SA - SBR^{-1}B^T S + Q. \quad (3.4-6)$$

The ARE can have several solutions, and these may be real or complex, positive definite, negative definite, etc. If $S(T)$ is symmetric, then the Riccati solution $S(t)$ is symmetric and at least positive semi-definite for all $t \leq T$. $S(\infty)$ is always a solution to the ARE, but all ARE solutions are not limiting Riccati solutions for some $S(T)$.

It is worth mentioning that a real solution to

$$0 = A^T S^T + SA - SBR^{-1}B^T S^T + Q, \quad (3.4-7)$$

where S is not required to be symmetric, is given by

$$S = \sqrt{A^T(BR^{-1}B^T)^{-1}A + QL} \sqrt{(BR^{-1}B^T)^{-1} + A^T(BR^{-1}B^T)^{-1}}, \quad (3.4-8)$$

where L is any orthogonal matrix (i.e., $LL^T = I$). Of course, this assumes $|BR^{-1}B^T| \neq 0$. See Schultz and Melsa (1967). In the scalar case, this reduces to the well-known formula for solving a quadratic equation! (Show that for S to be symmetric, the matrix L must satisfy a Lyapunov equation.)

If $S(\infty)$ exists, then the corresponding steady-state feedback gain is

$$K(\infty) = R^{-1}B^T S(\infty). \quad (3.4-9)$$

Under some circumstances it may be acceptable to use the time-invariant feedback law (3.4-2) with a gain of $K(\infty)$ as an alternative to the time-varying optimal feedback. The suboptimal cost associated with this control law is given by (3.4-4), where K in (3.4-5) is $K(\infty)$.

To examine the consequences of this simplified control strategy, let us discuss the limiting behavior of the closed-loop system (3.4-3) using the optimal feedback in Table 3.3-1.

Limiting Behavior of the Riccati-equation Solution

This subsection applies only for time-invariant plant and cost matrices. It is worthwhile reviewing the discrete case discussed in Section 2.4, as we shall not repeat the comments designed to impart some insight and motivation for our work.

The plant is (3.4-1) with cost index in Table 3.3-1. By defining a *fictitious output*

$$y(t) = \begin{bmatrix} C \\ 0 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ D \end{bmatrix} u(t), \quad (3.4-10)$$

where C and D are defined as any matrices such that $Q = C^T C$ and $R = D^T D$, we can write the cost as

$$J(t_0) = \frac{1}{2} x^T(T) S(T) x(T) + \frac{1}{2} \int_{t_0}^T y^T y \, dt. \quad (3.4-11)$$

We should first like to know when there is a finite limiting solution $S(\infty)$ to the Riccati equation. The next theorem gives us the answer.

Theorem 3.4-1. Let (A, B) be stabilizable. Then for every $S(T)$ there is a bounded limiting solution $S(\infty)$ to the Riccati equation. Furthermore, $S(\infty)$ is a positive semi-definite solution to the ARE. ■

The proof of this theorem is similar to that of Theorem 2.4-1. Note that, in general, $S(\infty)$ is different for different $S(T)$. Remember that the free-final-state LQ regulator does not require any controllability assumptions on the plant. However, such assumptions guarantee desirable properties as the control interval $[t_0, T]$ becomes large.

If we intend to use the simplified suboptimal control law (3.4-2) with a gain of $K(\infty)$, we should certainly like for the resulting closed-loop system to be stable! The next theorem tells when we can be sure of this. It is a strengthened version of the previous result and depends on the observability of the plant by the fictitious output.

Theorem 3.4-2. Let C be any matrix so that $Q = C^T C$.

Suppose (A, C) is observable. Then (A, B) is stabilize if and only if

1. There is a unique positive definite **limiting solution $S(\infty)$** to the Riccati equation. Furthermore, $S(\infty)$ is the unique positive definite solution to the ARE (3.4-6).
2. The closed-loop plant (3.4-3) is **asymptotically stable**, where $K = K(\infty)$ is given by (3.4-9). ■

The comments following Theorem 2.4-2 are relevant here. The observability condition in the theorem is not really needed. If (A, C) is detectable, the result still holds, but then $S(\infty)$ can be guaranteed only to be positive semidefinite.

What these theorems tell us is that if the plant is stabilizable and if we *select* Q so that $(A, \sqrt{Q})$ is observable, then the suboptimal feedback gain (3.4-9)

results in a stable closed-loop plant. Note that (3.4-9) is *the optimal* control law for the *infinite horizon* performance index

$$J_\infty = \frac{1}{2} \int_0^\infty (x^T Q x + u^T R u) dt. \quad (3.4-12)$$

Thus, as the control interval $[t_0, T]$ gets larger, it makes more and more sense to use a constant feedback with gain of $K(\infty)$.

A useful side result of these theorems is that we have a way of stabilizing any multivariable plant. Let Q and R be *any* positive definite matrices with the correct dimensions. Then $u = -K_\infty x$, where $K_\infty = R^{-1} B^T S$, with S the positive definite solution to (3.4-6), will result in a stable closed-loop plant. Different Q and R will result in different closed-loop poles for $(A - BK(\infty))$, but these poles will always be in the open left half-plane. Later we show examples of how the closed-loop poles move as Q and R vary.

Example 3.4-1. Steady-state Control of a System Obeying Newton's Law

Let the plant

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u \quad (1)$$

have the infinite-horizon cost

$$J(0) = \frac{1}{2} \int_0^\infty \left(x^T \begin{bmatrix} q_d & 0 \\ 0 & q_v \end{bmatrix} x + r u^2 \right) dt. \quad (2)$$

We could do a computer simulation using MATLAB, and such a simulation would have results similar to those of Example 2.4-3. However, let us instead capitalize on the simple form of the continuous LQ regulator to get some analytic solutions.

Using A, B, Q, r in the ARE (3.4-6) yields the three coupled scalar algebraic equations:

$$0 = -\frac{s_2^2}{r} + q_d, \quad (3)$$

$$0 = s_1 - \frac{s_2 s_3}{r}, \quad (4)$$

$$0 = 2s_2 - \frac{s_3^2}{r} + q_v, \quad (5)$$

where

$$S = \begin{bmatrix} s_1 & s_2 \\ s_2 & s_3 \end{bmatrix}. \quad (6)$$

These are easily solved to yield

$$s_2 = \sqrt{q_d r}, \quad (7)$$

$$s_3 = \sqrt{q_v r + 2r\sqrt{q_d r}}, \quad (8)$$

$$s_1 = \sqrt{q_d q_v + 2q_d \sqrt{q_d r}}, \quad (9)$$

where we have selected the positive definite solution $S(\infty)$. (Show this.)

The optimal feedback gain is

$$K(\infty) = R^{-1} B^T S(\infty) = \begin{bmatrix} \sqrt{\frac{q_d}{r}} & \sqrt{\frac{q_v}{r} + 2\sqrt{\frac{q_d}{r}}} \end{bmatrix}. \quad (10)$$

Since this depends only on the ratios q_d/r and q_v/r , let us now assume that $r = 1$. The closed-loop plant is

$$a^{\text{cl}} = (A - BK(\infty)) = \begin{bmatrix} 0 & 1 \\ -\sqrt{q_d} & -\sqrt{q_v + 2\sqrt{q_d}} \end{bmatrix}, \quad (11)$$

whence the optimal closed-loop characteristic equation is

$$s^2 + \sqrt{q_v + 2\sqrt{q_d}}s + \sqrt{q_d} = 0. \quad (12)$$

Comparing this to $s^2 + 2\delta\omega_n s + \omega_n^2$, we conclude that the optimal closed-loop poles are a complex pair with a natural frequency and damping ratio of

$$\omega_n = (q_d)^{1/4}, \quad (13)$$

$$\delta = \frac{1}{\sqrt{2}} \sqrt{1 + \frac{q_v}{2\sqrt{q_d}}}. \quad (14)$$

In particular, if no velocity weighting is used ($q_v = 0$), the damping ratio is the familiar $1/\sqrt{2}$! Note that the natural frequency depends only on the position weighting, and the damping ratio only on the ratio of the velocity to the square root of the position weighting.

Knowing the relations (13) and (14), we can now pick the weights q_d and q_v that result in desirable closed-loop behavior. These values can even be used in a finite-horizon (i.e., finite final time T) performance index to design optimal time-varying feedbacks with a prescribed steady-state behavior.

It is worth remarking that if $q_d = 0$ so that $(A, \sqrt{Q})$ is not detectable, then one of the closed-loop poles is at $s = 0$ and the closed-loop plant is not stable. ■

An Analytic Solution to the Riccati Equation

The LQ regulator Hamiltonian system is

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = H \begin{bmatrix} x \\ \lambda \end{bmatrix}, \quad (3.4-13)$$

where the Hamiltonian matrix is

$$H = \begin{bmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{bmatrix}. \quad (3.4-14)$$

By assuming

$$\lambda(t) = S(t)x(t), \quad (3.4-15)$$

we were able to derive the formulation in Table 3.3-1, wherein $S(t)$ satisfies the Riccati equation. Instead of solving the Riccati equation, $S(t)$ can be found analytically in terms of the eigenvalues and eigenvectors of H .

To find an analytic expression for $S(t)$, it is first necessary to show that if μ is an eigenvalue of H , then so is $-\mu$. Define

$$J = \begin{bmatrix} 0 & I \\ -I & 0 \end{bmatrix}. \quad (3.4-16)$$

Then by direct multiplication we see that

$$H = JH^T J. \quad (3.4-17)$$

Therefore, if μ is an eigenvalue of H with eigenvector v ,

$$Hv = \mu v,$$

so that

$$\begin{aligned} JH^T Jv &= \mu v, \\ H^T Jv &= -\mu Jv \end{aligned}$$

(note $J^{-1} = -J$). Hence,

$$(Jv)^T H = -\mu (Jv)^T, \quad (3.4-18)$$

and (Jv) is a left eigenvector of H with eigenvalue $-\mu$.

Now we merely repeat the steps leading up to (2.4-41). The results are as follows.

Order the eigenvalues of H in a matrix

$$D = \begin{bmatrix} -M & 0 \\ 0 & M \end{bmatrix}, \quad (3.4-19)$$

where M is a diagonal matrix containing the right-half-plane eigenvalues. Let the modal matrix of eigenvectors, arranged in order to correspond to D , be

$$W \triangleq \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix}. \quad (3.4-20)$$

Thus,

$$\begin{bmatrix} W_{11} \\ W_{21} \end{bmatrix}$$

are the n eigenvectors of the stable eigenvalues of H .

If $S(T)$ is the Riccati-equation boundary condition, define

$$V(T) = -(W_{22} - S(T)W_{12})^{-1}(W_{21} - S(T)W_{11}) \quad (3.4-21)$$

and

$$V(t) = e^{-M(T-t)} V(T) e^{-M(T-t)}. \quad (3.4-22)$$

Then an analytic solution to the Riccati equation is given by

$$S(t) = (W_{21} + W_{22}V(t))(W_{11} + W_{12}V(t))^{-1}. \quad (3.4-23)$$

In the limiting case $(T - t) \rightarrow \infty$, a bounded positive definite solution $S(\infty)$ exists if (A, B) is stabilizable and $(A, \sqrt{Q})$ is observable. In this limit, $V(t) \rightarrow 0$ since $-M$ is stable, so

$$S(\infty) = W_{21}W_{11}^{-1}. \quad (3.4-24)$$

The ARE solution is thus constructed by using the stable eigenvectors of the Hamiltonian matrix.

Design of Steady-state Regulators by Eigenstructure Assignment

We have just discovered a way to determine the optimal steady-state cost kernel $S(\infty)$ in terms of the eigenstructure of the Hamiltonian matrix H . By pursuing this line of thought a little further, we can find a way to determine the optimal steady-state feedback gain $K(\infty)$ directly from the eigenstructure of H .

We assume that (A, B) is reachable and $(A, \sqrt{Q})$ is detectable. The optimal steady-state closed-loop plant is

$$\dot{x} = (A - BK(\infty))x. \quad (3.4-25)$$

This and the Hamiltonian system (3.4-13) are both ways of characterizing the optimal state trajectories. We can demonstrate that if μ_i is a stable eigenvalue of H with eigenvector $[X_i^T \Lambda_i^T]^T$, where $X_i \in R^n$, then μ_i is also an eigenvalue of

$(A - BK(\infty))$ with eigenvector X_i . The argument is a straightforward modification of the discrete-time argument in Chapter 2.

H can be written down by inspection. Therefore, in the single-input case we can use this result by finding the eigenvalues of H and then realizing that the stable eigenvalues are the poles of the optimal closed-loop plant. Given these desired poles, a technique such as Ackermann's formula can be used to find the optimal feedback gain $K(\infty)$.

In the multi-input case, the optimal feedback is not uniquely specified by the closed-loop poles, so it is necessary to find the eigenvectors of H as well. A derivation virtually identical to the one for the discrete case in Chapter 2 leads to the following result.

Let the eigenvectors of the stable eigenvalues of H be placed into the $2n \times n$ matrix $[X^T \Lambda^T]^T$, where $X \in R^n$. (We called this

$$\begin{bmatrix} W_{11} \\ W_{21} \end{bmatrix}$$

in (3.4-20).) Then the optimal steady-state feedback is given by

$$K(\infty) = R^{-1} B^T \Lambda X^{-1}. \quad (3.4-26)$$

Compare this result to the corresponding result in Chapter 2, which, interestingly enough, includes in addition a matrix of stable eigenvalues M . (Why?)

Example 3.4-2. Eigenstructure Design of Steady-state Regulator for Harmonic Oscillator

Let the plant

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u \quad (1)$$

have a cost index of

$$J(0) = \frac{1}{2} \int_0^\infty \left(x^T \begin{bmatrix} q_d & 0 \\ 0 & q_v \end{bmatrix} x + ru^2 \right) dt. \quad (2)$$

a. Optimal Closed-loop Poles

The Hamiltonian matrix is

$$H = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\omega_n^2 & 0 & 0 & -1/r \\ -q_d & 0 & 0 & \omega_n^2 \\ 0 & -q_v & -1 & 0 \end{bmatrix}, \quad (3)$$

whence we can compute

$$|sI - H| = s^4 + \left(2\omega_n^2 - \frac{q_v}{r}\right)s^2 + \left(\omega_n^4 + \frac{q_d}{r}\right). \quad (4)$$

Note the even form of the characteristic polynomial of H . This means that if s is a root, then so is $-s$. Since only the ratios q_v/r and q_d/r appear, we can assume that $r = 1$. Letting $\bar{s} \triangleq s^2$, (4) becomes

$$\bar{s}^2 + (2\omega_n^2 - q_v)\bar{s} + (\omega_n^4 + q_d), \quad (5)$$

which has a pair of complex roots $\bar{s}_1, \bar{s}_2$ with natural frequency of

$$\bar{\omega}_n = (\omega_n^4 + q_d)^{1/2} \quad (6)$$

and damping ratio of

$$\bar{\delta} = \frac{\omega_n^2 - q_v/2}{(\omega_n^4 + q_d)^{1/2}}. \quad (7)$$

The roots of (4) are given by $\pm\sqrt{\bar{s}_1}$ and $\pm\sqrt{\bar{s}_2}$. If the roots of (5) are represented as

$$\bar{s}_1 = \bar{\omega}_n e^{j\bar{\theta}_1}, \quad (8a)$$

$$\bar{s}_2 = \bar{\omega}_n e^{j\bar{\theta}_2}, \quad (8b)$$

where $\bar{\theta}_2 = -\bar{\theta}_1$, then the roots of (4) are

$$(\bar{\omega}_n)^{1/2} e^{\pm j\bar{\theta}_1/2}, \quad (9a)$$

$$(\bar{\omega}_n)^{1/2} e^{\pm j\bar{\theta}_2/2}, \quad (9b)$$

If $\bar{\delta} = -\cos \bar{\theta}_1$ is the damping ratio of a pole pair at angles of $\pm\bar{\theta}_1$ (i.e., $\bar{\theta}_1$ and $\bar{\theta}_2$), then

$$\pm\delta = \pm \frac{\sqrt{1 - \bar{\delta}}}{\sqrt{2}} \quad (10)$$

are the damping ratios of the two pole pairs at angles of $\pm\bar{\theta}_1/2$ and $\pm\bar{\theta}_2/2$. These four poles are symmetric about the imaginary axis, and the pole pair corresponding to $+\delta$ is stable, whereas the pole pair corresponding to $-\delta$ is unstable. Equation (10) follows from the trigonometric relationship

$$\cos \frac{\alpha}{2} = \frac{\sqrt{1 + \cos \alpha}}{\sqrt{2}}. \quad (11)$$

In our case the stable poles of H thus have a damping ratio of

$$\delta^{\text{cl}} = \frac{1}{\sqrt{2}} \sqrt{1 - \frac{\omega_n^2 - q_v/2}{(\omega_n^4 + q_d)^{1/2}}}. \quad (12)$$

The natural frequency of the stable poles of H is

$$\omega_n^{\text{cl}} = (\omega_n^4 + q_d)^{1/4}. \quad (13)$$

Since the stable poles of H are the optimal closed-loop poles, we can write down the characteristic polynomial of $(A - BK(\infty))$:

$$\begin{aligned} \Delta^{\text{cl}}(s) &= s^2 + 2\delta^{\text{cl}}\omega_n^{\text{cl}}s + (\omega_n^{\text{cl}})^2 \\ &= s^2 + \sqrt{2}\sqrt{(\omega_n^4 + q_d)^{1/2} + (q_v/2 - \omega_n^2)}s \\ &\quad + (\omega_n^4 + q_d)^{1/2}. \end{aligned} \quad (14)$$

b. Optimal Feedback Gain

According to Ackermann's formula

$$K(\infty) = [0 \quad 1] U_2^{-1} \Delta^{\text{cl}}(A). \quad (15)$$

The reachability matrix is

$$U_2 = [B \quad AB] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad (16)$$

and substituting A^2 , A , and I for s^2 , s^1 , and s^0 in (14) yields

$$\begin{aligned} \Delta^{\text{cl}}(A) &= \begin{bmatrix} -\omega_n^2 + (\omega_n^4 + q_d)^{1/2} & \sqrt{2}\sqrt{(\omega_n^4 + q_d)^{1/2} + (q_v/2 - \omega_n^2)} \\ -\omega_n^2\sqrt{2}\sqrt{(\omega_n^4 + q_d)^{1/2} + (q_v/2 - \omega_n^2)} & -\omega_n^2 + (\omega_n^4 + q_d)^{1/2} \end{bmatrix}. \end{aligned} \quad (17)$$

The optimal feedback gain is thus

$$K(\infty) = \begin{bmatrix} -\omega_n^2 + (\omega_n^4 + q_d)^{1/2} & \sqrt{2}\sqrt{(\omega_n^4 + q_d)^{1/2} + (q_v/2 - \omega_n^2)} \end{bmatrix} \quad (18)$$

If $\omega_n = 0$, these results agree with Example 3.4-3. ■

Time-varying Plant

If the plant is time varying, then we must redefine observability and reachability. Suppose the plant is (3.4-1) with cost index in Table 3.3-1, where A , B , Q , and R are time dependent. Let $\phi(t, t_0)$ be the state transition matrix of A .

We say the plant is *uniformly completely observable* if for every final time T the *observability gramian* satisfies

$$\alpha_0 I \leq \int_{t_0}^T \phi^T(\tau, t_0) Q(\tau) \phi(\tau, t_0) d\tau \leq \alpha_1 I \quad (3.4-27)$$

for some $t_0 < T$, $\alpha_0 > 0$, and $\alpha_1 > 0$. Compare this with (3.3-14). We say the plant is *uniformly completely reachable* if for every initial time t_0 the *reachability gramian* satisfies

$$\alpha_0 I \leq \int_{t_0}^T \phi(t, \tau) B(\tau) R^{-1}(\tau) B^T(\tau) \phi^T(t, \tau) d\tau \leq \alpha_1 I \quad (3.4-28)$$

for some $T > t_0$, $\alpha_0 > 0$, and $\alpha_1 > 0$. Compare this to the gramian (3.3-25).

Uniform complete observability and reachability (and boundedness of $A(t)$, $B(t)$, $Q(t)$, $R(t)$) guarantee that for large t the behavior of $P(t)$ is unique, independent of $P(0)$. They also guarantee the uniform asymptotic stability of the closed-loop plant $(A - BK(t))$.

3.5 FREQUENCY-DOMAIN RESULTS

Several methods for designing steady-state continuous regulators have been discussed. Here we present an approach that amounts to a root-locus design method. The plant and weighting matrices are assumed time invariant, with (A, B) reachable and $(A, \sqrt{Q})$ observable.

A Factorization Result

The optimal steady-state regulator is given by a constant feedback (3.4-2), where $K = R^{-1} B^T S$ and S is the unique positive definite solution to the ARE (3.4-6). The resulting closed-loop system (3.4-3) is asymptotically stable.

As in Section 2.5, we can show that

$$\Delta^{\text{cl}}(s) = [I + K(sI - A)^{-1} B] \Delta(s), \quad (3.5-1)$$

which relates the closed-loop characteristic polynomial $\Delta^{\text{cl}}(s) = |sI - A + BK|$ to the open-loop polynomial $\Delta(s) = |sI - A|$. According to Fig. 2.5-1 (with z^{-1} replaced by $1/s$), $I + K(sI - A)^{-1} B$ can be interpreted as a return-difference matrix (return difference = I -loop gain).

We can also show the factorization result

$$\begin{aligned} & B^T (-sI - A)^{-T} Q (sI - A)^{-1} B + R \\ &= (I + K)(-sI - A)^{-1} B^T R (I + K(sI - A)^{-1} B), \end{aligned} \quad (3.5-2)$$

which can be interpreted as follows. Let

$$H(s) = C(sI - A)^{-1} B \quad (3.5-3)$$

be the transfer function from $u(t)$ to $y^1(t) = Cx(t)$, the “top portion” of the fictitious output $y(t)$ in (3.4-10). Now examine Fig. 2.5-2 (with z^{-1} , z replaced

by $1/s$, $-1/s$, respectively). It is clear that $H^T(-s)$ is the transfer function from y^1 to the intermediate signal w . Therefore, (3.5-2) simply expresses the equivalence between a transfer-function product in the continuous version of Fig. 2.5-2 and a transfer-function product in the continuous version of Fig. 2.5-1. It is just another way of expressing the equivalence between the state-costate (3.4-13) and the closed-loop (3.4-3) formulations of the optimal LQ regulator.

Chang-Letov Design Procedure for the Steady-state LQ Regulator

According to (3.5-1) and (3.5-2) we can write

$$\Delta^{\text{cl}}(-s)\Delta^{\text{cl}}(s) = |H^T(-s)H(s) + R| \cdot \Delta(-s)\Delta(s) \cdot |R|^{-1}, \quad (3.5-4)$$

where $H(s)$ is given by (3.5-3). This is the *Chang-Letov equation* (Kailath 1980). It can be used to design optimal steady-state LQ regulators by a root-locus approach. Note that the entire right-hand side is known if the plant and weighting matrices are given, so we can use the Chang-Letov equation to determine the optimal closed-loop poles; since $(A - BK)$ is stable by Theorem 3.4-2, they are just the stable roots of the right-hand side. The roots of $\Delta^{\text{cl}}(-s)\Delta^{\text{cl}}(s)$ are always symmetric with respect to the imaginary axis; that is, if s_1 is a root, then so is $-s_1$.

In the single-input case with $Q = qI$, we have

$$H(s) = \frac{\sqrt{q}[\text{adj}(sI - A)]B}{\Delta(s)} \triangleq \sqrt{q} \frac{N(s)}{\Delta(s)}, \quad (3.5-5)$$

where $N(s)$ is a column vector. Then (3.5-4) becomes

$$\Delta^{\text{cl}}(-s)\Delta^{\text{cl}}(s) = \frac{q}{r} N^T(-s)N(s) + \Delta(-s)\Delta(s). \quad (3.5-6)$$

The roots of the right-hand side are the zeros of

$$1 + \left(\frac{q}{r}\right) \frac{N^T(-s)N(s)}{\Delta(-s)\Delta(s)} = 1 + \frac{q}{r} H^T(-s)H(s), \quad (3.5-7)$$

which is exactly the form required for a root-locus analysis as a function of the parameter q/r . This shows that as q/r varies from zero (no state weighting) to ∞ (no control weighting), the optimal closed-loop poles move from the stable poles of

$$G(s) = H^T(-s)H(s) \quad (3.5-8)$$

to its stable zeros. We can therefore select the ratio of cost weights q/r to yield suitable closed-loop poles.

It is worth remarking that the stable poles of $H^T(-s)H(s)$ are the poles of $H(s)$ with unstable poles reflected into the left half-plane (i.e., $\bar{s}_1 = -s_1$). The stable zeros of $H^T(-s)H(s)$ are the zeros of $H(s)$ with unstable zeros reflected into the left half-plane.

The next example illustrates these ideas. For a good discussion, see Kailath (1980) or Schultz and Melsa (1967).

Example 3.5-1. Chang-Letov Design of Aircraft Longitudinal Autopilot

The short-period longitudinal dynamics for a medium-sized jet with center of gravity unusually far aft might be described by the state equations

$$\begin{bmatrix} \dot{\alpha} \\ \dot{\rho} \end{bmatrix} = \begin{bmatrix} -1.417 & 1.0 \\ 2.860 & -1.183 \end{bmatrix} \begin{bmatrix} \alpha \\ \rho \end{bmatrix} + \begin{bmatrix} 0 \\ -3.157 \end{bmatrix} \delta_e, \quad (1)$$

where α is the angle of attack, ρ the pitch rate, and δ_e the elevator deflection (Blakelock 1965). (We shall show only three decimal places.) The open-loop characteristic polynomial is

$$\Delta(s) = |sI - A| = s^2 + 2.6s - 1.183, \quad (2)$$

so the open-loop poles are

$$s = -2.995, 0.395. \quad (3)$$

Evidently, the center of gravity is so far aft that the short-period poles, which usually constitute a lightly damped complex pair, have become one stable and one unstable pole.

To stabilize the plant and keep the pitch rate small, we might select the performance index

$$J(0) = \frac{1}{2} \int_0^\infty (q\rho^2 + r\delta_e^2) dt, \quad (4)$$

so that

$$Q = \begin{bmatrix} 0 & 0 \\ 0 & q \end{bmatrix} \quad (5)$$

and a root of Q is

$$C = [0 \quad \sqrt{q}]. \quad (6)$$

Since (A, B) is reachable and (A, C) is observable (if $q \neq 0$), we know the steady-state LQ regulator results in a stable closed-loop plant.

Transfer function (3.5-3) is

$$H(s) = \frac{-\sqrt{q}(3.157s + 4.473)}{s^2 + 2.6s - 1.183}. \quad (7)$$

The Chang-Letov design procedure is based on the rational function

$$G(s) = H(-s)H(s). \quad (8)$$

Since we know the closed-loop characteristic polynomial $\Delta^{\text{cl}}(s)$ is stable, its roots are the stable zeros of

$$1 + \frac{q}{r} G(s). \quad (9)$$

In our case

$$\begin{aligned}
 G(s) &= \frac{(3.157s - 4.473)(-3.157s - 4.473)}{(s^2 - 2.6s - 1.183)(s^2 + 2.6s - 1.183)} \\
 &= \frac{-9.97(s - 1.417)(s + 1.417)}{(s + 0.395)(s - 2.995)(s - 0.395)(s + 2.995)}.
 \end{aligned} \quad (10)$$

The poles and zeros of $G(s)$ are plotted in Fig. 3.5-1. Note that they are symmetrically placed about the imaginary axis. Also shown is the root locus as q/r varies from 0 to ∞ . (Note that since the gain in (10) is *negative*, we are really plotting a root locus for negative gains $-9.97q/r$.)

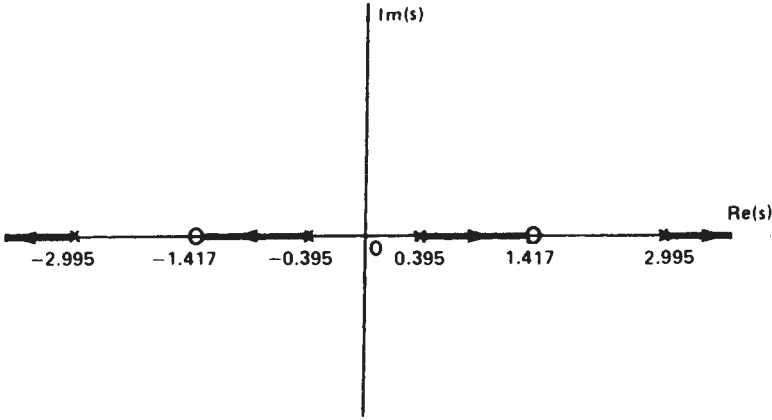


FIGURE 3.5-1 Chang-Letov root locus.

If we select $q/r = 1$, then the stable zeros of (9) are

$$s = -1.094, -4.231, \quad (11)$$

so the closed-loop characteristic polynomial is

$$\begin{aligned}
 \Delta^{cl}(s) &= (s + 1.094)(s + 4.231) \\
 &= s^2 + 5.324s + 4.627.
 \end{aligned} \quad (12)$$

Now Ackermann's formula can be used to find the optimal feedback gain K . ■

PROBLEMS

Section 3.2

3.2-1. We want to minimize

$$J = \int_0^\pi \dot{x}^2 dt.$$

Formulate this as an optimal control problem. Find and sketch the optimal $x(t)$. Find J^* .

3.2-2. Determining optimal control by approximation. Consider the nonlinear plant

$$\dot{x} = -x^3 + u, \quad x(0) = \frac{1}{2}, \quad (1)$$

with quadratic performance index

$$J = \frac{1}{2}x^2(2) + \frac{1}{2} \int_0^2 (x^2 + u^2) dt. \quad (2)$$

- a. Write state and costate equations, stationarity condition, and boundary conditions. Eliminate $u(t)$ from the state and costate equations.
- b. Prove that if $x(t)$ is small on $[0, 2]$, then an approximate solution for the costate is

$$\lambda = x - x^3. \quad (3)$$

- c. For this approximate costate, find the state solution $x(t)$.
- d. Find the approximate optimal control. (Note that applying this control to the plant will tend to keep $x(t)$ small, so that approximation (3) is valid.)

3.2-3. A model of an automobile suspension system is given by

$$m\ddot{y} + ky = u, \quad (1)$$

where m is the mass, k the spring constant, u the upward force on the frame, and y the vertical position.

- a. Write the state equation if the state is $x = [y \quad \dot{y}]^T$.
To conduct a durability test, we repetitively apply a force $u(t)$ and suddenly remove it until failure occurs. To compute the force, we can solve the following problem. Find $u(t)$ to move the automobile from $y(0) = 0$, $\dot{y}(0) = 0$ to a final position of $y(T) = h$, $\dot{y}(T) = 0$ at a given final time T . Minimize the control energy

$$J = \frac{1}{2} \int_0^T u^2 dt. \quad (2)$$

- b. Write the state and costate equations, stationarity condition, and boundary conditions. Eliminate u from the state and costate equations.
- c. Solve for the costate in terms of the as yet unknown $\lambda(0)$. Solve for the state in terms of the unknown $\lambda(0)$ and the known $x(0)$.
- d. Use the boundary conditions to find $\lambda(0)$. Let $m = k = 1$, $T = 2$, $h = -3$ (i.e., down) for the remainder of the problem.
- e. Find optimal control and optimal state trajectory.
- f. Verify that $x^*(T) = [h \quad 0]^T$ as required.

3.2-4. See Fig. P3.2-1.

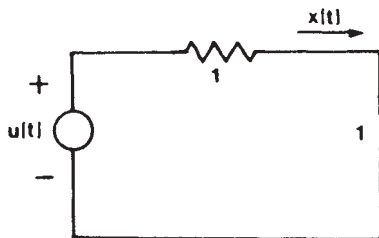


FIGURE P3.2-1

- Find the state equation.
- It is desired to charge up the inductance to $x(T) = 2A$ at $T = 1$ if $x(0) = 0$ while minimizing

$$J = \int_0^1 u^2 dt. \quad (1)$$

Find the optimal control.

- Find the optimal state trajectory.

3.2-5. **Optimal control of a bilinear system.** Let

$$\dot{x} = Ax + Dxu + bu, \quad (1)$$

where $x \in R^n$, $u \in R$, and

$$J = \frac{1}{2}x^T(T)S(T)x(T) + \frac{1}{2} \int_0^T (x^T Qx + ru^2) dt. \quad (2)$$

Show that the optimal control consists of a state–costate inner product. Find state and costate equations after eliminating u . These cubic differential equations are very hard to solve.

3.2-6. **Numerical solution of bilinear system.** Let

$$\begin{aligned} \dot{x}_1 &= x_1 x_2 + u, \\ \dot{x}_2 &= x_1^2 + x_2^2, \end{aligned}$$

with cost

$$J = \frac{s_1 x_1^2(T)}{2} + \frac{s_2 x_2^2(T)}{2} + \frac{1}{2} \int_0^T (q_1 x_1^2 + q_2 x_2^2 + ru^2) dt,$$

where T is fixed. Derive Euler's equations and boundary conditions. Write subroutines *fcni*, *fcnbf* for use with *tpoint*.

3.2-7. Unit solution method. Consider the optimal control problem formulated by (3.2-1) and (3.2-14)–(3.2-21), where the plant is linear. We want to formalize the unit solution method by finding a system of equations to solve for the initial costate $\lambda(t_0)$.

Let $r(t_0)$ be the given initial state. Let $x_0(t)$, $\lambda_0(t)$ be the solutions of the Hamiltonian system when $x(t_0) = r(t_0)$, $\lambda(t_0) = 0$. Let $x_i(t)$, $\lambda_i(t)$ for $i = 1, \dots, n$ be the unit solutions when $x(t_0) = 0$, $\lambda(t_0) = e_i$, where e_i is the i th column of the $n \times n$ identity matrix. Then the solutions for the general initial conditions (3.2-26) with c_i scalar constants, are given by (3.2-27).

a. *Final state fixed:* If the final condition is $x(T) = r(T)$ given, show that the initial costate $\lambda(t_0)$ that guarantees that this final condition holds is given by solving

$$[x_1(T) \cdots x_n(T)] \lambda(t_0) = r(T) - x_0(T). \quad (1)$$

b. *Final state free:* If the final state is free, show that the optimal initial costate $\lambda(t_0)$ is given by solving

$$\begin{aligned} & -[\lambda_1(T) - S(T)x_1(T) \cdots \lambda_n(T) - S(T)x_n(T)] \lambda(t_0) \\ & = -(\lambda_0(T) - S(T)x_0(T)r(T)). \end{aligned} \quad (2)$$

3.2-8. Unit solution method. Let

$$\dot{x}_1 = x_2 + u, \quad (1a)$$

$$x_2 = x_1 - x_2, \quad (1b)$$

with

$$J = 5x_1^2(T) + \frac{1}{2} \int_0^T (x_1^2 + x_2^2 + u^2) dt, \quad (2)$$

where T is fixed and we require

$$x_2(T) = 1. \quad (3)$$

Solve for the optimal control by the unit solution method.

Section 3.3

3.3-1. The scalar plant

$$\dot{x} = u \quad (1)$$

has performance index

$$J(t_0) = \frac{1}{2}sx^2(T) + \frac{1}{2} \int_{t_0}^T ru^2 dt. \quad (2)$$

- a. First we want a closed-loop control minimizing J .
 - i. Solve the Riccati equation using separation of variables.
 - ii. Find the optimal control.
 - iii. Sketch optimal feedback gain versus t .
- b. Now we want an open-loop control that minimizes J and drives the state to a given $x(T)$. Initial state $x(t_0)$ is known.
 - i. Find the weighted reachability gramian.
 - ii. Find the optimal control as a function of $x(t_0)$ and $x(T)$. Sketch it.
 - iii. Find and sketch the optimal state trajectory.
- c. The open-loop control formulation of part b can be used to find a state feedback control law.
 - i. Use part b.iii to solve for $x(t_0)$ in terms of $x(t)$ and $x(T)$.
 - ii. Substitute this into your result in part b.ii to find $u^*(t)$ in the form of $u^*(t) = g(t)x(t) + h(t)$. Compare to the closed-loop control in part a.
 - iii. Compare this control to the optimal control that minimizes the *cost to go* $J(t)$ on $[t, t + T]$.

3.3-2. See Fig. P3.2-1.

- a. Write the state equation.
- b. Solve Problem 3.2-4 using the reachability gramian.
- c. Now let

$$J = \frac{1}{2}10x^2(T) + \frac{1}{2} \int_0^T (x^2 + u^2) dt.$$

- i. Solve the Riccati equation using separation of variables.
- ii. Find the optimal control.
- iii. Sketch optimal feedback gain versus t .

3.3-3. Optimal control of newton's system. Let

$$\dot{x}_1 = x_2, \tag{1a}$$

$$\dot{x}_2 = u \tag{1b}$$

have performance index

$$J = \frac{1}{2}x^T(T)x(T) + \frac{1}{2} \int_0^T (x^T x + ru^2) dt, \tag{2}$$

where $x = [x_1 \ x_2]^T$.

- a. Find the Riccati equation. Write it as three scalar differential equations. Find the feedback gain in terms of the scalar components of $S(t)$.
- b. Write subroutines to find and simulate the optimal control using MATLAB.
- c. Find analytic expressions for the steady-state Riccati solution and gain.

3.3-4. Uncontrolled Newton's system. Consider the system of Problem 3.3-3. Solve the Lyapunov equation (3.3-9) to find the cost kernel $S(t)$ if $u = 0$. Sketch the scalar components of $S(t)$.

3.3-5. Uncontrolled harmonic oscillator. Repeat problem 3.3-4 for the system in Example 3.3-5. Let $S(T) = I$, $Q = I$, $\omega_n^2 = 1$, $\delta = 0.5$.

3.3-6. Cross-weighting cost terms. Repeat Problem 3.3-3 if a term like $2x_1(t)u(t)$ is added under the integral in J . Assume $r > 1$. (Why?)

3.3-7. Writing the Lyapunov equation as a vector equation. Show that Lyapunov equation (3.3-9) can be written as the vector equation

$$-\frac{d}{dt}s(S) = [(A^T \otimes I) + (I \otimes A^T)]s(S) + s(Q),$$

where the Kronecker product and stacking operator are defined in Appendix A.

3.3-8. Prove that (3.3-42) and the Joseph-stabilized equation (3.3-46) are equivalent to (3.3-40).

3.3-9. Open-loop control with function of final state fixed. Let $\dot{x} = Ax + Bu$ with

$$J = \frac{1}{2} \int_{t_0}^T u^T R u dt.$$

It is required to drive $x(t_0)$ to a final state $x(T)$ such that

$$Cx(T) = r(T)$$

for a given $r(T)$.

Show that the optimal control is given by

$$u(t) = -R^{-1}B^T e^{A^T(T-t)}C^T [CG(t_0, T)C^T]^{-1} (Ce^{A(t-t_0)}(t_0) - r(T)).$$

Note that if $C \neq I$, there may be a solution even if the system is not reachable.

3.3-10. Hamiltonian system. Let V , W be $n \times n$ solutions to the linear equation

$$\begin{bmatrix} \dot{V} \\ \dot{W} \end{bmatrix} = \begin{bmatrix} A & -BR^{-1}B^T \\ -Q & -A^T \end{bmatrix} \begin{bmatrix} V \\ W \end{bmatrix},$$

with boundary condition $W(T) = S(T)V(T)$. Show that the solution to (3.3-40) is given by $S(t) = W(t)V^{-1}(t)$. (Hint: Note that $d(V^{-1})/dt = -V^{-1}\dot{V}V^{-1}$.)

3.3-11. Writing the Riccati equation as a vector equation. Show that (3.3-40) can be written as the nonlinear vector differential equation

$$-\frac{d}{dt}s(S) = [(A^T \otimes I) + (I \otimes A^T)]s(S) + (S \otimes S)s(BR^{-1}B^T) + s(Q).$$

3.3-12. Relation between state and costate. Show that if $u(t) = 0$, $Q = 0$, then the inner product of the state with the costate is a constant independent of time in the continuous linear quadratic regulator.

Section 3.4

3.4-1. Newton's system. Let

$$\dot{x} = x_2, \quad (1a)$$

$$\dot{x}_2 = u, \quad (1b)$$

$$J = \frac{1}{2} \int_0^\infty (x_1^2 + 2vx_1x_2 + qx_1^2 + u^2) dt, \quad (2)$$

where $(q - v^2) > 0$.

- Find the solution to the ARE.
- Find the optimal control.
- Find the optimal closed-loop system.
- Plot loci of closed-loop poles as q varies from 0 to ∞ . For which values of q is the system stable?

3.4-2. Suboptimal control. Let

$$\dot{x} = u, \quad (1)$$

$$J = \frac{1}{2}sx^2(T) + \frac{1}{2} \int_0^T (qx^2 + ru^2) dt. \quad (2)$$

- Optimal control:
 - Set up and solve the Riccati equation. Sketch solution $s^*(t)$.
 - Find the optimal feedback $K(t)$. Show that the steady-state gain is

$$K_\infty = \sqrt{q/r}. \quad (3)$$

- Suboptimal control: We shall now use the suboptimal constant feedback

$$u(t) = -K_\infty x(t). \quad (4)$$

Find the resulting kernel $s(t)$. Sketch $s^*(t)$ and $s(t)$ on the same graph. Note $s^*(t) \leq s(t)$ for all t , but $s^*(0) = s(0)$ so the two limiting values are the same.

- c. Find the steady-state closed-loop plant and plot its pole as q/r goes from 0 to ∞ .

3.4-3. ARE solutions. Let

$$\dot{x}_1 = x_2, \quad (1a)$$

$$\dot{x}_2 = -ax_1 - 2x_2 + u, \quad (1b)$$

and

$$J = \frac{1}{2} \int_0^\infty (2x_1^2 + x_2^2 + u^2) dt. \quad (2)$$

- Plot the root locus of the open-loop system as a varies from $-\infty$ to ∞ . For which values of a is (1) stable?
- Find all symmetric ARE solutions in terms of a . How do you know there is a unique positive definite solution?
- In the remainder of the problem, let $a = -8$. Find open-loop poles. Is (1) stable?
- Find all (4) symmetric ARE solutions.
- Find the unique, negative definite solution and the unique, positive definite solution.
- Find the steady-state Kalman gain.
- Find the closed-loop poles.
- What can you say about the closed-loop poles for *any* value of a ?

3.4-4. Suboptimal control of Newton's system. In Example 3.4-3, the steady-state gain for Newton's system was found to be of the form

$$K_\infty = [\omega_n^2 \quad 2\delta\omega_n^2], \quad (1)$$

where δ and ω_n depend on the cost weights. In a suboptimal control scheme we apply the constant feedback

$$u(t) = -K_\infty x(t). \quad (2)$$

Determine the resulting suboptimal cost kernel $S(t)$. Compare this with Problem 3.3-3.

3.4-5. Let

$$\dot{x} = x + u, \quad (1)$$

$$J = \frac{1}{2} \int_0^\infty (x^2 + u^2) dt. \quad (2)$$

- Use the eigenstructure of the Hamiltonian matrix to determine the steady-state Riccati solution s_∞ . Hence, find the steady-state gain K_∞ .

- b. Find the solution to the ARE to check part a.
- c. Use the eigenstructure of the Hamiltonian matrix to determine optimal closed-loop poles. Hence, find K_∞ by Ackermann's formula.

3.4-6. Repeat Problem 3.4-5 for the plant

$$\dot{x}_1 = x_1 + u, \quad (1a)$$

$$\dot{x}_2 = -x_1 + x_2 \quad (1b)$$

if

$$J = \frac{1}{2} \int_0^\infty (x_1^2 + x_2^2 + ru^2) dt, \quad (2)$$

with $r = \frac{1}{10}$.

3.4-7. ARE solutions

- a. Find all symmetric solutions to (3.4-6) if

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C = [1 \quad 0], \quad R = 1, \quad \text{and} \quad Q = C^T C.$$

- b. How do you know a priori that there is a unique, positive definite solution? Check all your solutions for definiteness.
- c. Some of your solutions are complex. For each real solution S , check the stability of the closed-loop system $(A - BK)$, where $K = R^{-1}B^T S$. Hence, identify the *stabilizing* and *destabilizing* solutions to the algebraic Riccati equation as the positive definite and negative definite solutions, respectively.

3.4-8. Observability and reachability for time-varying systems

- a. Show that (3.4-27) holds if and only if there is a bounded positive definite solution for some $t < T$ to

$$-\dot{S} = A^T S + SA + Q, \quad S(T) = 0. \quad (1)$$

Show that (3.4-28) holds if and only if there is a bounded positive definite solution $S^{-1}(t)$ for some $t > 0$ to

$$\frac{d}{dt}(S^{-1}) = AS^{-1} + S^{-1}A^T + BR^{-1}B^T, \quad S^{-1}(0) = 0. \quad (2)$$

- b. Derive discrete counterparts to these tests.

Section 3.5**3.5-1.** Prove (3.5-2).**3.5-2. Change-Letov design**

- a. For the system of Problem 3.4-6, use the Chang-Letov equation and Ackermann's formula to find the optimal feedback gain and closed-loop plant if $r = \frac{1}{10}$.
- b. Plot a root locus for the closed-loop plant as r varies from ∞ to 0.

3.5-3. Polynomial decomposition

- a. Show that any polynomial $P(s)$ with real coefficients can be decomposed as

$$P(s) = P_1(s) + P_2(s) \quad (1)$$

where

$$\begin{aligned} P_1(s) &= \frac{1}{2}(P(s) + P(-s)), \\ P_2(s) &= \frac{1}{2}(P(s) - P(-s)). \end{aligned} \quad (2)$$

Show that

$$P_1(s) = P_1(-s), \quad (3)$$

$$P_2(s) = -P_2(-s). \quad (4)$$

- b. Given the roots s_i of $P(s) = 0$, how can you find the roots of $P(-s) = 0$?

3.5-4. Chang-Letov design for systems in reachable canonical form. Let

$$\begin{aligned} \dot{x} &= \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\delta\omega_n \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u, \\ J &= \frac{1}{2} \int_0^\infty (qx^T x + ru^2) dt. \end{aligned}$$

- a. Using the Chang-Letov procedure, plot the loci of the optimal closed-loop poles as q/r goes from 0 to ∞ . Show that the poles move from the “stabilized” poles of the plant (i.e., if they are unstable, reflect them into the left half- s -plane replacing s by $-s$) to values of $s = -1, \infty$.
- b. Find the optimal gain by Ackermann's formula if $\omega_n = 1, \delta = -\frac{1}{2}, q/r = 2$.

6

DYNAMIC PROGRAMMING

The purpose of this chapter is to present a brief introduction to *dynamic programming*, which is an alternative to the variational approach to optimal control discussed in Chapters 2 and 3. We show how some of our results from those chapters can be derived by this new approach.

Dynamic programming was developed by R. E. Bellman in the later 1950s (Bellman 1957, Bellman and Dreyfus 1962, Bellman and Kalaba 1965). It can be used to solve control problems for nonlinear, time-varying systems, and it is straightforward to program. The optimal control is expressed as a state-variable feedback in graphical or tabular form.

Additional references for this material include Kirk (1970), Bryson and Ho (1975), Schultz and Melsa (1967), Athans and Falb (1966), and Elbert (1984).

6.1 BELLMAN'S PRINCIPLE OF OPTIMALITY

Dynamic programming is based on Bellman's *principle of optimality*:

An optimal policy has the property that no matter what the previous decision (i.e., controls) have been, the remaining decisions must constitute an optimal policy with regard to the state resulting from those previous decisions.

(6.1-1)

We shall see that the principle of optimality plays a role similar to that played by Pontryagin's minimum principle (5.2-6) in the variational approach to system control. It serves to limit the number of potentially optimal control strategies that must be investigated. It also implies that optimal control strategies must

be determined by working *backward* from the final stage; the optimal control problem is inherently *a backward-in-time problem*.

A simple routing example will serve to demonstrate all the points made so far.

Example 6.1-1. An Aircraft Routing Problem

An aircraft can fly from left to right along the paths shown in Fig. 6.1-1. Intersections $a, b, c, \dots$ represent cities, and the numbers represent the fuel required to complete each path. Let us use the principle of optimality to solve the minimum-fuel problem.

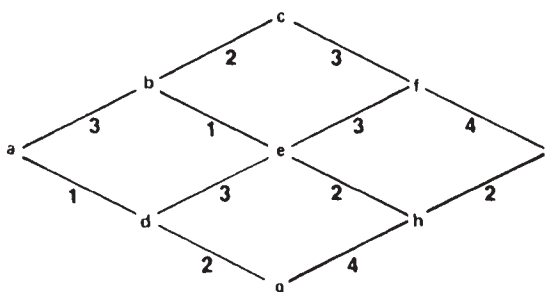


FIGURE 6.1-1 Aircraft routing network.

We can quickly construct a state-variable feedback that shows both the optimal cost and the optimal control from *any* node to i . First, however, we must define what we mean by *state* in this example.

We can label stages $k = 0$ through $k = N = 4$ of the decision-making process as shown in Fig. 6.1-2. (The arrowheads and numbers in parentheses should initially be disregarded: we shall show how to fill them in later.) At each stage $k = 0, 1, \dots, N - 1$ a decision is required, and N is the final stage.

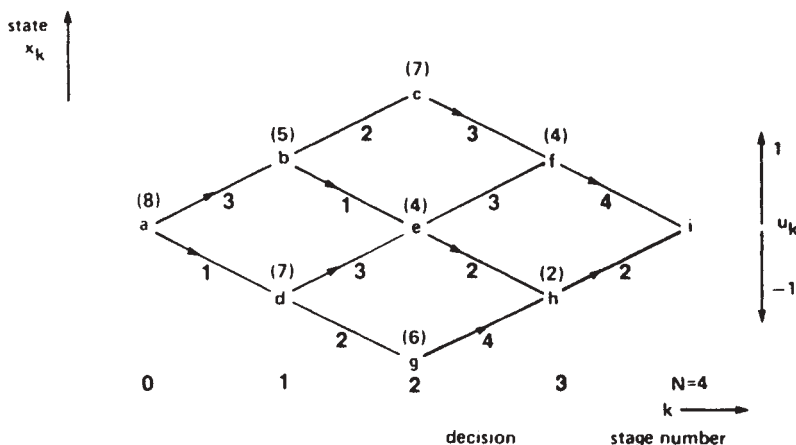


FIGURE 6.1-2 Minimum-fuel state feedback control law.

The current *state* is the node where we are making the current decision. Thus, the initial state is $x_0 = a$. At stage 1, the stage can be $x_1 = b$ or $x_1 = d$. Similarly $x_2 = c$, e , or g ; $x_3 = f$ or h ; and the final state is constrained to be $x_N = x_4 = i$. The control u_k at stage k can be considered to be $u_k = \pm 1$, where $u_k = 1$ results in a move up, and $u_k = -1$ results in a move down at stage $k + 1$. Now it is clear that all we have on our hands is a minimum-fuel problem with fixed final state and constrained control *and* state values.

To find a minimum-fuel feedback control law using the principle of optimality, start at $k = N = 4$. No decision is required here, so decrement k to 3. If $x_3 = f$, the optimal (only) control is $u_3 = -1$, and the cost is then 4. This is indicated by placing (4) above node f , and placing an arrowhead on path $f \rightarrow i$. If $x_3 = h$, the optimal control is $u_3 = 1$, with a cost of 2, which is now indicated on the figure.

Now decrement k to 2. If $x_2 = c$, then $u_2 = -1$ with a cost to go of $4 + 3 = 7$. This information is added to the figure. If $x_2 = e$, then we must make a decision. If we apply $u_2 = 1$ to get to f , and then go via the optimal path to i , the cost is $4 + 3 = 7$. On the other hand, if we apply $u_2 = -1$ at e and go to h , the cost is $2 + 2 = 4$. Hence, at e the optimal decision is $u_2 = -1$ with a cost to go of 4. Add this information to the figure. If $x_2 = g$, there is only one choice: $u_2 = 1$ with a cost to go of 6.

By successively decrementing k and continuing to compare the control possibilities allowed by the principle of optimality, we can fill in the remainder of the control decisions (arrowheads) and optimal costs to go shown in Fig. 6.1-2. It should be clearly realized that the only control sequences we are allowed to consider are those whose *last portions* are *optimal sequences*. Note that when $k = 0$, a control of either $u_0 = 1$ or $u_0 = -1$ yields the same cost to go of 8; the optimal control for $k = 0$ is not unique.

To examine what we have just constructed, suppose we now are told to find the minimum-fuel path from node d to the destination i . All we need to do is begin at d and follow the arrows! The optimal control u_k^* and the cost to go at each stage k are determined if we know the value of x_k . This, however, is exactly the meaning of state-variable feedback. Therefore, the grid labeled with arrows as in Fig. 6.1-2 is just a graphical *state feedback control law* for the minimum-fuel problem!

This control law formulation should be compared to those in the examples of Section 5.2. Our feedback law tells us how to get from *any* state to the fixed final state $x_4 = x_N = i$. If we change the *final* state, however (e.g., to $x_3 = x_N = f$), then the entire grid must be redone.

Several points should be noted about this example. First, note that, according to Fig. 6.1-2, there are two paths from a to i with the same cost of 8: $a \rightarrow b \rightarrow e \rightarrow h \rightarrow i$ and $a \rightarrow d \rightarrow e \rightarrow h \rightarrow i$. Evidently, the optimal solution found by dynamic programming may not be unique.

Second, suppose we had attempted, in ignorance of the optimality principle, to determine an optimal route from a to i by working *forward*. Then a near-sighted decision maker at a would compare the costs of traveling to b and d , and decide to go to d . The next myopic decision would take him to g . From there on there is no choice: he must go via h to i . The net cost of this strategy is $1 + 2 + 4 + 2 = 9$, which is nonoptimal. We can say that “any portion of an optimal path is optimal.” For example, an optimal route from a to e is $a \rightarrow b \rightarrow e$, while the optimal route from e to i is $e \rightarrow h \rightarrow i$.

Finally, let us point out that Bellman’s principle of optimality has reduced the number of required calculations by *restricting the number of decisions that must be made*. We could determine the optimal route from a to i by comparing *all* possible paths from a to i . This would require many more calculations than we used. ■

Example 6.1-2. Computational Savings

In Example 6.1-1, determine the number of calculations we used to solve the minimum fuel problem. Now, determine the number of calculations that would be required to solve the problem by comparing *all* possible paths from *a* to *i*. ■

Example 6.1-3. Maximizing a Performance Index

For the routing network in Fig. 6.1-1, find the *maximum* fuel route from *a* to *i*. ■

Example 6.1-4. Target Set of Final States

In Fig. 6.1-3, the cost of traveling from one node to another is given by the numbers, and motion is allowed only from left to right.

- Find the minimum cost path from node *a* to the desired final state *j*.
- Now, define the *target set* *S* as $\{h, i, j\}$. That is, any value of the final state within this set is acceptable. Find the minimum cost path from *a* to *S*.
- Find the minimum cost path from *b* to *S*.

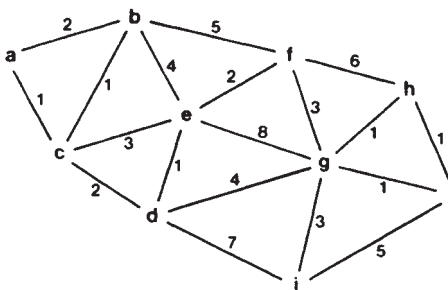


FIGURE 6.1-3 Irregular routing grid. ■

We now discuss the application of Bellman's principle of optimality to the control of dynamical systems.

6.2 DISCRETE-TIME SYSTEMS

In Chapters 2–5 we discussed the variational approach to optimal control for dynamical systems. We discovered that for nonlinear systems the state and costate equations are hard to solve, and that constraints on the control variables further complicate things. Dynamic programming, on the other hand, can easily be applied to nonlinear systems, and the more constraints there are on the control and state variables, the easier the solution!

Let the plant be

$$x_{k+1} = f^k(x_k, u_k), \quad (6.2-1)$$

where the superscript k on f indicates that it can be time varying. Suppose we associate with this plant the performance index

$$J_i(x_i) = \phi(N, x_N) + \sum_{k=i}^{N-1} L^k(x_k, u_k), \quad (6.2-2)$$

where $[i, N]$ is the time interval of interest. We have shown the dependence of J on the initial time and state.

We want to use the principle of optimality (6.1-1) to select the control sequence u_k to minimize (6.2-2). First, we need to determine what (6.1-1) looks like in this situation. Let us express it in a more mathematical form.

Suppose we have computed the optimal cost $J_{k+1}^*(x_{k+1})$ from time $k+1$ to the terminal time N for all possible states x_{k+1} , and that we have also found the optimal control sequences from time $k+1$ to N for all x_{k+1} . The optimal cost results when the optimal control sequence $u_{k+1}^*, u_{k+2}^*, \dots, u_{N-1}^*$ is applied to the plant with a state of x_{k+1} . (Note that the optimal control sequence depends on x_{k+1} .)

If we apply any arbitrary control u_k at time k and then use the known optimal control sequence from $k+1$ on, the resulting cost will be

$$L^k(x_k, u_k) + J_{k+1}^*(x_{k+1}), \quad (6.2-3)$$

where x_k is the state at time k , and x_{k+1} is given by (6.2-1). According to Bellman, the optimal cost from time k on is equal to

$$J_k^*(x_k) = \min_{u_k} (L^k(x_k, u_k) + J_{k+1}^*(x_{k+1})), \quad (6.2-4)$$

and the optimal control u_k^* at time k is the u_k that achieves this minimum.

Equation (6.2-4) is the principle of optimality for discrete systems. Its importance lies in the fact that it allows us to optimize over only one control vector at a time by working backward from N . It is called the *functional equation of dynamic programming* and is the basis for computer implementation of Bellman's method. In general, we can specify additional constraints, such as the requirement that u_k belong to some admissible control set.

Without realizing it, we worked Example 6.1-1 by applying (6.2-4). Let us consider next a systems-oriented example.

Example 6.2-1. Optimal Control of a Discrete System Using Dynamic Programming

Let the plant

$$x_{k+1} = x_k + u_k \quad (1)$$

have an associated performance index of

$$J_0 = x_N^2 + \frac{1}{2} \sum_{k=0}^{N-1} u_k^2, \quad (2)$$

where the final time N is 2. The control is constrained to take on values of

$$u_k = -1, -0.5, 0, 0.5, 1, \quad (3)$$

and the state is constrained to take on only the values

$$x_k = 0, 0.5, 1.0, 1.5. \quad (4)$$

The control value constraint is not unreasonable, since we found in Section 5.2 that a minimum-time optimal control takes on only values of ± 1 , whereas a minimum-fuel optimal control takes on only values of $0, \pm 1$. The state value constraint in this problem is also reasonable, since if the state initially takes on one of the admissible values (4), then under the influence of the allowed controls (3) subsequent states will take on integer or half-integer values. An equivalent constraint to (4) is therefore $x_0 = 0, 0.5, 1.0, 1.5$, and

$$0 \leq x_k \leq 1.5. \quad (5)$$

This is a positivity condition and a magnitude constraint on the state, which is often reasonable in physical situations. In an actual application N would almost certainly be greater than 2, but to handle large N we would use a computer program designed on the basis of our work in this example.

Now, the optimal control problem is to find an admissible control sequence u_0^*, u_1^* that minimizes J_0 while resulting in an admissible state trajectory x_0^*, x_1^*, x_2^* . We should like for u_k^* to be specified as a state feedback control law.

To solve this problem, we can use the principle of optimality in the form (6.2-4). First, we set up a grid of x_k versus k as shown in Fig. 6.2-1. (Disregard the arrowheads, numbers above the lines, and numbers in parentheses for now.) The lines on the figure are drawn according to the state equation (1). For each admissible x_k , the lines extend toward $x_{k+1} = x_k + u_k$, with the control value u_k written at the end of each line. Only u_k that are both admissible by (3) and result in admissible values of x_{k+1} given in (4) are considered. Thus, for example, if $x_0 = 1.0$, then the admissible u_0 are $-1, -0.5, 0, 0.5$, which result, respectively, in

$$x_1 = x_0 + u_0 = 1 - 1 = 0,$$

$$x_1 = 1 - 0.5 = 0.5,$$

$$x_1 = 1 + 0 = 1,$$

$$x_1 = 1 + 0.5 = 1.5.$$

The lines emanating from the node $x_0 = 1.0$ are directed toward these values of x_1 .

Figure 6.2-1 incorporates all of the information in the state equation (1) and the constraints (3), (4). To compute the optimal control using (6.2-4), we need to work with the cost function (2). Write

$$J_k = \frac{1}{2}u_k^2 + J_{k+1}^* \quad (6)$$

as the *admissible costs* at time k ; an admissible cost J_k is one whose last portion J_{k+1}^* is optimal. In terms of these J_k , (6.2-4) becomes

$$J_k^* = \min_{u_k}(J_k). \quad (7)$$

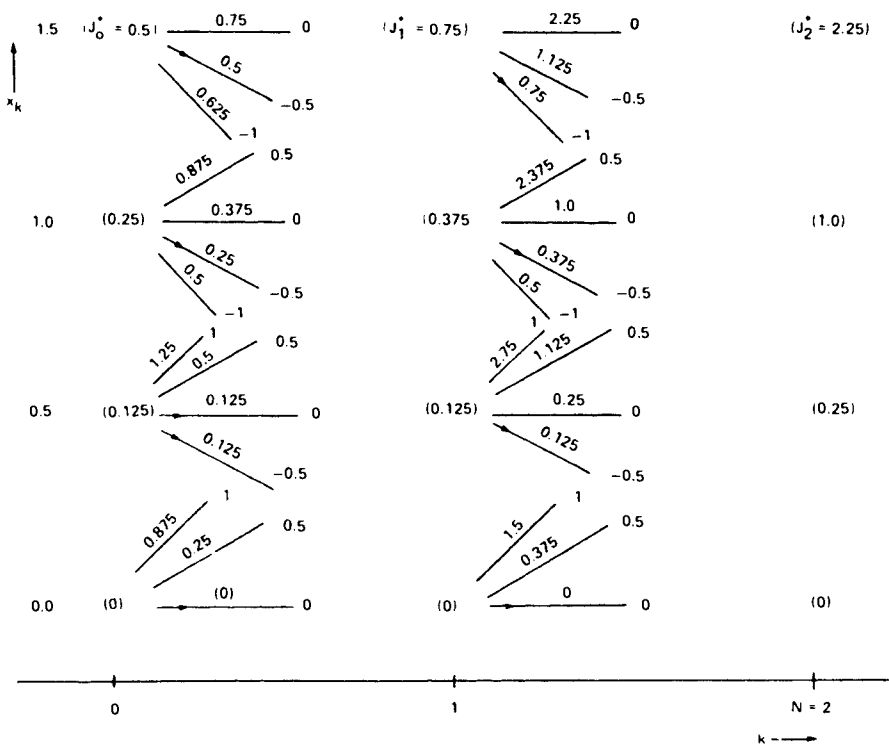


FIGURE 6.2-1 Decision grid for solving the optimal control problem by dynamic programming.

(The dependence of J_k and J_k^* on x_k is not explicitly shown.) Write down (6) for each k beneath the grid in Fig. 6.2-1.

To compute u_k^* and J_k^* for each x_k using (7), we need to work backward from $k = N$. First, let $k = N = 2$. The final costs

$$J_N^* = x_N^2 \quad (8)$$

must be evaluated for each admissible final state. J_N^* represents the penalty for being in the final state with value x_N . These costs are written down in parentheses in Fig. 6.2-1 at the location of each of the final states $x_N = 0$, $x_N = 0.5$, $x_N = 1.0$, and $x_N = 1.5$. They are $J_2^*(x_2 = 1.5) = 1.5^2 = 2.25$, $J_2^*(x_2 = 1.0) = 1.0$, and so on.

Now decrement to $k = 1$. For each possible state x_1 and for each admissible control u_1 we must now compute (6). Begin with $x_1 = 1.5$. If we apply $u_1 = 0$, then

$x_2 = x_1 + u_1 = 1.5$, so that $J_2^* = 2.25$. According to (6),

$$J_1 = u_1^2/2 + J_2^* = 0^2/2 + 2.25 = 2.25. \quad (9)$$

This cost is written above the line representing $u_1 = 0$ at $x_1 = 1.5$.

Now suppose we apply $u_1 = -0.5$ when $x_1 = 1.5$. Then $x_2 = x_1 + u_1 = 1.0$, and so $J_2^* = 1.0$. Thus,

$$J_1 = \frac{u_1^2}{2} + J_2^* = \frac{(-0.5)^2}{2} + 1.0 = 1.125. \quad (10)$$

Write this above the line for $u_1 = -0.5$ applied to $x_1 = 1.5$. If we apply $u_1 = -1$ when $x_1 = 1.5$, then $x_2 = x_1 + u_1 = 0.5$, so that $J_2^* = 0.25$. Thus, in this case,

$$J_1 = \frac{u_1^2}{2} + J_2^* = \frac{(-1)^2}{2} + 0.25 = 0.75. \quad (11)$$

Having computed J_1 for each control possibility with $x_1 = 1.5$, we now need to decide on the value of u_1^* if $x_1 = 1.5$. According to (7), u_1^* is simply the value of u_1 that yields the smallest J_1 . Therefore,

$$u_1^* = -1, \quad J_1^* = 0.75. \quad (12)$$

This can be shown in the figure by placing an arrowhead on the $u_1 = -1$ control path leading from $x_1 = 1.5$ and placing the value of J_1^* in parentheses at the location corresponding to $x_1 = 1.5$.

Now we focus our attention on $x_1 = 1.0$. Using (6) to determine J_1 for the admissible values of control $u_k = 0.5, 0, -0.5, -1$, we get the numbers shown above each line emanating from $x_1 = 1.0$ in the figure. The smallest value of J_1 is 0.375, which occurs for $u_1 = -0.5$. Hence, if $x_1 = 1.0$, then

$$u_1^* = -0.5, \quad J_1^* = 0.375. \quad (13)$$

This is indicated by placing an arrowhead on the $u_1 = -0.5$ path and $J_1^* = 0.375$ in parentheses at the location $x_1 = 1.0$. In a similar manner we obtain u_1^* and J_1^* for $x_1 = 0.5$ and $x_1 = 0.0$. See the figure for the results.

Now we decrement k to 0. Let $x_0 = 1.5$. For this value of the initial state, we use (6) with each of the possible control values $u_0 = 0, -0.5$, and -1 to compute the associated values of $J_0 = u_0^2/2 + J_1^*$, where J_1^* is the number in parentheses at the state x_1 resulting from $x_1 = 1.5 + u_0$. The values of J_0 found are shown in the figure. The smallest value of J_0 is 0.5, which occurs for $u_0 = -0.5$; so if $x_0 = 1.5$, then

$$u_0^* = -0.5, \quad J_0^* = 0.5. \quad (14)$$

In a similar fashion we compute u_0^* and J_0^* for $x_0 = 1.0, 0.5$, and 0.0 . The resulting values are indicated in the figure. Note that if $x_0 = 0.5$, then a control of either $u_0 = 0$ or $u_0 = -0.5$ results in the optimal cost of $J_0^* = 0.125$; the optimal control for this initial state is not unique.

To see what we have just constructed, we can redraw Fig. 6.2-1 showing only the optimal controls u_k^* and costs to go J_k^* . See Fig. 6.2-2. Now, suppose $x_0 = 1.0$. Then to find the optimal state trajectory we need only follow the arrows. It is

$$x_0^* = 1.0, \quad x_1^* = 0.5, \quad x_2^* = 0. \tag{15}$$

Before we even apply the control, we know the optimal cost will be

$$J_0^* = 0.25. \tag{16}$$

The optimal control sequence to be applied to the plant is

$$u_0^* = -0.5, \quad u_1^* = -0.5. \tag{17}$$

Whatever the initial state x_0 is, the optimal control u_0^* and cost to go J_0^* are determined by Fig. 6.2-2 once x_0 is given. The current control u_k^* is given once the current state x_k is known; hence, the figure is just the optimal control law given in *state feedback* form.

In the variational approach to optimal control of Chapters 4 and 5, constraints make the problem solution more difficult. In the dynamic programming approach, however, more constraints mean that fewer control possibilities must be examined; constraints *simplify* the dynamic programming problem.

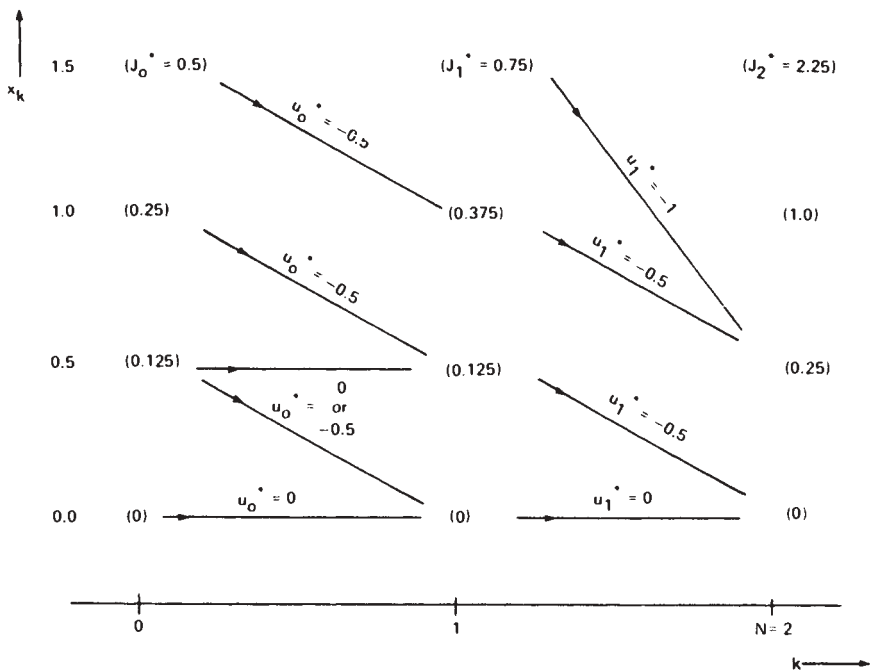


FIGURE 6.2-2 State feedback control law found by dynamic programming.

Figure 6.2-2 has the form of a *vector field* that tends to force the state down toward the origin as k increases. By changing the control weighting in J_0 in (2), the shape of the vector field changes. For instance, if we modify the cost index to

$$J_0 = x_2^2 + 2 \sum_{k=0}^1 u_k^2 \quad (18)$$

and repeat the above procedure, the state feedback control law of Fig. 6.2-3 results. A heavier control weighting has resulted in a tendency to use less control effort. The family of optimal state trajectories shown in figures like these is called a *field of extremals*.

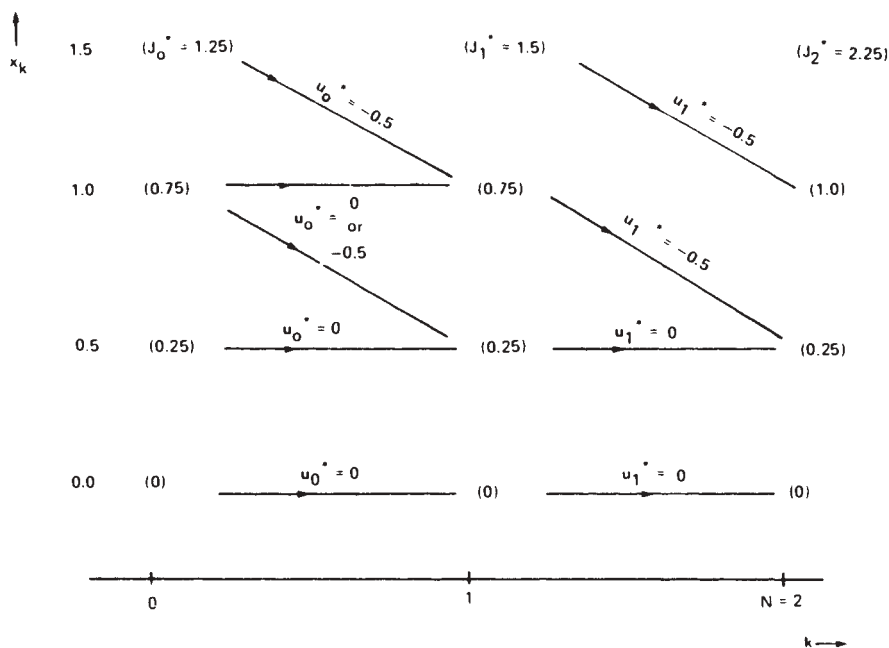


FIGURE 6.2-3 State feedback control for modified performance index.

This problem could be solved without using the optimality principle; but then *every possible* control sequence would need to be examined, and the required number of calculations would drastically increase. ■

In the problems we see how easy it is to extend the dynamic programming approach to nonlinear systems. It is instructive to compare the number of computations required to find the optimal control by investigating *all* possible control sequences to the number of computations required if dynamic programming is used. Let the plant be first order with a scalar control. Suppose the state has 10 admissible values and the control has 4 admissible values.

If the final time is $N = 1$, then comparing all $10 \times 4 = 40$ control possibilities requires 40 calculations. In this case, dynamic programming also requires 40

calculations. As N increases, however, the number of calculations required to investigate possible control sequences increases *exponentially* as $\sum_{k=1}^N 10 \cdot 4^k$. By contrast, the number of calculations required if we use the principle of optimality to restrict the set of potentially optimal controls increases *linearly* as $40N$ (Kirk 1970).

The next example shows that we can derive the linear quadratic regulator of Table 2.2-1 by using the dynamic programming approach.

Example 6.2-2. Discrete Linear Quadratic Regulator via Dynamic Programming

Let the plant

$$x_{k+1} = Ax_k + Bu_k \quad (1)$$

have associated performance index

$$J_i = \frac{1}{2} x_N^T S_N x_N + \frac{1}{2} \sum_{k=i}^{N-1} (x_k^T Q x_k + u_k^T R u_k), \quad (2)$$

with $S_N \geq 0$, $Q \geq 0$, and $R > 0$. (If the plant and weighting matrices are time varying, the development to follow still holds.) It is desired to find the optimal control u_k^* on the fixed time interval $[i, N]$ that minimizes J_i . The initial state x_i is given and the final state x_N is free.

The determination of u_k^* by using the principle of optimality (6.2-4) looks very much like our derivation of the zero-input cost (2.2-21), (2.2-22). To begin, let $k = N$ and write

$$J_N^* = \frac{1}{2} x_N^T S_N x_N, \quad (3)$$

which is the penalty for being in state x_N at time N . Now decrement k to $N-1$ and write

$$J_{N-1} = \frac{1}{2} x_{N-1}^T Q x_{N-1} + \frac{1}{2} u_{N-1}^T R u_{N-1} + \frac{1}{2} x_N^T S_N x_N. \quad (4)$$

According to (6.2-4), we need to find u_{N-1}^* by minimizing (4). To do this, use state equation 1 to write

$$\begin{aligned} J_{N-1} &= \frac{1}{2} x_{N-1}^T Q x_{N-1} + \frac{1}{2} u_{N-1}^T R u_{N-1} \\ &\quad + \frac{1}{2} (Ax_{N-1} + Bu_{N-1})^T S_N (Ax_{N-1} + Bu_{N-1}). \end{aligned} \quad (5)$$

Since there are no constraints, the minimum of J_{N-1} is found by setting

$$0 = \frac{\partial J_{N-1}}{\partial u_{N-1}} = Ru_{N-1} + B^T S_N (Ax_{N-1} + Bu_{N-1}). \quad (6)$$

Solving for the optimal control yields

$$u_{N-1}^* = -(B^T S_N B + R)^{-1} B^T S_N A x_{N-1}. \quad (7)$$

Defining the Kalman gain as

$$K_{N-1} \triangleq (B^T S_N B + R)^{-1} B^T S_N A, \quad (8)$$

we can write

$$u_{N-1}^* = -K_{N-1} x_{N-1}. \quad (9)$$

The optimal cost to go from $k = N - 1$ is found by substituting (9) into (5). If we do this and simplify, the result is

$$J_{N-1}^* = \frac{1}{2} x_{N-1}^T [(A - BK_{N-1})^T S_N (A - BK_{N-1}) + K_{N-1}^T R K_{N-1} + Q] x_{N-1}. \quad (10)$$

If we define

$$S_{N-1} \triangleq (A - BK_{N-1})^T S_N (A - BK_{N-1}) + K_{N-1}^T R K_{N-1} + Q, \quad (11)$$

this can be written

$$J_{N-1}^* = \frac{1}{2} x_{N-1}^T S_{N-1} x_{N-1}. \quad (12)$$

Now decrement to $k = N - 2$. Then

$$J_{N-2} = \frac{1}{2} x_{N-2}^T Q x_{N-2} + \frac{1}{2} u_{N-2}^T R u_{N-2} + \frac{1}{2} x_{N-1}^T S_{N-1} x_{N-1} \quad (13)$$

are the admissible costs for $N - 2$, since these are the costs that are optimal from $k = N - 1$ on. To determine u_{N-2}^* , according to (6.2-4), we must minimize (13).

Now, let us be a little tricky to save some work. Note that (13) is of the same form as (4). The optimal control and cost to go for $k = N - 2$ are therefore given by (8), (9) and (11), (12) with N there replaced by $N - 1$. If we continued to decrement k and apply the optimality principle, the result for each $k = N - 1, \dots, 1, 0$ is

$$K_k = (B^T S_{k+1} B + R)^{-1} B^T S_{k+1} A, \quad (14)$$

$$u_k^* = -K_k x_k, \quad (15)$$

$$S_k = (A - BK_k)^T S_{k+1} (A - BK_k) + K_k^T R K_k + Q, \quad (16)$$

$$J_k^* = \frac{1}{2} x_k^T S_k x_k, \quad (17)$$

where the final condition S_N for (16) is given in (2). Equation 16 is the Joseph stabilized Riccati equation, and so these results are identical to those found in Table 2.2-1 using the variational approach! ■

6.3 CONTINUOUS-TIME SYSTEMS

There are basically two approaches to the optimal control of continuous-time systems. We can discretize, solve for the optimal discrete control, and then use a zero-order hold to manufacture a digital control. Alternatively, we can solve the continuous optimal control problem to obtain a continuous input. Let us discuss both approaches.

Digital Control

Let the plant

$$\dot{x} = f(x, u, t) \quad (6.3-1)$$

have cost index of

$$J(0) = \phi(x(T), T) + \int_0^T L(x(t), u(t), t) dt. \quad (6.3-2)$$

To discretize the plant with a sampling period of τ sec, we can use the first order approximation

$$\dot{x}(k\tau) = (x_{k+1} - x_k)/\tau \quad (6.3-3)$$

to write (6.3-1) as

$$x_{k+1} = x_k + \tau f(x_k, u_k, k\tau), \quad (6.3-4)$$

where we have defined $x_k \triangleq x(k\tau)$, $u_k \triangleq u(k\tau)$ for notational convenience. Defining the discrete function

$$f^k(x_k, u_k) \triangleq x_k + \tau f(x_k, u_k, k\tau), \quad (6.3-5)$$

this can be written

$$x_{k+1} = f^k(x_k, u_k), \quad (6.3-6)$$

which is exactly (6.2-1).

To discretize the cost index, we can write

$$J(0) = \phi(x(T), T) + \sum_{k=0}^{N-1} \int_{k\tau}^{(k+1)\tau} L(x(t), u(t), t) dt, \quad (6.3-7)$$

where

$$N = \frac{T}{\tau}. \quad (6.3-8)$$

Using a first-order approximation to each integral yields

$$J(0) = \phi(x(T), T) + \sum_{k=0}^{N-1} \tau L(x_k, u_k, k\tau). \quad (6.3-9)$$

By defining the discrete functions

$$\begin{aligned} J_0 &\triangleq J(0), \\ \phi^s(N, x_N) &\triangleq \phi(x(N\tau), N\tau), \\ L^k(x_k, u_k) &\triangleq \tau L(x_k, u_k, k\tau), \end{aligned} \quad (6.3-10)$$

this becomes

$$J_0 = \phi^s(N, x_N) + \sum_{k=0}^{N-1} L^k(x_k, u_k), \quad (6.3-11)$$

which is just (6.2-2).

In the case of a **time-invariant linear plant** with **quadratic cost index**,

$$\dot{x} = Ax + Bu, \quad (6.3-12)$$

$$J(0) = \frac{1}{2}x^T(T)S(T)x(T) + \frac{1}{2} \int_0^T (x^T Q x + u^T R u) dt, \quad (6.3-13)$$

this first-order approximation to the discretized plant becomes

$$x_{k+1} = (I + A\tau)x_k + B\tau u_k, \quad (6.3-14)$$

$$J_0 = \frac{1}{2}x_N^T S_N x_N + \frac{1}{2} \sum_{k=0}^{N-1} (x_k^T Q^s x_k + u_k^T R^s u_k), \quad (6.3-15)$$

where

$$S_N \triangleq S(N\tau), \quad (6.3-16)$$

$$Q^s = Q\tau, \quad (6.3-17)$$

$$R^s = R\tau. \quad (6.3-18)$$

In this case, however, we can do better than the Euler's approximation (6.3-14) by using the **exact discrete representation of (6.3-12)** including a sampler and zero-order hold

$$x_{k+1} = A^s x_k + B^s u_k, \quad (6.3-19)$$

where

$$A^s = e^{A\tau}, \quad (6.3-20)$$

$$B^s = \int_0^\tau e^{A\tau} B dt. \quad (6.3-21)$$

Now that the plant has been discretized, dynamic programming can be used to compute u_k^* as in Section 6.2. The digital control to be applied to the actual plant (6.3-1) is then given by

$$u(t) = u_k^*, \quad k\tau \leq t < (k+1)\tau. \quad (6.3-22)$$

To use dynamic programming, the **state and control values must first be quantized**, that is, restricted to belong to some finite set of admissible values. The finer we quantize, the more accurate our digital control will be; however, as the

number of admissible values of x_k and u_k increases, so does the number of calculations required to find u_k^* . The problem can quickly become intractable even for a large digital computer. Bellman called this growth in the number of required calculations as the quantization is made finer and the number of state variables increases **the curse of dimensionality**.

An additional problem is the following. No matter how finely we quantize x_k , the relation $x_{k+1} = A^s x_k + B^s u_k$ might result in values for x_{k+1} that do not coincide with quantization levels. In this event we need to *interpolate* to find the values of $J_{k+1}^*(x_{k+1})$ to use in (6.2-4).

For further discussion on these topics, see Kirk (1970). Let us now do an example.

Example 6.3-1. Discretization, Quantization, and Interpolation

Consider the scalar plant

$$\dot{x} = ax + bu \quad (1)$$

with cost index

$$J(0) = \frac{1}{2}x^2(T) + \frac{1}{2} \int_0^T u^2(t) dt. \quad (2)$$

and a final time of $T = 1$. The state and control are constrained by

$$0 \leq x(t) \leq 1 \quad (3)$$

$$|u(t)| \leq 1. \quad (4)$$

To compute a control minimizing $J(0)$ by discrete dynamic programming, let us select a sampling period of $\tau = 0.5$. (In practice, a smaller τ should be selected for more accurate control, but this sampling period will serve for purposes of illustration.) Then the discretized plant is

$$x_{k+1} = e^{a\tau} x_k + \int_0^\tau e^{at} b dt \cdot u_k. \quad (5)$$

For simplicity, let $a = 0$ and $b = 1$ so that

$$x_{k+1} = x_k + \tau u_k = x_k + 0.5u_k. \quad (6)$$

The discretized performance index is

$$J_0 = \frac{1}{2}x_N^2 + \frac{1}{4} \sum_{k=0}^{N-1} u_k^2, \quad (7)$$

with $N = 2$.

We must now *quantize* x_k and u_k . Let us select the quantization levels

$$u_k = -1, -0.5, 0, 0.5, 1, \quad (8)$$

$$x_k = 0, 0.5, 1. \quad (9)$$

(While these levels for u_k might be satisfactory, in an actual application we would select a finer quantization for x_k .) To apply discrete dynamic programming, let us use the grid in Fig. 6.3-1, which is filled in by working backward as follows.

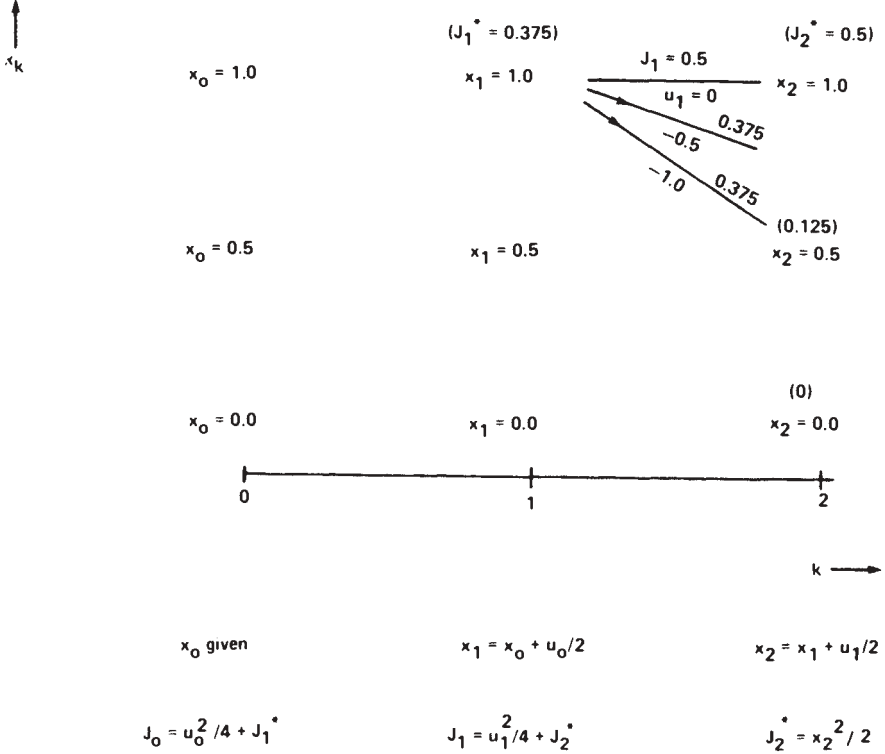


FIGURE 6.3-1 Discrete dynamic programming for continuous systems. Illustration of interpolation to find J_k^* .

Setting $k = N = 2$, we find $J_2^*(x_2)$ for admissible values of x_2 to be

$$J_2^*(x_2 = 1) = \frac{1^2}{2} = 0.5, \quad (10)$$

$$J_2^*(x_2 = 0.5) = \frac{0.5^2}{2} = 0.125, \quad (11)$$

$$J_2^*(x_2 = 0) = 0. \quad (12)$$

These values are indicated in the figure in parentheses above the respective values of x_2 .

Stepping back to $k = N - 1 = 1$, we first consider the possibility $x_1 = 1$. Applying each admissible control value as u_1 , the resulting values of x_2 are given by (6):

$$x_2 = x_1 + \frac{0}{2} = 1, \quad (13)$$

$$x_2 = x_1 - \frac{0.5}{2} = 0.75, \quad (14)$$

$$x_2 = x_1 - \frac{1}{2} = 0.5. \quad (15)$$

Lines are added to the figure between $x_1 = 1$ and these values of x_2 . Below each line is indicated the corresponding value of u_1 . Note that $x_2 = 0.75$ as given by (14) is not a quantization level for x_k .

We need to compute the admissible costs

$$J_1 = \frac{u_1^2}{4} + J_2^* \quad (16)$$

so that we can use (6.2-4) to find J_1^* and u_1^* when $x_1 = 1$. If $u_1 = 0$, then

$$J_1 = 0 + J_2^*(x_2 = 1) = 0.5. \quad (17)$$

This is placed above the line corresponding to $u_1 = 0$ at $x_1 = 1$.

Now we run into a problem. For $u_1 = -0.5$, we need to compute

$$J_1 = \frac{u_1^2}{4} + J_2^*(x_2 = 0.75). \quad (18)$$

Unfortunately, $J_2^*(x_2 = 0.75)$ was not calculated at stage $k = N = 2$, since 0.75 is not a value for x_k listed in (9). It can be approximated by *linear interpolation* as

$$J_2^*(x_2 = 0.75) = J_2^*(x_2 = 0.5) + \frac{J_2^*(x_2 = 1) - J_2^*(x_2 = 0.5)}{2} = 0.3125. \quad (19)$$

Then (18) yields

$$J_1 = 0.375. \quad (20)$$

Write this above the $u_1 = -0.5$ line in the figure.

For $u_1 = -1$, we obtain

$$J_1 = \frac{u_1^2}{4} + J_2^*(x_2 = 0.5) = 0.375. \quad (21)$$

Applying the principle of optimality (6.2-4), we select

$$\begin{aligned} u_1^* &= -0.5 \text{ or } -1.0, \\ J_1^* &= 0.375, \end{aligned} \quad (22)$$

when $x_1 = 1$. (The optimal control is not unique.) To indicate this in the figure, place J_1^* above $x_1 = 1$ in parentheses, and place arrowheads on the lines corresponding to the two possible optimal controls $u_1 = -0.5$ and $u_1 = -1$.

The remainder of the optimal control law is determined exactly as in Example 6.2-1, using interpolation as necessary to approximate $J_k^*(x_k)$ for the values of x_k not listed in (9). ■

Continuous Control and the Hamilton-Jacobi-Bellman Equation

The plant

$$\dot{x} = f(x, u, t) \quad (6.3-23)$$

has a performance index of

$$J(x(t_0), t_0) = \phi(x(T), T) + \int_{t_0}^T L(x, u, t) dt. \quad (6.3-24)$$

We are interested in determining a continuous optimal control $u^*(t)$ on a given interval $[t_0, T]$ that minimizes J and drives a given initial state $x(t_0)$ to a final state satisfying

$$\psi(x(T), T) = 0 \quad (6.3-25)$$

for a given function ψ . Let us first see what form the principle of optimality (6.1-1) takes on for this problem.

Suppose t is the current time and $t + \Delta t$ is a future time close to t . Then the cost to go $J(x(t), t)$ can be written

$$J(x, t) = \phi(x(T), T) + \int_{t+\Delta t}^T L(x, u, \tau) d\tau + \int_t^{t+\Delta t} L(x, u, \tau) d\tau. \quad (6.3-26)$$

(We are using τ as a dummy variable since t is the current time.) We can therefore say

$$J(x, t) = \int_t^{t+\Delta t} L(x, u, \tau) d\tau + J(x + \Delta x, t + \Delta t), \quad (6.3-27)$$

where $x + \Delta x$ is the state at time $t + \Delta t$ that results when the current $x(t)$ and $u(t)$ are used in (6.3-23). Note that, to first order,

$$\Delta x = f(x, u, t) \Delta t. \quad (6.3-28)$$

Equation (6.3-27) describes all possible costs to go from time t to final time T . According to the optimality principle (6.1-1), however, the only candidates for $J^*(x, t)$ are those costs $J(x, t)$ that are optimal from $t + \Delta t$ to T . Suppose that the optimal cost $J^*(x + \Delta x, t + \Delta t)$ is known for all possible $x + \Delta x$. Suppose also that the optimal control has been determined on the interval $[t + \Delta t, T]$ for each $x + \Delta x$. Then it remains only to select the current control $u(t)$ on the interval $[t, t + \Delta t]$. Hence,

$$J^*(x, t) = \min_{\substack{u(\tau) \\ t \leq \tau \leq t + \Delta t}} \left[\int_t^{t+\Delta t} L(x, u, \tau) d\tau + J^*(x + \Delta x, t + \Delta t) \right]. \quad (6.3-29)$$

This is the principle of optimality for continuous-time systems; it should be compared with (6.2-4).

Unfortunately, (6.3-29) does not provide a straightforward means of finding the optimal control and cost analytically, although some numerical solution methods use it. Let us therefore perform some further manipulations to find a way to compute these quantities.

Perform a Taylor series expansion of $J^*(x + \Delta x, t + \Delta t)$ about (x, t) and take an approximation to the integral in (6.3-29) to write, to first order,

$$J^*(x, t) = \min_{\substack{u(\tau) \\ t \leq \tau \leq t + \Delta t}} \left(L \Delta t + J^*(x, t) + \left(\frac{\partial J^*}{\partial x} \right)^T \Delta x + \frac{\partial J^*}{\partial t} \Delta t \right). \quad (6.3-30)$$

Now use (6.3-28) and note that J^* and $J_t^* \Delta t$ are independent of $u(\tau)$, $t \leq \tau \leq t + \Delta t$, to see that

$$J^*(x, t) = J^*(x, t) + J_t^* \Delta t \quad \Rightarrow \quad \min_{\substack{u(\tau) \\ t \leq \tau \leq t + \Delta t}} (L \Delta t + (J_x^*)^T f \Delta t), \quad (6.3-31)$$

or

$$-J_t^* \Delta t = \min_{\substack{u(\tau) \\ t \leq \tau \leq t + \Delta t}} (L \Delta t + (J_x^*)^T f \Delta t). \quad (6.3-32)$$

Letting $\Delta t \rightarrow 0$ yields finally

$$-\frac{\partial J^*}{\partial t} = \min_{u(t)} \left(L + \left(\frac{\partial J^*}{\partial x} \right)^T f \right). \quad (6.3-33)$$

This is a partial differential equation for the optimal cost $J^*(x, t)$. It is called the **Hamilton-Jacobi-Bellman (HJB)** equation. It is solved backward in time from $t = T$, and by setting $t_0 = T$ in (6.3-24), its boundary condition is seen to be

$$J^*(x(T), T) = \Phi(x(T), T) \quad \text{on the hypersurface } \psi(x(T), T) = 0. \quad (6.3-34)$$

If we define the **Hamiltonian function** as

$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^T f(x, u, t), \quad (6.3-35)$$

then the HJB equation can be written

$$-\frac{\partial J^*}{\partial t} = \min_u (H(x, u, J_x^*, t)). \quad (6.3-36)$$

The HJB equation provides the solution to the optimal control problem for general nonlinear systems; however, it is in most cases impossible to solve analytically. When it can be solved it provides an optimal control in state-variable feedback (i.e., closed-loop) form.

The next examples show how to use the HJB equation to find the optimal cost and control. They also demonstrate that the nonlinear optimal control scheme of Table 3.2-1 and the linear quadratic regulator of Table 3.3-1 can both be retrieved from the HJB equation.

Example 6.3-2. Optimal Control by Solution of HJB Equation

The scalar plant

$$\dot{x} = x + u \quad (1)$$

has a cost index of

$$J(t_0) = \frac{1}{2}x^2(T) + \frac{1}{2} \int_{t_0}^T ru^2 dt. \quad (2)$$

The initial time is $t_0 = 0$, the final time T is fixed, and the final state $x(T)$ is free. It is desired to find the control $u^*(t)$ minimizing $J(0)$.

The Hamiltonian is

$$H(x, u, \lambda, t) = \frac{ru^2}{2} + \lambda(x + u) \quad (3)$$

and the HJB equation is

$$-J_t^* = \min_u \left(\frac{ru^2}{2} + J_x^*(x + u) \right). \quad (4)$$

Since there are no constraints on the control u , we can determine the value of u minimizing the right-hand side of (4) by setting

$$0 = \frac{\partial H(x, u, J_x^*, t)}{\partial u} = ru + J_x^*. \quad (5)$$

The optimal control is therefore given in terms of J_x^* , the derivative of the optimal cost with respect to x , by

$$u^*(t) = -\frac{1}{r} \frac{\partial J^*(t)}{\partial x}. \quad (6)$$

Since

$$\frac{\partial^2 H}{\partial u^2} = r > 0, \quad (7)$$

u^* in (6) does indeed minimize $H(x, u, J_x^*, t)$. To find J_x^* , use (6) to evaluate

$$H^* \triangleq H(x, u^*, J_x^*, t) = \frac{r}{2} \left(\frac{J_x^*}{r} \right)^2 + J_x^* \left(x - \frac{J_x^*}{r} \right) = xJ_x^* - \frac{(J_x^*)^2}{2r}. \quad (8)$$

Using H^* as the right-hand side of (4) yields

$$-J_t^* = xJ_x^* - \frac{(J_x^*)^2}{2r}. \quad (9)$$

We must now solve the HJB equation (9) for the optimal cost J^* . Evaluating (2) at $t_0 = T$, we see that the boundary condition for (9) is

$$J^*(T) = \frac{x^2(T)}{2}, \quad (10)$$

a quadratic function of the state. Let us assume that $J^*(t)$ is a quadratic function of the state for all $t \leq T$. Then

$$J^*(t) = \frac{1}{2}s(t)x^2(t) \quad (11)$$

for some function $s(t)$ that is yet to be determined.

Taking partial derivatives of (11) gives

$$\frac{\partial J^*}{\partial x} = sx, \quad (12)$$

$$\frac{\partial J^*}{\partial t} = \frac{\dot{s}x^2}{2}, \quad (13)$$

which we substitute into (9) to obtain

$$\frac{-\dot{s}x^2}{2} = sx^2 - \frac{s^2x^2}{2r}. \quad (14)$$

This must hold for all $x(t)$, so we obtain the Riccati equation in $s(t)$

$$\dot{s} = \frac{s^2}{r} - 2s. \quad (15)$$

Comparing (10) and (11), the boundary condition for (14) is found to be

$$s(T) = 1. \quad (16)$$

Equation (15) can be integrated using separation of variables:

$$\int_t^T \frac{ds}{s^2/r - 2s} = \int_t^T dt. \quad (17)$$

The result is

$$s(t) = \frac{2re^{2(T-t)}}{e^{2(T-t)} + (2r-1)}. \quad (18)$$

This consistent solution for $s(t)$ shows that assumption (11) was valid.

Collecting our results, we see that the optimal cost is given by (11) with $s(t)$ given by (18). The optimal control (6) is

$$u^*(t) = -\frac{1}{r}J_x^*(t) = -\frac{s(t)}{r}x(t). \quad (19)$$

This is in the form of a time-varying state-variable feedback. ■

Example 6.3-3. Deriving the Euler Equations from the HJB Equation

We can easily show that the function

$$\lambda(t) \triangleq \frac{\partial J^*}{\partial x} \quad (1)$$

is a costate for the plant (6.3-23) if $J^*(x, t)$ satisfies the HJB equation

$$-\frac{\partial J^*}{\partial t} = \min_u (L + \lambda^T f). \quad (2)$$

See Athans and Falb (1966) and Bryson and Ho (1975).

Since $u(t)$ is unconstrained, the minimization in (2) can be carried out by setting

$$0 = \frac{\partial L}{\partial u} + \frac{\partial f^T}{\partial u} \lambda. \quad (3)$$

This is just the stationarity condition from Table 3.2-1.

To examine the dynamics of $\lambda(t)$, write

$$-\frac{d\lambda}{dt} = -\frac{\partial^2 J^*}{\partial t \partial x} - \frac{\partial^2 J^*}{\partial x^2} \dot{x}. \quad (4)$$

Next, take the partial derivative of (2) with respect to x , realizing that $u = u(x, t)$ to see that

$$-\frac{\partial^2 J^*}{\partial x \partial t} = \min_u \left(\frac{\partial L}{\partial x} + \frac{\partial u^T}{\partial x} \frac{\partial L}{\partial u} + \frac{\partial \lambda^T}{\partial x} f + \frac{\partial f^T}{\partial x} \lambda + \left(\frac{\partial f}{\partial u} \frac{\partial u}{\partial x} \right)^T \lambda \right). \quad (5)$$

At the minimum, (3) holds so that

$$-\frac{\partial^2 J^*}{\partial x \partial t} = \frac{\partial L}{\partial x} + \frac{\partial \lambda^T}{\partial x} f + \frac{\partial f^T}{\partial x} \lambda. \quad (6)$$

Use (1), (6), and $\dot{x} = f(x, u, t)$ in (4) to obtain

$$-\frac{d\lambda}{dt} = \frac{\partial f^T}{\partial x} \lambda + \frac{\partial L}{\partial x}, \quad (7)$$

which is exactly the costate equation in Table 3.2-1.

If $u(t)$ is constrained, then we must select the optimal control $u^*(t)$ from the set of admissible controls to minimize $H(x, u, \lambda^*, t)$. It is apparent that the right-hand side of the HJB equation is just Pontryagin's minimum principle from Section 5.2. ■

Example 6.3-4. Deriving the Continuous Linear Quadratic Regulator from the HJB Equation

Let the plant be

$$\dot{x} = Ax + Bu \quad (1)$$

with cost index

$$J(t_0) = \frac{1}{2}x^T(T)S(T)x(T) + \frac{1}{2}\int_{t_0}^T (x^T Qx + u^T Ru)dt. \quad (2)$$

It is desired to find the optimal control to drive the state from a given $x(t_0)$ so that $J(t_0)$ is minimized. The final time T is fixed and the final state $x(T)$ is free. We can derive the Riccati-equation-based linear quadratic regulator of Table 3.3-1 from the HJB equation as follows.

Form the Hamiltonian

$$H = \frac{1}{2}(x^T Qx + u^T Ru) + \lambda^T (Ax + Bu). \quad (3)$$

To minimize H , set

$$0 = \frac{\partial H}{\partial u} = Ru + B^T \lambda, \quad (4)$$

so the optimal control is

$$u^* = -R^{-1}B^T \lambda. \quad (5)$$

Since

$$\frac{\partial^2 H}{\partial u^2} = R > 0, \quad (6)$$

$u^*(t)$ is a *minimizing* control. Use (5) in (3) to find the optimal Hamiltonian

$$H^* = \frac{1}{2}x^T Qx + \lambda^T Ax - \frac{1}{2}\lambda^T BR^{-1}B^T \lambda. \quad (7)$$

Setting $\lambda = J_x^*$, the HJB equation is

$$-J_t^* = \frac{1}{2}x^T Qx + (J_x^*)^T Ax - \frac{1}{2}(J_x^*)^T BR^{-1}B^T J_x^*. \quad (8)$$

The boundary condition is

$$J^*(T) = \frac{1}{2}x^T(T)S(T)x(T). \quad (9)$$

Since $J^*(T)$ is quadratic in the state, let us assume that this is true for $J^*(t)$, $t \leq T$. That is, assume there is a symmetric matrix $S(t)$ such that

$$J^*(t) = \frac{1}{2}x^T(t)S(t)x(t) \quad (10)$$

for all $t \leq T$. Substituting (10) into (8) yields

$$0 = \frac{1}{2}x^T \dot{S}x + \frac{1}{2}x^T Qx + x^T SAx - \frac{1}{2}x^T SBR^{-1}B^T Sx, \quad (11)$$

or

$$x^T(\dot{S} + 2SA - SBR^{-1}B^T S + Q)x = 0. \quad (12)$$

The matrix in parentheses, which we shall call M , is not symmetric, since SA is not symmetric (the other terms are). Write

$$M = M_S + M_a \quad (13)$$

where M_S is symmetric, $M_S = M_S^T$, and M_a is antisymmetric, $M_a = -M_a^T$. Then

$$x^T M x = x^T M_S x + x^T M_a x. \quad (14)$$

But, since (14) is a scalar,

$$x^T M_a x = x^T M_a^T x = -x^T M_a x = 0, \quad (15)$$

hence,

$$x^T M x = x^T M_S x. \quad (16)$$

The symmetric part of a matrix is given by

$$M_S = (M + M^T)/2. \quad (17)$$

Therefore, (12) is equivalent to

$$x^T (\dot{S} + A^T S + SA - SBR^{-1}B^T S + Q)x = 0 \quad (18)$$

Since (18) holds for all $x(t_0)$ and hence for all state trajectories $x(t)$, we require

$$-\dot{S} = A^T S + SA - SBR^{-1}B^T S + Q. \quad (19)$$

This is the Riccati equation in Table 3.3-1. The optimal control is given by (5) as the state feedback

$$u^*(t) = -R^{-1}B^T Sx(t), \quad (20)$$

and the optimal cost is given by (10). ■

PROBLEMS

Section 6.1

6.1-1. A routing network is shown in Fig. P6.1-1. Find the optimal path from x_0 to x_6 if only movement from left to right is permitted. Now find the optimal path from any node as a state-variable feedback.

Section 6.2

6.2-1. Dynamic programming for bilinear system. Consider the scalar bilinear system

$$x_{k+1} = x_k u_k + u_k^2, \quad (1)$$

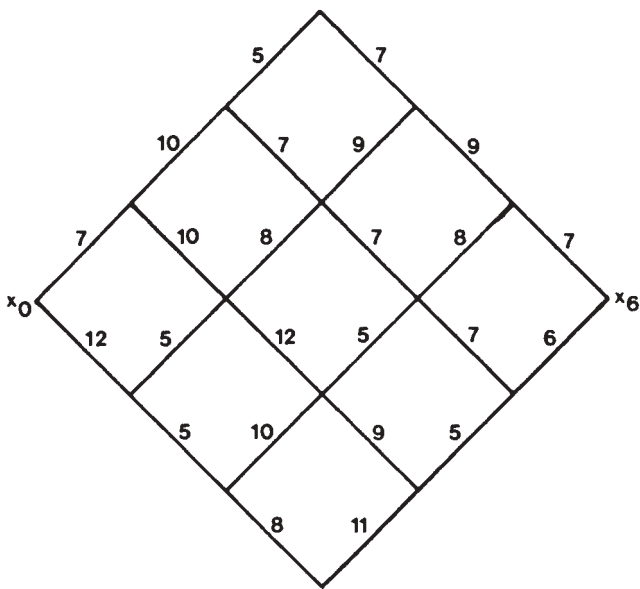


FIGURE P6.1-1 A routing network.

with cost index

$$J_0 = x_N^2 + \sum_{k=0}^{N-1} x_k u_k. \quad (2)$$

Let $N = 2$. The control is constrained to take on values of

$$u_k = -1 \text{ or } 1 \quad (3)$$

and the state to take on values of

$$x_k = -1, 0, 1, \text{ or } 2. \quad (4)$$

- Use dynamic programming to find an optimal state feedback control law.
- Let $x_0 = 2$. Find the optimal cost, control sequence, and state trajectory.

6.2-2. Software for dynamic programming. Write a MATLAB program to generate Fig. 6.2.2.

6.2-3. Effect of control weighting on field of extremals. Redo Example 6.2-1 if

$$J_0 = x_N^2 + \frac{1}{2} \sum_{k=0}^{N-1} r u_k^2$$

for $r = \frac{1}{4}, 4, 8$ and compare the resulting fields of extremals.

6.2-4. Cargo loading. A vessel is to be loaded with stocks of N items. Each item $k = 1, \dots, N$ has a weight of w_k and a known value of v_k . The maximum allowed weight of the cargo is W . We want to determine the most valuable cargo load without overloading the vessel.

Define

u_k = number of units of item loaded,

x_k = weight capacity available for items $k, k + 1, \dots, N$.

Then the problem becomes the following: Maximize

$$J_1 = \sum_{k=1}^N v_k u_k$$

subject to the constraint

$$\sum_{k=1}^N w_k u_k \leq W.$$

- Write the state equation.
- List the constraints for x_k and u_k .

6.2-5. Cargo loading. A ship is to be loaded with stocks of three items, whose weights and values are given in the following table:

Item	Weight (lb)	Value (\$)
Dishwashers	100	360
Washing machines	125	475
Refrigerators	250	1000

- Let
 - u_1 = number of refrigerators loaded,
 - u_2 = number of washing machines loaded,
 - u_3 = number of dishwashers loaded.

Use the results of Problem 6.2-4 to find the optimal loading policy if the maximum weight allowed is 730 lb.

- By ordering the decision stages in order of increasing value per weight, show how to solve the same problem in a trivial manner.

Section 6.3

6.3-1. Software for dynamic programming

- Write a MATLAB program to generate a feedback law in the tabular form of Fig. 6.2-2 for the scalar system

$$x_{k+1} = ax_k + bu_k, \quad (1)$$

with cost index

$$J_0 = \frac{1}{2}s_N x_N^2 + \frac{1}{2} \sum_{k=0}^{N-1} (qx_k^2 + ru_k^2). \quad (2)$$

The state x_k can take on values of

$$XA(1), \quad XA(2), \quad \dots, \quad XA(NS) \quad (3)$$

and the control u_k can take on values

$$UA(1), \quad UA(2), \quad \dots, \quad UA(NU), \quad (4)$$

all of which are specified. Assume that x_{k+1} can take on only the values (3).

- b. Add to your program an interpolation scheme to find $J_{k+1}^*(x_{k+1})$ in the event that x_{k+1} takes on a value not listed in (3).

6.3-2. Optimal control by solution of the HJB equation. Let

$$\ddot{x} = u, \quad (1)$$

$$J(t_0) = \frac{1}{2}x^2(T) + \frac{1}{2} \int_{t_0}^T ru^2 dt, \quad (2)$$

with $t_0 = 0$, $T = 2$.

- a. Write the HJB equation, eliminating $u(t)$.
b. Assume that

$$J^*(t) = \frac{1}{2}s_1(t)x^2(t) + s_2(t)x(t)\dot{x}(t) + \frac{1}{2}s_3(t)\dot{x}^2(t) \quad (3)$$

for some $s_1(t)$, $s_2(t)$, $s_3(t)$. Use the HJB equation to determine coupled differential equations for these $s_i(t)$. Compare with Example 3.3-5. Find boundary conditions for the equations. Express the optimal control as a linear state feedback in terms of the $s_i(t)$.

6.3-3. Continuous-time tracker via the HJB equation. Use the HJB equation to solve the linear quadratic tracking problem of Section 4.1. The procedure should basically follow Example 6.3-4. (Assume $J(x, t) = \frac{1}{2}x^T s x - x^T v + w$ and find equations for $s(t)$, $v(t)$, and $w(t)$.)

11

REINFORCEMENT LEARNING AND OPTIMAL ADAPTIVE CONTROL

In this book we have presented a variety of methods for the analysis and design of optimal control systems. Design has generally been based on solving matrix design equations assuming full knowledge of the system dynamics. Optimal control is fundamentally a backward-in-time problem, as we have seen especially clearly in Chapter 6 Dynamic Programming. Optimal controllers are normally designed offline by solving Hamilton-Jacobi-Bellman (HJB) equations, for example, the Riccati equation, using complete knowledge of the system dynamics. The controller is stored and then implemented online in real time. In the linear quadratic case, this means the feedback gains are stored. Determining optimal control policies for nonlinear systems requires the offline solution of nonlinear HJB equations, which are often difficult or impossible to solve analytically.

In this book, we have developed many matrix design equations whose solutions yield various sorts of optimal controllers. This includes the Riccati equation, the HJB equation, and the design equations in Chapter 10 for differential games. These equations are normally solved offline. In this chapter we give practical methods for solving these equations online in real time using data measured along the system trajectories. Some of the methods do not require knowledge of the system dynamics.

In practical applications, it is often important to be able to design controllers online in real time without having complete knowledge of the plant dynamics. Modeling uncertainties may exist, including inaccurate parameters, unmodeled high-frequency dynamics, and disturbances. Moreover, both the system dynamics and the performance objectives may change with time. A class of controllers known as adaptive controllers learn online to control unknown systems using data measured in real time along the system trajectories. While learning the control solutions, adaptive controllers are able to guarantee stability and system

performance. Adaptive control and optimal control represent different philosophies for designing feedback controllers. Adaptive controllers are not usually designed to be optimal in the sense of minimizing user-prescribed performance functions. Indirect adaptive controllers use system identification techniques to identify the system parameters, then use the obtained model to solve optimal design equations (Ioannou and Fidan 2006). Adaptive controllers may satisfy certain inverse optimality conditions, as shown in Li and Krstic (1997).

In this chapter we show how to design optimal controllers online in real time using data measured along the system trajectories. We present several adaptive control algorithms that converge to optimal control solutions. Several of these algorithms do not require full knowledge of the plant dynamics. In the LQR case, for instance, this amounts to using adaptive control techniques to learn the solution of the algebraic Riccati equation online without knowing the plant matrix A . In the nonlinear case, these algorithms allow the approximate solution of complicated HJ equations that cannot be exactly solved using analytic means. Design of optimal controllers online allows the performance objectives, such as the LQR weighting matrices, to change slowly in real time as control objectives change.

The framework we use in this chapter is the theory of Markov decision processes (MDP). It is shown here that MDP provide a natural framework that connects reinforcement learning, optimal control, adaptive control, and cooperative control (Tsitsiklis 1984, Jadbabaie et al. 2003, Olfati-Saber and Murray 2004). Some examples are given of cooperative decision and control of dynamical systems on communication graphs.

11.1 REINFORCEMENT LEARNING

Dynamic programming (Chapter 6) is a method for determining optimal control solutions using **Bellman's principle** (Bellman 1957) by working **backward in time** from some desired goal states. Designs based on dynamic programming yield **offline solution algorithms**, which are then stored and implemented online forward in time. In this chapter we show that techniques based on reinforcement learning allow the design of optimal decision systems that learn optimal solutions online and forward in time. **This allows both the system dynamics and the performance objectives to vary slowly with time**. The methods studied here depend on solving a certain equation, known as Bellman's equation (Sutton and Barto 1998), whose solution both evaluates the performance of current control policies and provides methods for improving those policies.

Reinforcement learning (RL) refers to a class of learning methods that allow the design of **adaptive controllers** that learn online, **in real time**, the solutions to user-prescribed optimal control problems. In machine learning, reinforcement learning (Mendel and MacLaren 1970, Werbos 1991, Werbos 1992, Bertsekas and Tsitsiklis 1996, Sutton and Barto 1998, Powell 2007, Cao 2007, Busoniu et al. 2009) is a method for **solving optimization problems that involves an actor or agent that interacts with its environment and modifies its actions, or control policies, based on stimuli received in response to its actions**. Reinforcement

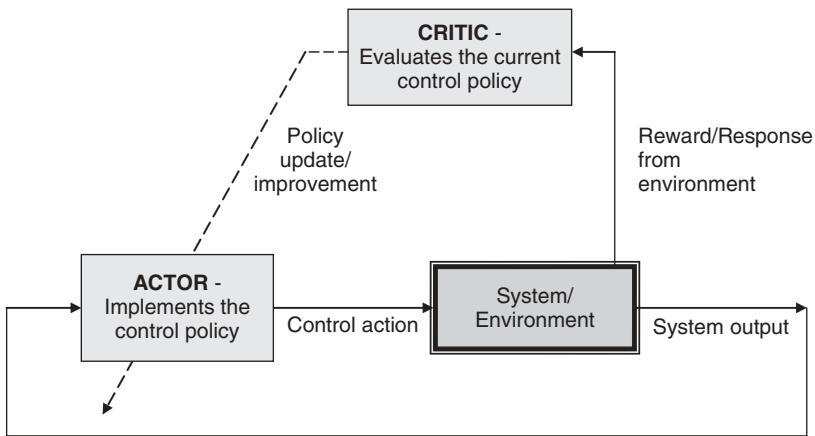


FIGURE 11.1-1 Reinforcement learning with an actor–critic structure. This structure provides methods for learning optimal control solutions online based on data measured along the system trajectories.

learning is inspired by natural learning mechanisms, where animals adjust their actions based on reward and punishment stimuli received from the environment. Other reinforcement learning mechanisms operate in the human brain, where the dopamine neurotransmitter in the basal ganglia acts as a reinforcement informational signal that favors learning at the level of the neuron (Doya et al. 2001, Schultz 2004).

The actor–critic structures shown in Figure 11.1-1 (Barto et al. 1983) are one type of reinforcement learning algorithm. These structures give forward-in-time algorithms for computing optimal decisions that are implemented in real time where an actor component applies an action, or control policy, to the environment, and a critic component assesses the value of that action. The learning mechanism supported by the actor–critic structure has two steps, namely, policy evaluation, executed by the critic, followed by policy improvement, performed by the actor. The policy evaluation step is performed by observing from the environment the results of applying current control actions. These results are evaluated using a performance index that quantifies how close to optimal the current action is. Performance can be defined in terms of optimality objectives, such as minimum fuel, minimum energy, minimum risk, or maximum reward. Based on the assessment of the performance, one of several schemes can then be used to modify or improve the control policy in the sense that the new policy yields a performance value that is improved relative to the previous value. In this scheme, reinforcement learning is a means of learning optimal behaviors by observing the real-time responses from the environment to nonoptimal control policies.

Direct adaptive controllers tune the controller parameters to directly identify the controller. Indirect adaptive controllers identify the system, and the identified model is then used in design equations to compute a controller. Actor–critic

schemes are a logical extension of this sequence in that they identify the performance value of the current control policy, and then use that information to update the controller.

This chapter presents the main ideas and algorithms of reinforcement learning and applies them to design adaptive feedback controllers that converge online to optimal control solutions relative to prescribed cost metrics. Using these techniques, we can solve online in real time the Riccati equation, the HJB equation, and the design equations in Chapter 10 for differential games. Some of the methods given here do not require knowledge of the system dynamics.

We start from a discussion of MDP and specifically focus on a family of techniques known as approximate or adaptive dynamic programming (ADP) (Werbos 1989, 1991, 1992) or neurodynamic programming (Bertsekas and Tsitsiklis 1996). We show that the use of reinforcement learning techniques provides optimal control solutions for linear or nonlinear systems using adaptive control techniques.

This chapter shows that reinforcement learning methods allow the solution of HJB design equations online, forward in time, and without knowing the full system dynamics. Specifically, the drift dynamics is not needed, but the input coupling function is needed. In the linear quadratic case, these methods determine the solution to the algebraic Riccati equation online, without solving the equation and without knowing the system A matrix. This chapter presents an expository development of ideas from reinforcement learning and ADP and their applications in feedback control systems. Surveys of ADP are given in Si et al. (2004), Lewis, Lendaris, and Liu (2008), Balakrishnan et al. (2008), Wang et al. (2009), and Lewis and Vrabie (2009).

11.2 MARKOV DECISION PROCESSES

A natural framework for studying RL is provided by Markov decision processes (MDP). Many dynamical decision problems can be cast into the framework of MDP. Included are feedback control systems for human engineered systems, feedback regulation mechanisms for population balance and survival of species (Darwin 1859, Luenberger 1979), decision-making in multiplayer games, and economic mechanisms for regulation of global financial markets. Therefore, we provide a development of MDP here. References for this material include (Bertsekas and Tsitsiklis 1996, Sutton and Barto 1998, Busoniu et al. 2009).

Consider the Markov decision process (MDP) (X, U, P, R) , where X is a set of states and U is a set of actions or controls. The transition probabilities $P: X \times U \times X \rightarrow [0, 1]$ give for each state $x \in X$ and action $u \in U$ the conditional probability $P_{x,x'}^u = \Pr\{x'|x, u\}$ of transitioning to state $x' \in X$ given the MDP is in state x and takes action u . The cost function $R: X \times U \times X \rightarrow \mathbb{R}$ gives the expected immediate cost $R_{xx'}^u$ paid after transition to state $x' \in X$ given the MDP starts in state $x \in X$ and takes action $u \in U$. The Markov property refers to the fact that transition probabilities $P_{x,x'}^u$ depend only on the current state x and not on the history of how the MDP attained that state. An MDP is shown in Figure 11.2-1.

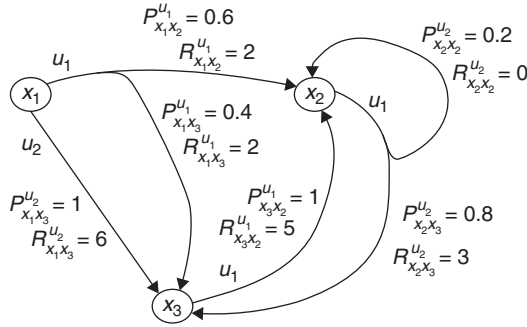


FIGURE 11.2-1 MDP shown as a finite state machine with controlled state transitions and costs associated with each transition.

The basic problem for MDP is to find a mapping $\pi: X \times U \rightarrow [0, 1]$ that gives for each state x and action u the conditional probability $\pi(x, u) = \Pr\{u|x\}$ of taking action u given the MDP is in state x . Such a mapping is termed a (closed-loop) control or action strategy or policy.

The strategy or policy $\pi(x, u) = \Pr\{u|x\}$ is called *stochastic* or *mixed* if there is a nonzero probability of selecting more than one control when in state x . We can view mixed strategies as probability distribution vectors having as component i the probability of selecting the i th control action while in state $x \in X$. If the mapping $\pi: X \times U \rightarrow [0, 1]$ admits only one control (with probability 1) when in every state x , it is called a *deterministic* policy. Then, $\pi(x, u) = \Pr\{u|x\}$ corresponds to a function mapping states into controls $\mu(x): X \rightarrow U$.

Most work on reinforcement learning has been done for MDP that have finite state and action spaces. These are termed *finite MDP*.

Optimal Sequential Decision Problems

Dynamical systems evolve causally through time. Therefore, we consider sequential decision problems and impose a discrete stage index k such that the MDP takes an action and changes states at nonnegative integer stage values k . The stages may correspond to time or more generally to sequences of events. We refer to the stage value as the time. Denote state values and actions at time k by x_k, u_k . MDP traditionally evolve in discrete time.

Naturally occurring systems, including biological organisms and living species, have available a limited set of resources for survival and increase. Natural systems are therefore optimal in some sense. Likewise, human engineered systems should be optimal in terms of conserving resources such as cost, time, fuel, energy. Thus, it is important to capture the notion of optimality in selecting control policies for MDP.

Define, therefore, a *stage cost* at time k by $r_k = r_k(x_k, u_k, x_{k+1})$. Then $R_{xx'}^u = E\{r_k | x_k = x, u_k = u, x_{k+1} = x'\}$, with $E\{\cdot\}$ the expected value operator. Define

a performance index as the sum of future costs over time interval $[k, k + T]$

$$J_{k,T} = \sum_{i=0}^T \gamma^i r_{k+i} = \sum_{i=k}^{k+T} \gamma^{i-k} r_i, \quad (11.2-1)$$

where $0 \leq \gamma < 1$ is a discount factor that reduces the weight of costs incurred further in the future. T is a planning horizon over which decisions are to be made.

Traditional usage of MDP in the fields of computational intelligence and economics consider r_k as a reward incurred at time k , also known as utility, and $J_{k,T}$ as a discounted return, also known as strategic reward. We refer instead to stage costs and discounted future costs to be consistent with objectives in the control of dynamical systems. We may sometimes loosely call r_k the utility.

Consider that an agent selects a control policy $\pi_k(x_k, u_k)$ and uses it at each stage k of the MDP. We are primarily interested in stationary policies, where the conditional probabilities $\pi_k(x_k, u_k)$ are independent of k . Then $\pi_k(x, u) = \pi(x, u) = \Pr\{u|x\}$, for all k . Nonstationary deterministic policies have the form $\pi = \{\mu_0, \mu_1, \dots\}$, where each entry is a function $\mu_k(x): X \rightarrow U; k = 0, 1, \dots$. Stationary deterministic policies are independent of time so that $\pi = \{\mu, \mu, \dots\}$.

Select a fixed stationary policy $\pi(x, u) = \Pr\{u|x\}$. Then the ("closed-loop") MDP reduces to a Markov chain with state space X . That is, the transition probabilities between states are fixed with no further freedom of choice of actions. The transition probabilities of this Markov chain are given by

$$p_{x,x'} \equiv P_{x,x'}^\pi = \sum_u \Pr\{x'|x, u\} \Pr\{u|x\} = \sum_u \pi(x, u) P_{x,x'}^u, \quad (11.2-2)$$

where we have used the Chapman-Kolmogorov identity.

Under the assumption that the Markov chain corresponding to each policy (with transition probabilities given as in (11.2-2)) is ergodic, it can be shown that every MDP has a stationary deterministic optimal policy (Wheeler and Narendra 1986, Bertsekas and Tsitsiklis 1996). A Markov chain is ergodic if all states are positive recurrent and aperiodic (Luenberger 1979). Then, for a given policy there exists a stationary distribution $p_\pi(x)$ over X that gives the steady-state probability the Markov chain is in state x . We shall soon discuss more about the closed-loop Markov chain.

The value of a policy is defined as the conditional expected value of future cost when starting in state x at time k and following policy $\pi(x, u)$ thereafter,

$$V_k^\pi(x) = E_\pi\{J_{k,T}|x_k = x\} = E_\pi\left\{\sum_{i=k}^{k+T} \gamma^{i-k} r_i | x_k = x\right\}. \quad (11.2-3)$$

Here, $E_\pi\{\cdot\}$ is the expected value given that the agent follows policy $\pi(x, u)$. $V^\pi(x)$ is known as the value function for policy $\pi(x, u)$. It tells the value of being in state x given that the policy is $\pi(x, u)$.

An important objective of MDP is to determine a policy $\pi(x, u)$ to **minimize the expected future cost**

$$\pi^*(x, u) = \arg \min_{\pi} V_k^{\pi}(s) = \arg \min_{\pi} E_{\pi} \left\{ \sum_{i=k}^{k+T} \gamma^{i-k} r_i | x_k = x \right\}. \quad (11.2-4)$$

This is termed the **optimal policy**, and the corresponding **optimal value** is given as

$$V_k^*(x) = \min_{\pi} V_k^{\pi}(x) = \min_{\pi} E_{\pi} \left\{ \sum_{i=k}^{k+T} \gamma^{i-k} r_i | x_k = x \right\}. \quad (11.2-5)$$

In computational intelligence and economics, when we talk about utilities and rewards, we are interested in maximizing the expected performance index.

A Backward Recursion for the Value

By using the Chapman-Kolmogorov identity and the Markov property we may write the value of policy $\pi(x, u)$ as

$$V_k^{\pi}(x) = E_{\pi} \{ J_k | x_k = x \} = E_{\pi} \left\{ \sum_{i=k}^{k+T} \gamma^{i-k} r_i | x_k = x \right\} \quad (11.2-6)$$

$$V_k^{\pi}(x) = E_{\pi} \left\{ r_k + \gamma \sum_{i=k+1}^{k+T} \gamma^{i-(k+1)} r_i | x_k = x \right\} \quad (11.2-7)$$

$$V_k^{\pi}(x) = \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u \left[R_{xx'}^u + \gamma E_{\pi} \left\{ \sum_{i=k+1}^{k+T} \gamma^{i-(k+1)} r_i | x_{k+1} = x' \right\} \right]. \quad (11.2-8)$$

Therefore, the value function for policy $\pi(x, u)$ satisfies

$$V_k^{\pi}(x) = \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_{k+1}^{\pi}(x')]. \quad (11.2-9)$$

This provides a backward recursion for the value at time k in terms of the value at time $k + 1$.

Dynamic Programming

The optimal cost can be written as

$$V_k^*(x) = \min_{\pi} V_k^{\pi}(x) = \min_{\pi} \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_{k+1}^{\pi}(x')]. \quad (11.2-10)$$

Bellman's optimality principle (Bellman 1957) states that "an optimal policy has the property that **no matter what the previous control actions have been, the remaining controls constitute an optimal policy with regard to the state resulting**

from those previous controls.” Therefore, we may write

$$V_k^*(x) = \min_{\pi} \sum_{\mathbf{u}} \pi(x, \mathbf{u}) \sum_{x'} P_{xx'}^{\mathbf{u}} [R_{xx'}^{\mathbf{u}} + \gamma V_{k+1}^*(x')]. \quad (11.2-11)$$

Suppose we now apply an arbitrary control $\mathbf{u}$ at time k and the optimal policy from time $k+1$ on. Then Bellman’s optimality principle says that the optimal control at time k is given by

$$\mathbf{u}_k^* = \arg \min_{\mathbf{u}} \sum_{\mathbf{u}} \pi(x, \mathbf{u}) \sum_{x'} P_{xx'}^{\mathbf{u}} [R_{xx'}^{\mathbf{u}} + \gamma V_{k+1}^*(x')]. \quad (11.2-12)$$

Under the assumption that the Markov chain corresponding to each policy with transition probabilities given as in (11.2-2) is ergodic, every MDP has a stationary deterministic optimal policy. Then we can equivalently minimize the conditional expectation over all actions $\mathbf{u}$ in state x . Therefore,

$$V_k^*(x) = \min_{\mathbf{u}} \sum_{x'} P_{xx'}^{\mathbf{u}} [R_{xx'}^{\mathbf{u}} + \gamma V_{k+1}^*(x')], \quad (11.2-13)$$

$$\mathbf{u}_k^* = \arg \min_{\mathbf{u}} \sum_{x'} P_{xx'}^{\mathbf{u}} [R_{xx'}^{\mathbf{u}} + \gamma V_{k+1}^*(x')]. \quad (11.2-14)$$

The backward recursion (11.2-11), (11.2-13) forms the basis for dynamic programming (DP), which gives offline methods for working backward in time to determine optimal policies. DP was discussed in Chapter 6. It is an offline procedure for finding the optimal value and optimal policies that requires knowledge of the complete system dynamics in the form of transition probabilities $P_{xx'}^{\mathbf{u}} = \Pr\{x'|x, \mathbf{u}\}$ and expected costs $R_{xx'}^{\mathbf{u}} = E\{r_k|x_k = x, u_k = \mathbf{u}, x_{k+1} = x'\}$. Once the optimal control has been found offline using DP, it is stored and implemented on the system online forward in time.

Bellman Equation and Bellman Optimality Equation (HJB)

Dynamic programming is a backward-in-time method for finding the optimal value and policy. By contrast, reinforcement learning is concerned with finding optimal policies based on causal experience by executing sequential decisions that improve control actions based on the observed results of using a current policy. This requires the derivation of methods for finding optimal values and optimal policies that can be executed forward in time. Here we develop the Bellman equation, which is the basis for such methods.

To derive forward-in-time methods for finding optimal values and optimal policies, set now the time horizon T to infinity and define the infinite-horizon cost

$$J_k = \sum_{i=0}^{\infty} \gamma^i r_{k+i} = \sum_{i=k}^{\infty} \gamma^{i-k} r_i. \quad (11.2-15)$$

The associated (infinite-horizon) value function for policy $\pi(x, u)$ is

$$V^\pi(x) = E_\pi\{J_k|x_k = x\} = E_\pi\left\{\sum_{i=k}^{\infty} \gamma^{i-k} r_i | x_k = x\right\}. \quad (11.2-16)$$

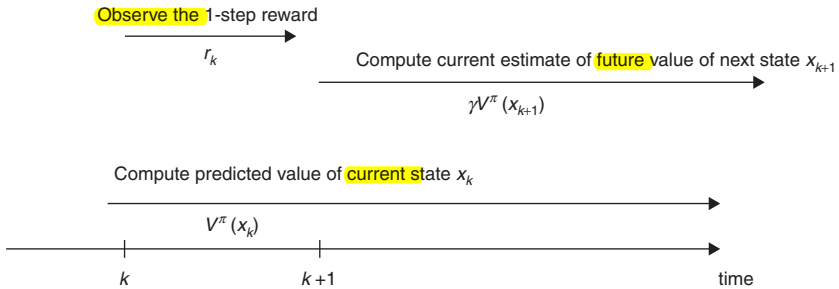
By using (11.2-8) with $T = \infty$ we see that the value function for policy $\pi(x, u)$ satisfies the *Bellman equation*

$$V^\pi(x) = \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')]. \quad (11.2-17)$$

This equation is of extreme importance in reinforcement learning. It is important that the same value function appears on both sides. This is due to the fact that the infinite-horizon cost was used. Therefore, (11.2-17) can be interpreted as a consistency equation that must be satisfied by the value function at each time stage. The Bellman equation expresses a relation between the current value of being in state x and the value(s) of being in the next state x' given that policy $\pi(x, u)$ is used.

The Bellman equation forms the basis for a family of reinforcement learning algorithms for finding optimal policies by using causal experiences received stagewise forward in time. In this context, the meaning of the Bellman equation is shown in Figure 11.2-2, where $V^\pi(x)$ may be considered as a predicted performance, $\sum_u \pi(x, u) \sum_{x'} P_{xx'}^u R_{xx'}^u$ the observed one-step reward, and $V^\pi(x')$

1. Apply control action



2. Update predicted value to satisfy the Bellman equation

$$V^\pi(x_k) = r_k + \gamma V^\pi(x_{k+1})$$

3. Improve control action

FIGURE 11.2-2 Temporal difference interpretation of Bellman equation, showing how the Bellman equation captures the action, observation, evaluation, and improvement mechanisms of reinforcement learning.

a current estimate of future behavior. These notions are capitalized on in the subsequent discussion of temporal difference learning, which uses them to develop adaptive control algorithms that can learn optimal behavior online in real-time applications.

If the MDP is finite and has N states, then the Bellman equation (11.2-17) is a system of N simultaneous linear equations for the value $V^\pi(x)$ of being in each state x given the current policy $\pi(x, u)$. The optimal value satisfies

$$V^*(x) = \min_{\pi} V^\pi(x) = \min_{\pi} \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')]. \quad (11.2-18)$$

Bellman's optimality principle then yields the Bellman optimality equation

$$V^*(x) = \min_{\pi} V^\pi(x) = \min_{\pi} \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^*(x')]. \quad (11.2-19)$$

Equivalently, under the ergodicity assumption on the Markov chains corresponding to each policy, we have

$$V^*(x) = \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^*(x')]. \quad (11.2-20)$$

If the MDP is finite and has N states, then the Bellman optimality equation is a system of N nonlinear equations for the optimal value $V^*(x)$ of being in each state. The optimal control is given by

$$u^* = \arg \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^*(x')]. \quad (11.2-21)$$

Though the ideas just introduced may not seem familiar to the control engineer, it is shown in the next examples that they correspond to some familiar notions in feedback control system theory.

Example 11.2-1. Bellman Equation for Discrete-time Linear Quadratic Regulator (DT LQR)

This example studies the Bellman equation for the discrete-time LQR and shows that it is closely related to ideas developed in Chapter 2.

a. MDP Dynamics for Deterministic DT Systems

Consider the discrete-time (DT) linear quadratic regulator (LQR) problem where the MDP is deterministic and satisfies the state transition equation

$$x_{k+1} = Ax_k + Bu_k, \quad (11.2-22)$$

with k the discrete time index. The associated infinite horizon performance index has deterministic stage costs and is

$$J_k = \frac{1}{2} \sum_{i=k}^{\infty} r_i = \frac{1}{2} \sum_{i=k}^{\infty} (x_i^T Q x_i + u_i^T R u_i). \quad (11.2-23)$$

In this example, the state space $X = \mathbb{R}^n$ and action space $U = \mathbb{R}^m$ are infinite and continuous.

b. Bellman Equation for DT LQR: The Lyapunov Equation

The performance index J_k depends on the current state x_k and all future control inputs $u_k, u_{k+1}, \dots$. Select a fixed stabilizing policy $u_k = \mu(x_k)$ and write the associated value function as

$$V(x_k) = \frac{1}{2} \sum_{i=k}^{\infty} r_i = \frac{1}{2} \sum_{i=k}^{\infty} (x_i^T Q x_i + u_i^T R u_i). \quad (11.2-24)$$

The value function for a fixed policy depends only on the initial state x_k . A difference equation equivalent to this infinite sum is given by

$$\begin{aligned} V(x_k) &= \frac{1}{2} (x_k^T Q x_k + u_k^T R u_k) + \frac{1}{2} \sum_{i=k+1}^{\infty} (x_i^T Q x_i + u_i^T R u_i) \\ &= \frac{1}{2} (x_k^T Q x_k + u_k^T R u_k) + V(x_{k+1}). \end{aligned} \quad (11.2-25)$$

That is, the positive definite solution $V(x_k)$ to this equation that satisfies $V(0) = 0$ is the value given by (11.2-24). Equation (11.2-25) is exactly the Bellman equation (11.2-17) for the LQR.

Assuming the value is quadratic in the state so that

$$V_k(x_k) = \frac{1}{2} x_k^T P x_k, \quad (11.2-26)$$

for some kernel matrix P yields the Bellman equation form

$$2V(x_k) = x_k^T P x_k = x_k^T Q x_k + u_k^T R u_k + x_{k+1}^T P x_{k+1}, \quad (11.2-27)$$

which, using the state equation, can be written as

$$2V(x_k) = x_k^T Q x_k + u_k^T R u_k + (Ax_k + Bu_k)^T P (Ax_k + Bu_k). \quad (11.2-28)$$

Assuming a constant (that is, stationary) state feedback policy $u_k = \mu(x_k) = -Kx_k$ for some stabilizing gain K , we write

$$2V(x_k) = x_k^T P x_k = x_k^T Q x_k + x_k^T K^T R K x_k + x_k^T (A - BK)^T P (A - BK) x_k. \quad (11.2-29)$$

Since this holds for all state trajectories, we have

$$(A - BK)^T P (A - BK) - P + Q + K^T R K = 0. \quad (11.2-30)$$

This is a Lyapunov equation. That is, the Bellman equation (11.2-17) for the DT LQR is equivalent to a Lyapunov equation. Since the performance index is undiscounted ($\gamma = 1$), we must select a stabilizing gain K , that is, a stabilizing policy.

The formulations (11.2-25), (11.2-27), (11.2-29), (11.2-30) for the Bellman equation are all equivalent. Note that forms (11.2-25) and (11.2-27) do not involve the system dynamics (A, B) . On the other hand, the Lyapunov equation (11.2-30) can be used only if we know the state dynamics (A, B) . Optimal controls design using matrix equations is the

standard procedure in control systems theory. Unfortunately, by assuming that (11.2-29) holds for all trajectories and going to (11.2-30), we lose all possibility of applying any sort of reinforcement learning algorithms to solve for the optimal control and value online by observing data along the system trajectories. By contrast, it will be shown that by employing the form (11.2-25) or (11.2-27) for the Bellman equation, we can devise RL algorithms for learning optimal solutions online by using temporal difference methods. That is, RL allows us to solve the Lyapunov equation online without knowing A or B .

c. Bellman Optimality Equation for DT LQR: The Algebraic Riccati Equation

The DT LQR Hamiltonian function is

$$H(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k + (A x_k + B u_k)^T P (A x_k + B u_k) - x_k^T P x_k. \quad (11.2-31)$$

This is known as the temporal difference error in MDP. A necessary condition for optimality is the stationarity condition $\partial H(x_k, u_k) / \partial u_k = 0$, which is equivalent to (11.2-21). Solving this yields the optimal control

$$u_k = -K x_k = -(B^T P B + R)^{-1} B^T P A x_k.$$

Putting this into (11.2-31) yields the DT algebraic Riccati equation (ARE)

$$A^T P A - P + Q - A^T P B (B^T P B + R)^{-1} B^T P A = 0. \quad (11.2-32)$$

This is exactly the Bellman optimality equation (11.2-19) for the DT LQR. ■

The Closed-loop Markov Chain

References for this section include (Wheeler and Narendra 1986, Bertsekas and Tsitsiklis 1996, Luenberger 1979). Consider a finite MDP with N states. Select a fixed stationary policy $\pi(x, u) = \Pr\{u|x\}$. Then the “closed-loop” MDP reduces to a Markov chain with state space X . The transition probabilities of this Markov chain are given by (11.2-2). Enumerate the states using index $i = 1, \dots, N$ and denote the transition probabilities (11.2-2) by $p_{ij} = p_{x=i, x'=j}$. Define the transition matrix $P = [p_{ij}] \in R^{N \times N}$. Denote the expected costs $r_{ij} = R_{ij}^\pi$ and define the cost matrix $R = [r_{ij}] \in R^{N \times N}$. Array the scalar values of the states $V(i)$ into a vector $V = [V(i)] \in R^N$.

With this notation, we may write the Bellman equation (11.2-17) as

$$V = \gamma P V + (P \odot R) \underline{1}, \quad (11.2-33)$$

where $\odot$ is the Hadamard (element-by-element) matrix product, $P \odot R = [p_{ij} r_{ij}] \in R^{N \times N}$ is the cost-transition matrix, and $\underline{1}$ is the N -vector of 1's. This is a system of N linear equations for the value vector V of using the selected policy.

Transition matrix P is stochastic, that is, all row sums are equal to one. If the Markov chain is irreducible, that is, all states communicate with each other with nonzero probability, then P has a single eigenvalue at 1 and all other eigenvalues

inside the unit circle (Gershgorin theorem). Then, if the discount factor is less than one, $I - \gamma P$ is nonsingular and there is a unique solution to (11.2-33) for the value, namely $V = (I - \gamma P)^{-1}(P \odot R)\underline{1}$.

Define vector $p \in R^N$ as a probability distribution vector (that is, its elements sum to 1) with element $p(i)$ being the probability that the Markov chain is in state i . Then, the evolution of p is given by

$$p_{k+1}^T = p_k^T P, \quad (11.2-34)$$

with k the time index and starting at some initial distribution p_0 . The solution to this equation is

$$p_k^T = p_0^T P^k. \quad (11.2-35)$$

The limiting value of this recursion is the invariant or steady-state distribution, given by solving

$$p_\infty^T = p_\infty^T P, \quad p_\infty^T (I - P) = 0. \quad (11.2-36)$$

Thus, p_∞ is the left eigenvector of the eigenvalue $\lambda = 1$ of P . Since P has all row sums equal one, the right eigenvector of $\lambda = 1$ is given by $\underline{1}$. If the Markov chain is irreducible, the eigenvalue $\lambda = 1$ is simple, and vector p_∞ has all elements positive. That is, all states have a nonzero probability of being occupied in steady state.

Define the average cost per stage and its value under the selected policy as

$$\bar{V}(i) = \lim_{T \rightarrow \infty} \frac{1}{T} E \left(\sum_{k=0}^{T-1} r_k | x_0 = i \right). \quad (11.2-37)$$

Define the vector $\bar{V} = [\bar{V}(i)] \in R^N$. Then we have

$$\bar{V} = (P \odot R)\underline{1}. \quad (11.2-38)$$

Define the expected value over all the states as $\bar{\bar{V}} = E_i\{\bar{V}(i)\}$. Then

$$\bar{\bar{V}} = p_\infty^T (P \odot R)\underline{1}. \quad (11.2-39)$$

The left eigenvector $p_\infty = [p(i)]$ for $\lambda = 1$ has received a great deal of attention in the literature about cooperative control (Jadbabaie et al. 2003, Olfati-Saber and Murray 2004). The meaning of its elements $p(i)$ is interesting. Element $p(i)$ is the probability that the Markov chain is in state i at steady state, and also the percentage of time the Markov chain is in state i at steady state. Starting in state i , the time at which the Markov chain first returns to state i is a random variable known as the *hitting time* or first return time. Its expected value T_i is known as the *expected return time* to state i . If the Markov chain is irreducible, $T_i > 0, \forall i$; then $p(i) = 1/T_i$.

11.3 POLICY EVALUATION AND POLICY IMPROVEMENT

Given a current policy $\pi(x, u)$, we can determine its value (11.2-16) by solving the Bellman equation (11.2-17). As will be discussed, there are several methods of doing this, several of which can be implemented online in real time. This procedure is known as *policy evaluation*.

Moreover, given the value function for any policy $\pi(x, u)$, we can always use it to find another policy that is better, or at least no worse. This step is known as *policy improvement*. Specifically, suppose $V^\pi(x)$ satisfies (11.2-17). Then define a new policy $\pi'(x, u)$ by

$$\pi'(x, u) = \arg \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')]. \quad (11.3-1)$$

Then it is easy to show that $V^{\pi'}(x) \leq V^\pi(x)$ (Bertsekas and Tsitsiklis 1996, Sutton and Barto 1998). The policy determined as in (11.3-1) is said to be *greedy with respect to value function $V^\pi(x)$* .

In the special case that $V^{\pi'}(x) = V^\pi(x)$ in (11.3-1), then $V^{\pi'}(x)$, $\pi'(x, u)$ satisfy (11.2-20) and (11.2-21) so that $\pi'(x, u) = \pi(x, u)$ is the optimal policy and $V^{\pi'}(x) = V^\pi(x)$ the optimal value. That is, (only) an optimal policy is greedy with respect to its own value. In computational intelligence, *greedy* refers to quantities determined by optimizing over short or 1-step horizons, without regard to potential impacts far into the future.

Now let us consider algorithms that repeatedly interleave the two procedures.

Policy evaluation by Bellman equation:

$$V^\pi(x) = \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')], \quad \text{for all } x \in S \subseteq X. \quad (11.3-2)$$

Policy improvement:

$$\pi'(x, u) = \arg \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')], \quad \text{for all } x \in S \subseteq X. \quad (11.3-3)$$

S is a suitably selected subspace of the state space, to be discussed later. We call an application of (11.3-2) followed by an application of (11.3-3) one *step*. This is in contrast to the decision time stage k defined above.

At each step of such algorithms, one obtains a policy that is no worse than the previous policy. Therefore, it is not difficult to prove convergence under fairly mild conditions to the optimal value and optimal policy. Most such proofs are based on the Banach fixed-point theorem. Note that (11.2-20) is a fixed-point equation for $V^*(\cdot)$. Then the two equations (11.3-2), (11.3-3) define an associated map that can be shown under mild conditions to be a contraction map (Bertsekas and Tsitsiklis 1996 and Powell 2007). Then, it converges to the solution of (11.2-20) for any initial policy.

There is a large family of algorithms that implement the policy evaluation and policy improvement procedures in various ways, or **interleave** them differently, or **select subspace $S \subseteq X$** in different ways, to determine the optimal value and optimal policy. We shall soon outline some of them.

The importance for feedback control systems of this discussion is that these **two procedures** can be implemented for dynamical systems online in real time by observing data **measured along the system trajectories**. This yields a family of adaptive control algorithms that converge to optimal control solutions. These algorithms are of the **actor-critic class** of reinforcement learning systems, shown in Figure 11.1-1. There, a **critic agent evaluates the current control policy using methods based on (11.3-2)**. After this has been completed, **the action is updated by an actor agent based on (11.3-3)**.

Policy Iteration

One method of reinforcement learning for using (11-3.2), (11-3.3) to find the optimal value and optimal policy is *policy iteration (PI)*.

POLICY ITERATION (PI) ALGORITHM

Initialize.

Select an initial policy $\pi_0(x, u)$. Do **for $j = 0$** until convergence:

Policy evaluation (value update) 

$$V_j(x) = \sum_u \pi_j(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_j(x')] \quad \text{for all } x \in X. \quad (11.3-4)$$



Policy improvement (policy update):

$$\pi_{j+1}(x, u) = \arg \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_j(x')], \quad \text{for all } x \in X. \quad (11.3-5)$$

At each step j the PI algorithm determines the solution of the Bellman equation (11.3-4) to compute the value $V_j(x)$ of using the current policy $\pi_j(x, u)$. This value corresponds to the infinite sum (11.2-16) for the current policy. Then the policy is improved using (11.3-5). The steps are continued until there is no change in the value or the policy.

Note that j is not the time or stage index k , but a PI step iteration index. It will be seen how to implement PI for dynamical systems online in real time by observing data measured along the system trajectories. Generally, data for multiple times k is needed to solve the Bellman equation (11.3-4) at each step j .

If the MDP is finite and has N states, then the policy evaluation equation (11.3-4) is **a system of N simultaneous linear equations**, one for each state. The PI algorithm must be suitably initialized to converge. The initial policy $\pi_0(x, u)$ and value V_0 must be selected so that $V_1 \leq V_0$. Then, for finite Markov chains

with N states, PI converges in a finite number of steps (less than or equal to N) because there are only a finite number of policies.

The Bellman equation (11.3-4) is a system of simultaneous equations. Instead of directly solving the Bellman equation, we can solve it by an iterative policy evaluation procedure. Note that (11.3-4) is a fixed-point equation for $V_j(\cdot)$. It defines the iterative policy evaluation map

$$V_j^{i+1}(x) = \sum_u \pi_j(x, u) \sum_{x'} P_{xx'}^u \left[R_{xx'}^u + \gamma V_j^i(x') \right], \quad i = 1, 2, \dots \quad (11.3-6)$$

which can be shown to be a contraction map under rather mild conditions. By the Banach fixed-point theorem the iteration can be initialized at any non-negative value of $V_j^1(\cdot)$ and it will converge to the solution of (11.3-4). Under certain conditions, this solution is unique. A good initial value choice is the value function $V_{j-1}(\cdot)$ from the previous step $j - 1$. On (close enough) convergence, we set $V_j(\cdot) = V_j^i(\cdot)$ and proceed to apply (11.3-5).

Index j in (11.3-6) refers to the step number of the PI algorithm. By contrast i is an iteration index. It is interesting to compare iterative policy evaluation (11.3-6) to the backward-in-time recursion (11.2-9) for the finite-horizon value. In (11.2-9), k is the time index. By contrast, in (11.3-6), i is an iteration index. Dynamic programming is based on (11.2-9) and proceeds backward in time. The methods for online optimal adaptive control described in this chapter proceed forward in time and are based on PI and similar algorithms.

The usefulness of these concepts is shown in the next example, where we use the theory of MDP and iterative policy evaluation to derive the relaxation algorithm for solution of Poisson's equation.

Example 11.3-1. Solution of Partial Differential Equations: Relaxation Algorithms

Consider Poisson's equation in two dimensions

$$\Delta V(x, y) = \nabla^2 V(x, y) = \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) V(x, y) = f(x, y), \quad (11.3-7)$$

where $\Delta = \nabla^2$ is the Laplacian operator and ∇ the gradient. Function $f(x, y)$ is a forcing function, often specified on the boundary of a region. Discretizing the equation on a uniform mesh as shown in Figure 11.3-1 with grid size h in the (x, y) plane we write in terms of the forward difference

$$\begin{aligned} \frac{\partial V(x, y)}{\partial x} &= \frac{1}{h} (V(x + h, y) - V(x, y)) + O(h), \\ \frac{\partial^2 V(x, y)}{\partial x^2} &= \frac{1}{h^2} (V(x + h, y) + V(x - h, y) - 2V(x, y)) + O(h^2), \\ \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) V(x, y) &= \frac{1}{h^2} (V(x + h, y) + V(x - h, y) + V(x, y + h) \\ &\quad + V(x, y - h) - 4V(x, y)) \approx f(x, y). \end{aligned}$$

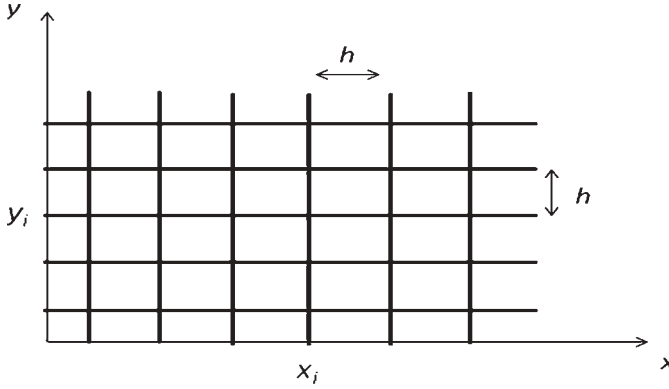


FIGURE 11.3-1 Sampling in the (x, y) plane with a uniform mesh of size h .

Indexing the (x, y) positions with an index i and denoting $X_i = (x_i, y_i)$ we have approximately to order h^2

$$\frac{1}{h^2} (V(X_{i,R}) + V(X_{i,L}) + V(X_{i,U}) + V(X_{i,D}) - 4V(X_i)) = f(X_i), \quad (11.3-8)$$

for states not on the boundary and a similar equation for boundary states. $X_{i,R}$ denotes the (x, y) location of the state to the right of X_i , and similarly for states to the left, up, and down relative to X_i . This is a set of N simultaneous equations.

We can interpret (11.3-8) as a Bellman equation (11.2-17) for a properly defined underlying MDP. Specifically, define an MDP with stage costs equal to zero and $\gamma = 1$. Then, we may interpret the state transitions as deterministic and the control as the equiprobable control with probabilities of $1/4$ (for nonboundary nodes) for moving up, right, down, and left. Alternatively, we may interpret the control as deterministic and the state transition probabilities as equiprobable with probabilities of $1/4$ for moving up, right, down, and left. In either case, the Bellman equation for the MDP is (11.3-8). Functions $f(X_i)$ may be interpreted as stage costs.

Now the iterative policy evaluation method of solution (11.3-6) performs the iterations

$$V^{m+1}(X_i) = \frac{1}{4} V^m(X_{i,U}) + \frac{1}{4} V^m(X_{i,R}) + \frac{1}{4} V^m(X_{i,D}) + \frac{1}{4} V^m(X_{i,L}) - \frac{h^2}{4} f(X_i). \quad (11.3-9)$$

The theory of MDP guarantees that this algorithm will converge to the solution of (11.3-8). These updates may be done for all nodes or states simultaneously. Then, this is nothing but the relaxation method for numerical solution of Poisson's equation. The relaxation algorithm converges, but may do so slowly. Variants have been developed to speed it up. ■

Value Iteration

A second method for using (11.3-2), (11.3-3) in reinforcement learning is value iteration (VI), which is easier to implement than policy iteration.

VALUE ITERATION (VI) ALGORITHM*Initialize.*Select an initial policy $\pi_0(x, u)$. Do for $j = 0$ until convergence—**Value update:**

$$V_{j+1}(x) = \sum_u \pi_j(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_j(x')], \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-10)$$

Policy improvement:

$$\pi_{j+1}(x, u) = \arg \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_{j+1}(x')], \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-11)$$

VI can combine the value update and policy improvement into one equation to obtain the equivalent form for VI

$$V_{j+1}(x) = \min_{\pi} \sum_u \pi(x, u) \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_j(x')], \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-12)$$

or, equivalently under the ergodicity assumption, in terms of deterministic policies

$$V_{j+1}(x) = \min_u \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_j(x')], \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-13)$$

Note that, now, (11.3-10) is a simple one-step recursion, not a system of linear equations as is (11.3-4) in the PI algorithm. In fact, VI uses simply one iteration of (11.3-6) in its value update step. It does not find the value corresponding to the current policy, but takes only one iteration toward that value. Again, j is not the time index, but the VI step index.

It will be seen later how to implement VI for dynamical systems online in real time by observing data measured along the system trajectories. Generally, data for multiple times k is needed to solve the update (11.3-10) for each step j .

Asynchronous Value Iteration. Standard VI takes the update set as $S_j = X$, for all j . That is, the value and policy are updated for all states simultaneously. Asynchronous VI methods perform the updates on only a subset of the states at each step. In the extreme case, one may perform the updates on only one state at each step.

It is shown in Bertsekas and Tsitsiklis (1996) that standard VI ($S_j = X$, for all j) converges for finite MDP for any initial conditions when the discount factor satisfies $0 < \gamma < 1$. When $S_j = X$, for all j and $\gamma = 1$ an absorbing state is added and a “properness” assumption is needed to guarantee convergence to

the optimal value. When a single state is selected for value and policy updates at each step, the algorithm converges, for any choice of initial value, to the optimal cost and policy if each state is selected for update infinitely often. More general algorithms result if value update (11.3-10) is performed multiple times for various choices of S_j prior to a policy improvement. Then updates (11.3-10) and (11.3-11) must be performed infinitely often for each state, and a monotonicity assumption must be satisfied by the initial starting value.

Considering (11.2-20) as a fixed-point equation, VI is based on the associated iterative map (11.3-10), (11.3-11), which can be shown under certain conditions to be a contraction map. In contrast to PI, which converges under certain conditions in a finite number of steps, VI generally takes an infinite number of steps to converge (Bertsekas and Tsitsiklis 1996). Consider finite MDP and consider the transition probability graph having probabilities (11.2-2) for the Markov chain corresponding to an optimal policy $\pi^*(x, u)$. If this graph is acyclic for some $\pi^*(x, u)$, then VI converges in at most N steps when initialized with a large value.

Having in mind the dynamic programming equation (11.2-9) and examining the VI value update (11.3-10), we can interpret $V_j(x')$ as an approximation or estimate for the future stage cost-to-go from the future state x' . See Figure 11.2-2. Those algorithms wherein the future cost estimate are themselves costs or values for some policy are called *rollout algorithms* in Bertsekas and Tsitsiklis (1996). These policies are forward looking and self-correcting. They are closely related to receding horizon control, as shown in Zhang et al. (2009).

MDP, policy iteration, and value iteration provide connections between optimal control, decisions on finite graphs, and cooperative control of networked systems, as amplified in the next examples. The first example shows how to use VI to derive the Bellman Ford algorithm for finding the shortest path in a graph to a destination node. The second example uses MDP and iterative policy evaluation to develop control protocols familiar in cooperative control of distributed dynamical systems. The third example shows that for the discrete-time LQR, policy iteration and value iteration can be used to derive algorithms for solution of the optimal control problem that are quite familiar in the feedback control systems community, including Hwer's algorithm.

Example 11.3-2. Deterministic Shortest Path Problems: The Bellman Ford Algorithm

A special case of finite MDP is the *shortest path problems* (Bertsekas and Tsitsiklis 1996), which are undiscounted $\gamma = 1$, and have a cost-free termination or absorbing state. These effectively have the time horizon T finite since the number of states is finite. The setup is shown in Figure 11.3-2.

Consider a directed graph $G = (V, E)$ with a nonempty finite set of N nodes $V = \{v_1, \dots, v_N\}$ and a set of edges or arcs $E \subseteq V \times V$. We assume the graph is simple, that is, it has no repeated edges and $(v_i, v_i) \notin E, \forall i$ no self-loops. Let the edges have weights $e_{ij} \geq 0$, with $e_{ij} > 0$ if $(v_i, v_j) \in E$ and $e_{ij} = 0$ otherwise. Note $e_{ii} = 0$. The set of neighbors of a node v_i is $N_i = \{v_j: (v_i, v_j) \in E\}$, that is, the set of nodes with arcs coming out from v_i .

Consider an agent moving through the graph along edges between nodes. Interpret the control at each node as the decision on which edge to follow leading out of that

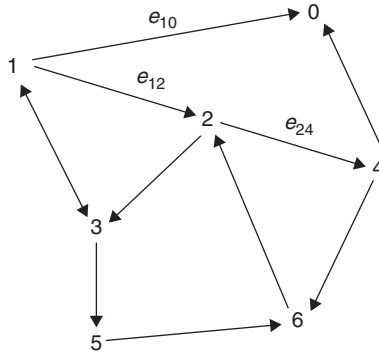


FIGURE 11.3-2 Sample graph for shortest path routing problem. The absorbing state 0 has only incoming edges. The objective is to find the shortest path from all nodes to node 0 using only local neighborhood computations.

node. Assume deterministic controls and state transitions. Interpret the edge weights e_{ij} as deterministic costs incurred by moving along that link. Then, the value iteration algorithm (11.3-13), (11.3-11) in this general digraph is

$$V_i(k+1) = \min_{j \in N_i} (e_{ij} + V_j(k)), \quad (11.3-14)$$

$$u_i(k+1) = \arg \min_{j \in N_i} (e_{ij} + V_j(k)), \quad (11.3-15)$$

with k an iteration index. We have changed the notation to conform to existing practice in cooperative control theory. Endow the graph with one node, v_0 , which uses as its update law $V_0(k+1) = V_0(k) = 0$. This corresponds to an absorbing state in MDP parlance. It is interpreted as a node having only incoming edges, none outgoing.

The VI algorithm in this scenario is nothing but the Bellman-Ford algorithm, which finds the shortest path from any node in the graph to the absorbing node v_0 . In the terminology of cooperative systems, each node v_i is endowed with two state variables. The node state variable $V_i(k)$ keeps track of the value, that is, the shortest path length to node v_0 , while node state variable $u_i(k)$ keeps track of which direction to follow while leaving the i th node in order to follow the shortest path.

The VI update iterations may be performed on all nodes simultaneously. Alternatively, updates may be performed on one state at a time, as in asynchronous VI. With simultaneous updates at all nodes, according to results about VI (Bertsekas and Tsitsiklis 1996) it is known that this algorithm converges to the solution to the shortest path problem in a finite number of steps (less than or equal to N) if there is a path from every node to node v_0 and the graph is acyclic. It is known further that the algorithm converges, possibly in an infinite number of iterations, if there are no cycles with net negative gain (that is, the product of gains around the cycle is not negative). With asynchronous updates, the algorithm converges under these connectivity conditions if each node is selected for update infinitely often.

What Is a State?

Note that in the terminology of MDP, the nodes are termed states, so that the state space is finite. On the other hand, in the terminology of feedback control systems theory, the

state (variable) of each node is $V_i(m)$, which is a real number so that the state space is continuous. ■

Example 11.3-3. Cooperative Control Systems

Consider the directed graph $G = (V, E)$ with N nodes $V = \{v_1, \dots, v_N\}$ and edge weights $e_{ij} \geq 0$, with $e_{ij} > 0$ if $(v_i, v_j) \in E$ and $e_{ij} = 0$ otherwise. The set of neighbors of a node v_i is $N_i = \{v_j: (v_i, v_j) \in E\}$, that is, the set of nodes with arcs coming out from v_i . Define the out-degree of node i in graph G as $d_i = \sum_{j \in N_i} e_{ij}$. Figure 11.3-3 shows a representative graph topology.

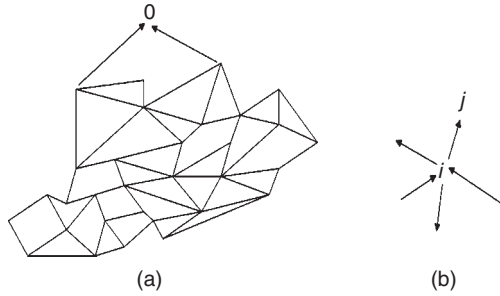


FIGURE 11.3-3 Cooperative control of multiple systems linked by a communication graph structure. The objective is for all nodes to reach consensus to the value of control node 0 by using only local neighborhood communication. (a) Representative graph structure. (b) Local neighborhood of node i . Each node has edges incoming and outgoing. The outgoing edges show the states reached from node i in one step.

On the graph G , define a MDP that has a stochastic policy $\pi(v_i, u_{ij}) = \Pr\{u_{ij}|v_i\}$, where u_{ij} means the control action that takes the MDP from node v_i to node v_j . Let the transition probabilities be deterministic. Endow the graph with one absorbing node v_0 that is connected to a few of the existing nodes. If there is an edge from node v_i to node v_0 , define $b_i \equiv e_{i0}$ as its edge weight. Let actions u_{ij} have probabilities $u_{ii} = 1/(1 + d_i + b_i)$, $u_{ij} = e_{ij}/(1 + d_i + b_i)$, that is, the MDP may return to the same state in one step. Let the stage costs all be zero. Then, the Bellman equation (11.2-17) is

$$V_i = \frac{1}{1 + d_i + b_i} \left[V_i + \sum_{j \in N_i} e_{ij} V_j + b_i V_0 \right], \quad (11.3-16)$$

where V_i is the value of node v_i . This is a set of N simultaneous equations in the values $V_i, i = 1, N$ of the nodes.

Iterative policy evaluation (11.3-6) can be used to solve this set of equations. The iterative policy evaluation algorithm is written here as

$$V_i(k+1) = \frac{1}{1 + d_i + b_i} \left[V_i(k) + \sum_{j \in N_i} e_{ij} V_j(k) + b_i V_0 \right], \quad (11.3-17)$$

with k the iteration index, which can be thought of here as a time index. Node v_0 keeps its value constant at V_0 .

The updates in (11.3-17) may be done simultaneously at all nodes. Then the theory of MDP shows that it is guaranteed to converge to the solution of the set of equations (11.3-16). Alternatively, one may use the theory of asynchronous VI or generalized PI to motivate other update schemes. For instance, if only one node is updated at each value of k , the theory of MDP shows that the algorithm still converges as long as each node is selected for update infinitely often.

Algorithm (11.3-17) can be written as the local control protocol

$$V_i(k+1) = V_i(k) + \frac{1}{1+d_i+b_i} \left[\sum_{j \in N_i} e_{ij} (V_j(k) - V_i(k)) + b_i (V_0 - V_i(k)) \right]. \quad (11.3-18)$$

On convergence $V_i(k+1) = V_i(k)$ so that the term in square brackets converges to zero. Assuming all nodes have a path to the absorbing node v_0 , it is easy to show that this guarantees that $\|V_j(k) - V_i(k)\| \rightarrow 0$, $\|V_0 - V_i(k)\| \rightarrow 0$, for all i, j ; that is, all nodes reach the same consensus value, namely the value of the absorbing node (Jadbabaie et al. 2003, Olfati-Saber and Murray 2004).

The term in square brackets in (11.3-18) is known as the *temporal difference error* for this MDP.

Routing Graph vs. Control Graph

The graphs used in routing problems have edges from node i to node j if the transition probabilities (11.2-2) in the MDP using a fixed policy $\pi(x_i, u)$, namely $p_{ij} \equiv P_{x_i, x_j}^\pi = \sum_u \pi(x_i, u) P_{x_i, x_j}^u$, are nonzero. The edge weights are taken as $e_{ij} = p_{ij}$. Then, e_{ij} is nonzero if there is an edge coming *out* of node i to node j . By contrast, the edge weights a_{ij} for graphs in cooperative control problems are generally taken as nonzero if there is an edge coming *in* from node j to node i , that is, $a_{ij} > 0$ iff $(v_j, v_i) \in E$. This is interpreted to mean that information from node j is available to node i for its decision process in computing its control input.

It is convenient to think of the former as motion or routing graphs, and the latter as information flow or decision and control graphs. In fact, the routing graph is the reverse of the decision graph. The reverse graph of a given graph $G = (V, E)$ is the graph with the same node set, but all edges reversed. Note that the Bellman-Ford protocols (11.3-14), (11.3-15) assume that node i gets information from its neighbor node j , but that the shortest path problem is, in fact, solved for motion in the reverse graph having edges e_{ij} .

Define the matrix of transition probabilities $P = [p_{ij}]$ and the adjacency matrix $A = [a_{ij}]$. Then $A = P^T$. ■

Example 11.3-4. Policy Iteration and Value Iteration for the DT LQR

The Bellman equation (11.2-17) is equivalent for the DT LQR to all the formulations (11.2-25), (11.2-27), (11.2-29), (11.2-30) in Example 11.2-1. We may use any of these to implement policy iteration and value iteration.

a. Policy Iteration: Hewer's Algorithm

With step index j , and using superscripts to denote algorithm steps and subscripts to denote the time k , the PI policy evaluation step (11.3-4) applied on (11.2-25) in

Example 11.2-1 yields

$$V^{j+1}(x_k) = \frac{1}{2} (x_k^T Q x_k + u_k^T R u_k) + V^j(x_{k+1}). \quad (11.3-19)$$

PI applied on (11.2-27) yields

$$x_k^T P^{j+1} x_k = x_k^T Q x_k + u_k^T R u_k + x_{k+1}^T P^j x_{k+1}, \quad (11.3-20)$$

and PI on (11.2-30) yields the Lyapunov equation

$$0 = (A - BK^j)^T P^{j+1} (A - BK^j) - P^{j+1} + Q + (K^j)^T R K^j. \quad (11.3-21)$$

In all cases the PI policy improvement step is

$$\mu^{j+1}(x_k) = K^{j+1} x_k = \arg \min (x_k^T Q x_k + u_k^T R u_k + x_{k+1}^T P^{j+1} x_{k+1}), \quad (11.3-22)$$

which can be written explicitly as

$$K^{j+1} = -(B^T P^{j+1} B + R)^{-1} B^T P^{j+1} A. \quad (11.3-23)$$

PI algorithm format (11.3-21), (11.3-23) relies on repeated solutions of Lyapunov equations at each step, and is nothing but Hewer's algorithm, well known in control systems theory. It was proven in Hewer (1971) to converge to the solution of the Riccati equation (11.2-32) in Example 11.2-1 if (A, B) is reachable and $(A, \sqrt{Q})$ is observable. It is an offline algorithm that requires complete knowledge of the system dynamics (A, B) to find the optimal value and control. It requires that the initial gain K^0 be stabilizing.

b. Value Iteration: Lyapunov Recursions

Applying VI (11.3-10) to Bellman equation format (11.2-27) in Example 11.2-1 yields

$$x_k^T P^{j+1} x_k = x_k^T Q x_k + u_k^T R u_k + x_{k+1}^T P^j x_{k+1}, \quad (11.3-24)$$

and on format (11.2-30) in Example 11.2-1 yields the Lyapunov recursion

$$P^{j+1} = (A - BK^j)^T P^j (A - BK^j) + Q + (K^j)^T R K^j. \quad (11.3-25)$$

In both cases the policy improvement step is still given by (11.3-22), (11.3-23).

VI algorithm format (11.3-25), (11.3-23) is simply a Lyapunov recursion, which is easy to implement and does not, in contrast to PI, require Lyapunov equation solutions. This algorithm was shown in Lancaster and Rodman (1995) to converge to the solution of the Riccati equation (11.2-32) in Example 11.2-1. It is an offline algorithm that requires complete knowledge of the system dynamics (A, B) to find the optimal value and control. It does not require that the initial gain K^0 be stabilizing, and can be initialized with any feedback gain.

c. Online Solution of the Riccati Equation without Knowing Plant Matrix A

Hewer's algorithm and the Lyapunov recursion algorithm are both offline methods for solving the algebraic Riccati equation (11.2-32) in Example 11.2-1. Full knowledge of the

plant dynamics (A, B) is needed to implement these algorithms. By contrast, it will be seen that both PI Algorithm format (11.3-20), (11.3-22) and VI Algorithm format (11.3-24), (11.3-22) can be implemented online to determine the optimal value and control *in real time* using data measured along the system trajectories, and without knowing the system A matrix. This is accomplished through the temporal difference methods to be presented. That is, RL allows the solution of the algebraic Riccati equation online without knowing the system A matrix.

d. *Iterative Policy Evaluation*

Given a **fixed policy K** , the iterative policy evaluation procedure (11.3-6) becomes

$$P^{j+1} = (A - BK)^T P^j (A - BK) + Q + K^T R K. \quad (11.3-26)$$

This **recursion** converges to the solution to the Lyapunov equation $P = (A - BK)^T P (A - BK) + Q + K^T R K$ if $(A - BK)$ is stable, for any choice of initial value P^0 . ■

Generalized Policy Iteration

In PI one **fully solves the system of linear equations** (11.3-4) at each step to compute the value (11.2-16) of using the current policy $\pi_j(x, u)$. This can be accomplished by running iterations (11.3-6) until convergence at each step j . By contrast, **in VI one takes only one iteration** of (11.3-6) in the value update step (11.3-10). **Generalized policy iteration (GPI) algorithms make several iterations (11.3-6) in their value update step.**

Generally, PI converges to the optimal value in fewer steps j , since it does more work in solving equations at each step. On the other hand, VI is the easiest to implement as it takes only one iteration of a recursion as per (11.3-10). GPI provides a suitable compromise between computational complexity and convergence speed. GPI is a special case of the VI algorithm given above, where we select $S_j = X$, for all j and perform value update (11.3-10) multiple times before each policy update (11.3-11).

Q Function

The conditional expected value in (11.2-13)

$$Q_k^*(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V_{k+1}^*(x')] = E_\pi \{r_k + \gamma V_{k+1}^*(x') | x_k = x, u_k = u\}. \quad (11.3-27)$$

is known as the **optimal Q (quality) function** (Watkins 1989, Watkins and Dayan 1992). This has also been called the action-value function (Sutton and Barto 1998). It is equal to the expected return for taking an arbitrary action u at time k in state x and thereafter following an optimal policy. It is a function of the current state x and the action u .

In terms of the Q function, the Bellman optimality equation has the particularly simple form

$$V_k^*(x) = \min_u Q_k^*(x, u), \quad (11.3-28)$$

$$u_k^* = \arg \min_u Q_k^*(x, u). \quad (11.3-29)$$

Given any fixed policy $\pi(x, u)$, define the Q function for that policy as

$$Q_k^\pi(x, u) = E_\pi \{ r_k + \gamma V_{k+1}^\pi(x') | x_k = x, u_k = u \} = \sum_{x'} P_{xx'}^\pi [R_{xx'}^\pi + \gamma V_{k+1}^\pi(x')], \quad (11.3-30)$$

where we have used (11.2-9). This is equal to the expected return for taking an arbitrary action u at time k in state x and thereafter following the existing policy $\pi(x, u)$. The meaning of the Q function is elucidated by the next example.

Example 11.3-5. Q Function for the DT LQR

The Q function following a given policy $u_k = \mu(x_k)$ is defined in (11.3-30). For the DT LQR in Example 11.2-1 the Q function is

$$Q(x_k, u_k) = \frac{1}{2} (x_k^T Q x_k + u_k^T R u_k) + V(x_{k+1}), \quad (11.3-31)$$

where the control u_k is arbitrary and the policy $u_k = \mu(x_k)$ is followed for $k+1$ and subsequent times. Writing

$$Q(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k + (Ax_k + Bu_k)^T P (Ax_k + Bu_k), \quad (11.3-32)$$

with P the Riccati solution yields the Q function for the DT LQR:

$$Q(x_k, u_k) = \frac{1}{2} \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T \begin{bmatrix} A^T P A + Q & B^T P A \\ A^T P B & B^T P B + R \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \quad (11.3-33)$$

Define

$$Q(x_k, u_k) \equiv \frac{1}{2} \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T S \begin{bmatrix} x_k \\ u_k \end{bmatrix} = \frac{1}{2} \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T \begin{bmatrix} S_{xx} & S_{xu} \\ S_{ux} & S_{uu} \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix}, \quad (11.3-34)$$

for kernel matrix S .

Applying $\partial Q(x_k, u_k) / \partial u_k = 0$ to (11.3-34) yields

$$u_k = -S_{uu}^{-1} S_{ux} x_k, \quad (11.3-35)$$

and to (11.3-33) yields

$$u_k = -(B^T P B + R)^{-1} B^T P A x_k. \quad (11.3-36)$$

The latter equation requires knowledge of the system dynamics (A, B) to perform the policy improvement step of either PI or VI. On the other hand, the former equation requires knowledge only of the Q-function matrix kernel S . We shall subsequently show how to use RL temporal difference methods to determine the kernel matrix S online in real time without knowing the system dynamics (A, B) using data measured along the system trajectories. This provides a family of Q learning algorithms that can solve the algebraic Riccati equation online without knowing the system dynamics (A, B) . ■

Note that $V_k^\pi(x) = Q_k^\pi(x, \pi(x, u))$ so that (11.3-30) may be written as the backward recursion in the Q function:

$$Q_k^\pi(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma Q_{k+1}^\pi(x', \pi(x', u'))]. \quad (11.3-37)$$

The Q function is a 2-dimensional function of both the current state x and the action u . By contrast, the value function is a 1-dimensional function of the state. For finite MDP, the Q function can be stored as a 2-D lookup table at each state/action pair. Note that direct minimization in (11.2-11), (11.2-12) requires knowledge of the state transition probabilities $P_{xx'}^u$ (system dynamics) and costs $R_{xx'}^u$. By contrast, the minimization in (11.3-28), (11.3-29) requires knowledge only of the Q function and not the system dynamics.

The importance of the Q function is twofold. First, it contains information about control actions in every state. As such, the best control in each state can be selected using (11.3-29) by knowing only the Q function. Second, the Q function can be estimated online in real time directly from data observed along the system trajectories, without knowing the system dynamics information (that is, the transition probabilities). We shall see how this is accomplished later.

The infinite horizon Q function for a prescribed fixed policy is given by

$$Q^\pi(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma V^\pi(x')]. \quad (11.3-38)$$

The Q function also satisfies a Bellman equation. Note that, given a fixed policy $\pi(x, u)$,

$$V^\pi(x) = Q^\pi(x, \pi(x, u)), \quad (11.3-39)$$

whence according to (11.3-38) the Q function satisfies the Bellman equation

$$Q^\pi(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma Q^\pi(x', \pi(x', u'))]. \quad (11.3-40)$$

It is important that the same Q function Q^π appears on both sides of this equation. The Bellman optimality equation for the Q function is

$$Q^*(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma Q^*(x', \pi^*(x', u'))], \quad (11.3-41)$$

$$Q^*(x, u) = \sum_{x'} P_{xx'}^u \left[R_{xx'}^u + \gamma \min_{u'} Q^*(x', u') \right]. \quad (11.3-42)$$

Compare (11.2-20) and (11.3-42), where the minimum operator and the expected value operator occurrences are reversed.

Based on Bellman equation (11.3-40), PI and VI are especially easy to implement in terms of the Q function, as follows.

PI USING Q FUNCTION

Policy evaluation (value update):

$$Q_j(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma Q_j(x', \pi_j(x', u'))], \quad \text{for all } x \in X. \quad (11.3-43)$$

Policy improvement:

$$\pi_{j+1}(x, u) = \arg \min_u Q_j(x, u), \quad \text{for all } x \in X. \quad (11.3-44)$$

VI USING Q FUNCTION

Value update:

$$Q_{j+1}(x, u) = \sum_{x'} P_{xx'}^u [R_{xx'}^u + \gamma Q_j(x', \pi_j(x', u'))], \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-45)$$

Policy improvement:

$$\pi_{j+1}(x, u) = \arg \min_u Q_{j+1}(x, u), \quad \text{for all } x \in S_j \subseteq X. \quad (11.3-46)$$

Combining both steps of VI yields the form

$$Q_{j+1}(x, u) = \sum_{x'} P_{xx'}^u \left[R_{xx'}^u + \gamma \min_{u'} Q_j(x', u') \right], \quad \text{for all } x \in S_j \subseteq X, \quad (11.3-47)$$

which should be compared to (11.3-13)

As shall be seen, the importance of the Q function is that these algorithms can be implemented online in real time, without knowing the system dynamics, by measuring data along the system trajectories. They yield optimal adaptive control algorithms—that is, adaptive control algorithms that converge online to optimal control solutions.

Methods for Implementing PI and VI

There are different methods for performing the value and policy updates for PI and VI (Bertsekas and Tsitsiklis 1996, Sutton and Barto 1998, Powell 2007). The main three are exact computation, Monte Carlo methods, and temporal difference (TD) learning. The last two methods can be implemented without knowledge of the system dynamics. TD learning is the means by which optimal adaptive control algorithms may be derived for dynamical systems. Therefore, TD is covered in the next section.

Exact computation. PI requires solution at each step of Bellman equation (11.3-4) for the value update. For a finite MDP with N states, this is a set of linear equations in N unknowns (the values of each state). VI requires performing the one-step recursive update (11.3-10) at each step for the value update. Both of these can be accomplished exactly if we know the transition probabilities $P_{x,x'}^u = \Pr\{x'|x, u\}$ and costs $R_{xx'}^u$ of the MDP. This corresponds to knowing full system dynamics information. Likewise, the policy improvements (11.3-5), (11.3-11) can be explicitly computed if the dynamics are known. It was shown in Example 11.2-1 that, for the DT LQR, the exact computation method for computing the optimal control yields the Riccati equation solution approach. As shown in Example 11.3-4, PI and VI boil down to repetitive solutions of Lyapunov equations or Lyapunov recursions. In fact, PI becomes nothing but Hewer's method (Hewer 1971), and VI becomes a well-known Lyapunov recursion scheme that was shown to converge in Lancaster and Rodman (1995). These are offline methods relying on matrix equation solutions and requiring complete knowledge of the system dynamics.

Monte Carlo learning is based on the definition (11.2-16) for the value function, and uses repeated measurements of data to approximate the expected value. The expected values are approximated by averaging repeated results along sample paths. An assumption on the ergodicity of the Markov chain with transition probabilities (11.2-2) for the given policy being evaluated is implicit. This is suitable for episodic tasks, with experience divided into episodes (Sutton and Barto 1998)—namely, processes that start in an initial state and run until termination, and are then restarted at a new initial state. For finite MDP, Monte Carlo methods converge to the true value function if all states are visited infinitely often. Therefore, to ensure good approximations of value functions, the episode sample paths must go through all the states $x \in X$ many times. This is called the problem of maintaining exploration. There are several ways of ensuring this, one of which is to use exploring starts, in which every state has nonzero probability of being selected as the initial state of an episode.

Monte Carlo techniques are useful for dynamic systems control because the episode sample paths can be interpreted as system trajectories beginning in a prescribed initial state. However, no updates to the value function estimate or the control policy are made until after an episode terminates. In fact, Monte Carlo learning methods are closely related to repetitive or iterative learning control (ILC) (Moore 1993). They do not learn in real time along a trajectory, but learn as trajectories are repeated.

11.4 TEMPORAL DIFFERENCE LEARNING AND OPTIMAL ADAPTIVE CONTROL

The temporal difference (TD) method (Sutton and Barto 1998) for solving Bellman equations leads to a family of **optimal adaptive controllers**—that is, adaptive controllers that learn online the solutions to optimal control problems without knowing the full system dynamics. TD learning is true online reinforcement learning, wherein control actions are improved in real time based on estimating their value functions by observing data measured along the system trajectories. In the context of TD learning, the interpretation of the Bellman equation is shown in Figure 11.2-2, where $V^\pi(x)$ may be considered a predicted performance, $\sum_u \pi(x, u) \sum_{x'} P_{xx'}^u R_{xx'}^u$, the observed one-step reward, and $V^\pi(x')$ a current estimate of future behavior.

Temporal Difference Learning along State Trajectories

PI requires solution at each step of N linear equations (11.3-4). VI requires performing the recursion (11.3-10) at each step. Temporal difference RL methods are based on the Bellman equation, and solve equations such as (11.3-4) and (11.3-10) **without using systems dynamics knowledge, but using and data observed along a single trajectory of the system**. This makes them extremely applicable for feedback control applications. TD updates the value at each time step as observations of data are made along a trajectory. Periodically, the new value is used to update the policy. TD methods are related to adaptive control in that they adjust values and actions online in real time along system trajectories.

TD methods can be considered to be **stochastic approximation techniques**, whereby the **Bellman equation** (11.2-17), or its variants (11.3-4), (11.3-10), is **replaced by its evaluation along a single sample path of the MDP**. This turns the Bellman equation, into a deterministic equation, which allows the definition of a so-called **temporal difference error**.

Equation (11.2-9) was used to write the Bellman equation (11.2-17) for the infinite-horizon value (11.2-16). According to (11.2-7)–(11.2-9), an **alternative form for the Bellman equation** is

$$V^\pi(x_k) = E_\pi\{r_k|x_k\} + \gamma E_\pi\{V^\pi(x_{k+1})|x_k\}. \quad (11.4-1)$$

This equation forms the **basis for TD learning**.

Temporal difference RL uses one sample path, namely the current system trajectory, to update the value. That is (11.4-1) is replaced by the deterministic Bellman equation

$$V^\pi(x_k) = r_k + \gamma V^\pi(x_{k+1}), \quad (11.4-2)$$

which holds for each observed data experience set (x_k, x_{k+1}, r_k) at each time stage k . This set consists of the current state x_k , the observed cost incurred r_k ,

and the next state x_{k+1} . The **temporal difference error** is defined as

$$e_k = -V^\pi(x_k) + r_k + \gamma V^\pi(x_{k+1}), \quad (11.4-3)$$

and the value estimate is updated to make the temporal difference error small.

The Bellman equation can be interpreted as a **consistency equation**, which holds if the current estimate for the value $V^\pi(x_k)$ is correct. Therefore, TD methods update the value estimate $\hat{V}^\pi(x_k)$ to make the TD error small. The idea is that if the deterministic version of Bellman's equation is used repeatedly, then on average one will converge toward the solution of the stochastic Bellman equation.

11.5 OPTIMAL ADAPTIVE CONTROL FOR DISCRETE-TIME SYSTEMS

A family of optimal adaptive control algorithms can now be developed for dynamical systems. Physical analysis of dynamical systems using Lagrangian mechanics, Hamiltonian mechanics, etc. produces system descriptions in terms of nonlinear ordinary differential equations. Discretization yields nonlinear difference equations. The bulk of research in RL has been conducted for systems that operate in discrete time (DT). Therefore, we cover DT dynamical systems first, then continuous-time systems.

TD learning is a stochastic approximation technique based on the deterministic Bellman's equation (11.4-2). Therefore, we lose little by considering deterministic systems here. Consider a class of discrete-time systems described by deterministic nonlinear dynamics in the affine state space difference equation form

$$x_{k+1} = f(x_k) + g(x_k)u_k, \quad (11.5-1)$$

with state $x_k \in R^n$ and control input $u_k \in R^m$. We use this form because its analysis is convenient. The following development can be generalized to the general sampled-data form $x_{k+1} = F(x_k, u_k)$.

A control policy is defined as a function from state space to control space $h(\cdot): R^n \rightarrow R^m$. That is, for every state x_k , the policy defines a control action

$$u_k = h(x_k). \quad (11.5-2)$$

That is, a policy is simply a feedback controller.

Define a cost function that yields the value function

$$V^h(x_k) = \sum_{i=k}^{\infty} \gamma^{i-k} r(x_i, u_i) = \sum_{i=k}^{\infty} \gamma^{i-k} (Q(x_i) + u_i^T R u_i), \quad (11.5-3)$$

with $0 < \gamma \leq 1$ a discount factor, $Q(x_k) > 0$, $R > 0$, and $u_k = h(x_k)$ a prescribed feedback control policy. The stage cost

$$r(x_k, u_k) = Q(x_k) + u_k^T R u_k. \quad (11.5-4)$$

is taken to be quadratic in u_k to simplify developments, but can be any positive definite function of the control. We assume the system is stabilizable on some set $\Omega \in R^n$; that is, there exists a control policy $u_k = h(x_k)$ such that the closed-loop system $x_{k+1} = f(x_k) + g(x_k)h(x_k)$ is asymptotically stable on Ω . A control policy $u_k = h(x_k)$ is said to be *admissible* if it is stabilizing and yields a finite cost $V^h(x_k)$ for trajectories in Ω .

For the deterministic DT system, the optimal value is given by Bellman's optimality equation

$$V^*(x_k) = \min_{h(\cdot)} (r(x_k, h(x_k)) + \gamma V^*(x_{k+1})). \quad (11.5-5)$$

This is just the discrete-time Hamilton-Jacobi-Bellman (HJB) equation. One then has the optimal policy as

$$h^*(x_k) = \arg \min_{h(\cdot)} (r(x_k, h(x_k)) + \gamma V^*(x_{k+1})). \quad (11.5-6)$$

In this setup, the deterministic Bellman's equation (11.4-2) is

$$V^h(x_k) = r(x_k, u_k) + \gamma V^h(x_{k+1}) = Q(x_k) + u_k^T R u_k + \gamma V^h(x_{k+1}), \quad V^h(0) = 0. \quad (11.5-7)$$

This is nothing but a difference equation equivalent of the value (11.5-3). That is, instead of evaluating the infinite sum (11.5-3), one can solve the difference equation (11.5-7), with boundary condition $V(0) = 0$, to obtain the value of using a current policy $u_k = h(x_k)$.

The DT Hamiltonian function is

$$H(x_k, h(x_k), \Delta V_k) = r(x_k, h(x_k)) + \gamma V^h(x_{k+1}) - V^h(x_k), \quad (11.5-8)$$

where $\Delta V_k = \gamma V_h(x_{k+1}) - V_h(x_k)$ is the forward difference operator. The Hamiltonian function captures the energy content along the trajectories of a system. In fact, the Hamiltonian is the temporal difference error (11.4-3). The Bellman equation requires that the Hamiltonian be equal to zero for the value associated with a prescribed policy.

For the DT linear quadratic regulator (DT LQR) case,

$$x_{k+1} = A x_k + B u_k, \quad (11.5-9)$$

$$V^h(x_k) = \frac{1}{2} \sum_{i=k}^{\infty} \gamma^{i-k} (x_i^T Q x_i + u_i^T R u_i), \quad (11.5-10)$$

and the Bellman equation is written in several ways, as seen in Example 11.2-1.

Policy Iteration and Value Iteration Using Temporal Difference Learning

It has been seen that two forms of reinforcement learning can be based on policy iteration and value iteration. For TD learning, PI is written as follows in terms of the deterministic Bellman equation.

POLICY ITERATION USING TD LEARNING

Initialize.

Select any admissible control policy $h_0(x_k)$. Do for $j = 0$ until convergence—

Policy evaluation:

$$V_{j+1}(x_k) = r(x_k, h_j(x_k)) + \gamma V_{j+1}(x_{k+1}). \quad (11.5-11)$$

Policy improvement:

$$h_{j+1}(x_k) = \arg \min_{h(\cdot)} (r(x_k, h(x_k)) + \gamma V_{j+1}(x_{k+1})), \quad (11.5-12)$$

or equivalently

$$h_{j+1}(x_k) = -\frac{\gamma}{2} R^{-1} g^T(x_k) \nabla V_{j+1}(x_{k+1}), \quad (11.5-13)$$

where $\nabla V(x) = \partial V(x)/\partial x$ is the gradient of the value function, interpreted here as a column vector.

VI is similar, but the policy evaluation procedure is performed as follows.

VALUE ITERATION USING TD LEARNING

Value update step:

Update the value using

$$V_{j+1}(x_k) = r(x_k, h_j(x_k)) + \gamma V_j(x_{k+1}). \quad (11.5-14)$$

Also, in VI we may select any initial control policy $h_0(x_k)$, not necessarily admissible or stabilizing.

It has been shown in Example 11.2-1 that for DT LQR the Bellman equation (11.5-7) is nothing but a linear Lyapunov equation and that (11.5-5) is the DT algebraic Riccati equation (ARE). In Example 11.3-4 it was seen that for the DT LQR the policy evaluation step (11.5-11) in PI is a Lyapunov equation and PI exactly corresponds to Hewer's algorithm (Hewer 1971) for solving the DT ARE. Hewer proved that it converges under stabilizability and detectability assumptions. For DT LQR, VI is a Lyapunov recursion, which has been shown to converge to the solution to the DT ARE under the stated assumptions by (Lancaster and Rodman 1995).

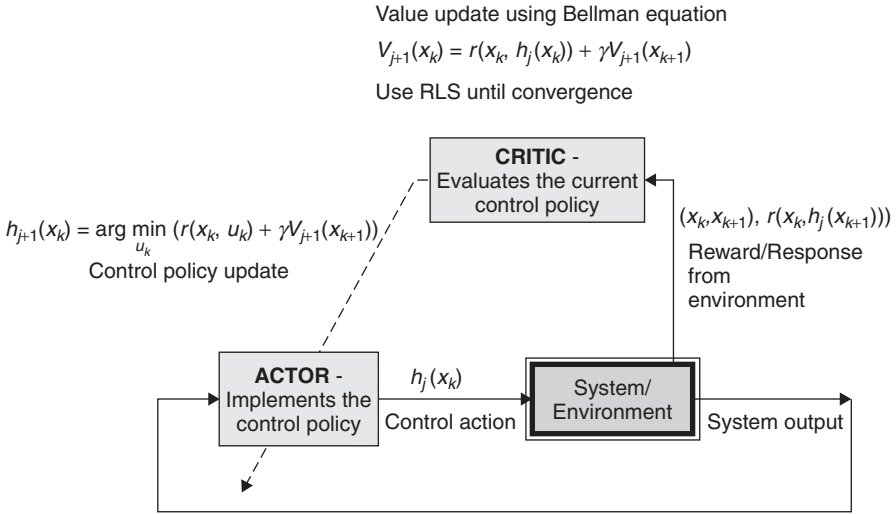


FIGURE 11.5-1 Temporal difference learning using policy iteration. At each time one observes the current state, the next state, and the cost incurred. This is used to update the value estimate. Based on the new value, the action is updated.

The online implementation of PI using temporal difference learning is shown in Figure 11.5-1. PI and VI using TD learning have an actor–critic structure, as shown in Figure 11.1-1. The critic evaluates the current policy by solving the Bellman equation (11.5-11). Then, the actor updates the policy using (11.5-12).

Value Function Approximation

PI and VI can be implemented for finite MDP by storing and updating look-up tables. The key to practical schemes for implementing PI and VI online for dynamical systems with infinite state and action spaces is to approximate the value function by a suitable approximator structure in terms of unknown parameters. Then, the unknown parameters are tuned online exactly as in system identification. This idea of **value function approximation (VFA)** was used by Werbos (1989, 1991, 1992) and called **approximate dynamic programming (ADP)** or adaptive dynamic programming. It was used by Bertsekas and Tsitsiklis (1996) and called neuro-dynamic programming (NDP). See Powell (2007) and Busoniu et al. (2009).

In the LQR case it is known that the value is quadratic in the state so that

$$V(x_k) = \frac{1}{2} x_k^T P x_k = \frac{1}{2} (\text{vec}(P))^T (x_k \otimes x_k) \equiv \bar{p}^T \bar{x}_k \equiv \bar{p}^T \phi(x_k), \quad (11.5-15)$$

for some kernel matrix P . The Kronecker product $\otimes$ (Brewer 1978) allows us to write this quadratic form as linear in the parameter vector $\bar{p} = \text{vec}(P)$, which is formed by stacking the columns of the P matrix. The vector $\phi(x_k) = \bar{x}_k \otimes \bar{x}_k$

x_k is the quadratic polynomial vector containing all possible pairwise products of the n components of x_k . Noting that P is symmetric and has only $n(n+1)/2$ independent elements, we remove the redundant terms in $x_k \otimes x_k$ to define a quadratic basis set $\phi(x_k)$ with $n(n+1)/2$ independent elements.

For general nonlinear systems (11.5-1) the value function contains higher-order nonlinearities. Then, we assume the Bellman equation (11.5-7) has a local smooth solution (van der Schaft 1992). According to the Weierstrass higher-order approximation theorem, there exists a dense basis set $\{\varphi_i(x)\}$ such that

$$V(x) = \sum_{i=1}^{\infty} w_i \varphi_i(x) = \sum_{i=1}^L w_i \varphi_i(x) + \sum_{i=L+1}^{\infty} w_i \varphi_i(x) \equiv W^T \phi(x) + \varepsilon_L(x), \quad (11.5-16)$$

where basis vector $\phi(x) = [\varphi_1(x) \ \varphi_2(x) \ \cdots \ \varphi_L(x)]: R^n \rightarrow R^L$, and $\varepsilon_L(x)$ converges uniformly to zero as the number of terms retained $L \rightarrow \infty$. In the Weierstrass Theorem, standard usage takes a polynomial basis set. In the neural network (NN) community, approximation results have been shown for various other basis sets including sigmoid, hyperbolic tangent, Gaussian radial basis functions, etc. There, standard results show that the NN approximation error $\varepsilon_L(x)$ is bounded by a constant on a compact set. L is referred to as the number of hidden-layer neurons, $\varphi_i(x)$ as the NN activation functions, and w_i as the NN weights.

Optimal Adaptive Control Algorithms for DT Systems

We are now in a position to present several adaptive control algorithms based on TD RL that converge online to the optimal control solution.

The parameters in $\bar{p}$ or W are unknown. Substituting the value function approximation into the value update (11.5-11) in PI we obtain the following algorithm.

OPTIMAL ADAPTIVE CONTROL USING A POLICY ITERATION ALGORITHM

Initialize.

Select any admissible control policy $h_0(x_k)$. Do for $j = 0$ until convergence—

Policy evaluation step:

Determine the least-squares solution W_{j+1} to

$$W_{j+1}^T (\phi(x_k) - \gamma \phi(x_{k+1})) = r(x_k, h_j(x_k)) = Q(x_k) + h_j^T(x_k) R h_j(x_k). \quad (11.5-17)$$

Policy improvement step:

Determine an improved policy using

$$h_{j+1}(x_k) = -\frac{\gamma}{2} R^{-1} g^T(x_k) \nabla \phi^T(x_{k+1}) W_{j+1}. \quad (11.5-18)$$

This algorithm is easily implemented online by standard system identification techniques (Ljung 1999). In fact, note that (11.5-17) is a scalar equation, whereas the unknown parameter vector $W_{j+1} \in R^L$ has L elements. Therefore, data from multiple time steps is needed for its solution. At time $k + 1$ one measures the previous state x_k , the control $u_k = h_j(x_k)$, the next state x_{k+1} , and computes the resulting utility $r(x_k, h_j(x_k))$. This gives one scalar equation. This is repeated for subsequent times using the same policy $h_j(\cdot)$ until we have at least L equations, at which point we may determine the LS solution W_{j+1} . We may use batch LS for this.

Alternatively, note that equations of the form (11.5-17) are exactly those solved by **recursive least-squares (RLS)** techniques (Ljung 1999). Therefore, we may run RLS **online until convergence**. Write (11.5-17) as

$$W_{j+1}^T \Phi(k) \equiv W_{j+1}^T (\phi(x_k) - \gamma \phi(x_{k+1})) = r(x_k, h_j(x_k)), \quad (11.5-19)$$

with $\Phi(k) \equiv (\phi(x_k) - \gamma \phi(x_{k+1}))$ a **regression vector**. At step j of the PI algorithm, one fixes the control policy at $u = h_j(x)$. Then, at each time k one measures the data set $(x_k, x_{k+1}, r(x_k, h_j(x_k)))$. One step of RLS is then performed. This is repeated for subsequent times until convergence to the parameters corresponding to the value $V_{j+1}(x) = W_{j+1}^T \phi(x)$.

Note that for RLS to converge, the regression vector $\Phi(k) \equiv (\phi(x_k) - \gamma \phi(x_{k+1}))$ must be **persistently exciting**.

As an alternative to RLS, we could use a gradient descent tuning method such as

$$W_{j+1}^{i+1} = W_{j+1}^i - \alpha \Phi(k) \left(\left(W_{j+1}^i \right)^T \Phi(k) - r(x_k, h_j(x_k)) \right), \quad (11.5-20)$$

with $\alpha > 0$ a tuning parameter. The step index j is held fixed, and index i is incremented at each increment of the time index k . Note that the quantity inside the large brackets is just the temporal difference error.

Once the value parameters have converged, the control policy is updated according to (11.5-18). Then, the procedure is repeated for step $j + 1$. This entire procedure is repeated until convergence to the optimal control solution.

This provides an online reinforcement learning algorithm for solving the optimal control problem using policy iteration by measuring data along the system trajectories. Likewise, an online reinforcement learning algorithm can be given based on value iteration. Substituting the value function approximation into the value update (11.5-14) in VI we obtain the following algorithm.

OPTIMAL ADAPTIVE CONTROL USING A **VALUE ITERATION ALGORITHM**

Initialize

Select any control policy $h_0(x_k)$, not necessarily admissible or stabilizing. Do for $j = 0$ until convergence—

Value update step:

Determine the least-squares solution W_{j+1} to

$$W_{j+1}^T \phi(x_k) = r(x_k, h_j(x_k)) + W_j^T \gamma \phi(x_{k+1}). \quad (11.5-21)$$

Policy improvement step:

Determine an improved policy using (11.5-18).

To solve (11.5-21) in real-time we can use batch LS, RLS, or gradient-based methods based on data $(x_k, x_{k+1}, r(x_k, h_j(x_k)))$ measured at each time along the system trajectories. Then the policy is improved using (11.5-18).

Note that the old weight parameters are on the right-hand side of (11.5-21). Thus, the regression vector is now $\phi(x_k)$, which must be persistently exciting for convergence of RLS.

Online Solution of Lyapunov and Riccati Equations

It is important to note that the PI and VI adaptive optimal control algorithms just given actually solve the Bellman equation (11.5-7) and the HJB equation (11.5-5) online in real time by using data measured along the system trajectories. The system drift function $f(x_k)$ (or the A matrix in the LQR case) is not needed in these algorithms. That is, these algorithms solve the Riccati equation online in real time without knowledge of the A matrix. The online implementation of PI is shown in Figure 11.5-1.

According to Example 11.3-4, for DT LQR policy iteration, this means that the Lyapunov equation

$$0 = (A - BK^j)^T P^{j+1} (A - BK^j) - P^{j+1} + Q + (K^j)^T RK^j, \quad (11.5-22)$$

has been replaced by (11.5-17) or

$$\bar{p}_{j+1}^T (\bar{x}_{k+1} - \bar{x}_k) = r(x_k, h_j(k)) = x_k^T (Q + K_j^T RK_j) x_k, \quad (11.5-23)$$

which is solved for the parameters $\bar{p}_{j+1} = \text{vec}(P^{j+1})$ using, for instance, RLS by measuring the data set $(x_k, x_{k+1}, r(x_k, h_j(x_k)))$ at each time. For this step the dynamics (A, B) can be unknown, as they are not needed. For DT LQR value iteration, the Lyapunov recursion

$$P^{j+1} = (A - BK^j)^T P^j (A - BK^j) + Q + (K^j)^T RK^j \quad (11.5-24)$$

has been replaced by (11.5-21) or

$$\bar{p}_{j+1}^T \bar{x}_{k+1} = r(x_k, h_j(k)) + \bar{p}_j^T \bar{x}_k = x_k^T (Q + K_j^T RK_j) x_k + \bar{p}_j^T \bar{x}_k, \quad (11.5-25)$$

which may be solved for the parameters $\bar{p}_{j+1} = \text{vec}(P_{j+1})$ using RLS without knowing A, B .

Introduction of a Second “Actor” Neural Network

Using value function approximation allows standard system identification techniques to be used to find the value function parameters that approximately solve the Bellman equation. The approximator structure just described that is used for approximation of the value function is known as the critic NN (neural network), as it determines the value of using the current policy. Using VFA, the PI and VI reinforcement learning algorithm solve the Bellman equation during the value update portion of each iteration step j by observing only the data set $(x_k, x_{k+1}, r(x_k, h_j(x_k)))$ at each time along the system trajectory and solving (11.5-17) or (11.5-21).

However, according to Example 11.3-4, in the LQR case, the policy update (11.5-18) is given by

$$K^{j+1} = -(B^T P^{j+1} B + R)^{-1} B^T P^{j+1} A, \quad (11.5-26)$$

which requires full knowledge of the dynamics (A, B) . Note further that the embodiment (11.5-18) cannot easily be implemented in the nonlinear case because it is implicit in the control, since x_{k+1} depends on u_k and is the argument of a nonlinear activation function.

These problems are both solved by introducing a *second neural network* for the control policy, known as the actor NN (Werbos, 1989, 1991, 1992). Introduce a parametric approximator structure for the control action

$$u_k = h(x_k) = U^T \sigma(x_k), \quad (11.5-27)$$

with $\sigma(x): R^n \rightarrow R^M$ a vector of M activation or basis functions and $U \in R^{M \times m}$ a matrix of weights or unknown parameters. After convergence of the critic NN parameters to W_{j+1} in PI or VI, it is required to perform the policy update (11.5-18). To achieve this we may use a *gradient descent method* for tuning the actor weights U such as

$$U_{j+1}^{i+1} = U_{j+1}^i - \beta \sigma(x_k) \left(2R \left(U_{j+1}^i \right)^T \sigma(x_k) + \gamma g(x_k)^T \nabla \phi^T(x_{k+1}) W_{j+1} \right)^T, \quad (11.5-28)$$

with $\beta > 0$ a tuning parameter. The tuning index i may be incremented with the time index k . On convergence, set the updated policy to $h_{j+1}(x_k) = (U_{j+1}^i)^T \sigma(x_k)$.

Several items are worthy of note at this point. First, the *tuning of the actor* NN requires *observations at* each time k of the data set (x_k, x_{k+1}) , that is, the current state and the next state. However, as per the formulation (11.5-27), the actor NN yields the control u_k at time k in terms of the state x_k at time k . The next state x_{k+1} is not needed in (11.5-27). Thus, after (11.5-28) has converged, (11.5-27) is a legitimate feedback controller. Second, in the LQR case, the actor NN (11.5-27) embodies the feedback gain computation (11.5-26). This is highly intriguing, for the latter contains the state internal dynamics A , but the former

does not. This means that the A matrix is not needed to compute the feedback control. The reason is that the actor NN has learned information about A in its weights, since (x_k, x_{k+1}) are used in its tuning.

Finally, note that **only the input function $g(\cdot)$** (in the LQR case, the B matrix) is **needed in (11.5-28) to tune the actor NN**. Thus, **introducing a second actor NN has completely avoided the need for knowledge of the state drift dynamics $f(\cdot)$ (or A in the LQR case).**

Example 11.5-1. Discrete-time Optimal Adaptive Control of Power System Using Value Iteration

In this simulation we show how to use DT value iteration to solve the DT ARE online without knowing the system matrix A . We simulated the online VI algorithm (11.5-21), (11.5-28) for load frequency control of an electric power system. Power systems are complex nonlinear systems. However, during normal operation the system load, which gives the nonlinearity, has only small variations. As such, a linear model can be used to represent the system dynamics around an operating point specified by a constant load value. A problem rises from the fact that in an actual plant the parameter values are not precisely known, as reflected in an unknown system A matrix, yet an optimal control solution is sought.

The model of the system that is considered here is $\dot{x} = Ax + Bu$, where

$$A = \begin{bmatrix} -1/T_p & K_p/T_p & 0 & 0 \\ 0 & -1/T_T & 1/T_T & 0 \\ -1/RT_G & 0 & -1/T_G & -1/T_G \\ K_E & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 1/T_G \\ 0 \end{bmatrix}.$$

The system state is $x(t) = [\Delta f(t) \quad \Delta P_g(t) \quad \Delta X_g(t) \quad \Delta E(t)]^T$, where $\Delta f(t)$ is the incremental frequency deviation (Hz), $\Delta P_g(t)$ is the incremental change in generator output (p.u. MW), $\Delta X_g(t)$ is the incremental change in governor position (p.u. MW), and $\Delta E(t)$ is the incremental change in integral control. The system parameters are T_G , the governor time constant; T_T , turbine time constant; T_p , plant model time constant; K_p , plant model gain; R , speed regulation due to governor action; and K_E , integral control gain.

The values of the CT system parameters were randomly picked within specified operating ranges so that

$$A = \begin{bmatrix} -0.0665 & 8 & 0 & 0 \\ 0 & -3.663 & 3.663 & 0 \\ -6.86 & 0 & -13.736 & -13.736 \\ 0.6 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 13.7355 & 0 \end{bmatrix}.$$

The discrete-time dynamics was obtained using the zero-order hold method with a sampling period of $T = 0.01$ sec. The solution to the DT ARE with cost function weights $Q = I$, $R = I$, and $\gamma = 1$ is

$$P_{\text{DARE}} = \begin{bmatrix} 0.4750 & 0.4766 & 0.0601 & 0.4751 \\ 0.4766 & 0.7831 & 0.1237 & 0.3829 \\ 0.0601 & 0.1237 & 0.0513 & 0.0298 \\ 0.4751 & 0.3829 & 0.0298 & 2.3370 \end{bmatrix}.$$

In this simulation, only the time constant T_G of the governor, which appears in the B matrix, is considered to be known, while the values for all the other parameters appearing in the system A matrix are not known. That is, the A matrix is needed only to simulate the system and obtain the data and is not needed by the control algorithm.

For the DT LQR, the value is quadratic in the states $V(x) = \frac{1}{2}x^T Px$, as in (11.5-15). Therefore, the basis functions for the critic NN in (11.5-16) are selected as the quadratic polynomial vector in the state components. Since there are $n = 4$ states, this vector has $n(n+1)/2 = 10$ components. The control is linear in the states $u = -Kx$, so the basis functions for the actor NN (11.5-27) are taken as the state components.

The online implementation of VI may be done by setting up a batch least-squares problem to solve for the 10 critic NN parameters, that is the Riccati solution entries $\bar{p}_{j+1} \equiv W_{j+1}$ in (11.5-21), for each step j . In this simulation the matrix P^{j+1} is determined after collecting 15 points of data $(x_k, x_{k+1}, r(x_k, u_k))$ for each least-squares problem. Therefore, a least-squares problem for the critic weights is solved each 0.15 sec. Then the actor NN parameters (that is, the feedback gain matrix entries) are updated using (11.5-28). The simulations were performed over a time interval of 60 sec.

The system states trajectories are given in Figure 11.5-2, which shows that they are regulated to zero as desired. The convergence of the Riccati matrix parameters is shown in Figure 11.5-3. The final values of the critic NN parameter estimates are

$$P_{\text{critic NN}} = \begin{bmatrix} 0.4802 & 0.4768 & 0.0603 & 0.4754 \\ 0.4768 & 0.7887 & 0.1239 & 0.3834 \\ 0.0603 & 0.1239 & 0.0567 & 0.0300 \\ 0.4754 & 0.3843 & 0.0300 & 2.3433 \end{bmatrix}.$$

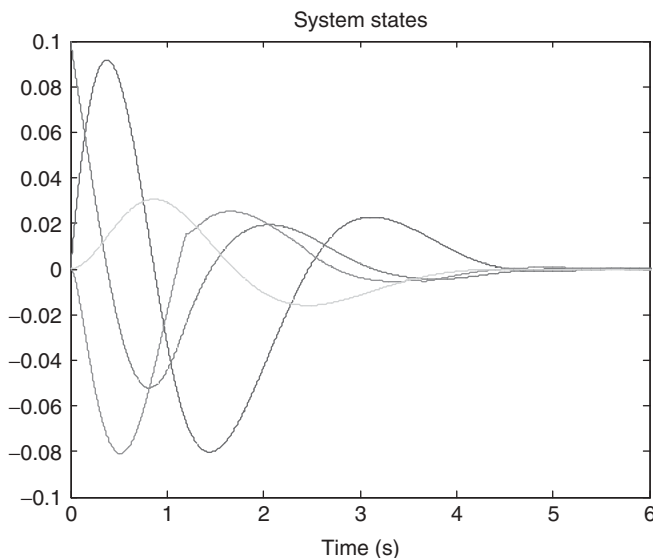


FIGURE 11.5-2 System states during the first 6 sec. This figure shows that even though the A matrix of the power system is unknown, the adaptive controller based on value iteration keeps the states stable and regulates them to zero.

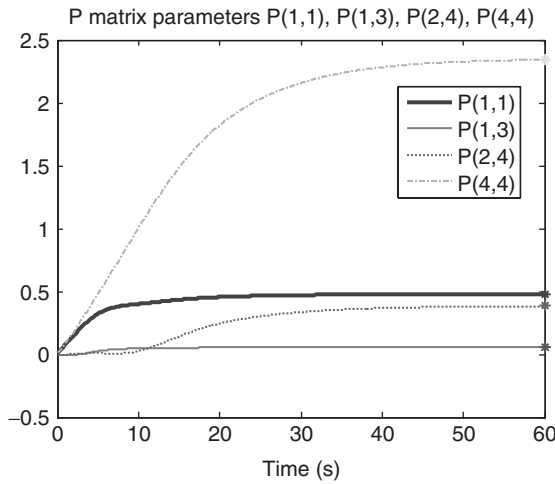


FIGURE 11.5-3 Convergence of selected ARE solution parameters. This figure shows that the adaptive controller based on VI converges to the Riccati-equation solution in real time without knowing the system A matrix.

The optimal adaptive control VI algorithm has converged to the optimal control solution, as given by the ARE solution. This has been accomplished in real time without knowing the system A matrix. ■

Actor–Critic Implementation of DT Optimal Adaptive Control

Two algorithms for optimal adaptive control of DT systems based on RL have been given. A PI algorithm is implemented by solving (11.5-17) using RLS or batch LS to determine the value of the current policy, then the policy is updated by running (11.5-28). A VI algorithm is implemented by solving (11.5-21) using RLS or batch LS to determine the value of the current policy, then the policy is updated by running (11.5-28).

The implementation of reinforcement learning using two NNs, one as a critic and one as an actor, yields the actor–critic RL structure shown in Figure 11.1-1. In this control system, the critic and the actor are tuned online using the observed data $(x_k, x_{k+1}, r(x_k, h_j(x_k)))$ along the system trajectory. The critic and actor are tuned sequentially in both the PI and the VI algorithms. That is, the weights of one NN are held constant while the weights of the other are tuned until convergence. This procedure is repeated until both NN have converged. Then, the controller has learned the optimal controller online. Thus, this is an online adaptive optimal control system wherein the value function parameters are tuned online and the convergence is to the optimal value and control. The convergence of value iteration using two NN for the DT nonlinear system (11.5-1) was proven in Al-Tamimi et al. (2008).

Q Learning for Optimal Adaptive Control

The Q learning RL method gives an adaptive control algorithm that converges online to the optimal control solution for completely unknown systems. That is, it solves the Bellman equation (11.5-7) and the HJB equation (11.5-5) online in real time by using data measured along the system trajectories, without any knowledge of the dynamics $f(x_k), g(x_k)$.

Q learning is a simple method for reinforcement learning that works for unknown MDP, that is, for systems with completely unknown dynamics. It was developed by Watkins (1989) and Watkins and Dayan (1992) and Werbos (1989, 1991, 1992), who called it action-dependent heuristic dynamic programming (ADHDP), since the Q function depends on the control input. Q learning learns the Q function (11.3-38) using TD methods by performing an action u_k and measuring at each time stage the resulting data experience set (x_k, x_{k+1}, r_k) consisting of the current state, the next state, and the resulting stage cost. Writing the Q function Bellman equation (11.3-40) along a sample path gives

$$Q^\pi(x_k, u_k) = r(x_k, u_k) + \gamma Q^\pi(x_{k+1}, h(x_{k+1})), \quad (11.5-29)$$

which defines a TD error

$$e_k = -Q^\pi(x_k, u_k) + r(x_k, u_k) + \gamma Q^\pi(x_{k+1}, h(x_{k+1})). \quad (11.5-30)$$

The VI algorithm for Q function is given as (11.3-47). Based on this, the Q function is updated using the algorithm

$$Q_k(x_k, u_k) = Q_{k-1}(x_k, u_k) + \alpha_k \left[r(x_k, u_k) + \gamma \min_u Q_{k-1}(x_{k+1}, u) - Q_{k-1}(x_k, u_k) \right]. \quad (11.5-31)$$

This Q learning algorithm is similar to stochastic approximation methods of adaptive control or parameter estimation used in control systems. It was developed for finite MDP and the convergence proven by Watkins (1989) using stochastic approximation methods. It was shown that the algorithm converges for finite MDP provided that all state–action pairs are visited infinitely often and

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \quad \sum_{k=1}^{\infty} \alpha_k^2 < \infty, \quad (11.5-32)$$

that is, standard stochastic approximation conditions. On convergence, the TD error is (approximately) equal to zero. For finite MDP, Q learning requires storing a 2-D lookup table in terms of all the states x and actions u .

The requirement that all state–action pairs are visited infinitely often translates to the problem of maintaining sufficient exploration during learning. Let us now derive methods for Q learning for dynamical systems that yield adaptive

control algorithms that converge to optimal control solutions. PI and VI algorithms have been given using the Q function in (11.3-43)–(11.3-47). A Q learning algorithm is easily developed for DT dynamical systems using Q function approximation (Werbos 1989, 1991, 1992; Bradtke et al. 1994). It was shown in Example 11.3-5 that for DT LQR the Q function is a quadratic form in terms of $z_k \equiv [x_k^T \ u_k^T]^T \in R^{n+m}$. Assume, therefore, that for nonlinear systems the Q function is parameterized as

$$Q(x, u) = W^T \phi(z),$$

for some unknown parameter vector W and basis set vector $\phi(z)$. For the DT LQR, $\phi(z)$ is the quadratic basis set formed from the state and input components. Substituting the Q function approximation into the TD error (11.5-30) yields

$$e_k = -W^T \phi(z_k) + r(x_k, u_k) + \gamma W^T \phi(z_{k+1}), \quad (11.5-33)$$

upon which either PI or VI algorithms can be based. Considering the PI algorithm (11.3-43), (11.3-44) yields the Q function evaluation step

$$W_{j+1}^T (\phi(z_k) - \gamma \phi(z_{k+1})) = r(x_k, h_j(x_k)) \quad (11.5-34)$$

and the policy improvement step

$$h_{j+1}(x_k) = \arg \min_u \left(W_{j+1}^T \phi(x_k, u) \right), \quad \text{for all } x \in X. \quad (11.5-35)$$

Q learning using VI (11.3-45) is given by

$$W_{j+1}^T \phi(z_k) = r(x_k, h_j(x_k)) + \gamma W_j^T \phi(z_{k+1}), \quad (11.5-36)$$

and (11.5-35). These equations do not require knowledge of the dynamics $f(\cdot)$, $g(\cdot)$.

For online implementation, batch LS or RLS can be used to solve (11.5-34) for the parameter vector W_{j+1} given the regression vector $(\phi(z_k) - \gamma \phi(z_{k+1}))$, or (11.5-36) using regression vector $\phi(z_k)$. The observed data at each time instant is $(z_k, z_{k+1}, r(x_k, u_k))$ with $z_k \equiv [x_k^T \ u_k^T]^T$. We take $z_{k+1} \equiv [x_{k+1}^T \ u_{k+1}^T]^T$, with $u_{k+1} = h_j(x_{k+1})$ and $h_j(\cdot)$ the current policy. Probing noise must be added to the control input to obtain persistence of excitation.

After convergence of the Q function parameters, the action update (11.5-35) is performed. This is easily accomplished without knowing the system dynamics due to the fact that the Q function contains u_k as an argument so that $\partial (W_{j+1}^T \phi(x_k, u)) / \partial u$ can be explicitly computed. In fact,

$$\frac{\partial Q(x, u)}{\partial u} = \left(\frac{\partial z}{\partial u} \right)^T \left(\frac{\partial \phi(z)}{\partial z} \right)^T W = [0_{m,n} \quad I_m] \nabla \phi^T W, \quad (11.5-37)$$

where $0_{m,n} \in R^{m \times n}$ is a matrix of zeros. For the specific case of the DT LQR, for instance, from Example 11.3-5 we have

$$Q(x, u) \equiv \frac{1}{2} \begin{bmatrix} x \\ u \end{bmatrix}^T S \begin{bmatrix} x \\ u \end{bmatrix} = \frac{1}{2} z^T S z = \frac{1}{2} \text{vec}^T(S)(z \otimes z) \equiv W^T \phi(z), \quad (11.5-38)$$

with $\otimes$ the Kronecker product (Brewer 1978) and $\text{vec}(S) \in R^{(n+m)^2}$ the vector formed by stacking the columns of the S matrix. The basis vector $\phi(z) = z \otimes z \in R^{(n+m)^2}$ is the quadratic polynomial vector containing all possible pairwise products of the $n + m$ components of z . Define $N = n + m$. Then,

$$\nabla \phi^T = \frac{\partial \phi^T}{\partial z} = (I_N \otimes z + z \otimes I_N)^T \in R^{N \times N^2} \quad (11.5-39)$$

Using these equations we obtain a form equivalent to the control update (11.5-35) given in Example 11.3-5, which can be done knowing the Q function parameters S without knowing system matrices A, B .

If the control can be computed explicitly from the action update (11.5-35), as in the DT LQR case, an actor NN is not needed for Q learning and it can be implemented using only one critic NN for Q function approximation.

11.6 INTEGRAL REINFORCEMENT LEARNING FOR OPTIMAL ADAPTIVE CONTROL OF CONTINUOUS-TIME SYSTEMS

Reinforcement learning is considerably more difficult for continuous-time (CT) systems than for discrete-time systems, and its development has lagged. See (Abu-Khalaf et al. 2006) for the development of a PI method for CT systems. Using a method known as *integral reinforcement learning* (IRL) (Vrabie et al. Vrabie and Lewis 2009) allows the application of RL to formulate online optimal adaptive control methods for CT systems.

Consider the continuous-time nonlinear dynamical system

$$\dot{x} = f(x) + g(x)u, \quad (11.6-1)$$

with state $x(t) \in R^n$, control input $u(t) \in R^m$, and the usual assumptions required for existence of unique solutions and an equilibrium point at $x = 0$, e.g., $f(0) = 0$ and $f(x) + g(x)u$ Lipschitz on a set $\Omega \subseteq R^n$ that contains the origin. We assume the system is stabilizable on Ω ; that is, there exists a continuous control function $u(t)$ such that the closed-loop system is asymptotically stable on Ω .

Define a performance measure or cost function that has the value associated with the feedback control policy $u = \mu(x)$ given by

$$V^\mu(x(t)) = \int_t^\infty r(x(\tau), u(\tau)) d\tau, \quad (11.6-2)$$

with utility $r(x, u) = Q(x) + u^T R u$, $Q(x)$ positive definite, that is, $Q(x) > 0$ for all x and $x = 0 \Rightarrow Q(x) = 0$, and $R > 0$ a positive definite matrix. For the CT linear quadratic regulator (LQR) we have

$$\dot{x} = Ax + Bu, \quad (11.6-3)$$

$$V^\mu(x(t)) = \frac{1}{2} \int_t^\infty (x^T Q x + u^T R u) d\tau. \quad (11.6-4)$$

A policy is called **admissible** if it is continuous, stabilizes the system, and has a finite associated cost. If the cost is smooth, then **an infinitesimal equivalent** to (11.6-2) can be found by differentiation to be the nonlinear equation

$$0 = r(x, \mu(x)) + (\nabla V^\mu)^T (f(x) + g(x)\mu(x)), \quad V^\mu(0) = 0, \quad (11.6-5)$$

where ∇V^μ (a column vector) denotes the gradient of the cost function V^μ with respect to x . This is the **CT Bellman equation**. It is defined based on the **CT Hamiltonian function**

$$H(x, \mu(x), \nabla V^\mu) = r(x, \mu(x)) + (\nabla V^\mu)^T (f(x) + g(x)\mu(x)). \quad (11.6-6)$$

The optimal value satisfies the CT **Hamilton-Jacobi-Bellman (HJB)** equation (Chapter 6),

$$0 = \min_{\mu} H(x, \mu(x), \nabla V^*), \quad (11.6-7)$$

and the optimal control satisfies

$$\mu^* = \arg \min_{\mu} H(x, \mu(x), \nabla V^*). \quad (11.6-8)$$

We now see the problem with CT systems immediately. Compare the CT Bellman Hamiltonian (11.6-6) to the DT Hamiltonian (11.5-8). The former contains the **full system dynamics** $f(x) + g(x)u$, while the **DT Hamiltonian does not**. This means the CT Bellman equation (11.6-5) cannot be used as a basis for reinforcement learning unless the full dynamics are known.

Reinforcement learning methods based on (11.6-5) can be developed (Baird 1994, Doya 2000, Murray et al. 2001, Mehta and Meyn 2009, Hanselmann et al. 2007). These have limited use for adaptive control purposes because the system dynamics must be known. In another approach, one can use **Euler's method** to **discretize the CT** Bellman equation (Baird 1994). Noting that

$$0 = r(x, \mu(x)) + (\nabla V^\mu)^T (f(x) + g(x)\mu(x)) = r(x, \mu(x)) + \dot{V}^\mu, \quad (11.6-9)$$

We use **Euler's method to discretize** this to obtain

$$0 = r(x_k, u_k) + \frac{V^\mu(x_{k+1}) - V^\mu(x_k)}{T} \equiv \frac{r_S(x_k, u_k)}{T} + \frac{V^\mu(x_{k+1}) - V^\mu(x_k)}{T}, \quad (11.6-10)$$

with sample period T so that $t = kT$. The discrete sampled utility is $r_S(x_k, u_k) = r(x_k, u_k)T$, where it is important to multiply the CT utility by the sample period.

Now note that the discretized CT Bellman equation (11.6-10) has the same form as the DT Bellman equation (11.5-7). Therefore, all the reinforcement learning methods just described for DT systems can be applied.

However, this is an approximation only. An alternative exact method for CT reinforcement learning was given by Vrabie et al. (2009) and Vrabie and Lewis (2009). This is termed integral reinforcement learning (IRL). Note that we may write the cost (11.6-2) in the integral reinforcement form

$$V^\mu(x(t)) = \int_t^{t+T} r(x(\tau), u(\tau)) d\tau + V^\mu(x(t+T)), \quad (11.6-11)$$

for any $T > 0$. This is exactly in the form of the DT Bellman equation (11.5-7). According to Bellman's principle, the optimal value is given in terms of this construction as (Chapter 6)

$$V^*(x(t)) = \min_{\bar{u}(t:t+T)} \left(\int_t^{t+T} r(x(\tau), u(\tau)) d\tau + V^*(x(t+T)) \right),$$

where $\bar{u}(t:t+T) = \{u(\tau) : t \leq \tau < t+T\}$. The optimal control is

$$\mu^*(x(t)) = \arg \min_{\bar{u}(t:t+T)} \left(\int_t^{t+T} r(x(\tau), u(\tau)) d\tau + V^*(x(t+T)) \right).$$

It is shown in (Vrabie et al. 2009) that the nonlinear equation (11.6-5) is exactly equivalent to the integral reinforcement form (11.6-11). That is, the positive definite solution of both that satisfies $V(0) = 0$ is the value (11.6-2) of the policy $u = \mu(x)$. Therefore, integral reinforcement form (11.6-11) also serves as a Bellman equation for CT systems and serves as a fixed-point equation. Thus, we can define the temporal difference error for CT systems as

$$e(t:t+T) = \int_t^{t+T} r(x(\tau), u(\tau)) d\tau + V^\mu(x(t+T)) - V^\mu(x(t)). \quad (11.6-12)$$

This does not involve the system dynamics.

Now, policy iteration and value iteration can be directly formula for CT systems. The following algorithms are termed integral reinforcement learning for CT systems (Vrabie et al. 2009). They both give optimal adaptive controllers for CT systems, that is, adaptive control algorithms that converge to optimal control solutions.

IRL OPTIMAL ADAPTIVE CONTROL USING POLICY ITERATION (PI)

Initialize

Select any admissible control policy $\mu_0(x)$. Do for $j = 0$ until convergence—

Policy evaluation step:

Solve for $V_{j+1}(x(t))$ using

$$V_{j+1}(x(t)) = \int_t^{t+T} r(x(s), \mu_j(x(s))) ds + V_{j+1}(x(t+T)), \quad \text{with } V_{j+1}(0) = 0. \quad (11.6-13)$$

Policy improvement step:

Determine an improved policy using

$$\mu_{j+1} = \arg \min_u [H(x, u, \nabla V_{j+1})], \quad (11.6-14)$$

which explicitly is

$$\mu_{j+1}(x) = -\frac{1}{2} R^{-1} g^T(x) \nabla V_{j+1}. \quad (11.6-15)$$

IRL OPTIMAL ADAPTIVE CONTROL USING VALUE ITERATION (VI)

Initialize

Select any control policy $\mu_0(x)$, not necessarily stabilizing. Do for $j = 0$ until convergence—

Policy evaluation step:

Solve for $V_{j+1}(x(t))$ using

$$V_{j+1}(x(t)) = \int_t^{t+T} r(x(s), \mu_j(x(s))) ds + V_j(x(t+T)). \quad (11.6-16)$$

Policy improvement step:

Determine an improved policy using (11.6-15).

Note that neither algorithm requires knowledge about the system drift dynamics function $f(x)$. That is, they work for partially unknown systems. Convergence of PI is proved in Vrabie et al. (2009).

The time is incremented at each iteration by the reinforcement learning time interval T . This time interval need not be the same at each iteration. T can be changed depending on how long it takes to get meaningful information from the observations. T is not a sample period in the standard meaning.

The measured data at each time increment is $(x(t), x(t+T), \rho(t:t+T))$, where

$$\rho(t:t+T) = \int_t^{t+T} r(x(\tau), u(\tau)) d\tau, \quad (11.6-19)$$

is the **integral reinforcement measured on each time interval**. This can be implemented by introducing an integrator $\dot{\rho} = r(x(t), u(t))$, as shown in Figure 11.6-1. That is, the integral reinforcement $\rho(t)$ is added as an extra continuous-time state. It functions as the memory or controller dynamics. The remainder of the controller is a sampled data controller.

Note that the control policy $\mu(x)$ is updated periodically after the critic weights have converged to the solution of (11.6-17) or (11.6-18). Therefore, the policy is piecewise constant in time. On the other hand, the control varies continuously with the state between each policy update. IRL for CT systems is, in fact, a hybrid CT/DT adaptive controller that converges to the optimal control solution in real time without knowing the drift dynamics $f(x)$. Due to the fact that the policy update (11.6-15) for CT systems does not involve the drift dynamics $f(x)$, no actor NN is needed in IRL. Only a critic NN is needed for VFA.

Online Solution of the Algebraic Riccati Equation without Full Plant Dynamics

It can be shown that the integral reinforcement form (11.6-11) is equivalent to the nonlinear Lyapunov (11.6-5) (Vrabie et al. 2009). Thus, the IRL controller solves the Lyapunov equation online without knowing the drift dynamics $f(x)$. Moreover, it converges to the optimal control so that it solves the HJB equation (11.6-7).

In the CT LQR case (11.6-3), (11.6-4) we have linear state feedback control policies $u = -Kx$. Then, equation (11.6-5) is

$$(A - BK)^T P + P(A - BK) + Q + K^T R K = 0, \quad (11.6-20)$$

which is a Lyapunov equation. The HJB equation (11.6-7) becomes the CT ARE

$$A^T P + PA + Q - PBR^{-1}B^T P = 0. \quad (11.6-21)$$

Thus, IRL solves both the Lyapunov equation and the ARE online in real time, using data measured along the system trajectories, without knowing the A matrix.

For the CT LQR, (11.6-13) is equivalent to a Lyapunov equation at each step, so that policy iteration is exactly the same as Kleinman's algorithm (Kleinman 1968) for solving the CT Riccati equation. This is a Newton method for finding

the optimal value. CT value iteration, on the other hand, is a new algorithm that solves the CT ARE based on iterations on certain discrete-time Lyapunov equations that are equivalent to (11.6-16).

Example 11.6-1. Continuous-time Optimal Adaptive Control Using IRL

This example shows the hybrid control nature of the IRL optimal adaptive controller. Consider the DC motor model

$$\dot{x} = Ax + Bu = \begin{bmatrix} -10 & 1 \\ -0.002 & -2 \end{bmatrix} x + \begin{bmatrix} 0 \\ 2 \end{bmatrix} u,$$

with cost weight matrices $Q = I$, $R = I$. The solution to the CT ARE is computed to be

$$P = \begin{bmatrix} 0.05 & 0.0039 \\ 0.0039 & 0.2085 \end{bmatrix}.$$

In this simulation we used the IRL-based CT VI algorithm. This algorithm does not require knowledge of the system A matrix. For the CT LQR, the value is quadratic in the states. Therefore, the basis functions for the critic NN are selected as the quadratic polynomial vector in the state components, $\phi(x) = [x_1^2 \ x_1 x_2 \ x_2^2]$. The IRL time interval was selected as $T = 0.04$ sec. To update the 3 critic weights $\bar{p}_{j+1} \equiv W_{j+1}$ (that is, the ARE solution elements) using (11.6-18), a batch LS solution can be obtained. Measurements of the data set $(x(t), x(t+T), \rho(t:t+T))$ are taken over 3 time intervals of $T = 0.04$ sec. Then, provided that there is enough excitation in the system, after each 0.12 sec enough data are collected from the system to solve for the value of the matrix P . Then a greedy policy update is performed using (11.6-15), that is, $u = -R^{-1}B^T P x \equiv -Kx$.

The states are shown in Figure 11.6-2, which shows the good regulation achieved. The control input and feedback gains are shown in Figure 11.6-3. Note that the control

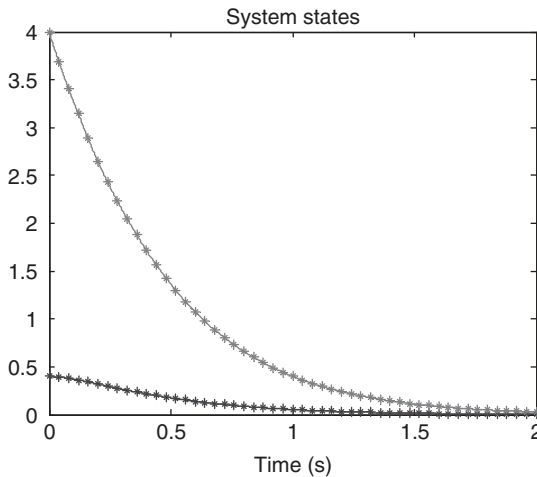


FIGURE 11.6-2 System states during the first 2 sec, showing that the continuous time IRL adaptive controller regulates the states to zero without knowing the system A matrix.

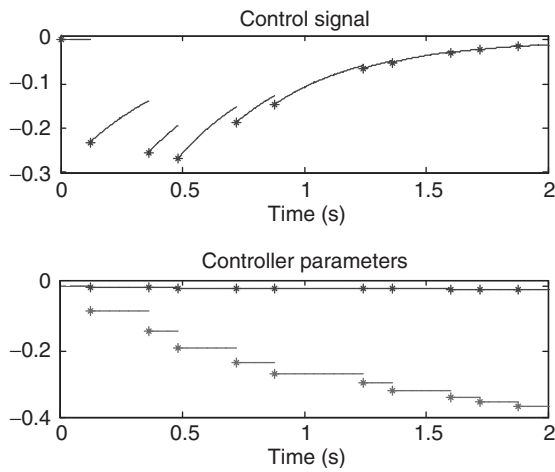


FIGURE 11.6-3 Control input and feedback gains, showing the hybrid nature of the IRL optimal adaptive controller. The controller gain parameters are discontinuous and piecewise constant, while the control signal itself is continuous between the gain parameter updates.

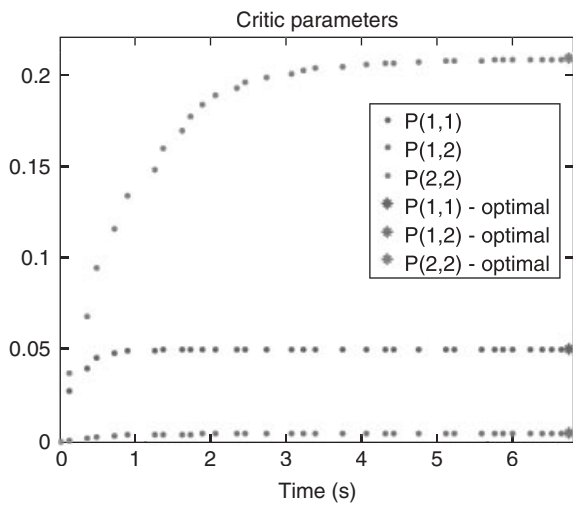


FIGURE 11.6-4 P matrix parameter estimates, showing that the IRL adaptive controller converges online to the optimal control Riccati equation solution without knowing the system A matrix.

gains are piecewise constant, while the control input is a continuous function of the state between policy updates. The critic NN parameter estimates are shown in Figure 11.6-4. They converge almost exactly to the entries in the Riccati solution matrix P . Thus, the ARE has been solved online without knowing the system A matrix. ■

Example 11.6-2. Continuous-time Optimal Adaptive Control for Power System Using IRL

In this example we simulate the CT IRL optimal adaptive control based on VI for the electric power system in Example 11.5-1. The same system matrices and performance index were used. The solution to the CT ARE is computed to be

$$P_{\text{ARE}} = \begin{bmatrix} 0.4750 & 0.4766 & 0.0601 & 0.4751 \\ 0.4766 & 0.7831 & 0.1237 & 0.3829 \\ 0.0601 & 0.1237 & 0.0513 & 0.0298 \\ 0.4751 & 0.3829 & 0.0298 & 2.3370 \end{bmatrix}.$$

This is close to the DT ARE solution presented in Example 11.5-1, since the sample period used there is small.

The VI IRL algorithm was simulated, which does not require knowledge of the system A matrix. The IRL time interval was taken as $T = 0.1$ sec. (Note the IRL interval is not related at all to the sample period used to discretize the system in Example 11.5-1.) Fifteen data points $(x(t), x(t+T), \rho(t:t+T))$ were taken to compute each batch LS update for the critic parameters $\bar{p}_{j+1} \equiv W_{j+1}$ (e.g., the elements of the ARE solution P) using (11.6-18). Hence, the value estimate was updated every 1.5 sec. Then the policy was computed using (11.6-15), that is, $u = -R^{-1}B^TPx \equiv -Kx$.

The state trajectories are similar to those presented in Example 11.5-1. The critic parameter estimates for the P matrix entries are shown in Figure 11.6-5. They converge to the true solution to the CT ARE. Thus, the ARE has been solved online without knowing the system A matrix.

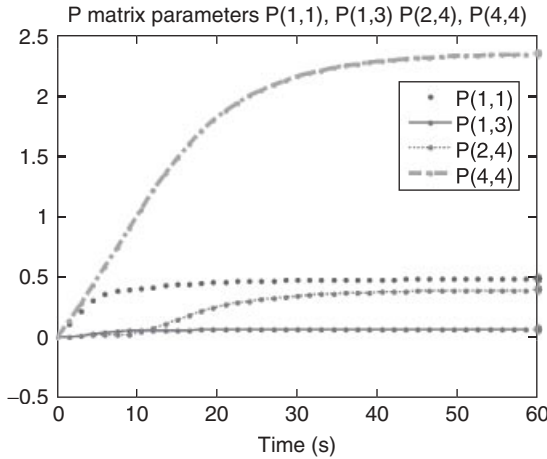


FIGURE 11.6-5 P matrix parameter estimates, showing that the IRL adaptive controller converges online to the optimal control Riccati equation solution without knowing the system A matrix.

Note that far less computation is needed using this IRL algorithm on the CT dynamics than was used in Example 11.5-1 for the DT optimal adaptive control algorithm. There, the critic parameter estimates were updated every 0.15 sec. Yet, the parameter estimates for the P matrix entries almost overlay each other. ■

Example 11.6-3. Continuous-time IRL Optimal Adaptive Control for Nonlinear System

In this example we show that IRL can solve the HJB equation for nonlinear CT systems by using data measured along the trajectories in real time. This example was developed using the converse HJB approach (Nevistic and Primbs 1996), which allows construction of nonlinear systems starting from the known optimal cost function.

Consider the nonlinear system given by the equations

$$\begin{cases} \dot{x}_1 = -x_1 + x_2 + 2x_2^3 \\ \dot{x}_2 = f(x) + g(x)u \end{cases}, \quad (11.6-22)$$

with $f(x) = -\frac{1}{2}(x_1 + x_2) + \frac{1}{2}x_2(1 + 2x_2^2)\sin^2(x_1)$, $g(x) = \sin(x_1)$. If we define $Q(x) = x_1^2 + x_2^2 + 2x_2^4$, $R = 1$, then the optimal cost function for this system is $V^*(x) = \frac{1}{2}x_1^2 + x_2^2 + x_2^4$ and the optimal controller is $u^*(x) = -\sin(x_1)(x_2 + 2x_2^3)$. It can be verified that for these choices the HJB equation (11.6-7) and the Bellman equation (11.6-5) are both satisfied.

The cost function was approximated by the smooth function $V_j(x(t)) = W_j^T \phi(x(t))$ with $L = 8$ neurons and $\phi(x) = [x_1^2 \ x_1 x_2 \ x_2^2 \ x_1^4 \ x_1^3 x_2 \ x_1^2 x_2^2 \ x_1 x_2^3 \ x_2^4]^T$. The PI IRL algorithm (11.6-17), (11.6-15) was used. This does not require knowledge of the drift dynamics $f(x)$.

To ensure exploration so that the HJB solution is found over a suitable region, data were taken along five trajectories defined by five different initial conditions chosen randomly in the region $\Omega = \{-1 \leq x_i \leq 1; i = 1, 2\}$. The IRL time period was taken as $T = 0.1$ sec. At each iteration step we set up a batch least-squares problem to solve for the eight NN weights using 40 data points measured on each of the 5 trajectories in Ω . Each data point consists of $(x(t), x(t+T), \rho(t:t+T))$, with $\rho(t:t+T)$ the measured integral

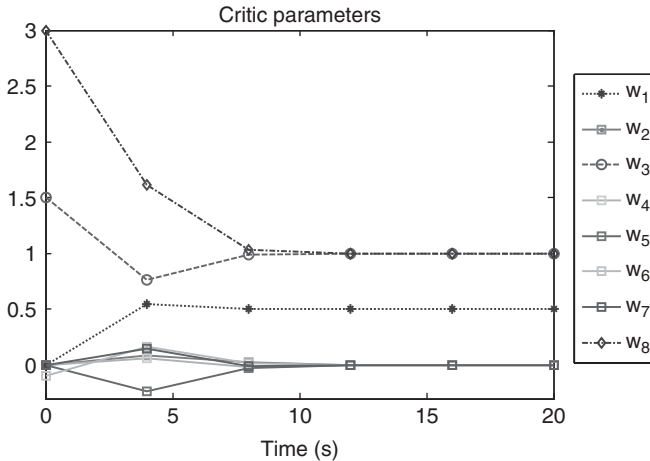


FIGURE 11.6-6 Convergence of critic NN parameters, showing that the IRL adaptive controller converges online to the optimal control Riccati equation solution without knowing the system A matrix.

reinforcement cost. In this way, at every 4 sec, the value was solved for and then a policy update was performed.

The result of applying the algorithm is presented in Figure 11.6-6, which shows that the parameters of the critic neural network converged to the coefficients of the optimal cost function $V^*(x) = \frac{1}{2}x_1^2 + x_2^2 + x_2^4$, that is, $W = [0.5 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1]^T$. We observe that after 3 iteration steps (that is, after 12 sec) the critic NN parameters have effectively converged. Then the controller is close to the optimal controller $u^*(x) = -\sin(x_1)(x_2 + 2x_2^3)$. The approximate solution to the HJB equation has been determined online and the optimal control has been found without knowing the system drift dynamics $f(x)$. Note that analytic solution of the HJB equation in this example would be intractable. ■

11.7 SYNCHRONOUS OPTIMAL ADAPTIVE CONTROL FOR CONTINUOUS-TIME SYSTEMS

The integral reinforcement learning controller just given tunes the critic NN to determine the value while holding the control policy fixed, then a policy update is performed. Now we develop an adaptive controller that has two NN, one for value function approximation and one to approximate the control. We could call these the critic NN and actor NN. These two NN are tuned simultaneously, that is, synchronously in time (Vamvoudakis and Lewis 2010a and b). This is more nearly in line with accepted practice in adaptive control. Though this synchronous controller does require knowledge of the dynamics, it converges to the approximate local solutions to the HJB equation and the Bellman equation online, yet does not require explicitly solving either one. The HJB is generally impossible to solve for nonlinear systems.

Based on the CT Hamiltonian (11.6-6) and the stationarity condition $0 = \partial H(x, u, \nabla V^\mu) / \partial u$, we could write a PI algorithm for CT systems based on the policy evaluation step

$$\begin{aligned} 0 &= H(x, \mu_j(x), \nabla V_{j+1}) = r(x, \mu_j(x)) + (\nabla V_{j+1})^T (f(x) + g(x)\mu_j(x)), \\ V_{j+1}(0) &= 0 \end{aligned} \quad (11.7-1)$$

and the policy improvement step

$$\mu_{j+1} = \arg \min_{\mu} H(x, \mu, \nabla V_{j+1}). \quad (11.7-2)$$

Unfortunately, the full dynamics $f(x)$, $g(x)$ are needed to implement this algorithm. Moreover, (11.7-1) is a nonlinear equation and cannot generally be solved.

However, this algorithm provides the structure needed to develop another adaptive control algorithm that can be implemented online using measured data along the trajectories and converges to the optimal control. Specifically, select a value function approximation (VFA), or critic NN, structure as

$$V(x) = W_1^T \phi(x) \quad (11.7-3)$$

and a control action approximation structure or actor NN as

$$u(x) = -\frac{1}{2}R^{-1}g^T(x)\nabla\phi^TW_2, \quad (11.7-4)$$

which could be, for instance, two neural networks with unknown parameters (weights) W_1 , W_2 , and $\phi(x)$ the basis set (activation functions) of the first NN. The structure of the second action NN comes from (11.6-15). Then, it can be shown that tuning the NN weights as

$$\dot{W}_1 = -\alpha_1 \frac{\sigma}{(\sigma^T\sigma + 1)^2} [\sigma^TW_1 + Q(x) + u^TRu], \quad (11.7-5)$$

$$\dot{W}_2 = -\alpha_2 \{ (F_2W_2 - F_1\bar{\sigma}^TW_1) - \frac{1}{4}D(x)W_2m^T(x)W_1 \}, \quad (11.7-6)$$

guarantees system stability as well as convergence to the optimal value and control (Vamvoudakis and Lewis, 2010a,b).

In these parameter estimation algorithms, $\alpha_1, \alpha_2, F_1, F_2$ are algorithm tuning parameters, $D(x) = \nabla\phi(x)g(x)R^{-1}g^T(x)\nabla\phi^T(x)$, $\sigma = \nabla\phi(f + gu)$, $\bar{\sigma} = \sigma/(\sigma^T\sigma + 1)$, and $m(x) = \sigma/(\sigma^T\sigma + 1)^2$. A PE condition on $\bar{\sigma}(t)$ is needed to get convergence to the optimal value.

This is an adaptive control algorithm that requires full knowledge of the system dynamics $f(x), g(x)$, yet converges to the optimal control solution. That is, it solves (locally approximately) the HJB equation, which is generally intractable for general nonlinear systems. In the CT LQR case, it solves the ARE using data measured along the trajectories (and knowledge of A, B). The importance of this algorithm is that it can approximately solve the HJB equation for nonlinear systems using data measured along the system trajectories in real time. The HJB is generally impossible to solve for nonlinear systems.

The VFA tuning algorithm for W_1 is based on gradient descent, while the control action tuning algorithm is a form of backpropagation (Werbos 1989), which is, however, also tuned by the VFA weights W_1 . The similarity to the actor-critic RL structure in Figure 11.1-1 is clear. However, in contrast to IRL, this algorithm is a CT optimal adaptive controller with two parameter estimators tuned simultaneously, that is, synchronously and continuously in time.

Example 11.7-1. Continuous-time Synchronous Optimal Adaptive Control

In this example we show that the synchronous optimal adaptive control algorithm can approximately solve the HJB equation for nonlinear CT systems by using data measured along the trajectories in real time. This example was developed using the method of Nevistic and Primbs (1996).

Consider the affine in the control input nonlinear system $\dot{x} = f(x) + g(x)u$, $x = [x_1 \ x_2]^T \in R^2$, where

$$f(x) = \begin{bmatrix} -x_1 + x_2 \\ -x_1^3 - x_2 - \frac{x_1^2}{x_2} + 0.25x_2(\cos(2x_1 + x_1^3) + 2)^2 \end{bmatrix}$$

$$g(x) = \begin{bmatrix} 0 \\ \cos(2x_1 + x_1^3) + 2 \end{bmatrix}.$$

We select $Q = I$, $R = 1$. Then the optimal value function that solves the HJB equation is $V^*(x) = \frac{1}{4}x_1^4 + \frac{1}{2}x_2^2$ and the optimal control policy is $u^*(x) = -\frac{1}{2}(\cos(2x_1 + x_1^3) + 2)x_2$.

We select the critic NN vector activation function as $\phi(x) = [x_1^2 \ x_2^2 \ x_1^4 \ x_2^4]^T$. The tuning algorithms (11.7-5), (11.7-6) were run for the critic NN and control actor NN, respectively, simultaneously in time. A probing noise was added to the control to guarantee persistence of excitation. This noise was decayed exponentially during the simulation. The evolution of the states is given in Figure 11.7-1. They are stable and approach zero as the probing noise decays to zero.

Figure 11.7-2 shows the critic parameters, denoted by $W_1 = [W_{c1} \ W_{c2} \ W_{c3} \ W_{c4}]^T$. After 80 sec the critic NN parameters converged to $W_1(t_f) = [0.0033 \ 0.4967 \ 0.2405 \ 0.0153]^T$, which is close to the true weights corresponding to the optimal value $V^*(x)$ that solves the HJB equation. The actor NN parameters converge to $W_2(t_f) = [0.0033 \ 0.4967 \ 0.2405 \ 0.0153]^T$. Thus, the control policy converges to

$$\hat{u}_2(x) = -\frac{1}{2} \begin{bmatrix} 0 \\ \cos(2x_1 + x_1^3) + 2 \end{bmatrix}^T \begin{bmatrix} 2x_1 & 0 & 4x_1^3 & 0 \\ 0 & 2x_2 & 0 & 4x_2^3 \end{bmatrix} \hat{W}_2(t_f).$$

This is the optimal control.

Figure 11.7-3 shows the 3-D plot of the difference between the approximated value function, by using the online synchronous adaptive algorithm, and the optimal value. The errors are small relative to the magnitude of the optimal value. Figure 11.7-4 shows the

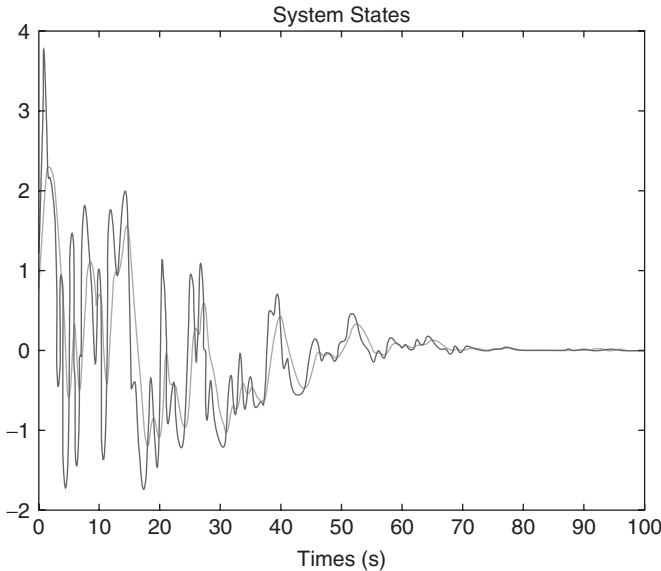


FIGURE 11.7-1 Evolution of the states, showing that the synchronous optimal adaptive controller ensures stability and regulates the states to zero.

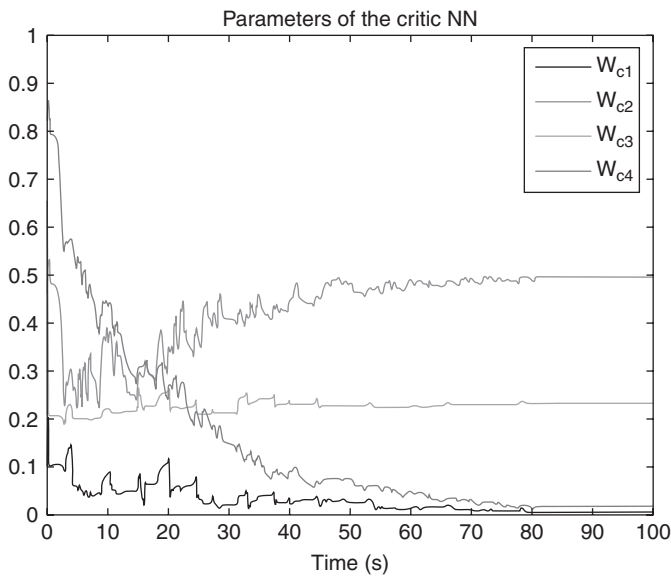


FIGURE 11.7-2 Convergence of critic NN parameters, showing that the optimal adaptive controller converges to the approximate solution of the nonlinear HJB equation.

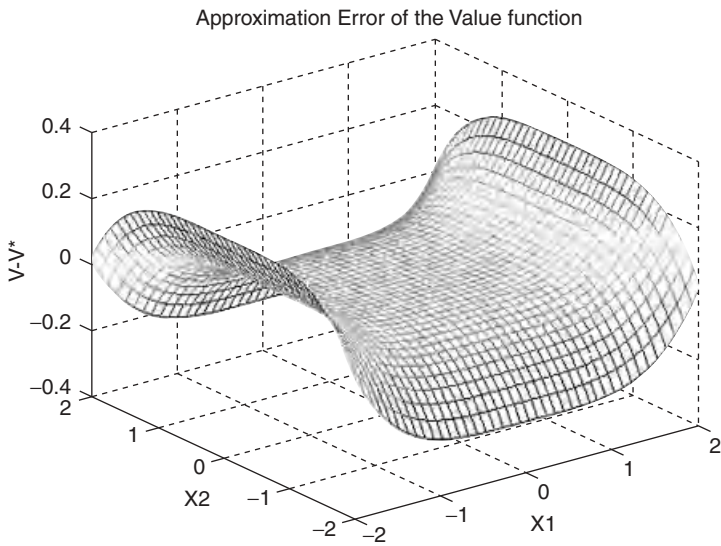


FIGURE 11.7-3 Error between optimal and approximated value function. This 3-D plot of the value function error shows that the synchronous optimal adaptive controller converges to a value function that is very close to the true solution of the HJB equation.

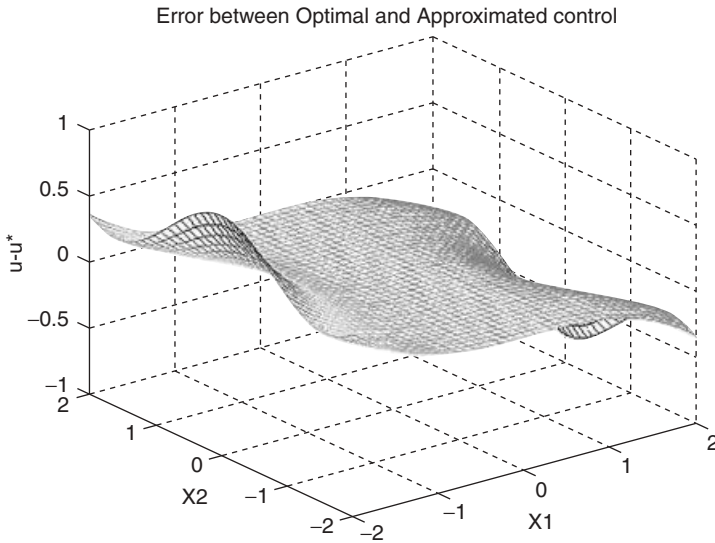


FIGURE 11.7-4 Error between optimal and approximated control input. This 3-D plot of the feedback control policy error shows that the synchronous optimal adaptive controller converges very close to the true optimal control policy.

3-D plot of the difference between the approximated feedback control policy found by using the online algorithm and the optimal control.

This example demonstrates that the synchronous optimal adaptive controller is capable of approximately solving the HJB equation online by using data measured along the system trajectories. The HJB equation for this example is intractable to solve analytically. ■